COMP3511 Midterm Cheat Book

Fall 2026 · Midterm 29-Oct-2026 (Thu) 8:00–10:00 PM · L1 **LTD** · L2 **LSKG012** · L3 **LTB**
Coverage: Chapters 1–6 (to Synchronization Tools) · open-book · calculator allowed · bring rough sheets
中文解釋 + English keywords / model answers · 所有 fork / scheduling / sync 答案已用程式驗證

點用（How to use）

1. **唔記得點用？翻 H（攻略）**：fork / scheduling / sync 每類題「認題型 → 翻邊頁 → 照做 → 照寫」。
2. **先睇 E（考場一頁通）**：公式卡、本科 conventions、陷阱表 — MC 前掃一次。
3. **計數題用 P（Playbooks）**：fork / pipe / scheduling / sync 步驟 + 預計表（Amdahl、powers、RM bound）。
4. **概念題 / short question / T-F** → 先翻 **X（Answer Bank）**：按章節排嘅預測題，英文答案直接抄；每章尾有 True/False 快速判斷。冇再去章節（1–6）📝 Model answer。
5. **PA / lab API / coding 題** → X 最後嘅 **完整 C programs（已編譯測試）** + L（Labs & PA1/PA2）+ A（API 速查）。
6. **見到似曾相識嘅題目** → Q（歷屆全解）/ M（MC key）。**搵字** → 最後嘅 Index（紅色 = 主要頁）。

Tab legend

H	How-to Guide · 攻略 • fork • sched • sync	4	E	Exam-Day Essentials · 考場一頁通：公式 • 慣例 • 陷阱	15
P	Playbooks · 解題流程 + 預計表	17	X	Answer Bank · 預測題 + 直接抄答案 • C programs	20
1	Ch1 Introduction · 引言	82	2	Ch2 OS Structures · 作業系統結構	89
3	Ch3 Processes · 進程 • fork • IPC • interrupt	99	4	Ch4 Threads · 線程與並行	112
5	Ch5 CPU Scheduling · CPU 排程	120	6	Ch6 Synchronization · 同步工具	132
L	Labs & PA1/PA2 · Lab 1–6 • PA 考點 • lab 例題	139	Q	Past Papers Solved · 歷屆長題全解	146
M	MC Answer Key · MC 題庫 + 答案	158	A	C / Pthreads API · Lab / PA API 速查	167
G	Glossary A-Z · 術語表	171	I	Index · 索引	175

題型 Router（見到 → 翻去）

Stuck on fork / scheduling / sync? → how-to guide	4
Formula card / course conventions / top traps	15
ANY short question → predicted Q + copy answer	20
Write C code: fork / pipe / dup2 / exec / pthreads / sem	74
Interrupt / DMA / trap / dual mode / privileged	99
Cache, locality, average access time	84
von Neumann, storage hierarchy, main memory	83
SMP / multicore / NUMA / multiprocessor	85
Multiprogramming vs timesharing; virtualization; cloud	86
API vs system call, printf→write, parameters	90
Linker vs loader, DLL	92
Monolithic / layered / microkernel / LKM / hybrid	93
Process states, PCB, schedulers, context switch	101
fork() counting / tree / outputs / exec cases	17
pipe / dup / open / read / write fill-in	18
Zombie / orphan / termination	105
IPC, message passing, pipes, sockets	108
Threads: shared vs private, benefits, models	112
Amdahl's Law (pre-computed table)	19
Signals, cancellation, fork/exec semantics	116
Gantt chart procedure + tie rules	18
Metrics: TAT / WT / response (course def)	120
MLFQ rules + example	123
RM / EDF / real-time	126
PCS / SCS, affinity, load balancing, CMT	124
Past scheduling answers (all papers)	129
Race condition possible values	136
Semaphores, spinlock with TAS / CAS / swap	133
Critical-section requirements	132
Lab 3 system calls, exec family, dup / dup2	140
PA1 simplified shell (what the exam may ask)	141
Named pipe / FIFO, popen, Pthreads (Lab 4)	142
Priority + RR with arrivals; PA2 MLFQ spec	143
Peterson + Lab 6 sync sample questions	144

≡ Contents 目錄

H · How-to Guide 4
攻略：fork / pipe 題點用 cheat book 攞分 4
攻略：CPU scheduling 題點用 cheat book 攞分 8
攻略：Synchronization 題點用 cheat book 攞分 12

E · Exam-Day Essentials 15
E1. 必背公式卡 Formula Card 15
E2. 本科 Conventions（考試照用，唔確定就寫 assumption） 15
E3. 最易中伏 Top Traps（MC 前睇一次） 15
E4. 答題模板 Answer Templates 16

P · Playbooks 17
P1. fork() Playbook 17
P2. Pipe / dup / exec 填空 Playbook 18
P3. CPU Scheduling Playbook 18
P4. Synchronization Playbook 18
P5. 預先計好嘅表 Pre-computed Tables 19

X · Answer Bank 20
題目搜尋表 Question finder 20
X1 · Ch1 Introduction 22
X2 · Ch2 OS Structures 28
X3 · Ch3 Processes · IPC · Interrupts 35
X4 · Ch4 Threads 45
X5 · Ch5 CPU Scheduling 53
X6 · Ch6 Synchronization 63
XI · Cross-chapter traces（綜合「發生咩事」題） 73
XC · Copy-ready C programs（Lab / PA coding，已編譯測... 74

1 · Ch1 Introduction 82
1. What is an Operating System?（OS 係乜？）[1.20–1.24] 82
2. Computer System Organization（電腦系統組織）[1.25–1... 82
3. Storage（儲存）[1.28–1.35] 83
4. Caching ⭐⭐（必考 short question）[1.32–1.33] 84
5. Processor Systems（處理器系統）[1.36–1.41] 85
6. Multiprogramming & Timesharing ⭐ [1.42] 86
7. Virtualization ⭐ [1.43–1.46] 86
8. Cloud Computing [1.47–1.48] 87
9. I/O Subsystem（Chapter 1 summary 提及） 87
10. 常見陷阱 & MC 速查（Traps） 87
11. Past-paper 題目（Ch1 相關） 88
12. 自我測驗（Self-test） 88

2 · Ch2 OS Structures 89
1. Operating System Services（OS 服務）[2.4–2.7] 89
2. User Interfaces [2.8–2.12] 90
3. System Calls ⭐⭐ [2.13–2.26] 90
4. System Programs（系統程式）[2.27–2.30] 92
5. Linkers and Loaders ⭐⭐（F19 Q1.1、F20 Q2.2 都考過... 92
6. OS Design and Implementation [2.33–2.37] 93
7. Operating System Structure ⭐⭐⭐（必考比較題）[2.... 93
8. History of OS [2.60] 97
9. 補充：Protection & Security（Chapter 2 summary） 97
10. 常見陷阱 & MC 速查 97
11. Past-paper 題目（Ch2） 97
12. 自我測驗 97

3 · Ch3 Processes 99
0. Computer-System Operation：I/O、Interrupt、DMA、D... 99
· 0.1 I/O 基本運作 [3.21] 99
· 0.2 Interrupts ⭐⭐ [3.22–3.24] 99
· 0.3 DMA (Direct Memory Access) ⭐⭐ [3.25–3.26] 99
· 0.4 Dual-mode Operation ⭐⭐ [3.27–3.31] 100
· 0.5 Three Types of Mode Transfer（user → kernel）⭐ [3.... 100
1. Process Concept（進程概念）[3.4–3.8] 100
· 1.1 Four Fundamental OS Concepts [3.4] 100
· 1.2 Process 嘅組成部分 [3.5] ⭐ 101
· 1.3 Loading a program into memory [3.7] 101
· 1.4 Memory Layout of a C Program [3.8] 101
2. Process State（進程狀態）⭐⭐ [3.9] 101
3. Process Control Block (PCB) ⭐ [3.10, 3.13] 102
4. Threads（簡介，詳見 Ch4）[3.11–3.12] 102
5. Process Scheduling（進程排程）⭐ [3.14–3.18] 102
· 5.1 基本 102
· 5.2 Scheduling Queues [3.15–3.16] 103
· 5.3 Schedulers ⭐⭐ [3.17–3.18] 103
6. Context Switch（上下文切換）⭐⭐ [3.19–3.20] 103
7. Process Creation（建立進程）⭐⭐⭐ [3.32–3.46] 104
· 7.1 概念 104
· 7.2 fork() ⭐⭐⭐ [3.35–3.39] 104
· 7.3 Copy-on-Write (COW) & clone() [3.46] 104
· 7.4 exec / wait / exit / signal [3.41–3.43] 105
8. Process Termination（終止）⭐⭐ [3.47–3.50] 105
· 8.1 Zombie ⭐⭐ 105
· 8.2 Orphan ⭐ 105
9. fork() 解題大法 ⭐⭐⭐（考試必用） 105
· 9.1 基本規則 105
· 9.2 常見 pattern 公式（全部已用真 C 程式驗證 ✅） 106
· 9.3 有 break / return 嘅 loop（追蹤「仲 active」嘅 proces... 106
· 9.4 Short-circuit 題 106
· 9.5 Nested / combined 106
· 9.6 Possible output orderings（輸出次序題）⭐ 107
· 9.7 fork + exec 成功/失敗情況（S26 Q3.2）⭐ 108
· 9.8 更多歷屆 fork 題（Lab 3 review 提供：F22 / F24 / S25... 108
· 9.9 Buffering 陷阱 108
10. Interprocess Communication (IPC) ⭐⭐ [3.51–3.64] 108
· 10.1 Independent vs Cooperating 108
· 10.2 Producer–Consumer Problem（Bounded Buffer, Shar... 108
· 10.3 Message Passing [3.57–3.64] 108
11. Pipes ⭐⭐⭐ [3.65–3.69] 109
· 11.1 設計問題（4 個） 109
· 11.2 Ordinary Pipes vs Named Pipes（必背表） 109
· 11.3 Pipe example（Parent → Child）[3.68] 110
· 11.4 dup / dup2 & I/O redirection（Lab 3 / PA1）⭐⭐ 110
12. Client–Server Communication：Sockets & RPC ⭐ [3.7... 110
13. 常見陷阱 & MC 速查 110
14. Past-paper 題目（Ch3） 111
15. 自我測驗 111

4 · Ch4 Threads 112
1. Motivation & Overview（動機）[4.4–4.6] 112
2. Benefits（四大好處）⭐⭐ [4.7] 113
3. Multicore Programming（多核心編程）⭐ [4.8–4.11] 113
4. Amdahl's Law ⭐⭐⭐（計數題必考）[4.12–4.13] 114
5. Multithreaded Process & TCB ⭐ [4.14–4.18] 114
6. User Threads vs Kernel Threads ⭐⭐ [4.19–4.20] 115
7. Multithreading Models ⭐⭐⭐ [4.21–4.26] 115
8. Threading Issues ⭐⭐ [4.27–4.33] 116
9. OS Examples [4.34–4.38] 117
10. Pthreads API 速查（Lab 4 / Ch6 都用） 117
11. 常見陷阱 & MC 速查 118
12. Past-paper 題目（Ch4） 118
13. 自我測驗 118

5 · Ch5 CPU Scheduling 120
1. Basic Concepts（基本概念）[5.4–5.7] 120
· 1.1 CPU–I/O Burst Cycle 120
· 1.2 When does scheduling happen? ⭐⭐ [5.6] 120
· 1.3 Dispatcher [5.7] 120
2. Scheduling Metrics（排程指標）⭐⭐⭐ [5.8, 5.12] 120
· 2.1 公式（本科 convention，必背）⭐ 121
3. FCFS — First-Come, First-Served ⭐ [5.9–5.10] 121
4. RR — Round Robin ⭐⭐⭐ [5.11–5.16] 121
· 4.1 Time quantum 大細 [5.14, 5.16] ⭐ 121
· 4.2 FCFS vs RR ⭐（F19 Q3.2 考過）[5.15] 121
· 4.3 RR 做 Gantt chart 規則（考試用） 122
5. SJF / SRTF — Shortest Job First ⭐⭐⭐ [5.17–5.24] 122
· 5.1 SJF（non-preemptive） 122
· 5.2 SRTF — Shortest-Remaining-Time-First（preemptive … 122
· 5.3 比較 [5.24] ⭐ 122
· 5.4 Exponential Averaging（預測下一個 CPU burst）⭐… 122
6. Priority Scheduling ⭐⭐ [5.25–5.27] 122
7. Multilevel Queue（多級佇列）⭐ [5.28–5.29] 123
8. MLFQ — Multilevel Feedback Queue ⭐⭐⭐（每年必… 123
· 8.1 Slide example 規則（考試照用）[5.31] ⭐ 124
· 8.2 Lecture example [5.32–5.34]（Q0 q=4, Q1 q=8, Q2 FC… 124
· 8.3 MLFQ advantages ⭐⭐（F19 Q3.3、F20 Q2.5 必背… 124
· 8.4 MLFQ 做題 checklist ✍ 124
9. Thread Scheduling ⭐⭐ [5.36–5.37] 124
10. Multiple-Processor Scheduling ⭐⭐ [5.38–5.46] 125
· 10.1 AMP vs SMP [5.38–5.39] 125
· 10.2 Multicore & Multithreaded cores (CMT) ⭐ [5.40–5.44] 125
· 10.3 Processor Affinity ⭐⭐ [5.45] 125
· 10.4 Load Balancing ⭐ [5.46] 126
11. Real-Time CPU Scheduling ⭐⭐⭐ [5.47–5.52] 126
· 11.1 Hard vs Soft ⭐ 126
· 11.2 Periodic tasks [5.48] 126
· 11.3 Rate-Monotonic (RM) ⭐⭐ [5.49–5.50] 126
· 11.4 EDF — Earliest Deadline First ⭐⭐ [5.51–5.52] 127
· 11.5 RM / EDF 做題方法 ✍ 127
12. Algorithm Evaluation [5.53–5.60] 128
13. Gantt Chart 萬用流程（考試步驟）✍⭐⭐⭐ 128
· 13.1 Tie-breaking（寫 assumption！） 129
14. Past-paper 計算題全集（全部用程式驗證 ✅） 129
· 14.1 S26 Q4.1 MLFQ（Q0 RR4, Q1 RR8, Q2 FCFS；preem… 129
· 14.2 S26 Q4.3 Dynamic priority（larger = higher；waiting… 129
· 14.3 S26 Q2.3 自訂 policy（揀 f_i 最大） 129
· 14.4 F19 Q3.5 LUF (Least Used First)（揀用 CPU 最少嘅；t… 129
· 14.5 F19 Q3.6（P1 0/8, P2 7/13, P3 9/1, P4 11/5, P5 20/7, … 130
· 14.6 F20 Q4.1（P1 0/9, P2 3/3, P3 8/2, P4 10/6, P5 16/8, P… 130
· 14.7 F20 Q4.2 Two-level feedback queue with aging（Q0 … 130
· 14.8 F20 Q4.3 Priority + RR（smaller = higher；preempti… 130
· 14.9 Sp21 Q2.1（P1 0/2, P2 4/12, P3 6/6, P4 9/4, P5 13/5） 130
· 14.10 Sp21 Q2.2 MLFQ（Q0 q=2, Q1 q=4, Q2 FCFS） 130
· 14.11 HW2（P1 0/7, P2 1/3, P3 4/5, P4 10/2） 130
15. 常見陷阱 & MC 速查 130
16. 自我測驗 131

6 · Ch6 Synchronization 132
1. Background（背景）[6.4–6.8] 132
2. Race Condition & Critical-Section Problem ⭐⭐⭐ [6.9… 132
3. Synchronization Hardware ⭐⭐ [6.13–6.21] 133
4. Mutex Locks ⭐⭐ [6.22–6.23] 135
5. Semaphores ⭐⭐⭐ [6.24–6.28] 135
6. 做題方法 ✍⭐⭐⭐ 136
7. 補充：Peterson（Lab 6）& classic problems（Ch7 預覽） 137
8. 常見陷阱 & MC 速查 138
9. Past-paper 題目（Ch6） 138
10. 自我測驗 138

L · Labs & PA1/PA2 139
L1. Lab 1 — Linux 環境 & Shell 指令 139
L2. Lab 2 — C in Linux、編譯流程、Ch1–3 Review 139
· L2.1 Compile pipeline（同 Ch2 linker/loader 掛鈎） 139
· L2.2 I/O Subsystem 補充（Lab 2 review slides） 139
· L2.3 Lab 2 fork 題（已驗證 ✅） 139
L3. Lab 3 — Linux Process & Pipe System Calls（⭐⭐⭐ … 140
· L3.1 本 lab 用到嘅 system calls 140
· L3.2 Zombie / Orphan demo（lab 示範） 140
· L3.3 File operations（低階 vs stdio） 140
· L3.4 Pipes 140
· L3.5 exec family 140
· L3.6 dup / dup2（redirection 核心） 141
· L3.7 Lab 3 fork 例題 + 更多歷屆 fork 題（全部驗證 ✅） 141
· L3.8 PA1 — Simplified Linux Shell（考試可能問） 141
L4. Lab 4 — Named Pipes & Pthreads 142
· L4.1 Anonymous vs Named pipes（lab 版本） 142
· L4.2 Exam example — 兩條 ordinary pipes 雙向通訊（填… 142
· L4.3 Pthreads 142
· L4.4 Exam example — thread vs process 共享 memory（… 143
L5. Lab 5 — Scheduling Review & PA2（MLFQ） 143
· L5.1 Review 例題（P1 7, P2 4, P3 1, P4 4，全部 t=0，已驗… 143
· L5.2 ⭐ Priority + RR 有唔同 arrival time（HW / Exam 熱… 143
· L5.3 RM / EDF 例題（deadline < period） 143
· L5.4 PA2 — N-level MLFQ simulator 143
L6. Lab 6 — Synchronization Review（Sample Questions） 144
· L6.1 Peterson's solution（software-only，2 processes） 144
· L6.2 POSIX primitives 速查 144
· L6.3 Sample Questions（官方答案，已驗證 ✅） 144
L7. 常見陷阱（Labs / PA） 145

Q · Past Papers Solved 146
Spring 2026 Midterm（lecturer 提供） 146
Fall 2019 Midterm 148
Fall 2020 Midterm 150
Spring 2021 Midterm 152
Homework 1（Fall 2026） 153
Homework 2（Fall 2026） 153
Fall 2022 Midterm（lab slides 引用） 154
Fall 2024 Midterm（lab slides 引用） 155
Spring 2025 Midterm（lab slides 引用） 155
Spring 2023 Midterm（lab slides 引用） 155
Lab 2 Exercises 155
Lab 4 Exam Example 156
Lab 6 Sample Questions（官方答案） 156

M · MC Answer Key 158
歷屆 MC 158
新練習 MC 163

A · C / Pthreads API 167
1. Process APIs 167
2. File I/O（POSIX，唔係 stdio） 168
3. Pipes 168
4. Pthreads（#include <pthread.h>，compile with -pthrea… 169
5. Compile / Run（Lab 1–2） 170

G · Glossary A-Z 171

I · Index 175

H How-to Guide 攻略 · fork · sched · sync

攻略：fork / pipe 題點用 cheat book 攞分

🎯 **呢頁唔係教原理，係教你點「查書 → 照做 → 照寫」**。所有頁數 = 印出嚟嗰本 cheat book。見到 fork / pipe 題：**先認類型（§1）→ 翻指定頁 → 跟步驟 → 用「答卷寫法」寫**。

0. 入場前：貼 5 張 sticky notes

貼喺	頁	用嚟
P1 fork Playbook（pattern 表 + 已驗證答案）	**p.17**	數 process / 數 print 次數
P2 pipe / dup 填空表	**p.18**	pipe / dup2 / open 填空
P5 Powers 表（2^n、3^n、4^n、6^n...）	**p.19**	唔使計次方，直接查
Ch3 §9 fork 解題大法	**p.105**	break / 短路 / 輸出次序 / exec
XC 完整 C programs	**p.74**	「寫 program」題直接抄改

1. 先認類型（睇題目字眼，10 秒）

題目字眼	類型	跳去
"How many processes are created / in total (including the parent)?"、"How many times is ... printed?"	**A 數目**	§2
"Draw the process tree"	**B Process tree**	§3
"List all possible outputs"、"Which output is (not) possible?"	**C 輸出次序**	§4
"If `exec()` succeeds / fails..."、"if `fork()` fails..."	**D exec / fail 情況**	§5
Code 有 BLANK，周圍有 `pipe` / `dup` / `open` / `read` / `write`	**E pipe 填空**	§6
"Write a program..."、"Complete the code so that..."	**F 寫 code**	§7
fork 同 `pthread_create` 一齊出現	**G fork + threads**	§8

2. 類型 A — 數 process / 數 print（最常見）

步驟（全部係查表，唔使理解）

1. 抄低 loop 入面嗰段 code（`for (i = 0; i < n; i++) { 呢度 }`）同 **n**。
2. 翻 **P1 Step 2 pattern 表（p.17）**：喺「每 iteration」一欄搵**一模一樣**嘅 code → 讀「n 次後」formula → 代 n。
3. 次方數唔使計：翻 **P5 Powers 表（p.19）**，行 = n，欄 = formula（2^n、3^n、4^n、6^n、$2^{n+1}-1$、$2(3^n+1)$、$(4^n-1)/3$ 都有）。
4. 表冇 → 翻 **P1 Step 3「其他已驗證答案」（p.17）** 同 **Ch3 §9.8（p.108）**：好多係歷屆原題（S26、F19、F20、Sp21、F22、F24、S25、Lab），**一樣就直接抄答案**。
5. 仍然冇 → 用「**一輪法**」（純機械）：
    - 喺 rough paper 由 **1 個 process** 開始，**只做一個 iteration**：照 **Ch3 §9.1 規則（p.105）** 逐句行（child 拎到 0；`A && B`：A 係 0 就唔做 B；`A || B`：A 非 0 就唔做 B）。
    - 數 **k = 做完一輪仲會繼續 loop 嘅 process 數**，同 **s = 停咗嘅**（`break` / `return` / `exit` / exec 成功）。
    - **s = 0**：答案 $= k^n$（查 P5）。
    - **s > 0**：翻 **Ch3 §9.3（p.106）** 嗰個「active 追蹤表」，逐輪填 *iteration · active · 新增 · total*。
6. 問 **print 幾多次**：
    - `printf` 喺 **loop 之後** → 次數 = 行到嗰句嘅 process 數（通常 = total；`return` / `exit` 咗嘅唔計）。
    - `printf` 喺 **loop 入面** → 每輪「執行嗰句嘅 process 數」加埋（P1 Step 3 有 F19 Q2.6 嘅 $2^{n+1}-2$ 例）。

答卷寫法（有 partial credit，S26 改法：tree 4 分 + 數目 1 分 + break 1 分）

1. 一句規則：*"In each iteration every process becomes 3 processes: the parent, its child, and the child's child."*
2. 畫頭 1–2 個 iteration 嘅 tree（§3）。
3. 寫 formula（例如 3^n）再代數。
4. 最後答案寫清楚：*"Total = 27 processes (including the original parent); X is printed 27 times."*

📝 **示範**：`for (i = 0; i < 3; i++) if (fork() == 0) fork();` 之後 `printf("X\n");`，問 X 印幾多次。

- 查 P1 表：row `if (fork() == 0) fork();` → 每 process ×3 → 3^n。
- 查 P5：$n=3 \Rightarrow 3^3=27$。
- `printf` 喺 loop 之後，每個 process 印一次 → **27**。
- 寫：*"In each iteration, fork() returns 0 in the child, so only the child forks again: each process becomes 3. After 3 iterations:* $3^3=27$ *processes, so X is printed 27 times."* + 1 個 iteration 嘅 tree（P0 → C1 → C2）。

📝 **例子 2（有 `break`）**：`for (i = 0; i < 3; i++) { if (fork()) fork(); else break; }`，問總共幾多個 process、幾多個 process 行 `break`。

- 查 **P1 表（p.17）**：row `if (fork()) fork(); else break;` → active ×2；total $2^{n+1}-1$；breaks 2^n-1（S26 Q3.1）。
- 查 **P5（p.19）** 嘅 $2^{n+1}-1$ 同 2^n-1 兩欄，n = 3 → **15 processes、7 breaks**。
- 想自己核對 → 一輪法表（每個 active process 每輪：第一個 child break；parent 同第二個 child 繼續）：

iteration	active（繼續 loop）	今輪新 break	total
開始	1	0	1
i = 0	2	1	3
i = 1	4	2	7
i = 2	8	4	15

- 寫：*"In each iteration the first child breaks out of the loop, while the parent and the second child continue, so the active processes double. Total =* $2^{n+1}-1$ *= 15 processes;* 2^n-1 *= 7 of them execute break."*

3. 類型 B — 畫 process tree

1. 翻 **P1 Step 4（p.17）**：每個 node 寫名（P0、P1、P2…），旁邊寫 **佢喺第幾個 iteration（i 值）出世**；child 由 fork 下一句開始、**繼承 i 值**繼續 loop。
2. 由 P0 開始，一個 iteration 畫一層：每個仍然 active 嘅 node 照嗰輪 code 逐句行，**邊個 process 執行 `fork()`，就由佢拉一條箭咀去新 child**（child 再 fork 就由 child 再拉，所以會有孫）。
3. 有 `break` / `return` → 嗰個 node 之後**唔再生**。例子：**Ch3 §9.3（p.106）**、原題 **S26 Q3.1（p.146）**。
4. **自我檢查**：node 總數 = §2 計到嘅 total。

答卷寫法：箭咀 parent → child，每條線標 *i = 0 / 1 / 2*；最底寫 *"Total: N processes"*。

📝 **例子**：`for (i = 0; i < 2; i++) fork();` 畫 process tree。

```
P0 --(i=0)--> P1 --(i=1)--> P3
P0 --(i=1)--> P2
```

- P0 喺 i = 0 生 P1、i = 1 生 P2；P1 由 i = 0 嗰個 fork 之後開始（i 仲係 0），所以 i = 1 再生 P3；P2、P3 出世時已經係最後一輪，唔再生。
- 檢查：node 數 = 4 = P1 表 `fork();` 嘅 2^n（n = 2）✓。答卷最底寫 *"Total: 4 processes."*

4. 類型 C — 所有可能輸出 / 次序

1. 翻 **P1 Step 5（p.17）** 同 **Ch3 §9.6（p.107）**。
2. 喺 rough paper：**每個 process 一行**，寫低佢印嘅字（同一 process 嘅次序**固定**）。
3. 加約束（用箭咀連）：
   - parent 喺 `wait()` 之後印嘅 → 一定喺 child（同 child 嘅子孫）所有輸出**之後**；
   - fork 之前已印嘅 → 最先；
   - `read()` 一條 pipe → 要等對方 `write()` 之後（例：**X3-50（p.43）**）。
4. 逐個位揀「而家可以印邊個」→ 列晒所有次序；數量可以用 Ch3 §9.6 嘅「插空位」方法計。
5. 同類原題：**F20 Q3.1（p.151）**、**Sp21 Q1.3（p.152）**；threads 版：**X4-24（p.48）**。

答卷寫法：*"Assume each printf is flushed immediately."* 然後 (1)、(2)、(3)… 逐個 output 列出；問「幾多個」就寫總數 + 點數。

📝 **例子**（假設 printf 即時 flush）：

```
printf("A");
if (fork() == 0) { printf("B"); printf("D"); }
else             { wait(NULL); printf("C"); printf("E"); }
```

- 每個 process 一行：child：B → D；parent：（wait）→ C → E；A 喺 fork 之前，**最先**。
- 約束：parent `wait(NULL)` → C、E 一定喺 B、D 之後 → **只有 1 個 output：ABDCE**。
- 如果冇 `wait(NULL)`：A 之後係 (B, D) 同 (C, E) 嘅任意合併 → $\binom{4}{2}=6$ 個：**ABDCE、ABCDE、ABCED、ACBDE、ACBED、ACEBD**。

5. 類型 D — exec 成功 / 失敗、fork 失敗

1. 翻 **P1 Step 6（p.18）** + **Ch3 §9.7（p.108）**；原題 **S26 Q3.2（p.147）**。
2. 照三個情況逐個寫：
   - **(I) fork ✓、exec ✓**：child 只印 exec **之前**嘅嘢（exec 之後嘅 code **永遠唔行**）+ parent 嘅輸出。
   - **(II) fork ✓、exec ✗**：exec 返回 −1，child **繼續行 exec 之後嘅 code**（loop 入面會繼續 fork！見 **X3-31（p.40）**）。
   - **(III) fork ✗**：冇 child，只有 `pid < 0` 嗰個 branch 嘅輸出。

答卷寫法：三個 bullet，每個寫「印咩 + 一句原因」（*"exec replaces the process image, so the code after it never runs"*）。

📝 **例子**：

```
printf("1");
pid_t pid = fork();
if (pid < 0)       printf("F");
else if (pid == 0) { execlp("ls", "ls", NULL); printf("2"); }
else               { wait(NULL); printf("3"); }
```

- **(I) fork ✓、exec ✓：1、ls 嘅輸出、3**（"2" 永遠唔印：exec 已經換咗成個 program）。
- **(II) fork ✓、exec ✗：1 2 3**（exec 返回 −1，child 繼續行 `printf("2")`；parent wait 完先印 3）。
- **(III) fork ✗：1 F**（冇 child）。

6. 類型 E — pipe / dup / open 填空

1. 翻 **P2 表（p.18）**，逐個 BLANK 對「要做」一欄：

BLANK 喺邊	通常填
`fork()` **之前**	`pipe(fd);`
會 `write` 嘅 process	`close(fd[0]);` → `write(fd[1], &x, sizeof(x));` → `close(fd[1]);`
會 `read` 嘅 process	`close(fd[1]);` → `read(fd[0], &x, sizeof(x));` → `close(fd[0]);`
要 stdout 去 pipe / file	`dup2(fd[1], 1);` 或 `dup2(fo, 1);`
要 stdin 由 pipe / file	`dup2(fd[0], 0);` 或 `close(0); dup(fin);`
開檔	`open(name, O_RDONLY)` / `open(name, O_WRONLY | O_CREAT | O_TRUNC, 0644)`

2. **點知邊個係 writer / reader**：睇 BLANK 之後嗰句用 `read` 定 `write`、用邊個 fd。
3. 兩條 pipe（雙向）→ 抄 **XC-07（p.76）** / 原題 **Lab 4 EX1（p.156）**。
4. 交卷前 checklist：
   - 每個 process 都 **close 自己唔用嗰端**（否則 reader 收唔到 EOF、永遠 block）；
   - `read` / `write` 第二個參數係 **address**（`&x` / `buf`），第三個係 **size**（`sizeof(x)`）；
   - **fd[0] = read，fd[1] = write**；
   - 個 BLANK 唔使填嘢 → 寫 `/* Nothing */`（**E4 模板，p.16**）。
5. 同類原題：**S26 Q3.3（p.147）**、**HW1 Q3.3（p.153）**、**HW1 Q2.5（p.153）**；完整程式：**XC-05（p.75）**、**XC-06（p.75）**、**XC-08（p.76）**、**XC-09（p.76）**。

📝 **例子**：parent 將 x = 6 經 pipe 傳畀 child，child 印出嚟。

```
int fd[2], x = 6, y;
BLANK1;
if (fork() == 0) { BLANK2; BLANK3; printf("%d\n", y); }
else             { BLANK4; BLANK5; wait(NULL); }
```

- child 印 `y` → child = **reader**；parent = **writer**（P2 表 Reader / Writer 兩行照抄）。
- **BLANK1** `pipe(fd);`（fork 之前）· **BLANK2** `close(fd[1]);` · **BLANK3** `read(fd[0], &y, sizeof(y));` · **BLANK4** `close(fd[0]);` · **BLANK5** `write(fd[1], &x, sizeof(x));` → 印 **6**。

7. 類型 F — 叫你寫 program

翻 **XC（p.74）**，搵最似嘅 program **成段抄**，再改字串 / 檔名 / 數目：

題目要求	抄
parent 等 child、印 exit status	**XC-01（p.74）**
開 N 個 children / 一條 chain	**XC-02（p.74）** / **XC-03（p.74）**
用 exec 行一個 command	**XC-04（p.74）**
parent → child 傳字串	**XC-05（p.75）**
child 計完傳結果畀 parent	**XC-06（p.75）**
雙向溝通	**XC-07（p.76）**
`cmd < in > out` redirection	**XC-08（p.76）**
`cmd1 | cmd2`（例如 `ls | wc -l`）	**XC-09（p.76）**
PA1 多段 pipe	**XC-10（p.77）**
簡單 shell / `cd` / Ctrl-C	**XC-11（p.77）**
named pipe (FIFO)	**XC-12（p.78）**
zombie / orphan 示範	**XC-19（p.81）**

答卷寫法：保留 comments（graders 睇得明 = 攞分）；記得 `wait()`、exec 之後 `perror(...); exit(1);`、close 唔用嘅 pipe ends。API 參數唔記得 → **A 部 API 速查（p.167）**。

📝 **例子**：「Write a program in which the parent sends "hello" to its child through a pipe, and the child prints it.」

- 翻 **XC-05**（**p.75**）（parent → child 傳字串）→ **成段抄**，只改一行：`const char *msg = "Greetings";` → `const char *msg = "hello";`。
- 其餘（`pipe` 喺 fork 之前、兩邊 close 唔用嗰端、`wait(NULL)`）保持不變 = 全部都係擺分位。

8. 類型 G — fork + pthread_create 混合

直接翻 **X4-32 至 X4-36**（**p.50–p.51**）：已解好「幾多 process / thread / 印幾多次」嘅五種寫法。規則（抄落答卷）：*"fork() in a multithreaded process duplicates only the calling thread (Linux)."*；*"Threads share globals, processes get a copy."*（**X4-34**（**p.50**））。

📝 **例子**：global `count = 0`；main 開一個 thread 做 `count++` 再 join；跟住 `fork()`；child 再開 thread 做 `count++`，印 count；parent `wait(NULL)` 之後印 count。→ 同 **X4-34** 一模一樣：**`child: 2` 先，之後 `parent: 1`**（thread 同 main 共用 count → 1；fork 之後 child 有自己一份 → 2；parent 嗰份仍然係 1）。

9. 交卷前 30 秒 checklist

- child 拎 **0**、parent 拎 **child PID**、失敗 **−1**（**E3 traps**，**p.15**）。
- `&&` / `||` 短路有冇計錯？
- `break` / `return` 只停**嗰個** process。
- exec 成功 → 之後唔行。
- 有冇寫 assumption：*"all fork() calls succeed; printf is flushed immediately."*

攻略：CPU scheduling 題點用 cheat book 攞分

🎯 **呢頁唔係教原理，係教你點「查書 → 照做 → 照寫」**。頁數 = 印出嚟嗰本 cheat book。Scheduling 係**最多分**嘅部分（歷屆約 27 分），而且 **Gantt chart 由左至右改，錯一格之後冇分**（F20 改卷規則），所以每一步都照書做、逐格對。

0. 入場前：貼 6 張 sticky notes

貼喺	頁	用嚟
E2 本科 Conventions	**p.15**	tie、RR 新 arrival、MLFQ 規則、response time 定義
E4 答題模板	**p.16**	Gantt + 表 + 平均數寫法
P3 Scheduling Playbook	**p.18**	每個 algorithm「揀邊個 / 幾時重揀」
P5 RM bound 表	**p.19**	RM 可唔可以排
Ch5 §13 Gantt 萬用流程	**p.128**	逐步 + tie-breaking
Ch5 §14 歷屆計算全集	**p.129**	同類已解題（對答案用）

1. 先認類型

題目字眼	跳去
FCFS / SJF / SRTF / RR / (non-)preemptive priority + "Gantt chart"、"average waiting / turnaround / response time"	**§2**
"priority scheduling ... processes with the same priority are scheduled RR"	**§3**
"multilevel feedback queue"、Q0 / Q1 / Q2、"aging"、"two-level"	**§4**
"rate-monotonic"、"EDF"、"period"、"deadline"、"CPU utilization"	**§5**
"predict the next CPU burst"（α、τ）、Little's formula、context-switch overhead、RR bound	**§6**
自訂 policy：dynamic priority（α、β）、"least used first"、公式 f_i	**§7**
冇數字：why / compare / which algorithm...	題目搜尋表 **p.20** → **X5**（**p.53**）直接抄

2. FCFS / SJF / SRTF / RR / Priority：Gantt + 平均數

Step 0 — 先寫 assumptions（照抄 E2，p.15），放喺答案最頂：*"Assumptions: ties are broken by FCFS, then smaller process ID; in RR a newly arrived process enters the ready queue before the preempted process; response time = end of the first run − arrival time."*

Step 1 — 喺 rough paper 抄個表：P、arrival、burst（、priority）。

Step 2 — 翻 **P3 表（p.18）**，搵你個 algorithm 嗰行，抄低兩樣嘢：**「揀邊個」**同**「幾時重新揀」**：

你個 algorithm	揀邊個	幾時重新揀
FCFS	最早到	有人完成
SJF	**已經到咗**嘅最短 burst	有人完成
SRTF	最短 **remaining**	有人完成 + **每個 arrival**（平手唔 preempt）
RR q	queue 頭	完成 / 用完 q（用完 → 排 queue 尾）
Priority NP / P	最細數字	完成 /（P：較高 priority 嘅 arrival）

Step 3 — 喺時間軸點出所有 **arrival 時間**；之後一路行一路加 **完成時間**同（RR）**quantum 用完時間**。

Step 4 — 每個 event 時間，寫低 **ready queue（照次序）**，再照 Step 2 揀下一個。RR：用完 q 嘅排**尾**；同一時刻有新 arrival → **新 arrival 先入**。

Step 5 — 寫 Gantt：`0 P1 3 P3 5 P1 8 …`（**每個 boundary 都寫**；CPU 冇嘢做都要寫 idle）。

Step 6 — 填表（公式喺 **E1，p.15**）：

```
P  | Arr | Burst | Completion | TAT = C - A | WT = TAT - Burst | RT = end of 1st run - A
```

- **RT 用本科定義**：第一次上 CPU 嗰段**完結**嘅時間 − arrival（non-preemptive 時 RT = TAT）。

Step 7 — 平均數連 working：*avg WT = (2 + 10 + 0 + 3) / 4 = 3.75*。

Step 8 — 自我檢查（P3 第 8 步）：最後完成時間 = 全部 burst 加埋 + idle；每個 process 喺 Gantt 上各段加埋 = 佢嘅 burst。

📝 **示範（SRTF）**：P1 (0, 6)、P2 (2, 5)、P3 (3, 2)、P4 (5, 4)。

- P3 表 SRTF：揀最短 remaining；每個 arrival 都重新揀；平手唔 preempt。
- t = 0：得 P1 → 行 P1。t = 2：P2 (5) vs P1 剩 4 → 繼續 P1。t = 3：P3 (2) vs P1 剩 3 → **轉 P3**。
- t = 5：P3 完；ready = P1 (3)、P2 (5)、P4 (4) → P1。t = 8：P1 完 → P4 (4) < P2 (5) → P4。t = 12 → P2 → 17。
- Gantt `0 P1 3 P3 5 P1 8 P4 12 P2 17`；WT = 2, 10, 0, 3 → **avg 3.75**。
- 書入面 **X5-22（p.56）** 有同一組數嘅 FCFS / SJF / SRTF / RR 全部答案 → **考前用佢練一次，對答案**。

📝 **例子 2（RR q = 2，同一組數）**：RR 最易錯 queue 次序，所以每個 event 都寫 ready queue（Step 4）：

t	event	ready queue（頭 → 尾）	行
0	P1 到	P1	P1 0–2
2	P2 到 + P1 用完 q	**P2**, P1（新 arrival 先入）	P2 2–4
3	P3 到	P1, P3	—
4	P2 用完 q	P1, P3, P2	P1 4–6
5	P4 到	P3, P2, P4	—
6	P1 用完 q	P3, P2, P4, P1	P3 6–8（完）
8	P3 完	P2, P4, P1	P2 8–10
10	P2 用完 q	P4, P1, P2	P4 10–12
12	P4 用完 q	P1, P2, P4	P1 12–14（完）
14	P1 完	P2, P4	P2 14–15（完），P4 15–17（完）

Gantt `0 P1 2 P2 4 P1 6 P3 8 P2 10 P4 12 P1 14 P2 15 P4 17`；RT（第一段完結 − arrival）= 2, 2, 5, 7 → **avg 4**；WT = 8, 8, 3, 8 → **avg 6.75**（同 X5-22 一樣 ✓）。

同類已解題（考試時見到似嘅就對住做）：X5-22（p.56）、X5-25 priority（p.56）、F19 Q3.6（p.150）、F20 Q4.1（p.151）、Sp21 Q2.1（p.152）、HW2 Q3.2（p.154）、Lab 5 review（p.143）。

3. Priority + RR（同 priority 輪流）

1. 照 §2 步驟，但**每個 priority level 一條 queue**（細數字 = 高，**E2 第 4 點**）。
2. 規則（**X5-26，p.56** 有完整示範）：
    - 被**更高 priority** 嘅 arrival preempt → 返**自己 level 嘅 queue 頭，保留剩餘 quantum**；
    - 用完 quantum → 排自己 level 嘅 **queue 尾**；
    - 同一 level 得一個 process → 佢繼續行。
3. 同類原題：**S23（p.155）、F20 Q4.3（p.151）、Lab 5 L5.2（p.143）**。
4. 最常錯：preempt 完重新畀返成個 quantum（**錯，書入面有標**）。

例子（X5-26 嘅數）：P1 (0, 5, pri 2)、P2 (0, 4, 2)、P3 (3, 2, 1)、P4 (4, 3, 2)；q = 2；細數字 = 高。

- t = 0：level 2 queue = P1, P2（同時到 → 細 ID 先）→ P1 0–2；用完 q → 排尾：[P2, P1]。
- t = 2：P2 行。t = 3：**P3（pri 1）preempt P2**（P2 用咗 1 unit）→ P2 返 **level 2 queue 頭，保留剩低 1 unit quantum**：[P2, P1]。
- t = 4：P4 到 → [P2, P1, P4]。t = 5：P3 完 → **P2 只行 5–6**（剩低嗰 1 unit）→ 排尾 → P1 6–8 → P4 8–10 → P2 10–12（完）→ P1 12–13（完）→ P4 13–14（完）。
- Gantt `0 P1 2 P2 3 P3 5 P2 6 P1 8 P4 10 P2 12 P1 13 P4 14`；WT = 8, 8, 0, 7 → avg = 23/4 = **5.75**。
- 錯法：t = 5 畀 P2 一個新 quantum（行 5–7）→ 之後成條 Gantt 錯晒。

4. MLFQ（每年必考）

1. 抄低題目規則（幾多條 queue、每條 quantum、最後一條 FCFS、有冇 aging）。題目冇講嘅 → 用 **E2 第 3 點（p.15）**，**寫做 assumption**。
2. Rough paper 畫 **P3 嘅 MLFQ 追蹤表（p.18）**：

```
t | event | Q0 | Q1 (rem q) | Q2 | running (queue, q left, burst left)
```

3. 每個 event 照呢幾條（E2 第 3 點）：
    - 新 job → **Q0 尾**；
    - 用完 quantum 仲未完 → **降一級，排尾**；
    - 被**高層 queue 新 arrival** preempt → 返**自己 queue 頭，保留剩餘 quantum，唔降級**；
    - 只係最高非空 queue 先可以行；Q0 job 唔會被新 arrival preempt。
4. Gantt 標 queue：`0 P1@Q0 4 P2@Q0 8 P1@Q1 …`（E2 第 8 點）。
5. 表 + 平均數同 §2 一樣。
6. 對答案用：**X5-34（p.57）** 完整示範（連 queue 狀態）、**Ch5 §8.2 lecture example（p.124）**、**§8.4 checklist（p.124）**；原題 **S26 Q4.1（p.147）**、**Sp21 Q2.2（p.152）**、**HW2 Q3.3（p.154）**；two-level + aging：**F20 Q4.2（p.151）**。
7. 純文字題（"why MLFQ"、"parameters of MLFQ"、"advantages"）→ **X5-31 至 X5-33（p.57）** 直接抄。

例子（X5-34）：Q0 RR q = 4、Q1 RR q = 8、Q2 FCFS；P1 (0, 14)、P2 (3, 5)、P3 (9, 3)、P4 (15, 2)。Rough paper 追蹤表：

t	event	Q0	Q1（剩 q）	Q2	行緊（queue, 剩 q, 剩 burst）
0	P1 到	P1			P1@Q0 q=4 b=14
3	P2 到（同層，唔 preempt）	P2			P1@Q0
4	P1 用完 q → 降 Q1		P1(8)		P2@Q0 q=4 b=5
8	P2 用完 q → 降 Q1		P1(8), P2(8)		P1@Q1 q=8 b=10
9	P3 到 → preempt P1	P3	**P1(7)**, P2(8)		P3@Q0（P1 返 Q1 頭，留 7）
12	P3 完		P1(7), P2(8)		P1@Q1 q=7 b=9

t	event	Q0	Q1（剩 q）	Q2	行緊（queue, 剩 q, 剩 burst）
15	P4 到 → preempt P1	P4	**P1(4)**, P2(8)		P4@Q0（P1 返頭，留 4）
17	P4 完		P1(4), P2(8)		P1@Q1 q=4 b=6
21	P1 用完 q → 降 Q2		P2(8)	P1	P2@Q1 b=1
22	P2 完			P1	P1@Q2 b=2
24	P1 完				—

Gantt `0 P1@Q0 4 P2@Q0 8 P1@Q1 9 P3@Q0 12 P1@Q1 15 P4@Q0 17 P1@Q1 21 P2@Q1 22 P1@Q2 24`；avg WT = (10 + 14 + 0 + 0)/4 = **6**；avg TAT = **12**；avg RT = (4 + 5 + 3 + 2)/4 = **3.5**。

- 錯法：每次 preempt 之後畀 P1 一個新嘅 8 → P1 會喺 Q1 做完，答案全錯。

5. RM / EDF（real-time）

1. **計 U**：$U = \sum_i t_i/p_i$（**E1，p.15**），寫晒條式。
2. **判斷**：
   - **RM**：查 **P5 RM bound 表**（**p.19**）（N = 2 → 0.828、N = 3 → 0.780…）。$U \le$ bound → "*schedulable by RM (guaranteed)*"；$U >$ bound → "*not guaranteed — draw the timeline*"。
   - **EDF**：$U \le 1$（deadline = period）→ "*schedulable by EDF*"。
   - $U > 1$ → "*overloaded: no algorithm can meet all deadlines*"。
3. **要畫 timeline**：照 **P3「RM / EDF 追蹤」**（**p.18**）：
   - 每個 task 列 **release 時間** $0, p, 2p, \ldots$ 同 **deadline = release + d**（題目 d < p 就用 d！見 **S26 Q4.2，p.147**）；
   - 逐個時間單位揀：**RM = period 最短**（固定）；**EDF = absolute deadline 最早**；平手 → 唔 preempt（E2 第 5 點）。
4. **Deadline miss**：完成時間 > deadline 就係 miss；soft real-time → **嗰個 job 繼續行完**，計一次 miss（E2 第 6 點）。
5. **答卷寫法**：U 條式 → 同 bound 比較一句 → Gantt → 逐個 miss 寫「P2 misses its deadline at t = 7」。

📝 **例子**（X5-53）：P1 (t = 2, p = d = 5)、P2 (t = 4, p = d = 7)，soft real-time，畫 0–14。

- U = 2/5 + 4/7 = 0.400 + 0.571 = **0.971**；查 P5：N = 2 嘅 RM bound = **0.828** → 0.971 > 0.828 → RM **not guaranteed**；0.971 ≤ 1 → **EDF schedulable**。
- Releases / deadlines：P1 @ 0, 5, 10（deadline 5, 10, 15）；P2 @ 0, 7（deadline 7, 14）。
- **RM**（P1 period 短 → 永遠優先）：`0 P1 2 P2 5 P1 7 P2 10 P1 12 P2 14` → P2 第一個 job 喺 t = 8 先完 → **miss deadline 7**（soft：繼續行完）。
- **EDF**：t = 5 時 P2 deadline 7 < P1 新 job deadline 10 → P2 繼續 → `0 P1 2 P2 6 P1 8 P2 12 P1 14` → **0 miss**。
- 寫：U 條式 → "*0.971 > 0.828, so RM is not guaranteed; 0.971 ≤ 1, so EDF can meet all deadlines.*" → 兩條 Gantt → "*Under RM, P2 misses its deadline at t = 7.*"

6. 對答案用：**X5-53**（**p.61**）（同一組 task 嘅 RM vs EDF timeline）、**X5-52**（**p.61**）（四組 U 判斷）、**Ch5 §11.5 做題方法**（**p.127**）、**Lab 5 L5.3**（**p.143**）；原題 **Sp21 Q2.3**（**p.152**）、**F19 Q3.4**（**p.149**）、**HW2 Q2.5**（**p.154**）。

6. 公式題（直接代入）

題目	公式頁	已解示範（照格式抄）
Exponential averaging（α、τ）	E1 **p.15**、P5 權重表 **p.19**	**X5-19**（**p.55**）、**X5-20**（**p.55**）
Little's formula $n = \lambda W$	E1	**X5-55**（**p.61**）（記得先轉單位）
RR + context switch：efficiency / utilization	—	**X5-13**（**p.54**）、**X5-14**（**p.55**）
RR 等待上限 $(n-1)q$	E1	**X5-11**（**p.54**）
Response time 定義	E2 第 1 點	**X5-07**（**p.54**）

答卷寫法：formula → 代數 → 答案（連 unit）。逐步寫，錯數都有 method 分。

📝 **例子**：α = 0.5、τ0 = 12 ms，實際 bursts 8、6、10、4 ms，預測 τ1–τ4。照 E1 公式 $\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$ 逐個代：

- τ1 = 0.5 × 8 + 0.5 × 12 = **10 ms**
- τ2 = 0.5 × 6 + 0.5 × 10 = **8 ms**
- τ3 = 0.5 × 10 + 0.5 × 8 = **9 ms**
- τ4 = 0.5 × 4 + 0.5 × 9 = **6.5 ms**（同 X5-19 一樣 ✓）

📝 **例子**（**Little**）：每分鐘 120 個 process 到，平均等 250 ms → 先轉單位：λ = 120/60 = 2 /s、W = 0.25 s → $n = \lambda W$ = 2 × 0.25 = **0.5**（X5-55）。

7. 自訂 policy（dynamic priority、LUF、f_i 公式）

1. P3 表有「Dynamic priority」同「LUF」兩行（**p.18**）。
2. 每個決策時間點**列一個表**：每個 process 嘅 priority / 用咗幾多 CPU / f_i 值 → 照題目規則揀 → 平手照**題目**講（例如 S26：larger ID）。
3. 對答案用：**Ch5 §14.2 至 §14.4**（**p.129**）；原題 **S26 Q4.3**（**p.147**）、**S26 Q2.3**（**p.146**）、**F19 Q3.5**（**p.149**）。

📝 **例子（表格格式，照抄落 rough paper / 答卷）**：每個決策時間點一行，**先計晒所有 process 嘅值先揀**：

t	P1 值	P2 值	P3 值	揀	原因
0	計	計	未到	…	題目規則：最大 / 最細值；平手照題目
下一個 event	重計	重計	計	…	…

數字完整示範睇 **S26 Q4.3**（dynamic priority：waiting 嘅 priority 以 α 變、running 以 β 變）同 **F19 Q3.5**（LUF：揀用 CPU 最少嗰個），兩題都係用呢個表格式做。

8. 交卷前 30 秒 checklist

- 有冇寫 **assumptions**（tie、RR 新 arrival、response time 定義）？
- Gantt **每個 boundary** 有冇寫？有冇漏 idle？
- 每個 process 嘅段數加埋 = burst？最後完成時間 = 總 burst + idle？
- RT 係「**第一段完結** − arrival」（唔係開始）？
- MLFQ：preempt 之後有冇**錯降級 / 錯 reset quantum**？

攻略：Synchronization 題點用 cheat book 攞分

🎯 **呢頁唔係教原理，係教你點「查書 → 照做 → 照寫」**。頁數 = 印出嚟嗰本 cheat book。Sync 題（歷屆約 8–15 分）幾乎全部係固定套路：**可能值、semaphore 次序、輸出序列、用 atomic instruction 寫 lock、三個 requirement 判斷**。

0. 入場前：貼 4 張 sticky notes

貼喺	頁	用嚟
P4 Synchronization Playbook	**p.18**	可能值方法、semaphore 次序規則、spinlock 表
Ch6 §6 做題方法	**p.136**	歷屆題型逐步示範
X6 答案庫（判斷表、填空、例題）	**p.63**	直接抄答案 / 對答案
XC mutex / semaphore programs	**p.79**	寫 code 題

1. 先認類型

題目字眼	跳去
"What are all the possible values of c / x / counter?"	**§2 可能值**
"Initialize the semaphores / fill in wait() and signal() so that S1 runs before S2..."、precedence graph	**§3 semaphore 次序**
"Which outputs are possible?"、"Is the sequence ... possible?"（有 semaphores）	**§4 輸出序列**
"Implement a lock using `test_and_set` / `compare_and_swap` / `swap` / 新 instruction"、填空	**§5 spinlock**
"Does this solution satisfy mutual exclusion / progress / bounded waiting?"	**§6 三個 requirements**
"Write a program with a mutex / semaphores"、producer–consumer、readers–writers	**§7 code**
定義 / 解釋：race condition、critical section、spinlock pros/cons、semaphore vs mutex	題目搜尋表 **p.20** → **X6（p.63）** 直接抄

2. 所有可能值（race condition）

照 **P4「Race 可能值」（p.18）** 四步，全部機械式：

1. **拆 T1 做 atomic steps**：`if (a > 0)` = 讀 a；`c = a + b` = 讀 a → 讀 b → 寫 c（**由左至右讀**）；`c = b - a` = 讀 b → 讀 a → 寫 c。寫成 *R1, R2, R3, W*。
2. **T2 嘅 writes 次序固定**（例如 `a = -1;` 先，`b = 10;` 後）；佢哋可以插入 T1 每個空位（R1 之前、R1–R2 之間...）。
3. **逐個插法計**：每個 read 見到新值定舊值 → 行邊個 branch → 計結果。如果 T2 都寫同一個變數（例如兩邊都寫 c），**最後寫嗰個贏**，記得加埋呢啲 case。
4. 收集所有**唔同**嘅值。

答卷寫法（**E4 模板，p.16**）：逐個 case 列：*"Case 1: T1 reads a = 3, then T2 runs a = −1; b = 10; T1 reads b = 10 → c = 13."* → 最後 *"Possible values: {4, 6, 8, 9, 11, 13}"*。**每個 distinct 值都有分**，所以寧願多列 case。

📝 **例子**（X6-41）：`a = 3, b = 5, c = 0`；T1：`if (a > 0) c = a + b; else c = b - a;`；T2：`a = -1; b = 10;`

- 拆 T1：**R1** 讀 a（if）→ then：**R2** 讀 a、**R3** 讀 b、**W** 寫 c；else：讀 b、讀 a、寫 c。
- T2 兩句（`a = -1` 一定喺 `b = 10` 之前）插入唔同空位：
  - 兩句都喺 T1 之後 → then，a = 3、b = 5 → **8**
  - `a = -1` 喺 R1 後、R2 前；`b = 10` 喺 R3 後 → a = −1、b = 5 → **4**
  - 兩句都喺 R1 後、R2 前 → a = −1、b = 10 → **9**
  - 兩句都喺 R2 後、R3 前 → a = 3、b = 10 → **13**
  - `a = -1` 喺 R1 前 → 行 else（a = −1）：讀 b 見 5 → 5 − (−1) = **6**；讀 b 見 10 → 10 − (−1) = **11**
- 答：**c ∈ {4, 6, 8, 9, 11, 13}**，每個值配返一個 interleaving。

📝 **完整示範：X6-41（p.69）**（6 個值，連表）。同類原題：**F19 Q4.3（p.150）**、**F20 Q5（p.151）**、**Lab 6 Q7（p.157）**、**HW1 Q3.1（p.153）**；`counter++ / counter--` 版：**X6-02（p.63）**。

3. Semaphore 次序 / precedence graph 填空

照 **P4「Semaphore 次序」（p.18）**：

1. 畫出要求嘅次序：每個「X 要喺 Y 之前」畫一條箭咀 X → Y。
2. **每條箭咀 = 一個 semaphore，初始值 0：X 做完之後** `signal(s)`**；Y 開始之前** `wait(s)`。
3. 一個 statement 有兩個前置 → **兩個** `wait`（例：**X6-37，p.69**）。
4. **輪流**（T1 → T2 → T3 → T1 ...）→ 第一個 semaphore **= 1**，其餘 **= 0**；每個 thread `wait(自己)` ... `signal(下一個)`（**X6-38，p.69**；**Lab 6 Q1，p.156**）。
5. 「最多 k 個同時」→ 一個 semaphore = **k**；mutual exclusion → **1**（**X6-24 / X6-26，p.66**）。
6. 唔使填嘢嘅 BLANK → 寫 `/* Nothing */`。

答卷寫法：每個 semaphore 寫「名 = 初始值」，再逐個 BLANK 填；加一句 "*S1 signals s after it finishes; S2 waits on s before it starts, so S2 always follows S1.*" 同類原題：**S26 Q5（p.147）**、**Lab 6 Q2（p.156）**；示範 **X6-25（p.67）**。

📝 **例子 1**：要 S1（P1）→ S2（P2）→ S3（P3）。

- 兩條箭咀 → 兩個 semaphore，init 0：`s12 = 0, s23 = 0`。
- P1：`S1; signal(s12);` P2：`wait(s12); S2; signal(s23);` P3：`wait(s23); S3;`
- 寫："*s12 and s23 start at 0, so P2 blocks until P1 signals after S1, and P3 blocks until P2 signals after S2.*"

📝 **例子 2（輪流，Lab 6 Q1）**：T1 → T2 → T3 → T1 ... 永遠咁輪。

- 三個 semaphore：**S1 = 1、S2 = 0、S3 = 0**（得第一個 = 1）。
- T1：`wait(S1); CS; signal(S2);` T2：`wait(S2); CS; signal(S3);` T3：`wait(S3); CS; signal(S1);`

4. 有 semaphores 嘅輸出序列

「呢個序列可唔可能？」 → 照 **P4「Semaphore 序列可行性」**：

1. 寫低每個 semaphore 嘅**初始值**。
2. 跟住序列逐個字：搵出邊個 process 印 → 行到嗰度之前要經過嘅 `wait`，**當時個值一定要 > 0**（否則嗰個 process 要等，序列就唔得）；`signal` 就 +1。
3. 全部行得通 → possible；卡住 → impossible，寫低**卡喺邊個 `wait`、個值係幾多**。

「列出所有可能輸出」 → **X6-43（p.70）** 方法：

1. 每個 process 自己嘅次序固定；
2. 每對 `signal(s)` → `wait(s)` 加一條「之前」約束；
3. 列晒符合約束嘅次序；
4. 檢查會唔會 deadlock（**X6-44，p.70**：數 permits，追到全部人卡住為止）。

同類原題：**F19 Q4.4（p.150）**、**Lab 6 Q4（p.156）**、**Lab 6 Q5（p.157）**。

📝 **例子**（X6-43）：`s = 0, t = 0`；P1：`print A; signal(s); wait(t); print C;`；P2：`print B; wait(s); signal(t); print D;`

- 約束：A 先過 C、B 先過 D（同一 process）；D 要等 P2 過 `wait(s)` → 要 P1 `signal(s)`（A 之後）；C 要等 `signal(t)`（B 之後）。
- 所以 A、B 都喺 C、D 之前 → **ABCD、ABDC、BACD、BADC**（4 個）。
- 「**BDAC** 得唔得？」→ 唔得："*D requires P2 to pass wait(s), but s stays 0 until P1 signals it after printing A, so D cannot be printed before A.*"

5. 用 atomic instruction 寫 spinlock / 填空

1. 讀題目畀嘅 instruction 定義，問自己：**「返回咩值代表我攞到 lock？」**
2. 套 **P4 spinlock 表（p.18）** 嘅格式：

```
while (instr(...) 返回「攞唔到」嘅值)
    ;                       /* spin */
/* critical section */
lock = 「free」嘅值;        /* release */
```

3. 表入面已經有四隻：`test_and_set`、`compare_and_swap`、`swap`、`compare_and_reset`；新 instruction（例如 `fetch_and_add`）→ **X6-40（p.69）**。
4. 填空版 → **X6-39（p.69）**；bounded-waiting TAS 填空 / 改錯 → **P4「Bounded-waiting TAS 檢查點」** + **X6-15 至 X6-17（p.65）**。
5. 同類原題：**S26 Q2.4 swap（p.146）**、**Sp21 Q3.1 compare_and_reset（p.152）**、**Sp21 Q3.2（p.152）**。

答卷寫法：code + 一句 "*The lock is acquired only when the instruction returns the old value 'free'; the release stores the free value back.*"

📝 **例子**：題目畀 `int swap(int *v, int new)`：**原子地將** `*v` 設做 `new`，返回**舊值**。用佢寫 lock（`lock = 0` = free）。

- 問自己：返回 **0（舊值 = free）** 先代表我攞到 lock → 返回 1 就繼續 spin。

```
int lock = 0;                 /* shared, 0 = free */
while (swap(&lock, 1) == 1)   /* old value 1: someone holds it, keep spinning */
    ;
/* critical section */
lock = 0;                     /* release: store the free value */
```

- 同 P4 表 `swap` 嗰行一樣（S26 Q2.4 原題）。

6. 「滿唔滿足 ME / progress / bounded waiting？」

1. 先翻 **X6-32 判斷表（p.68）**：九種常見 protocol（只用 `turn`、flag 先 check 後 set、flag 先 set 後 check、Peterson、Peterson 調轉、TAS、CAS、bounded-waiting TAS、非 atomic acquire）。**搵到一樣嘅就抄嗰行 Yes/No + reason**。
2. 搵唔到 → 用 **X6-33 至 X6-36（p.68）** 嘅「反例」寫法逐個試：
    - **ME**：試搵一個 interleaving 令**兩個都入到 CS**（兩邊都過咗 check 先改變數）→ 搵到 = violated。

  - **Progress**：試一個 process 留喺 **remainder section**（唔想入），另一個入唔入到？入唔到 = violated；或者兩個互相等（deadlock） = violated。
  - **Bounded waiting**：一個 process 等緊，另一個可唔可以**無限次**再入？可以 = violated。

3. 定義原文：**X6-06（p.63）**（三個 requirements + 點樣會違反）。

答卷寫法：每個 requirement 一行："*Mutual exclusion: satisfied, because … / violated: P0 … then P1 …, both enter.*" 有反例就寫出 interleaving（攞分關鍵）。

📝 **例子**：共享 `int turn = 0;`；Pi：`while (turn != i) ; /* CS */ turn = j;`（j = 1 − i）

- X6-32 表第一行 → ME **Yes**、Progress **No**、BW Yes。
- 寫：
  - "*Mutual exclusion: satisfied — Pi enters only when turn == i, and turn has a single value.*"
  - "*Progress: violated — if turn == 1 and P1 stays in its remainder section, P0 cannot enter although the critical section is free (strict alternation).*"
  - "*Bounded waiting: satisfied — after Pi requests, Pj can enter at most once before turn passes to Pi.*"

7. 寫 code（mutex / semaphores / classic problems）

題目要求	抄
兩個 thread 加同一個 counter，要保證正確	**XC-15（p.79）**
用 semaphores 保證 T1 → T2 → T3	**XC-16（p.79）**
Producer–consumer（bounded buffer）	**XC-17（p.80）**、pseudo-code **X6-47（p.70）**
Readers–writers	**XC-18（p.80）**、**X6-48（p.71）**、Lab 6 版 **Lab 6 Q6（p.157）**
Mutex 填空（`pthread_mutex_lock` …）	**Lab 6 Q3（p.156）**
Condition variable	**X6-50（p.71）**

📝 **例子**：「Two threads each add 1 to a shared counter 100000 times. Make sure the final value is always 200000.」

- 翻 **XC-15（p.79）** → **成段抄，唔使改**（mutex lock → `counter++` → unlock）。
- 如果題目指定用 semaphore：`sem_t s; sem_init(&s, 0, 1);`，將 `pthread_mutex_lock(&lock)` / `pthread_mutex_unlock(&lock)` 換做 `sem_wait(&s)` / `sem_post(&s)`。

8. 交卷前 30 秒 checklist

- 可能值：有冇**由左至右**讀 operands？有冇漏「T2 全部喺前 / 全部喺後」兩個極端？
- Semaphore 次序：所有「之前」關係都用 **init 0** 嘅 semaphore？輪流嗰個係咪得**第一個 = 1**？
- Spinlock：`test_and_set` / `compare_and_swap` 返回 **old value**（**E3 traps，p.15**）。
- `wait` / `signal` 調轉或者漏咗：**X6-45（p.70）** 有晒每種後果。

E Exam-Day Essentials 考場一頁通：公式・慣例・陷阱

E1. 必背公式卡 Formula Card

主題	公式	例
Average access time [1.32]	$\mathrm{AMAT} = h\,T_{\mathrm{hit}} + (1-h)\,T_{\mathrm{miss}}$（若畀 penalty：$T_{\mathrm{miss}} = T_{\mathrm{hit}} + \mathrm{penalty}$）	$0.9 \times 2 + 0.1 \times 50 =$ **6.8 ns**
Amdahl's Law [4.12]	$\mathrm{speedup} \le \dfrac{1}{S + (1-S)/N}$；$N \to \infty : \dfrac{1}{S}$；$S$ = **serial**	$S = 0.25,\ N = 2 \Rightarrow$ **1.6**
Amdahl 反解	$S = \dfrac{1/X - 1/N}{1 - 1/N}$	$X = 3,\ N = 4 \Rightarrow S = 1/9$
Turnaround [5.8]	$\mathrm{TAT} = C - A$（completion – arrival）	
Waiting [5.8]	$\mathrm{WT} = \mathrm{TAT} - \mathrm{burst}$（= Gantt 上嘅 gaps 總和）	
Response（本科） [5.12]	$\mathrm{RT} = (\text{end of FIRST run}) - A$（RR/MLFQ：第一個 quantum 完；non-preemptive：= TAT）	RR4：P1=4, P2=7, P3=10
Exponential averaging [5.19]	$\tau_{n+1} = \alpha\,t_n + (1-\alpha)\,\tau_n$	$\alpha = .25,\ t_n = 6,\ \tau_n = 10 \Rightarrow$ **9**
Little's formula [5.57]	$n = \lambda \times W$	$\lambda = 3/\mathrm{s},\ n = 12 \Rightarrow W =$ **4 s**
RR bound [5.11]	冇 process 等多過 $(n-1)\,q$	
RT utilization [5.49]	$U = \sum_i \dfrac{t_i}{p_i}$；$U > 1 \Rightarrow$ **一定 miss**	$25/50 + 35/80 = 0.94$
RM bound（課本）	保證可排：$U \le N\left(2^{1/N} - 1\right)$：N=1 1.000、2 0.828、3 0.780、4 0.757、5 0.743、∞ 0.693	
EDF [5.52]	$U \le 1$（$D = T$ 時）理論上可排	
fork 數目	n 個無條件 fork → 2^n；`if(fork()) fork();` 每輪 ×3 → 3^n	見 Playbook P1
Bounded buffer (in/out) [3.54]	最多 **BUFFER_SIZE – 1** items	
Semaphore（blocking） [6.26]	$\mathrm{value} < 0 \Rightarrow \lvert\mathrm{value}\rvert$ = 等緊嘅 processes 數	
Storage units [1.28]	KB $= 2^{10}$ B；MB $= 2^{20}$；GB $= 2^{30}$；networks 用 **bits**	

E2. 本科 Conventions（考試照用，唔確定就寫 assumption）

1. **Response time = completion of the first run – arrival**（slide 5.12；S26 官方答案照用）。
2. **RR**：quantum 用完 → **tail**；同一時刻有新 arrival → **新 arrival 先入 queue**。
3. **MLFQ**：新 job → **Q0 tail**；用晒 quantum 仍未完 → **降一級 tail**；**被高 queue 新 arrival preempt → 返自己 queue 嘅 HEAD，保留剩餘 quantum（Q1）/ 剩餘 burst（Q2），唔會降級**；Q0 job 唔會被 preempt；Q1 job 唔會 preempt Q2 job。
4. **Priority：細數字 = 高 priority**（除非題目話 larger = higher，例如 S26 dynamic priority）。
5. **Ties**：題目有講就跟（S26：smaller ID；dynamic priority：larger ID）；冇講：FCFS（早到先）→ 細 ID；**SRTF / EDF 平手 → 唔 preempt running process**。
6. **Soft real-time**：miss 咗 deadline 嘅 job **繼續跑完**，但計一次 miss。
7. **fork 題**：除非另有說明，假設 **printf 即時 flush**、**fork 全部成功**。
8. **Gantt chart**：每段寫 **start / process / end**；idle 都要畫；MLFQ 標 queue（P1@Q0）。

E3. 最易中伏 Top Traps（MC 前睇一次）

✗ 錯誤講法	✓ 正確
Main memory is nonvolatile / registers slower than memory	volatile；registers 最快；**main memory byte addressable**
von Neumann：instructions 同 data 分開存	**same memory**（stored-program）
Speedup with N CPUs = N	**< N**（contention / overhead）
NUMA：所有 memory 同樣 latency	**remote memory slower**
VM 一定比 native 快	minor slowdown
printf() 係 system call	printf **calls write()** system call
API 比 system call 快 / 可以 bypass protection	優點 = **portability + hide details（convenient）**
Microkernel performance 好過 monolithic	**worse**（message passing）
Layered：任何 layer 可 call 任何 layer	嗰個係 **LKM**；layered 只 call 下層
LKM 要 recompile kernel + reboot	可以 **run time load/remove**

✗ 錯誤講法	**✓ 正確**
Policy = how	**Policy = what；mechanism = how**
Interrupts 全部同一 priority / 只由 I/O 產生	有 **priority levels；trap = software interrupt**
DMA 每個 byte interrupt 一次	**block 完先 interrupt 一次**
Mode bit 1 = kernel	**1 = user，0 = kernel**
Context switch = user→kernel mode switch	context switch = **save/restore process state**；越複雜 OS 越慢
fork() 令 parent 等 child	要 **wait()**
child 改 global variable → parent 都變	各自 **copy**
exec() 成功會 return	**唔會 return**
Zombie 仲行緊 code / 冇 PID	terminated，**PID + PCB 仍在**，等 parent wait()
Ordinary pipe 任何 process 用到	**parent–child only**、單向、用完消失
Named pipe 用完自動消失	**explicitly delete**；UNIX FIFO **half-duplex**
Threads 有自己 address space / files	**share** code、data、heap、files；**own** PC、registers、**stack**
Concurrency = 同時執行	同時 = **parallelism**；concurrency 單核都得
Many-to-many 易 implement / user thread map 去多個 kernel threads	**difficult**；冇 one-to-many
所有 signals deliver 畀所有 threads	sync → causing thread；部分 async → all
Asynchronous cancellation = 定期 check	嗰個係 **deferred**（Pthreads default）
FCFS 可以 preemptive	**FCFS must be non-preemptive**
RR 一定比 FCFS 好	同長度 jobs 時 **TAT 更差**
q 越大 TAT 越好	唔一定；q 太大 → **FCFS**
MLFQ 要知 burst / 永不 starve	**唔使知**；可能 starve（要 aging/reshuffle）
Multilevel queue 可搬 queue	只有 **MLFQ** 可以
Multicore 簡化 scheduling	**complicates**（faster + less power）
Hard RT 遲咗都得 / RT scheduler 唔使 preemption	遲 = 冇服務；要 **priority + preemption**
EDF 要 periodic	**唔需要**；只需 announce deadline
Little's formula 只適用 FCFS	**any algorithm**
"Synchronization" 係 CS 三條件之一	**ME + progress + bounded waiting**
test_and_set 返回新值 / 會 release lock	返回 **old value**；release = `lock = false`
Binary semaphore = mutex	可當 mutex，但**更通用**（init 0 做 ordering）

E4. 答題模板 Answer Templates

Short question（2–4 分）：*[Definition sentence with keywords]. [Why / how — one sentence]. [Example or advantage/disadvantage].*

Gantt + averages：

```
Gantt:  0 P1 4 P2 8 P3 12 ...          (每個 boundary 都寫)
P | Arr | Burst | Comp | TAT=C-A | WT=TAT-B | RT=end1-A
avg WT  = (w1 + w2 + ... + wn) / n = x.xx
avg TAT = (t1 + ... + tn) / n = x.xx
Assumptions: ties -> ..., new arrival enters queue before preempted process.
```

fork 題：*Draw tree → count per level → formula → final number*；output 題：*constraints (wait / same process order) → list all interleavings*。

Race values：*Case 1 (branch ...): interleaving ... → c = ...* 逐個列。

Semaphore 填空：用 init-0 semaphore：**先做者** `signal(s)`**，後做者開頭** `wait(s)`；冇需要嘅 blank 寫 `/* Nothing */`。

P Playbooks 解題流程 + 預計表

P1. fork() Playbook

Step 1 — 規則

- fork() 返回：**child 0 / parent child-PID (>0) / fail −1（只有 parent，冇 child）**。
- Child 由 **fork() 下一個 operation** 開始，複製晒 variables（包括 loop 嘅 `i`）。
- 短路：`A && B` → A 係 0（child）唔做 B；`A || B` → A 非 0（parent）唔做 B。
- `return` / `exit` / `break` 會停止該 process 嘅 loop；`exec` 成功 = 之後 code 唔執行。

Step 2 — Pattern 表（每個 iteration、所有 process 都繼續 loop）

每 iteration	每 process 變	n 次後	例
`fork();`	×2	2^n	n=3 → 8；n=10 → 1024
`fork(); fork();`	×4	4^n	n=3 → 64
`if (fork()) fork();`	×3	3^n	n=3 → 27
`if (fork() == 0) fork();`	×3	3^n	n=3 → 27（Sp21 MC8）
`fork() && fork();`	×3	3^n	parent 做兩次
`fork() || fork();`	×3	3^n	child 做第二次
`if (fork() == fork()) printf`	×4	4^n；lines $(4^n - 1)/3$	F22 Q3
`if (fork()) {fork(); fork();} else fork();`	×6	6^n	Lab 3 Ex3
`if (fork()) fork(); else break;`	active ×2	$2^{n+1} - 1$ total	breaks $2^n - 1$（S26 Q3.1）
`if (fork()) printf; else printf;`	lines 2^i per iter	$2^{n+1} - 2$ lines	F19 Q2.6
`pid = fork(); if (!pid) for (j<N) fork();`（M 輪）	×$(2^N + 1)$	$(2^N + 1)^M$	F24 P3.1
`if (fork() == 0) break;`（fan）	+1 child	$n + 1$	只 parent 繼續
`if (fork() > 0) break;`（chain）	+1 child	$n + 1$	只 child 繼續

Step 3 — 其他已驗證答案

Code	答案
`if (fork() || fork()) fork();`	**5** processes
`if (fork() && fork()) fork();`	**4**（P0: T&&T → fork → 2；C1 false → 1；C2 false → 1）
`fork(); fork() && fork(); fork();`	**12**
`pid1=fork(); pid2=fork(); if(pid1>0) if(pid2==0) fork();`	**5**
`if (fork()==fork() || fork()!=fork()) fork(); printf`	**23** lines
S26 MC10：loop 3：`if(fork()==0){fork(); printf X; return;}` 之後 printf Y	**7 processes, X = 6, Y = 1**
S26 Q3.1：`if(fork()) fork(); else break;`	$2^{n+1} - 1$ processes, $2^n - 1$ breaks（n=2：7, 3）
F20 Q3.2：`if(!fork()){loop n: if(fork()) fork();} fork(); printf a`	$2(3^n + 1)$
Sp21 Q1.2：`if(fork()==0){loop 5: if(fork()==0) fork();} fork();`	$(3^5 + 1) \times 2 = 488$
F19 Q2.6：loop n，parent/child 都 printf	$2^{n+1} - 2$ lines
Slide 3.44：loop 10 `fork(); fork();`	$4^{10} = 1{,}048{,}576$
Slide 3.45：loop 10 `if (fork()) fork();`	$3^{10} = 59{,}049$
Lab 3 Ex3：loop n `if(fork()){fork();fork();} else fork();`	每輪 ×6 → 6^n
F22 Q3：loop n `if (fork() == fork()) printf`	processes 4^n；lines $(4^n - 1)/3$（1, 5, 21）
F24 P3.1：loop M `pid=fork(); if(!pid) for(j<N) fork();`	$(2^N + 1)^M$（line Y 由 **child** 執行；M=N=1 → 3）
S25 MC12：`if (fork() || fork()) { if (fork() && !fork()) fork(); }`	**9** processes
Lab 2：`pid=fork(); if(pid==0){cnt+=10; pid=fork(); if(pid){cnt+=10; pid=fork();}}` 印 pid、cnt	非 0 pid × **2**；'0' × **3**；'10' × **1**

Step 4 — Process tree：每個 node 寫 PID 名（P0, P1...）同佢喺邊個 iteration 出世；child 繼承 parent 嘅 `i` 值繼續。

Step 5 — Output ordering：

1. 同一 process 內嘅 print 次序固定。
2. `wait()`：parent 之後嘅 print > 所有 child（及其子孫，如 child 等咗佢哋）嘅 print。
3. fork 之前印嘅（已 flush）最先。

4. 列所有符合約束嘅排列（topological orders）；可以用「插空位」數：k 個自由字元插入固定序列。

Step 6 — exec 情況：(I) fork ✓ exec ✓：child exec 前嘅 print + parent 嘅 print；(II) fork ✓ exec ✗：再加 exec 後嘅 print；(III) fork ✗：只有 `ret < 0` branch。

P2. Pipe / dup / exec 填空 Playbook

要做	Code
建 pipe（**fork 之前**）	`int fd[2]; pipe(fd);` → **fd[0] read，fd[1] write**
Writer	`close(fd[0]); write(fd[1], &x, sizeof(x)); close(fd[1]);`
Reader	`close(fd[1]); read(fd[0], &x, sizeof(x)); close(fd[0]);`
stdin ← file	`int fin = open(name, O_RDONLY); close(0); dup(fin);`（或 `dup2(fin, 0)`）
stdout → file	`int fo = open(name, O_WRONLY|O_CREAT|O_TRUNC, 0644); dup2(fo, 1); close(fo);`
stdout → pipe	`dup2(fd[1], 1); close(fd[0]); close(fd[1]); execlp(...)`
stdin ← pipe	`dup2(fd[0], 0); close(fd[0]); close(fd[1]); execlp(...)`
等指定 child	`waitpid(pid, &status, 0);`
exec 一個 program	`execlp("ls", "ls", "-l", (char *)NULL);` / `execvp(argv[0], argv);`（argv 尾要 NULL）
數偶數（S26）	`if (number % 2 == 0) cnt++;`

P3. CPU Scheduling Playbook

萬用步驟：(1) 抄表；(2) 由 t=0 起逐個 event（arrival / completion / quantum expiry / check point）；(3) 每個 event 寫 ready queue 內容；(4) 照算法揀；(5) 畫 Gantt（每個 boundary）；(6) C → TAT → WT → RT；(7) 平均連 working；(8) 檢查：最後完成時間 $= \sum \text{burst} + \text{idle}$。

Algorithm	揀邊個	幾時重新揀	注意
FCFS	最早到	完成時	non-preemptive；convoy effect
SJF	**已到達**嘅最短 burst	完成時	一旦開始跑到完
SRTF	最短 **remaining**	完成 + **每個 arrival**	tie → keep running
RR (q)	queue head	完成 / quantum 用完	新 arrival 先入 queue，再放被 preempt 嘅
Priority NP	最高 priority（細數字）	完成時	tie → FCFS
Priority P	最高 priority	完成 + 高 priority arrival	
Priority + RR	最高 priority；同級 RR	完成 / quantum / 高 priority arrival	同級得一個就繼續跑
MLFQ	最高非空 queue 嘅 head	完成 / quantum / 高層 arrival	preempted → **head + 剩餘 quantum**
Dynamic priority	計 $\text{init} + \alpha \cdot \text{waited} + \beta \cdot \text{ran}$	每 check interval / 完成	tie 跟題目
LUF（F19）	用 CPU 最少	完成 + arrival	descheduled → tail
RM	period 最短	每個 release / 完成	static
EDF	absolute deadline 最早	每個 release / 完成	dynamic；tie → keep running

MLFQ 追蹤表（考試用 rough paper 畫）：

```
t   | event            | Q0       | Q1 (rem q)   | Q2       | run (queue, q left, burst left)
0   | P1 arrives       | P1       |              |          | P1@Q0 q=4 b=9
4   | P1 q used        | P2       | P1(8)        |          | P2@Q0 ...
```

RM / EDF 追蹤：列 release times：task i 喺 $0, p_i, 2p_i, \ldots$ release；deadline $=$ release $+ d_i$。逐個 time unit 揀；記低每個 job 完成時間，比較 deadline。

P4. Synchronization Playbook

Race 可能值（F19 Q4.3 / F20 Q5 type）

1. 將 T1 拆做 atomic steps：**read for condition** → **read b** → **read a** → **write c**（`c = b + a` 由左至右讀）。
2. T2 嘅 writes（次序固定）可以插入任何 gap。
3. 分 branch（condition read 前 / 後寫）→ 每個 read 見新值 / 舊值 → 計 c。
4. 列晒所有 **distinct** 值 + 每個值嘅 interleaving。

Semaphore 次序：每條 dependency「X 要喺 Y 前」→ semaphore s = 0：**X 尾** `signal(s)`、**Y 頭** `wait(s)`；多個前置 → 多個 `wait`。

Semaphore 序列可行性：逐字追蹤每個 semaphore 嘅值同每個 process 做到邊；任何 `wait` 時值必須 > 0；有 loop 嘅 process 可以重複。

Spinlock 任何 atomic instruction：`while (instr(...) 返回「攞唔到」);` + release 寫返 free 值：

Instruction（return）	acquire	release
`test_and_set(&lock)`（old）	`while (test_and_set(&lock)) ;`	`lock = false;`
`compare_and_swap(&lock,0,1)`（old）	`while (compare_and_swap(&lock,0,1) != 0) ;`	`lock = 0;`
`swap(locked,1)`（old）	`while (swap(locked,1) == 1) ;`	`locked = 0;`
`compare_and_reset(&v,0,1)`（1 = 成功）	`while (compare_and_reset(&v,0,1) == 0) ;`	`v = 0;`

Bounded-waiting TAS 檢查點：(1) **cyclic scan** `j=(i+1)%n … j!=i`；(2) **有人等就** `lock = false`；(3) 有人等就 `waiting[j] = false`（直接交 lock）；三個性質：ME（只有第一個 TAS 得 key=false / 只放一個 waiting）、progress（exit 一定放 lock 或者叫醒一個）、bounded waiting（≤ n−1 turns）。

P5. 預先計好嘅表 Pre-computed Tables

Amdahl speedup $= \dfrac{1}{S+(1-S)/N}$ （S = serial fraction；parallel $= 1 - S$）

$S\backslash N$	2	3	4	5	6	7	8	10	16	32	∞
0.05 (par 95%)	1.90	2.73	3.48	4.17	4.80	5.38	5.93	6.90	9.14	12.55	**20.00**
0.10 (par 90%)	1.82	2.50	3.08	3.57	4.00	4.38	4.71	5.26	6.40	7.80	**10.00**
0.15 (par 85%)	1.74	2.31	2.76	3.13	3.43	3.68	3.90	4.26	4.92	5.66	**6.67**
0.19 (par 81%)	1.68	2.17	2.55	2.84	3.08	3.27	3.43	3.69	4.16	4.64	**5.26**
0.20 (par 80%)	1.67	2.14	2.50	2.78	3.00	3.18	3.33	3.57	4.00	4.44	**5.00**
0.25 (par 75%)	1.60	2.00	2.29	2.50	2.67	2.80	2.91	3.08	3.37	3.66	**4.00**
0.30 (par 70%)	1.54	1.88	2.11	2.27	2.40	2.50	2.58	2.70	2.91	3.11	**3.33**
0.35 (par 65%)	1.48	1.76	1.95	2.08	2.18	2.26	2.32	2.41	2.56	2.70	**2.86**
0.39 (par 61%)	1.44	1.69	1.84	1.95	2.03	2.10	2.14	2.22	2.34	2.44	**2.56**
0.40 (par 60%)	1.43	1.67	1.82	1.92	2.00	2.06	2.11	2.17	2.29	2.39	**2.50**
0.45 (par 55%)	1.38	1.58	1.70	1.79	1.85	1.89	1.93	1.98	2.06	2.14	**2.22**
0.50 (par 50%)	1.33	1.50	1.60	1.67	1.71	1.75	1.78	1.82	1.88	1.94	**2.00**
0.60 (par 40%)	1.25	1.36	1.43	1.47	1.50	1.52	1.54	1.56	1.60	1.63	**1.67**
0.70 (par 30%)	1.18	1.25	1.29	1.32	1.33	1.35	1.36	1.37	1.39	1.41	**1.43**
0.80 (par 20%)	1.11	1.15	1.18	1.19	1.20	1.21	1.21	1.22	1.23	1.24	**1.25**

Powers（fork 用）

n	2^n	3^n	4^n	6^n	$2^{n+1}-1$	2^n-1	$2^{n+1}-2$	$2(3^n+1)$	$(4^n-1)/3$
1	2	3	4	6	3	1	2	8	1
2	4	9	16	36	7	3	6	20	5
3	8	27	64	216	15	7	14	56	21
4	16	81	256	1296	31	15	30	164	85
5	32	243	1024	7776	63	31	62	488	341
6	64	729	4096	46656	127	63	126	1460	1365
7	128	2187	16384	279936	255	127	254	4376	5461
8	256	6561	65536	1679616	511	255	510	13124	21845
9	512	19683	262144	10077696	1023	511	1022	39368	87381
10	1024	59049	1048576	60466176	2047	1023	2046	118100	349525

RM utilization bound $N\left(2^{1/N}-1\right)$

N	bound	N	bound
1	1.000	7	0.729
2	0.828	8	0.724
3	0.780	9	0.721
4	0.757	10	0.718
5	0.743	∞	0.693 $(\ln 2)$
6	0.735		

Exponential averaging（$\alpha = 1/2$）**權重**：t_n ½、t_{n-1} ¼、t_{n-2} ⅛、t_{n-3} 1/16、… τ_0 權重 $(1/2)^{n+1}$。$\alpha = 0.25$：t_n 0.25、t_{n-1} 0.1875、t_{n-2} 0.1406、… τ_0 權重 0.75^{n+1}。

X Answer Bank 預測題 + 直接抄答案 • C programs

🎯 **用法**：搵題目關鍵字 → 去對應 X1–X6 → 直接抄 **Answer**（英文，已按評分重點寫好，粗體 = 評分關鍵字）。計數題用 **P（Playbooks）**；歷屆原題用 **Q / M**；寫 code 用 **XC**（全部已編譯 + 跑過）。

任何 short question 都套得落嘅滿分寫法

1. **Explain / Why**：一句 direct answer → mechanism（點做）→ consequence / example。
2. **Compare A vs B**：3–4 行表（purpose • mechanism • advantage • disadvantage/overhead），每格一句。
3. **Advantages / disadvantages**：每點 = point + 一句 reason；分數幾多就寫幾多點（2 marks → 2 points）。
4. **Calculation**：formula → substitution → answer（連 unit）；Gantt 寫 `0 P1 5 P2 8 …`；唔清楚就寫 assumption（e.g. "Assume ties are broken by FCFS, then smaller process ID"）。
5. **Code**：close 唔用嘅 pipe ends；parent `wait()`；exec 之後加 `perror(...); exit(1);`；threads 要 `pthread_join`；shared data 用 mutex / semaphore 保護。
6. **True / False**：先答 T/F，再加一句 reason（用關鍵字）。

題目搜尋表 Question finder（按章節 → 題目關鍵字 → 頁數）

X1 • Ch1 Introduction

01 OS definition, intermediary, four components 22
02 OS goals in different computing environments 22
03 OS goals, resource allocator, control program 22
04 Scenario: which OS role 22
05 Kernel, middleware, system and application programs 22
06 Computer-system organization: bus, controllers, drivers 23
07 von Neumann architecture, stored-program concept 23
08 Instruction-execution cycle steps 23
09 Why programs must be in memory; MMU 23
10 I/O subsystem: buffering, caching, spooling, drivers 23
11 Storage units: bits, bytes, words, KB–PB 23
12 Storage hierarchy: speed, cost, capacity, volatility 23
13 Why secondary storage is needed 24
14 Main memory properties, bootstrap in EEPROM 24
15 Mechanical vs electrical (NVM) storage 24
16 Cache hit, miss, cache management 24
17 Where caching is used in systems 24
18 Average access time; miss time vs penalty 24
19 Higher hit ratio is not always better 24
20 Required hit ratio and caching speedup 24
21 Locality: temporal, spatial, when caching fails 24
22 Who manages each storage level 25
23 Timescales: why switch jobs during disk I/O 25
24 Single-processor system, special-purpose processors 25
25 Multiprocessor advantages; speed-up below N 25
26 SMP vs AMP comparison 25
27 Cache coherency with private caches 25
28 Multicore benefits for laptops and mobile devices 25
29 NUMA: motivation, structure, drawback 26
30 CPU, processor, core, multicore, multiprocessor 26
31 Multiprogramming: why, how, OS support 26
32 Multiprogramming vs time sharing compared 26
33 Virtualization: host, VMM, guest OS 26
34 Virtualization motivations: consolidation, testing scenario 26
35 Virtualization vs emulation; VM performance 26
36 Cloud computing; public, private, hybrid clouds 26
37 SaaS, PaaS, IaaS classification 27
TF True / False quick verdicts 27

X2 • Ch2 OS Structures

01 OS services: helpful to user, efficient operation 28
02 Scenario: identify the OS service 28
03 Protection vs security; access rights, domains 28
04 Two categories: protection, functionality beyond hardware 28
05 CLI vs GUI vs touch-screen interfaces 28
06 Shell: functions; kernel or system program 29
07 cp command: sequence of system calls 29
08 API definition, common APIs, read() example 29
09 Why use APIs; API call vs system call 29
10 System-call interface, RTE, system-call table 29
11 printf() vs write(); when kernel mode starts 29
12 Parameter passing: registers, block, stack 29
13 Six types of system calls, classification 30
14 System programs: categories and user view 30
15 System calls vs system programs vs applications 30
16 Compiler, linker, loader roles; relocation 31
17 Static linking vs dynamically linked libraries 31
18 Why applications are OS-specific; ABI 31
19 OS design goals: user vs system 31
20 Policy vs mechanism: definition, separation 31
21 Scenario: classify policy or mechanism 31
22 OS in a high-level language: pros, cons 31
23 MS-DOS vs FreeBSD: running a program 32
24 Simple structure: why MS-DOS was vulnerable 32
25 Monolithic kernel: pros, cons, Linux 32
26 Tightly vs loosely coupled; modular design 32
27 Layered approach: information hiding, debugging 32
28 Why few OSes are purely layered 32
29 Microkernel: file access by message passing 32
30 Microkernel: advantages, drawback, Windows NT 33
31 LKM approach; USB device scenario 33
32 LKM vs microkernel; LKM vs layered 33
33 Hybrid systems: Linux, Windows, macOS 33
34 Compare monolithic, layered, microkernel, LKM 33
35 Scenario: choose an OS structure 34
36 Mac OS X structure: Darwin, Mach, BSD 34
37 macOS vs iOS: similarities, differences 34
38 iOS vs Android: similarities, differences 34
39 Android architecture: ART, JNI, HAL, Bionic 34
40 History: from batch to time sharing 34
TF True / False quick verdicts 34

X3 • Ch3 Processes • IPC • Interrupts

01 Program vs process; loading a program 35
02 Memory layout: text, data, heap, stack 35
03 Process states and transition events 36
04 PCB contents; process table; Linux task_struct 36
05 Threads within a process 36
06 I/O-bound vs CPU-bound; good process mix 36
07 Scheduling queues and the queueing diagram 36
08 Long-, short- and medium-term schedulers compared 36
09 Swapping and the medium-term scheduler 37
10 Context switch: saved state and kernel actions 37
11 Context-switch overhead; multiple register sets 37
12 Context switch vs mode switch 37
13 Interrupt-driven I/O and device controllers 37
14 Interrupt handling and the interrupt vector 37
15 Maskable vs nonmaskable interrupts; priority levels 37
16 Programmed I/O vs DMA; DMA transfer steps 38
17 Dual-mode operation, mode bit, privileged instructions 38
18 Which operations must be privileged 38
19 Timer: OS always regains control 38
20 Three types of mode transfer 38
21 System-call path: printf() to write() 38
22 Beyond two modes: rings, VMM, machine switch 38
23 Process creation options; tree of processes 39
24 fork() return values and why two 39
25 fork() steps, copy-on-write, clone() 39
26 After fork(): copied, shared, different 39
27 Output: child modifies a global array 39
28 Output: fork() return value vs getpid() 39
29 Output: unflushed printf buffer copied by fork() 40
30 exec(): success, failure, execlp vs execvp 40
31 Output: exec in a loop succeeds/fails 40
32 Why a shell uses fork, exec, wait 40

33 PA1: cd and exit as built-ins **40**
34 PA1: why system() and popen() are banned **41**
35 PA1: Ctrl-C (SIGINT) handling with signal() **41**
36 Termination: exit, wait, abort, cascading **41**
37 Zombie vs orphan; role of init/systemd **41**
38 Zombie or orphan? Two scenarios **41**
39 Independent vs cooperating processes **41**
40 Shared memory vs message passing **42**
41 Bounded buffer: empty, full, BUFFER_SIZE − 1 **42**
42 Message passing: link and design choices **42**
43 Direct vs indirect communication; mailbox sharing **42**
44 Blocking vs non-blocking send/receive; rendezvous **42**
45 Buffering: zero, bounded, unbounded capacity **42**
46 Pipe design issues; ordinary vs named pipes **43**
47 When to use ordinary vs named pipes **43**
48 Why close unused pipe ends (EOF) **43**
49 What happens: blocking pipe and FIFO calls **43**
50 Output: parent-to-child pipe program **43**
51 Two-way communication with ordinary pipes **43**
52 Shell pipelines and multi-level pipes **44**
53 dup vs dup2; I/O redirection **44**
54 Sockets, ports and the 4-tuple **44**
55 Remote procedure calls (RPC) **44**
TF True / False quick verdicts **44**

X4 · Ch4 Threads

01 thread definition, thread vs process **45**
02 shared vs private per-thread state **45**
03 four benefits of multithreading, examples **46**
04 thread creation and switching cheaper **46**
05 disadvantages of threads vs processes **46**
06 multicore programming challenges **46**
07 concurrency vs parallelism **46**
08 data vs task parallelism, classify examples **46**
09 Amdahl's law statement, limit 1/S **46**
10 Amdahl speedup, 60% parallel **47**
11 Amdahl serial fraction from measured speedup **47**
12 Amdahl cores needed for target speedup **47**
13 doubling cores vs halving serial part **47**
14 why real speedup is lower **47**
15 multithreading on a single core **47**
16 Thread Control Block contents **47**
17 process state with multiple threads **47**
18 user threads vs kernel threads **47**
19 thread libraries, Pthreads specification **48**
20 pthread_create arguments, pthread_join purpose **48**
21 passing arguments, returning thread results **48**
22 passing &i to threads bug **48**
23 pthread_exit vs return vs exit **48**
24 Pthreads trace, possible outputs **48**
25 compare four multithreading models **48**
26 blocking system call, many-to-one vs one-to-one **49**
27 why most OSes use one-to-one **49**
28 many-to-many and two-level models **49**
29 LWP, scheduler activations, upcalls **49**
30 thread pool benefits **50**
31 fork() and exec() in multithreaded process **50**
32 fork + pthread_create loop with join **50**
33 fork + pthread_create in a branch **50**
34 thread shares, fork copies global **50**
35 fork called inside a thread **51**
36 fork + pthread_create loop, formula **51**
37 what is a signal, handling steps **51**
38 synchronous vs asynchronous signals **51**
39 signal delivery in multithreaded process **51**
40 asynchronous vs deferred cancellation **51**
41 pthread_cancel only requests cancellation **51**
42 thread-local storage **52**
43 Windows thread components, ETHREAD KTHREAD TEB **52**
44 Linux tasks, task_struct vs Windows **52**
45 clone() flags vs fork() **52**
TF True / False quick verdicts **52**

X5 · Ch5 CPU Scheduling

01 CPU–I/O burst cycle and burst distribution **53**
02 Distinguish I/O-bound from CPU-bound processes **53**
03 Four scheduling decision points, which preemptive **53**
04 Preemptive vs non-preemptive scheduling issues **53**
05 Dispatcher and dispatch latency **53**
06 Scheduling criteria to maximize or minimize **53**
07 Response time definition and computation **54**
08 Conflicting scheduling criteria **54**
09 FCFS and the convoy effect **54**
10 Round-robin mechanism and waiting bound **54**
11 Choosing q from the waiting bound **54**
12 Time quantum size effects, 80% rule **54**
13 RR CPU efficiency with context-switch overhead **54**
14 Utilization and throughput with switch cost **55**
15 RR versus FCFS with equal jobs **55**
16 SJF optimality and proof sketch **55**
17 SJF implementation difficulty and approximation **55**
18 Exponential averaging formula and alpha **55**
19 Exponential averaging calculation, alpha 0.5 **55**
20 Exponential averaging: alpha 0.25, solve alpha **55**
21 SRTF definition and comparison with SJF **55**
22 FCFS, SJF, SRTF, RR averages calculation **56**
23 Priority scheduling and its relation to SJF **56**
24 Starvation and aging **56**
25 Preemptive vs non-preemptive priority calculation **56**
26 Priority with RR, remaining quantum rule **56**
27 Dynamic priority alpha beta gives FCFS or LIFO **56**
28 Least-recent-CPU policy favours I/O-bound **57**
29 Multilevel queue scheduling **57**
30 Multilevel queue versus MLFQ **57**
31 MLFQ defining parameters **57**
32 MLFQ rules: demotion, preemption, head of queue **57**
33 MLFQ disadvantages and remedies **57**
34 MLFQ Gantt calculation, remaining quantum **57**
35 Algorithm property comparison table **58**
36 Which algorithm: quick judgement answers **58**
37 Equivalences between scheduling algorithms **58**
38 How FCFS, RR, MLFQ treat short jobs **58**
39 PCS versus SCS and LWP **59**
40 AMP versus SMP **59**
41 Common versus per-processor ready queues **59**
42 Memory stall and chip multithreading **59**
43 Two levels of multicore thread scheduling **59**
44 Processor affinity: soft, hard, NUMA **59**
45 Load balancing: push, pull, affinity conflict **60**
46 Hard versus soft real-time **60**
47 Event, interrupt and dispatch latency **60**
48 Periodic tasks, utilization, admission control **60**
49 Rate-monotonic scheduling and missed deadlines **60**
50 Earliest-deadline-first scheduling **60**
51 RM versus EDF comparison **60**
52 Schedulability test with utilization bound **61**
53 RM and EDF timelines, missed deadlines **61**
54 Algorithm evaluation methods **61**
55 Little's formula calculations **61**
TF True / False quick verdicts **61**

X6 · Ch6 Synchronization

01 Race condition: counter++ / counter-- interleaving **63**
02 All possible final values of counter **63**
03 Unprotected counter in a loop: lost updates **63**
04 Race condition inside the kernel **63**
05 Critical section; entry, exit, remainder sections **63**
06 Three requirements with violation examples **63**
07 Preemptive vs non-preemptive kernels **63**
08 Disabling interrupts: uniprocessor vs multiprocessor **64**
09 Peterson's solution: code and assumption (Lab 6) **64**
10 Peterson satisfies all three requirements **64**
11 Peterson on modern hardware: memory barriers **64**
12 Atomic hardware instructions **64**
13 test_and_set(): definition, lock, bounded waiting **64**
14 compare_and_swap(): definition and lock **65**
15 Bounded-waiting TAS: waiting[], key, lock **65**
16 Bounded-waiting TAS: sketch proof **65**
17 Broken variants of the bounded-waiting algorithm **65**
18 Atomic variables: increment() with CAS **65**
19 Mutex lock: why acquire() must be atomic **66**
20 Spinlocks: when to use, not on uniprocessors **66**
21 Semaphore: definition, wait/signal, P and V **66**
22 Why wait() and signal() must be atomic **66**
23 Counting vs binary semaphore; semaphore vs mutex **66**
24 Mutual exclusion with a semaphore: initial value **66**
25 Ordering S1 before S2 (synch = 0) **67**
26 Counting semaphore: at most k users **67**
27 Rendezvous and barrier with semaphores **67**
28 Semaphore without busy waiting: block() and wakeup() **67**
29 Tracing a semaphore value with a waiting queue **67**
30 Deadlock with two semaphores **67**
31 Starvation (indefinite blocking) vs deadlock **68**

32 Verdict table for classic CS protocols 68
33 Strict alternation using only turn 68
34 Flag checked before it is set 68
35 Flag set before checking: deadlock 68
36 Peterson variants that break mutual exclusion 69
37 Semaphores for a precedence graph 69
38 Strict alternation of threads with semaphores 69
39 Fill in the blanks: TAS, CAS, increment 69
40 Spinlock from a new instruction: fetch_and_add 69
41 Possible values of c: worked example 69
42 Can both threads read 0? 70
43 Possible outputs with two semaphores 70
44 Counting permits: what gets printed 70
45 Misusing wait() and signal() around a CS 70
46 Bounded buffer: wrong order of wait() 70
47 Bounded-buffer solution (Ch7 preview / Lab 6) 70
48 First readers–writers solution (Ch7 preview / Lab 6) 71
49 Dining philosophers: deadlock and remedies (Ch7 preview) 71
50 POSIX condition variables (Lab 6) 71
TF True / False quick verdicts 71

XI · Cross-chapter traces（綜合「發生咩事」題）

01 read() system call: user → kernel → DMA → interrupt → return 73
02 Timer interrupt ends a time quantum (RR) 73
03 How a shell runs `ls -l > out.txt` and `ls | wc -l` 73
04 Life cycle of a process through all states 73
05 From source file to running process (compiler, linker, loader, e... 73
06 What happens when you press Ctrl-C 73

XC · Copy-ready C programs（Lab / PA coding，已編譯測試）

01 fork + wait + exit status 74
02 N children (fan) + reap all 74
03 Chain of processes 74
04 fork + execvp a command line 74
05 Pipe: parent sends a string to child 75
06 Pipe: child computes, parent receives 75
07 Two pipes: two-way communication 76
08 Redirection: cmd < in > out 76
09 Pipeline: ls | wc -l 76
10 Multi-level pipe (PA1 Feature 5) 77
11 Mini shell loop: built-in cd, exit, Ctrl-C 77
12 Named pipe (FIFO) writer / reader 78
13 Pthreads: partial sums with N threads 78
14 Thread returns a value to main 79
15 Mutex-protected shared counter 79
16 Semaphores: force the order T1 → T2 → T3 79
17 Bounded buffer (producer–consumer) with semaphores 80
18 First readers–writers (semaphores) 80
19 Zombie and orphan demos 81

X1 · Ch1 Introduction

X1-01 OS definition, intermediary, four components

What is an operating system? Explain its role among the four components of a computer system. [1.20–1.21]

An OS is **system software that acts as an intermediary between users / application programs and the computer hardware**; it **controls and coordinates the use of computing resources among various applications and users**.

- **Four components**: **hardware** (CPU, memory, I/O devices) → **operating system** → **application programs** (define how resources are used to solve problems, e.g., editors, browsers, games) → **users** (people, machines, other computers).
- **Abstraction**: the OS **hides the complicated details of the hardware** and provides a **simple, easy-to-use interface**, so users deal with files, photos and web pages instead of bytes and blocks on devices.
- **Resource allocation**: it gives an **orderly and controlled allocation** of processors, memory, disks and I/O devices among programs.

X1-02 OS goals in different computing environments

How do OS design goals differ for shared computers, workstations, mobile devices, embedded systems and real-time systems? [1.22]

The OS design **first and foremost depends on the target device**:

Environment	Main OS goal / feature
Shared computers (mainframe, minicomputer)	keep all users satisfied: balance **performance vs. fairness**
Individual systems (workstations)	dedicated resources: **performance rather than fairness**; may use shared resources from servers
Mobile devices (smartphones, handhelds)	**resource constrained**; sensors; **touch screen and voice control** (e.g., Siri); optimized for **usability and battery life**
Embedded systems (air conditioners, cars, ships, spacecraft)	**little or no user interface**; OS **pre-installed in ROM**
Real-time systems	**strict time constraints**

X1-03 OS goals, resource allocator, control program

What are the main goals of an OS? Explain the views of the OS as a resource allocator and as a control program. [1.23]

Two main goals: **user convenience** (an environment to execute programs in a **convenient, safe, protected and efficient** manner) and **resource allocation** (allocate resources **fairly and efficiently**).

- **Resource allocator**: the OS **manages all resources**, hardware (CPU, memory, I/O) and software (e.g., signals, locks), and **decides between conflicting requests for efficient and fair resource use**.
- **Control program**: the OS **controls the execution of programs to prevent errors and improper use of the computer** (stops users misusing resources; handles I/O).
- In a nutshell, the OS **manages and controls the hardware and helps programs run on the computer**.

X1-04 Scenario: which OS role

Which OS role does each case show? (a) Two programs request the only printer at the same time. (b) A program writing into another program's memory is stopped. (c) A user opens a photo by file name, not by disk block numbers. [1.21, 1.23]

- (a) **Resource allocator**: the OS **decides between conflicting requests** (one job gets the printer, the other waits), so the device is shared **fairly and efficiently**.
- (b) **Control program**: the OS **controls program execution to prevent errors and improper use**; each program may access only its own memory.
- (c) **Abstraction / user convenience**: the OS **hides hardware details** (bytes and blocks) behind a simple interface (files).

X1-05 Kernel, middleware, system and application programs

Distinguish the kernel, middleware, system programs and application programs. Which of them are part of the OS? [1.23–1.24]

There is **no universally accepted definition** of an OS ("everything a vendor ships" is only an approximation); commonly, **OS = always-running kernel + middleware frameworks + system programs**.

Term	Meaning	Part of OS?	Example
Kernel	**"the one program running at all times on the computer"**; the essential functions	yes	process and memory management
Middleware	**software frameworks that provide additional services to application developers**	yes (popular in iOS, Android)	databases, multimedia, graphics
System programs	**ship with the OS but are not part of the kernel**; help manage the system while it runs	yes	shells, compilers, `ls`, `cp`
Application programs	**not associated with the OS**	no	apps from app stores, games

X1-06 Computer-system organization: bus, controllers, drivers

Describe the organization of a modern computer system. Why do the CPUs and devices compete for memory? [1.25, 1.27]

A computer has **one or more CPU cores and device controllers connected through a common bus that provides access to shared memory**.

- **CPUs and devices execute concurrently**, so they **compete for memory cycles through the shared bus** (a memory controller keeps the access orderly).
- Each **device controller** is in charge of a particular device; the OS controls it through a **device driver** (the kernel module that controls a device).
- Basic I/O hardware elements: **buses, device controllers and the devices themselves**.

X1-07 von Neumann architecture, stored-program concept

Describe the main components of the von Neumann architecture. What does "stored-program" mean? [1.26]

- **Processing unit (CPU)**: an **arithmetic logic unit (ALU)** and **processor registers**: PC (program counter), AC (accumulator), MAR (memory address register), MDR (memory data register).
- **Control unit**: an **instruction register (IR)** and the **program counter (PC)**.
- **Memory** that stores **both data and instructions** (with caches), **external mass storage** (secondary storage), and **input and output mechanisms**.

Stored-program: **instructions and data are stored in the same memory and accessed by address**, so **programs and data must be brought into memory before execution**.

X1-08 Instruction-execution cycle steps

Describe the steps of the instruction-execution cycle on a von Neumann machine. [1.26]

1. **Fetch instruction**: the instruction at the address in the **PC** is fetched from memory (or cache) into the **instruction register (IR)**.
2. **Decode instruction**.
3. **Fetch data**: the operands are fetched from memory (or cache) into **data registers**.
4. **Execute instruction**: the ALU operates on the operands.
5. **Write back (if any)**: the result **may be stored back in memory or cache**.

The cycle then repeats for the next instruction.

X1-09 Why programs must be in memory; MMU

Why must a program be loaded into main memory before it can run? What does the memory management unit (MMU) "see"? [1.26, 1.30]

Because **main memory is the only large storage the CPU can access directly**; the CPU accesses **secondary storage only indirectly**, so code and data on disk must first be **brought into memory**.

- In the von Neumann (stored-program) model, the CPU **fetches every instruction and operand from memory** (or cache / registers).
- The **MMU sees only a stream of memory addresses**; it **does not know how they are generated, or whether they are instructions or data**.

X1-10 I/O subsystem: buffering, caching, spooling, drivers

What services does the kernel I/O subsystem provide? Distinguish buffering, caching and spooling, and state what a device driver does. (Ch1 summary, Lab 2)

The kernel I/O subsystem provides **I/O scheduling, buffering, caching, spooling, device reservation and error handling**.

- **Buffering**: **stores data temporarily while it is being transferred** (e.g., between a device and an application).
- **Caching**: keeps a **copy of data on faster storage**. Difference: **a buffer may hold the only existing copy of a data item, whereas a cache only holds a copy of an item that resides elsewhere**.
- **Spooling**: **holds output for a device that cannot accept interleaved data** (e.g., a **printer**), coordinating concurrent output.
- **Device driver**: the **kernel module that controls a device**; it **hides device details behind a uniform device-access interface** for the I/O subsystem.

X1-11 Storage units: bits, bytes, words, KB–PB

(Calculation) A computer has 64-bit registers and addressing. (a) Bytes per word? (b) Words in 1 KB? (c) Bytes in 4 GB? (d) Time to send a 1 GB file over a 1 Gb/s link? [1.28]

Use **1 byte = 8 bits**; **KB = 2^{10} = 1,024 bytes, MB = 1024^2, GB = 1024^3, TB = 1024^4, PB = 1024^5 bytes**; a **word** is the architecture's **native unit of data**; **networks are measured in bits per second** (Mb/s, Gb/s).

- (a) 64 bits / 8 = **8 bytes per word**.
- (b) 1,024 / 8 = **128 words**.
- (c) $4 \times 2^{30} = 2^{32}$ = **4,294,967,296 bytes** (manufacturers round this to about 4 billion).
- (d) $2^{30} \times 8$ = 8,589,934,592 bits; at 10^9 bits/s this takes about **8.6 s** (about 8 s if 1 GB is rounded to 10^9 bytes). Always **multiply bytes by 8** before dividing by a network rate.

X1-12 Storage hierarchy: speed, cost, capacity, volatility

Describe the storage hierarchy. How do speed, cost, capacity and volatility change as we move down it? [1.29]

Storage is organized in a hierarchy by **speed, cost per byte, capacity and volatility**: **registers → cache → main memory → nonvolatile memory (e.g., SSD) → hard-disk drives → optical disk → magnetic tapes**.

- **Higher levels (closer to the CPU)** are **faster, smaller and more expensive per byte**.
- **Moving down**, **cost per byte decreases, access time increases (slower) and capacity increases**.
- **Registers, cache and main memory are volatile** (primary storage); **NVM and HDDs are nonvolatile** (secondary storage); optical disks and tapes are tertiary storage.

X1-13 Why secondary storage is needed

Why can't all programs and data be kept permanently in main memory? What is secondary storage for? [1.30–1.31]

Because **main memory is volatile** (it **loses its contents when power is turned off**) and is **too small and too costly per byte** to hold all programs and data.

- **Secondary storage** is an **extension of main memory that provides large nonvolatile storage capacity**, holding **large quantities of data permanently**.
- The most common devices are **hard-disk drives (HDDs)** and **nonvolatile memory (NVM) devices** such as SSDs; programs are loaded from them into memory when they run.

X1-14 Main memory properties, bootstrap in EEPROM

Describe main memory. Where is the bootstrap program stored, and why not in main memory? [1.30]

- **Main memory** is the **only large storage the CPU can access directly**; it is **volatile**, typically **DRAM**, accessed by **load and store instructions** to specific addresses, and **byte addressable (each address refers to one byte)**.
- The **bootstrap program**, the **first program run at power-on**, is stored in **EEPROM** (electrically erasable programmable read-only memory, i.e., firmware).
- Reason: **DRAM is volatile and empty at power-on**, so start-up code must be in **nonvolatile** memory; the bootstrap program **initializes the system and loads the OS kernel into memory**.

X1-15 Mechanical vs electrical (NVM) storage

Compare mechanical and electrical secondary storage. [1.31]

Mechanical storage is larger and cheaper per byte; electrical storage (NVM) is more costly, smaller, more reliable and faster.

	Mechanical	Electrical (NVM)
Examples	HDDs, optical disks, holographic storage, magnetic tape	flash memory, SSD, FRAM, NRAM
Capacity	**larger**	smaller
Cost per byte	**less expensive**	**more costly**
Speed	slower	**faster**
Reliability	lower (moving parts)	**more reliable**

Both are **nonvolatile** and hold programs and data permanently.

X1-16 Cache hit, miss, cache management

Explain what happens on a cache hit and on a cache miss. What are the main cache-management issues? [1.32]

Caching **temporarily copies a subset of information from a slower, larger storage into a faster, smaller storage**, making the **frequently used case faster**. Every access **first checks whether the information is in the cache**:

- **Hit**: it is in the cache, so it is **used directly from the cache (fast)**.
- **Miss**: it is not, so the **data is copied from the slower storage into the cache and used there**.
- **Cache management**: the **cache is much smaller than the storage being cached**, so the key issues are the **cache size** and the **replacement policy** (what to evict); the **major criterion is the cache hit ratio** (percentage of accesses found in the cache).

X1-17 Where caching is used in systems

Caching is "performed at many levels". Give examples of caching in a computer system or OS. [1.32]

Caching is used **whenever a subset of content can be kept on a faster device**:

- **Hardware caches** keep recently used parts of **main memory** close to the CPU.
- Recent **address translations** are cached for fast lookup.
- **File blocks, file names and file directories**: e.g., part of the **file directory** (a kernel data structure stored on disk) is **cached in main memory**, so main memory acts as a cache for the disk.
- Frequently used **network routes** are cached.

X1-18 Average access time; miss time vs penalty

(Calculation) Hit ratio 98%, hit time 1 ns, miss time 60 ns: find the average access time. What if 60 ns is the miss penalty instead? [1.32]

Average access time = (hit ratio × hit time) + (miss ratio × miss time), where miss ratio = 1 − hit ratio: $T_{avg} = h \times T_{hit} + (1 - h) \times T_{miss}$

- Miss time 60 ns: $T_{avg} = 0.98 \times 1 + 0.02 \times 60 = 0.98 + 1.2 =$ **2.18 ns**.
- If 60 ns is the **miss penalty** (extra time on top of the hit time), then $T_{miss} = 1 + 60 = 61$ ns and $T_{avg} = 0.98 \times 1 + 0.02 \times 61 = 0.98 + 1.22 =$ **2.20 ns** (same as $T_{hit} + (1-h) \times \text{penalty} = 1 + 0.02 \times 60$).

State which interpretation you use.

X1-19 Higher hit ratio is not always better

(Calculation) Cache A: hit ratio 95%, hit time 4 ns, miss time 100 ns. Cache B: hit ratio 90%, hit time 1 ns, miss time 60 ns. Which is better, and what does this show? [1.32]

Cache B is better, even though its hit ratio is lower.

- Formula: $T_{avg} = h \times T_{hit} + (1 - h) \times T_{miss}$.
- A: $0.95 \times 4 + 0.05 \times 100 = 3.8 + 5 =$ **8.8 ns**.
- B: $0.90 \times 1 + 0.10 \times 60 = 0.9 + 6 =$ **6.9 ns**.
- This shows that **hit ratio alone does not determine cache performance**: the **average access time** also depends on the **hit time** and the **miss time (miss penalty)**.

X1-20 Required hit ratio and caching speedup

(Calculation) Hit time 2 ns, miss time 82 ns, and an access without the cache takes 80 ns. What hit ratio gives an average access time of at most 6 ns? What is the speedup over no cache? [1.32]

A hit ratio of at least 95% is needed, giving a speedup of about 13.3 times.

- $h \times 2 + (1 - h) \times 82 \le 6 \Rightarrow 82 - 80h \le 6 \Rightarrow 80h \ge 76 \Rightarrow h \ge 0.95$.
- General form: $h = (T_{miss} - T_{target})/(T_{miss} - T_{hit})$ = (82 − 6) / (82 − 2) = 76 / 80 = 0.95.
- Check: $0.95 \times 2 + 0.05 \times 82 = 1.9 + 4.1 = 6$ ns; **speedup = 80 / 6 ≈ 13.3** compared with accessing memory directly.

X1-21 Locality: temporal, spatial, when caching fails

Why does caching work? Identify the temporal and spatial locality in a loop that sums an array. When would a cache not help? [1.33]

Caching works because programs show **locality of reference**:

```
for (i = 0; i < n; i++)
    sum += a[i];
```

- **Temporal locality** (locality in time): **recently accessed items are likely to be accessed again soon**; here `sum`, `i` and the **loop instructions** are reused in every iteration.
- **Spatial locality** (locality in space): **items near recently accessed items are likely to be accessed shortly**; `a[0]`, `a[1]`, `a[2]`, … are **contiguous** and accessed in order (instructions are also sequential), so one block brought into the cache serves the next accesses.
- **Without locality**, e.g., if **all items are accessed with equal probability**, **the cache would never work**: the hit ratio stays low and most accesses pay the miss time.

X1-22 Who manages each storage level

Which component manages each level of the storage hierarchy? Is movement between levels explicit or implicit? [1.34]

Registers are managed by the compiler, the cache by hardware, and main memory, SSD and disk by the OS.

Level	Access time (ns)	Managed by	Backed by
Registers	0.25–0.5	**compiler**	cache
Cache	0.5–25	**hardware**	main memory
Main memory	80–250	**operating system**	disk
Solid-state disk	25,000–50,000	**operating system**	disk
Magnetic disk	5,000,000	**operating system**	disk or tape

Movement between levels can be explicit or implicit: cache ↔ memory transfers are **implicit (done by hardware)**, while memory ↔ disk transfers (e.g., loading a program, swapping) are **explicitly controlled by the OS**.

X1-23 Timescales: why switch jobs during disk I/O

Using typical latencies, explain why the OS should run another job while one job waits for a disk access. [1.35, 1.42]

A disk access is about 100,000 times slower than a memory access, so a waiting CPU would waste millions of instruction times.

- **Main memory reference ≈ 100 ns**; **disk seek ≈ 10 ms** = 10^7 ns, so $10^7/10^2 = 10^5$ times slower (an L1 cache reference is only ≈ 0.5 ns).
- Therefore, under **multiprogramming**, when the running job waits for I/O the OS **switches the CPU to another job**, so the **CPU always has a job to execute**.

X1-24 Single-processor system, special-purpose processors

A PC has one single-core CPU plus a disk controller and a GPU that contain their own processors. Is it a multiprocessor system? Explain. [1.36]

No, it is still a single-processor system. It has **one general-purpose CPU with a single processing core**, which executes a **general-purpose instruction set**, including user processes.

- The disk controller and GPU are **special-purpose (device-specific) processors**: they **run a specific and limited instruction set** and **usually do not execute instructions from user processes**.

X1-25 Multiprocessor advantages; speed-up below N

Give three advantages of multiprocessor systems. Why is the speed-up with N processors less than N? What is the main disadvantage? [1.37]

- **Increased throughput**: more computing capability, so more work is done in less time.
- **Economy of scale**: processors **share I/O devices, power supplies, housings and peripherals**, so it costs less than several single-processor systems.
- **Increased reliability**: **graceful degradation** (service continues in proportion to the surviving hardware) or **fault tolerance** (operation continues despite a component failure).

Speed-up is less than N because of **overhead**, e.g., **contention for shared resources (bus or memory)** and coordination among processors. **Disadvantage**: **far more complex hardware design and software (parallel programming)**.

X1-26 SMP vs AMP comparison

Compare symmetric multiprocessing (SMP) and asymmetric multiprocessing (AMP). [1.37–1.38]

In AMP a master processor assigns tasks to slaves and handles I/O; in SMP all processors are equal and each performs all tasks.

	Asymmetric (AMP)	Symmetric (SMP)
Relationship	**master–slave**: the master assigns specific tasks to the slaves	**all processors treated equally**
OS functions and user processes	run as the master assigns	**each processor performs all tasks, including OS functions and user processes**
I/O	**handled by the master only**	**handled by any processor**
Weakness	**the master can become a bottleneck**	adding CPUs may not scale: **contention for the system bus**

In SMP, **each CPU has its own set of registers and a private (local) cache, but all processors share physical memory through the system bus**.

X1-27 Cache coherency with private caches

In an SMP system each CPU has its own private cache. What problem does this create, and who usually solves it? [1.38]

Cache coherency: **the same data item can be copied in several CPUs' private caches**, so after one CPU updates its copy, the **other caches may hold stale values**.

- The system must ensure that **an update in one cache is immediately reflected in all other caches** holding that item, so every CPU reads the **most recent value**.
- This is **usually handled by hardware**, below the OS level.

X1-28 Multicore benefits for laptops and mobile devices

Why is one multicore chip preferred to several single-core chips, especially in laptops and mobile devices? [1.39]

- **Faster communication**: cores on one chip communicate **on-chip, which is faster than between-chip communication**.
- **Lower power**: **one chip with multiple cores uses significantly less power than multiple single-core chips**, which matters for **battery-powered devices** such as mobile devices and laptops.

X1-29 NUMA: motivation, structure, drawback

Why was NUMA introduced? Describe its structure, its drawback and its implication for the OS. [1.40]

- **Motivation**: adding more CPUs to an SMP system **may not scale**, because **contention for the system bus becomes a bottleneck**.
- **Structure**: **each CPU (or group of CPUs) has its own local memory accessed via a small, fast local bus**; the CPUs are connected by a **shared system interconnect**, and **all CPUs share one physical memory address space**. Local memory is fast, remote memory is slower (**non-uniform** access time).
- **Drawback**: **increased latency when a CPU must access remote memory across the system interconnect**.
- **OS implication**: **scheduling and memory management** should keep a process on a CPU **close to the memory that holds its data**.

X1-30 CPU, processor, core, multicore, multiprocessor

Define CPU, processor, core, multicore and multiprocessor. [1.41]

A **processor** (chip) contains one or more **CPUs**, and the **core** is the unit that actually executes instructions:

Term	Definition
CPU	the **hardware that executes instructions**
Processor	a **physical chip that contains one or more CPUs**
Core	the **basic computation unit of the CPU**; it **executes instructions and has registers for storing data locally**
Multicore	**multiple computing cores on a single physical processor chip**
Multiprocessor	a system **including multiple processors**

X1-31 Multiprogramming: why, how, OS support

Why is multiprogramming needed? What happens when the running job waits for I/O, and what OS support does it need? [1.42]

A single program cannot keep the CPU or I/O devices busy at all times, so multiprogramming **keeps several jobs in memory at the same time and organizes them so the CPU always has one to execute**, raising **CPU utilization (efficiency)**.

- When the running job **waits for I/O**, the OS **switches the CPU to another job** in memory instead of leaving the CPU idle.
- OS support: **job scheduling** (choose queued jobs to load into memory), **CPU scheduling** (several jobs ready at the same time), memory management for several jobs, **swapping** (move processes in and out of memory when they do not fit) and **virtual memory** (run a process that is **not completely in memory**).

X1-32 Multiprogramming vs time sharing compared

Compare multiprogramming (batch) and time sharing (multitasking). [1.42]

Time sharing is a logical extension of multiprogramming: both keep several jobs in memory, but time sharing switches the CPU so often that users can interact with their programs.

	Multiprogramming (batch)	**Time sharing (multitasking)**
Goal	**efficiency: keep the CPU busy** (CPU utilization)	**fast response time** for interactive users
When the CPU switches	when the running job **must wait (e.g., for I/O)**	**frequently**, even if the job is not waiting (short time slices, enforced by a timer)
User interaction	none while jobs run (jobs are submitted and queued)	users **interact with each job while it is running** (interactive computing)
Requirement	job scheduling, CPU scheduling	CPU scheduling; **response time under a certain threshold**

X1-33 Virtualization: host, VMM, guest OS

What is virtualization? Describe the roles of the host, the VMM (hypervisor) and the guest OS. [1.43–1.44]

Virtualization abstracts the hardware of a single computer into multiple different execution environments (virtual machines, VMs), creating the **illusion that each user or program runs on its own "private computer"**. It **allows an OS to run as an application within another OS**, so **one physical machine can run multiple operating systems concurrently**.

- **Host**: the **underlying hardware system**.
- **Virtual machine manager (VMM) or hypervisor**: **creates and manages virtual machines** by **providing an interface identical to the host**; it is **in complete control of system resources**.
- **Guest**: a process given a **virtual copy of the host**, usually an **operating system** (guest OS) running its own applications.

X1-34 Virtualization motivations: consolidation, testing scenario

A company runs many lightly loaded servers, each needing a different OS or OS version, and its developers must test software on several OSes. How does virtualization help? [1.43]

- **Server consolidation**: virtualization **consolidates multiple OSes onto fewer hardware platforms**; each service runs in its own VM with the OS / version it needs, on far fewer physical machines.
- **Testing and debugging**: **developers can run many OS types and versions on just one machine**, which makes **testing and debugging easier**.
- **Desktop use**: a user can run one OS (e.g., Linux) and **still use native applications of another platform, such as Windows**, inside a VM.

X1-35 Virtualization vs emulation; VM performance

Distinguish virtualization from emulation. Does a program run faster inside a VM? [1.46]

- **Virtualization**: the **guest OS is natively compiled for the same CPU** as the host; most guest instructions run directly on the hardware under VMM control, so programs show **only minor performance decreases** (they pass through more layers of software).
- **Emulation**: **simulates a different CPU architecture in software** (source CPU differs from target CPU); every instruction must be translated, so it is **much slower**.
- So a program in a VM is **not faster** than on the physical machine; it is usually **slightly slower**. (History: virtualization began on **IBM mainframes (1972)**; in the late 1990s Intel CPUs became fast enough for **VMware and Xen** on PCs.)

X1-36 Cloud computing; public, private, hybrid clouds

What is cloud computing, and why is it a "logical extension of virtualization"? Distinguish public, private and hybrid clouds. [1.47–1.48]

Cloud computing delivers computing, storage and applications as a service over a network, on demand.

- It is a **logical extension of virtualization** because it **uses virtualization as the base for its functionality**; e.g., **Amazon EC2** has millions of servers running **tens of millions of VMs**, and users **pay based on usage**.
- **Public cloud**: available **via the Internet to anyone willing to pay**.
- **Private cloud**: **run by a company for the company's own use**.
- **Hybrid cloud**: **includes both public and private cloud components**.

X1-37 SaaS, PaaS, IaaS classification

Explain SaaS, PaaS and IaaS, and classify: (a) an online word processor; (b) a ready-to-use database server for your application; (c) rented storage for backups or virtual servers.

[1.48]

SaaS provides applications, PaaS a ready software stack, and IaaS servers or storage, all over the Internet.

Model	Provided over the Internet	Example
SaaS (Software as a Service)	**one or more applications**	(a) **online word processor**
PaaS (Platform as a Service)	**a software stack ready for application use**	(b) **a database server**
IaaS (Infrastructure as a Service)	**servers or storage**	(c) **storage for backup**, virtual servers (e.g., Amazon EC2)

Clouds increasingly offer other services too, e.g., **MaaS (Machine learning as a Service)**.

X1-TF True / False quick verdicts

Statement	T/F	Why (≤ 12 words)
The kernel is the one program running at all times on the computer.	**T**	Middleware and system programs are part of the OS, not kernel.
System programs ship with the OS and are part of the kernel.	**F**	They ship with the OS but are not part of the kernel.
There is a universally accepted definition of an operating system.	**F**	"Everything a vendor ships" is only an approximation.
Embedded systems usually have rich user interfaces.	**F**	Little or no UI; OS is pre-installed in ROM.
The MMU knows whether an address refers to an instruction or data.	**F**	The MMU sees only a stream of memory addresses.
The CPU can execute a program directly from the hard disk.	**F**	Programs must first be brought into main memory.
DRAM keeps its contents when the power is turned off.	**F**	Main memory (DRAM) is volatile.
The bootstrap program is stored in EEPROM.	**T**	Nonvolatile firmware; DRAM is empty at power-on.
Moving down the hierarchy, cost per byte and access time both decrease.	**F**	Cost per byte decreases, but access time increases.
NVM (flash, SSD) is larger and cheaper per byte than an HDD.	**F**	Mechanical storage is larger and cheaper; NVM is faster.
Cache memory is managed by the operating system.	**F**	Cache is managed by hardware; registers by the compiler.
Main memory, SSD and magnetic disk are managed by the OS.	**T**	Storage-characteristics table, levels 3 to 5.
A cache is usually larger than the storage being cached.	**F**	A cache is much smaller; it holds a subset.
A cache with a higher hit ratio always has a lower average access time.	**F**	Hit time and miss time also determine average access time.
Spatial locality: items near a recent access are likely accessed soon.	**T**	E.g., sequential array elements and sequential instructions.
Caching still works if all items are accessed with equal probability.	**F**	Without access locality, a cache would never work.
A buffer may hold the only existing copy of a data item.	**T**	A cache, by definition, only holds a copy.
Spooling holds output for a device that cannot accept interleaved data.	**T**	E.g., a printer shared by many processes.
A GPU in a single-CPU PC makes it a multiprocessor system.	**F**	Special-purpose processors do not execute user processes.
In AMP, any processor may handle I/O.	**F**	The master handles I/O; any processor only in SMP.
Multicore chips use more power than multiple single-core chips.	**F**	One multicore chip uses significantly less power.
In NUMA, each CPU has a separate physical address space.	**F**	All CPUs share one physical memory address space.
Cache coherency is usually handled by hardware.	**T**	It is handled below the operating-system level.
Multiprogramming lets users interact with each job while it runs.	**F**	That is time sharing (multitasking).
Emulation is used when guest and host have the same CPU type.	**F**	That is virtualization; emulation simulates a different CPU.
A private cloud is available to anyone willing to pay.	**F**	That is a public cloud; private is for one company.
Storage available over the Internet for backup is IaaS.	**T**	IaaS provides servers or storage over the Internet.
1 KB is 1,000 bytes in the lecture's notation.	**F**	1 KB = 1,024 bytes; manufacturers round off.
Network speeds are usually given in bytes per second.	**F**	Networks move data a bit at a time: bits/s.

X2 · Ch2 OS Structures

X2-01 OS services: helpful to user, efficient operation

List the OS services that are helpful to the user, and those that ensure the efficient operation of the system itself. [2.4–2.6]

Helpful to the user:

- **User interface**: CLI, GUI, touch screen.
- **Program execution**: load a program into memory, run it, and end execution **normally or abnormally (indicating error)**.
- **I/O operations**: a running program may require I/O on a file or an I/O device.
- **File-system manipulation**: read, write, create, delete and search files and directories; list file information; **permission management**.
- **Communications**: processes exchange information on the same computer or over a network, via **shared memory** or **message passing** (packets moved by the OS).
- **Error detection**: errors **may occur in the CPU and memory, in I/O devices and in user programs**; the OS takes appropriate action to ensure **correct and consistent computing** (plus **debugging facilities**).

Efficient operation of the system (via resource sharing): resource allocation (CPU cycles, memory, file storage, I/O devices), **logging** (keep track of **which users use how much and what kinds of resources**), and **protection and security**.

X2-02 Scenario: identify the OS service

Which OS service is shown? (a) CPU time used by each user is recorded for billing. (b) A disk read fails; the OS retries and reports it. (c) Two processes exchange data through shared memory. (d) A program is loaded, runs, and exits with an error code. (e) A user must log in with a password. [2.4–2.6]

- (a) **Logging (accounting)**: tracks which users use how much and what kinds of resources.
- (b) **Error detection**: the OS is constantly aware of possible errors (here in an I/O device) and takes appropriate action.
- (c) **Communications**, using the **shared-memory** model (the other model is message passing).
- (d) **Program execution**: load, run, and end execution normally or **abnormally (indicating error)**.
- (e) **Protection and security**: **security from outsiders requires user authentication**.

X2-03 Protection vs security; access rights, domains

Distinguish protection and security. What are an access right and a domain? [2.6] **(Ch2 summary)**

- **Protection**: **controlling the access of users or processes to the resources** (hardware and software) of the computer, **ensuring that all access to system resources is controlled**, so concurrent processes do not interfere with each other.
- **Security**: **defending the system against internal and external attacks** (e.g., **denial-of-service, worms, viruses**); **security from outsiders requires user authentication** and extends to defending external I/O devices from invalid access attempts.
- An **access right** is the **permission to perform certain operations on an object**; a **domain** is **a set of access rights for a set of objects**. A process executes in a domain and **cannot perform an operation on an object unless its domain allows it**.

X2-04 Two categories: protection, functionality beyond hardware

According to the chapter summary, OS services fall into two general categories. What are they? Give an example of each. (Ch2 summary)

1. **Enforcing protection among processes running concurrently**: e.g., **a process may access only the memory in its own address space**, and **may not access hardware devices directly without OS assistance and intervention**.
2. **Providing functionality not directly supported by the underlying hardware**: e.g., **memory management and file systems** (the disk only stores blocks; the OS builds files and directories on top of them).

X2-05 CLI vs GUI vs touch-screen interfaces

Compare the three approaches for users to interact with an OS: CLI, GUI and touch screen. [2.8–2.12]

CLI = typed commands, GUI = desktop metaphor with mouse and icons, touch screen = gestures and voice.

	CLI (command interpreter)	**GUI**	**Touch screen**
Interaction	user **enters commands directly**; the shell fetches, interprets and executes them	**user-friendly desktop metaphor**: mouse, keyboard, monitor; **icons** for files, programs, directories	**gestures**, **virtual keyboard**, **voice commands** (e.g., Siri)
Best for	experienced users; fast and scriptable	ease of learning and use	devices where a mouse is not feasible (phones, tablets)
Examples	UNIX/Linux shells: Bourne-Again (bash), C shell, Korn shell	Mac OS X **Aqua**, Windows, KDE / GNOME	iPhone, iPad, Android

The GUI was **invented at Xerox PARC (early 1970s)** and first widely used on the **Apple Macintosh (1984)**; **many systems now provide both CLI and GUI** (e.g., Windows GUI with a CLI "command" shell).

X2-06 Shell: functions; kernel or system program

What is the primary function of a shell? Why is it better to implement the command interpreter as a system program than inside the kernel? [2.8, 2.37]

A shell **(1) fetches a command from the user, (2) interprets it and (3) executes it**; in UNIX it runs a program by calling `fork()` to create a new process and `exec()` to load the program into it, then **waits** for it (or continues, for a background job).

The command interpreter is **sometimes implemented in the kernel, sometimes by system programs**. As a system program:

- **Several shells can coexist** (C shell, Bourne-Again shell, Korn shell), and users can choose or replace one **without changing the kernel**.
- Most commands are **separate system programs**: the shell just **uses the command name to find a file to load and execute**, so **new commands can be added without modifying the shell**.
- A bug in the shell **affects only the shell process, not the kernel**.

X2-07 cp command: sequence of system calls

Describe the sequence of system calls needed by `cp in.txt out.txt` to copy one file to another. [2.14]

Even this simple command needs **a sequence of system calls**:

1. **Acquire the input and output file names** (from the command line, or by writing a prompt to the screen and reading the keyboard).
2. **Open the input file**; if it does not exist, print an error and **abort**.
3. **Create the output file**; if it already exists, abort (or overwrite).
4. **Loop: read from the input file and write to the output file**, until the read reaches end of file (or an error occurs).
5. **Close the files**, write a completion message to the screen, and **terminate normally**.

X2-08 API definition, common APIs, read() example

What is an API? Name the three most common APIs, and explain the parameters and return value of `ssize_t read(int fd, void *buf, size_t count)`. [2.15–2.16]

An **Application Programming Interface (API) specifies a set of functions available to application programmers, including the parameters passed to each function and the return values it may expect**. The API functions **invoke the actual system calls on behalf of the programmer** (e.g., Windows `CreateProcess()` invokes the `NTCreateProcess()` system call). Programmers use an API through a **library provided by the OS**: **libc** for C programs on UNIX/Linux.

- Most common APIs: **Win32 API** (Windows), **POSIX API** (virtually all versions of UNIX, Linux and Mac OS X), **Java API** (Java virtual machine).
- `read()`: `fd` = the **file descriptor** to be read; `buf` = the **buffer** the data is read into; `count` = the **maximum number of bytes** to read. It returns the **number of bytes read**, **0 at end of file**, and **−1 if an error occurred**.

X2-09 Why use APIs; API call vs system call

Explain why programmers use an API rather than invoking system calls directly. Does every API call result in a system call? [2.17]

- **Program portability**: a program written to an API **can compile and run on any system that supports the same API**, although **the implementation of system calls varies from machine to machine**.
- **Hiding complex details**: actual system calls are **more detailed and difficult to work with**; the caller **need know nothing about how the system call is implemented**, only **obey the API and understand what the OS will do** as a result. E.g., `printf()` handles formatting and buffering, then calls the `write()` system call.
- **No**: an API function may **invoke one or more system calls, or none**; e.g., `strlen()` runs entirely in user mode.

X2-10 System-call interface, RTE, system-call table

How does the system-call interface link an API call to the kernel? Explain the roles of the run-time environment, the system-call number and the system-call table. [2.18–2.19]

The system-call interface, part of the run-time environment, maps each API call to a numbered system call and invokes it in the kernel.

- The **run-time environment (RTE)**, the full suite of software needed to run programs in a language (compilers or interpreters, **libraries**, loaders), provides a **system-call interface that serves as the link to the system calls** of the OS.
- **An identity number is associated with each system call**, and the system-call interface **maintains a table indexed by these numbers**.
- The interface **intercepts function calls in the API, invokes the necessary system call within the OS** (the call traps into kernel mode; the kernel uses the number to index the table and run the service routine), and **returns the status of the system call and any return values**.
- **The caller knows nothing about how the system call is implemented.**

X2-11 printf() vs write(); when kernel mode starts

Explain the relationship between `printf()` and `write()`. At what point does the program enter kernel mode? [2.26]

`printf()` is a standard C library (API) function; `write()` is a system call. The program **invokes the `printf()` library call, which calls the `write()` system call** to get the service from the OS.

- `printf()` runs in **user mode** (formatting the string and buffering the output).
- Only when the library **invokes `write()`** does the program **trap into kernel mode**; the kernel writes the data to the file or device and then **returns to user mode** with the result.
- The standard C library (**libc**) thus provides the **system-call interface** for UNIX and Linux.

X2-12 Parameter passing: registers, block, stack

Describe the three methods to pass system-call parameters to the OS. A call needs 7 parameters but only 4 registers are available: which method would you use? [2.20–2.21]

Use the block (or stack) method, because registers limit the number of parameters:

Method	How	Limit
Registers	parameters are placed **directly in CPU registers (simplest)**	**limited**: there may be more parameters than registers
Block (table)	parameters are **stored in a block, or table, in memory**, and **the address of the block is passed in a register**	**no limit** on number or length
Stack	parameters are **pushed onto the stack by the program and popped off by the OS**	**no limit** on number or length

E.g., store all 7 parameters in a memory block and pass its address in one register.

X2-13 Six types of system calls, classification

Name the six categories of system calls, and classify `fork()`, `wait()`, `open()`, `ioctl()`, `getpid()`, `sleep()`, `pipe()`, `mmap()`, `chmod()` and `umask()`. [2.22–2.25]

Process control, file management, device management, information maintenance, communications and protection.

Category	Typical operations	UNIX	Windows
Process control	create/terminate, load/execute, wait for time or event, allocate/free memory, dump memory, debugger, **locks**	**`fork()`**, `exit()`, **`wait()`**	`CreateProcess()`, `ExitProcess()`, `WaitForSingleObject()`
File management	create/delete, open/close, read/write/reposition, get/set file attributes	**`open()`**, `read()`, `write()`, `close()`	`CreateFile()`, `ReadFile()`, `WriteFile()`, `CloseHandle()`
Device management	request/release device, read/write, get/set device attributes, attach/detach	**`ioctl()`**, `read()`, `write()`	`SetConsoleMode()`, `ReadConsole()`, `WriteConsole()`
Information maintenance	get/set time or date, system data, process/file/device attributes	**`getpid()`**, `alarm()`, **`sleep()`**	`GetCurrentProcessID()`, `SetTimer()`, `Sleep()`
Communications	create/delete connection, send/receive messages, **shared-memory** regions	**`pipe()`**, `shm_open()`, **`mmap()`**	`CreatePipe()`, `CreateFileMapping()`, `MapViewOfFile()`
Protection	control access, get/set permissions, allow/deny user access	**`chmod()`**, **`umask()`**, `chown()`	`SetFileSecurity()`, `InitializeSecurityDescriptor()`

X2-14 System programs: categories and user view

What are system programs? List their main categories with examples. Why is the user's view of the OS defined by them? [2.27–2.30]

System programs (system services, system utilities) provide a convenient environment for program development and execution; some are simply **user interfaces to system calls**, others are considerably more complex.

- **File management**: create, delete, copy, rename, print, list files and directories.
- **Status information**: date, time, available memory or disk space, number of users; performance, logging and debugging information.
- **File modification**: text editors; commands to search file contents or transform text.
- **Programming-language support**: compilers, assemblers, debuggers, interpreters.
- **Program loading and execution**: absolute and relocatable loaders, linkage editors, overlay loaders, debugging systems.
- **Communications**: messages between users, web browsing, e-mail, remote login, file transfer.
- **Background services**: launched at boot; those running until the system halts are **services, subsystems or daemons** (disk checking, process scheduling, error logging).

Most users see the OS through application and system programs, not the actual system calls: a GUI and a UNIX shell **use the same set of system calls** but look and act very differently.

X2-15 System calls vs system programs vs applications

Compare system calls, system programs and application programs. [2.27–2.30]

System calls are the low-level programming interface; system programs are higher-level utilities built on them; application programs are not part of the OS.

	System calls	System programs	Application programs
Level	**low-level programming interface to OS services**	higher-level utilities shipped with the OS	user programs that solve problems
Used by	running programs, usually **through an API**	users, **without writing programs** (e.g., shell commands to create or delete a file)	end users
Requests	**basic functions: process control, file and device manipulation**	one request may be **translated into a sequence of system calls** with richer functions	use system calls through libraries
Part of the OS?	yes (kernel interface)	yes (ship with the OS, not part of the kernel)	**no**
Examples	`fork()`, `read()`, `write()`	`ls`, `cp`, compilers, text editors	games, spreadsheets

X2-16 Compiler, linker, loader roles; relocation

What does each of the compiler, linker and loader do? What is a relocatable object file, and what is relocation? [2.31–2.32]

The compiler produces relocatable object files, the linker combines them into one executable on disk, and the loader brings that executable into memory to run.

```
gcc -c main.c            # compiler -> main.o
(relocatable object file)
gcc -o main main.o -lm   # linker   -> main (executable;
-lm links the math library)
./main                   # loader   -> program loaded
into memory (+ DLLs if needed)
```

- A **relocatable object file** is compiled code **designed to be loaded into any physical memory location**.
- **Linker**: **combines the relocatable object files, along with libraries if needed, into a single binary executable file** stored on secondary storage.
- **Loader**: **brings the binary executable into memory**, possibly with **dynamically linked libraries**, where it **becomes eligible to run on an available CPU**.
- **Relocation assigns final addresses to the program parts and adjusts the code and data to match those addresses**. Object and executable files have **standard formats** (machine code + symbol table, e.g., ELF on Linux), so the OS knows how to load and start them.

X2-17 Static linking vs dynamically linked libraries

Compare static linking with dynamically linked libraries (DLLs). Why do modern OSes use DLLs? [2.31]

Modern general-purpose OSes do not link libraries statically; DLLs are loaded when needed and shared, which saves memory.

	Static linking	Dynamic linking (DLLs)
When linked	at link time; library code is **copied into every executable**	**loaded when needed**, at load or run time
Executable size	larger	smaller
Memory use	every program has its own copy	**loaded only once and shared by all programs using the same version of the library**
Unused code	libraries may be linked in even if never used	**avoids linking and loading libraries that end up not being used**

Trade-off: the program **depends on the right library version being present** at run time, and loading it costs a little time.

X2-18 Why applications are OS-specific; ABI

Why can an executable built for one OS usually not run on another OS? How can programs be made portable? [2.15, 2.31, 2.56]

Because each OS has its own system calls, executable format and binary conventions.

- **Different system calls / APIs**: **each OS provides its own set of system calls** (different numbers, parameters, meanings and results), e.g., Win32 vs POSIX.
- **Different executable formats**: each OS uses its **own standard binary format** so it knows how to load and start the program (e.g., **ELF** on Linux, **PE** on Windows).
- **Different CPUs have different instruction sets**: e.g., **iOS runs on ARM and does not run Mac OS X (Intel) applications natively**. The rules for how binary code interfaces with a given OS on a given architecture are called the **ABI (application binary interface)**.

Portability: write to a **standard API** (e.g., POSIX) and **recompile** on each system, or use an **interpreted or virtual-machine language** such as **Java (JVM)**.

X2-19 OS design goals: user vs system

What are user goals and system goals in OS design? What affects the design at the highest level? [2.33]

- At the highest level, the design is **affected by the choice of hardware and the type of system** (desktop/laptop, mobile, distributed or real-time); it starts by **defining goals and specifications**.
- **User goals**: the OS should be **convenient to use, easy to learn and use, reliable, safe and fast**.
- **System goals** (for designers and maintainers): it should be **easy to design, implement and maintain**, and **flexible, reliable, error-free and efficient**.
- OS design is **not "solvable"**; it is a **highly creative software-engineering task**, though some approaches have proven successful.

X2-20 Policy vs mechanism: definition, separation

Distinguish policy and mechanism with an example. Why should they be separated? [2.34]

Policy: what will be done. Mechanism: how to do it.

- Example: a **timer** is a **mechanism** for ensuring CPU protection, but **deciding how long the timer is set for a particular user** is a **policy** decision.
- **Why separate**: policies change over time; if policy and mechanism are properly separated, **a policy change needs minimal change to the mechanism**, which gives **flexibility**. The same mechanism can support **"I/O-intensive programs have priority over CPU-intensive ones"** or **the opposite policy**.
- In a nutshell, **design goals determine the policies, and the OS implements these policies through specific mechanisms**.

X2-21 Scenario: classify policy or mechanism

Policy or mechanism? (a) code that saves and restores registers to switch processes; (b) "interactive jobs get higher priority than batch jobs"; (c) a hardware timer; (d) "each user may run at most 10 ms at a time"; (e) checking permission bits on every `open()`; (f) "only the owner may write a file". [2.34]

- (a) **Mechanism**: how the CPU is switched between processes.
- (b) **Policy**: decides which jobs get priority.
- (c) **Mechanism**: a tool for ensuring CPU protection.
- (d) **Policy**: how long the timer is set for a user.
- (e) **Mechanism**: how access control is enforced.
- (f) **Policy**: what access is allowed.

Rule: **"what should be done / who gets what" = policy; "how it is done" = mechanism**.

X2-22 OS in a high-level language: pros, cons

What are the advantages and disadvantages of implementing an OS in a high-level language (C/C++) rather than assembly? [2.35]

The advantages outweigh the small costs, so most OSes are now written in **C or C++** (the **lowest levels of the kernel may still be in assembly**; e.g., Android's system libraries are in C/C++ and its application frameworks mostly in Java).

- **Advantages**: the code **can be written faster, is more compact, and is easier to understand and debug**; **advances in compiler technology improve the generated code for the entire OS by simple recompilation**; and the OS is **far easier to port** to other hardware.
- **Disadvantages**: possibly **reduced speed and increased storage requirements**, which are **not a major issue in today's computer systems**.

X2-23 MS-DOS vs FreeBSD: running a program

Compare how MS-DOS and FreeBSD run a user program. [2.36–2.37]

MS-DOS is single-tasking and runs a program without creating a process; FreeBSD is multitasking and runs each program in a new process (`fork()` + `exec()`).

	MS-DOS	FreeBSD (UNIX variant)
Tasking	**single-tasking**	**multitasking**
Running a program	the shell (invoked at boot) **loads the program into memory, overwriting all but the kernel; no process is created**	the shell **calls `fork()` to create a new process**, then `exec()` **to load the program** into it
Memory	**single memory space**	each process has its own address space
While it runs	only that program runs	the shell **waits for the process to terminate** or **continues with user commands**
When it exits	**the shell is reloaded**	the process terminates; the shell continues
OS support	simple	**CPU scheduling, process coordination, memory management**

X2-24 Simple structure: why MS-DOS was vulnerable

Why was MS-DOS vulnerable to buggy or malicious programs? [2.39]

Because nothing protected the OS and hardware from application programs.

- MS-DOS has a **simple structure**: it was **written to provide the most functionality in the least space**, was **not carefully divided into modules**, and its **interfaces and levels of functionality are not well separated**.
- **Application programs can access I/O directly**, bypassing the OS.
- It was written for the **Intel 8088, which had no dual mode (kernel vs. user mode) and no hardware protection**, so **one faulty program could crash the whole system**.

X2-25 Monolithic kernel: pros, cons, Linux

What is a monolithic kernel? Why do UNIX, Linux and Windows still use this structure despite its drawbacks? [2.40–2.42]

A **monolithic structure places all the functionality of the kernel into a single, static binary file that runs in a single address space**; in traditional UNIX, **the kernel is everything below the system-call interface and above the physical hardware** (UNIX = **kernel + system programs**).

- **Why still used**: a **distinct performance advantage**, since there is **very little overhead in the system-call interface, and communication within the kernel is fast**.
- **Drawback**: an enormous amount of functionality is **combined into one level**, making it **difficult to implement, debug and maintain**; it is **tightly coupled**.
- **Linux** is monolithic (it **runs entirely in kernel mode in a single address space**; applications use **glibc** to reach the system-call interface) but also **modular**: **loadable kernel modules** let the kernel be modified at run time.

X2-26 Tightly vs loosely coupled; modular design

Distinguish a tightly coupled system from a loosely coupled (modular) one. What is the advantage of modularity? [2.43]

- **Tightly coupled** (monolithic): essentially **one big program running in one address space**, where **changes to one part of the system affect other parts**.
- **Loosely coupled (modular)**: the system is **divided into separate, smaller components, each with specific and limited functionality**, with specified interactions between them.
- **Advantage**: **changes in one component affect only that component**, giving **more freedom and flexibility in creating and modifying each component** and **simplifying debugging and verification**. The **layered approach** is one way to make a system modular.

X2-27 Layered approach: information hiding, debugging

Describe the layered approach. What is information hiding, and why is a layered OS easy to construct and debug? [2.44–2.45]

The OS is **divided into layers, each built on top of lower layers**: **layer 0 (bottom) is the hardware; layer N (top) is the user interface**. Each layer has **data structures and a set of functions**, **uses the services of the layer below** and **offers services to the layer above**.

- **Information hiding**: a layer **needs to know only what the lower-layer operations do (their interface), not how they are implemented**; each layer **hides its own data structures, operations and hardware** from higher layers.
- **Main advantage, simplicity of construction and debugging**: layers are **debugged one at a time, starting from the lowest layer**; when a layer is tested, the layers below are already verified, so **an error found must be in the layer being tested**.
- Layering is used successfully in other systems such as **computer networks (TCP/IP)** and web applications.

X2-28 Why few OSes are purely layered

Why do few operating systems use a pure layered approach? What is the trend instead? [2.45, 2.53]

1. **Difficulty of defining the layers**: there are **significant challenges in appropriately defining the layers and their functionalities**, since each layer may use only lower layers.
2. **Performance**: the **overhead of requiring a program to traverse multiple layers to obtain a service**.

Trend: **fewer layers with more functionality** and **modularized code**, avoiding the problems of complex layer definition and interaction.

X2-29 Microkernel: file access by message passing

In a microkernel OS, describe how an application reads a file. Why does this hurt performance? [2.46–2.47]

The application and the file server never interact directly; they exchange messages through the microkernel.

- The microkernel keeps only **minimal process and memory management plus a communication facility (IPC)**; services such as the **file server** and device drivers run as **user-level programs in separate address spaces**.
- To read a file, the **application sends a request message to the microkernel, which passes it to the file server; the file server does the work and replies by message through the microkernel**.
- **Performance cost**: each request needs **several user-to-kernel transitions and message copies between separate address spaces** (**increased system-function overhead**), instead of one function call inside a monolithic kernel.

X2-30 Microkernel: advantages, drawback, Windows NT

Why is a microkernel OS easier to extend and port, and more secure and reliable? What is its main drawback? Give an example. [2.48]

- **Easier to extend**: **new services are added in user space without modifying the kernel**; when the kernel must change, **the changes are fewer because the kernel is much smaller**.
- **Easier to port**: the **much smaller kernel can be more easily ported from one hardware platform to another**.
- **More secure and reliable**: **less code runs in kernel mode**; most services run as **user processes**, so **if a service fails, the rest of the OS remains untouched**; a simpler kernel is also more reliable.
- **Main drawback**: **performance suffers** from **increased system-function overhead**: user-space to kernel-space communication and **copying messages between services in separate address spaces**. Example: **Windows NT** (a layered microkernel) performed worse than Windows 95, so **Windows XP became more monolithic**.

X2-31 LKM approach; USB device scenario

What is the loadable kernel module (LKM) approach? What happens when a USB device is plugged into a running Linux machine whose kernel lacks its driver? [2.49–2.51]

The kernel has a set of core components and links in additional services via modules, either at boot time or during run time; this is the **best current methodology** in OS design.

- **USB case**: the needed **driver module is loaded dynamically into the running kernel** (and can be **removed** later), with **no need to recompile the kernel or reboot**.
- **Why better**: linking services dynamically is **preferable to adding features directly to the kernel, which would require recompiling the entire kernel every time a change is made**; since it is hard to predict every needed feature, functionality can be **added and removed while the kernel is running**.
- Linux uses LKMs **mainly for device drivers and file systems** (Solaris also loads scheduling classes, loadable system calls, executable formats and STREAMS modules); the result is a **dynamic, modular kernel with the performance benefits of a monolithic system**.

X2-32 LKM vs microkernel; LKM vs layered

Compare the LKM approach with the microkernel approach (one similarity, one difference). How does LKM also resemble a layered design? [2.50]

- **Similarity with a microkernel**: **the primary module has only core functions and knowledge of how to load and communicate with other modules**; other services are added separately.
- **Difference**: LKM is **more efficient, because modules do not need to invoke message passing to communicate**; they run inside the kernel and call each other directly, whereas microkernel services run in **user space** and talk through **message passing**.
- **Versus layered**: like a layered design, **each kernel section has a well-defined, protected interface**, but LKM is **more flexible, since any module can call any other module**.

X2-33 Hybrid systems: Linux, Windows, macOS

Why are most modern OSes hybrid systems? Explain how Linux, Windows and macOS combine structures. [2.52–2.53]

Very few OSes adopt a single, strictly defined structure; they **combine different structures to address performance, security and usability needs**.

- **Linux**: **monolithic** (OS in a single address space, so **efficient performance**) and **modular** (new functionality is **dynamically added** via LKMs).
- **Windows**: **largely monolithic**, but keeps microkernel-like behaviour with **separate subsystems (personalities) running as user-mode processes**, and supports **dynamically loadable kernel modules**.
- **macOS / iOS**: **layered** (user interface, application environments, kernel environment) with the kernel **Darwin = Mach microkernel + BSD UNIX kernel**, plus **dynamically loadable kernel extensions (kexts)**.

X2-34 Compare monolithic, layered, microkernel, LKM

Compare the monolithic, layered, microkernel and loadable-kernel-module structures. [2.38–2.53]

Monolithic is fastest but hardest to maintain; microkernel is the most extensible and reliable but slowest; LKM combines modularity with near-monolithic speed.

	Monolithic	Layered	Microkernel	LKM (modular)
Idea	all kernel functions in **one static binary, one address space**	**layer 0 = hardware ... layer N = UI**; each layer uses only lower layers	**minimal kernel** (process and memory management, IPC); other services in **user space**	**core kernel + modules loaded or removed at boot or run time**
Communication	direct calls inside the kernel	calls to lower layers	**message passing** via the kernel	direct calls; **any module can call any other**
Performance	**best: little system-call overhead**	overhead of **traversing layers**	**worst: message-passing overhead**	**close to monolithic**
Main advantage	**efficiency**	**simple construction and debugging**	**easy to extend and port; more secure and reliable**	**flexible; no recompile or reboot**
Main drawback	**hard to implement, debug, maintain** (tightly coupled)	**hard to define the layers**	**performance suffers**	modules run in kernel mode, so a faulty module can still crash the kernel
Examples	**original UNIX**, Linux, Windows (largely)	TCP/IP networking; few pure OSes	**Mach**, Darwin, Windows NT	**Linux, Solaris**

X2-35 Scenario: choose an OS structure

Which structure would you choose, and why, for: (a) a server where speed matters most; (b) a safety-critical system where a failed service must not bring down the OS; (c) a desktop OS that must add drivers and file systems without rebooting; (d) an OS to be ported to many hardware platforms? [2.41–2.53]

- (a) **Monolithic** (possibly with LKMs): **very little overhead in the system-call interface and fast communication within the kernel**.
- (b) **Microkernel**: services run as **user processes in separate address spaces**, so **if a service fails, the rest of the OS remains untouched**; less code in kernel mode makes it **more secure and reliable**.
- (c) **Loadable kernel modules (modular / hybrid)**: drivers and file systems are **loaded or removed at run time without recompiling or rebooting**, while keeping monolithic performance.
- (d) **Microkernel**: the **small kernel is more easily ported** to new hardware (writing the OS in a **high-level language** also helps).

X2-36 Mac OS X structure: Darwin, Mach, BSD

Describe the structure of Mac OS X. What is Darwin, and what do Mach and BSD each provide? [2.54–2.57]

Mac OS X is **hybrid and layered**: **Aqua GUI** (for mouse or trackpad) → **application environments and services** (**Cocoa** programming environment with an **Objective-C API**; **Core frameworks** for graphics and media, e.g., QuickTime, OpenGL) → **kernel environment, Darwin**.

- **Darwin** = **Mach microkernel** + **BSD UNIX kernel** (the hybrid **XNU kernel**), with **two system-call interfaces: Mach system calls (known as traps) and BSD system calls (POSIX functionality)**, plus rich libraries (standard C library, networking, security, programming-language support).
- **Mach** provides **fundamental OS services: memory management, CPU scheduling and IPC facilities**.
- **BSD** provides the **POSIX API, networking, file system and security**.
- The kernel environment also has an **I/O Kit for device drivers** and **dynamically loadable modules called kernel extensions (kexts)**.

X2-37 macOS vs iOS: similarities, differences

Compare macOS and iOS. [2.54–2.56]

Similar: both share the same architecture of **user interface, programming (language) support, graphics and media services, and the kernel environment Darwin (Mach microkernel + BSD UNIX kernel)**; iOS is **structured on Mac OS X with added functionality**.

	macOS	**iOS**
User interface	**Aqua** (mouse or trackpad)	**Springboard** (touch devices)
Programming API	**Cocoa** (Objective-C API)	**Cocoa Touch** (Objective-C API for touch-screen devices)
Graphics and services	**Core frameworks** (QuickTime, OpenGL)	**Media services** (graphics, audio, video) and **Core services** (cloud computing, databases)
CPU	Intel	**ARM**; iOS **does not run Mac OS X applications natively**

X2-38 iOS vs Android: similarities, differences

Compare iOS and Android. [2.58]

Similarities: (1) both are **based on existing kernels** (Android on **Linux**, iOS on the **Mac OS X kernel**); (2) both use a **layered software-stack architecture**; (3) both provide **rich frameworks for developers** (graphics, audio, hardware features).

	iOS	**Android**
Source	**closed-source**; runs only on Apple mobile devices	**open-source**; runs on a variety of mobile platforms (Open Handset Alliance, led by Google)
App language	**Objective-C**	**Java**, using a separate **Android API**
Execution	**executes code natively**	apps run on a **virtual machine, the Android RunTime (ART)**

X2-39 Android architecture: ART, JNI, HAL, Bionic

Explain the roles of ART, JNI, HAL and Bionic in the Android architecture. [2.58–2.59]

- **ART (Android RunTime)**: the **virtual machine on which Java applications execute**, **optimized for mobile devices with limited memory and CPU processing capabilities**.
- **JNI (Java Native Interface)**: lets developers **bypass the virtual machine** and write Java programs that **access specific hardware features**; such programs are **generally not portable** from one hardware to another.
- **HAL (hardware abstraction layer)**: **abstracts all hardware** (camera, GPS chip, other sensors) and gives applications a **consistent view independent of specific hardware**.
- **Bionic**: Google's **standard C library for Android**, used **instead of glibc**. Underneath is a **modified Linux kernel** for mobile systems (e.g., **power management**); libraries include **WebKit**, **SQLite** and **SSL**.

X2-40 History: from batch to time sharing

Briefly describe how operating systems evolved, and why the focus moved from hardware utilization to user convenience. [2.60]

Early on, hardware was expensive and humans relatively cheap, so OSes maximized hardware utilization; later hardware became cheap and humans expensive, so OSes focused on users.

- **Hand-programmed machines (1945–1955)** had low hardware utilization, followed by **batch processing (1955–1965)** and **multiprogramming (1965–1980)**: multiple jobs in memory, **multiplexed over the CPU(s)**.
- **Time sharing**: users share servers through cheap terminals. **Personal computers**: utilization is not a major concern; the focus is on **user interfaces and APIs**. **Distributed OSes**: consistency, performance, reliability and resource management. **Real-time OSes**: guarantee or prioritize jobs.

X2-TF True / False quick verdicts

Statement	**T/F**	**Why (≤ 12 words)**
`printf()` is a system call provided by the kernel.	**F**	It is a library function that calls `write()`.
Every C library (API) function makes a system call.	**F**	Some, e.g., `strlen()`, run entirely in user mode.
Windows `CreateProcess()` invokes the `NTCreateProcess()` system call.	**T**	API functions invoke system calls for the programmer.
`read()` returns 0 at end of file and −1 on error.	**T**	Otherwise it returns the number of bytes read.
Using an API lets a program bypass kernel protection.	**F**	APIs only wrap system calls; protection still applies.

Statement	T/F	Why (≤ 12 words)
An API caller must know how the system call is implemented.	**F**	The caller only obeys the API and knows the result.
Each system call has a number that indexes a table.	**T**	The system-call interface maintains the table.
Block and stack methods limit the number of parameters.	**F**	Only registers are limited; block and stack are not.
With the stack method, the OS pushes and the program pops.	**F**	The program pushes parameters; the OS pops them.
The command interpreter is always part of the kernel.	**F**	Sometimes in the kernel, sometimes a system program.
Most users' view of the OS is defined by system calls.	**F**	It is defined by application and system programs.
Daemons are background services that run until the system halts.	**T**	Launched at boot, e.g., for error logging.
The loader combines object files into an executable.	**F**	The linker combines; the loader loads into memory.
A DLL is loaded once and shared by programs using it.	**T**	Same library version; saves memory.
Policy specifies how something is done.	**F**	Policy = what; mechanism = how.
A timer is a mechanism; its length for a user is a policy.	**T**	The slides' standard policy-versus-mechanism example.
Writing an OS in a high-level language makes it easier to port.	**T**	Recompile for new hardware; code is also easier to debug.
MS-DOS creates a new process for every program it runs.	**F**	Single-tasking: program overwrites all memory but the kernel.
MS-DOS on the Intel 8088 had dual mode and hardware protection.	**F**	Neither; applications could access I/O directly.
Monolithic kernels have very little system-call overhead.	**T**	Single address space; fast communication within the kernel.
In a layered OS, layer 0 is the user interface.	**F**	Layer 0 is the hardware; layer N is the UI.
In a microkernel, an app and the file server interact directly.	**F**	They exchange messages through the microkernel.
Windows XP was more microkernel-like than Windows NT.	**F**	XP became more monolithic for performance.
Loadable kernel modules communicate by message passing.	**F**	Modules call each other directly, so they are more efficient.
LKMs can be loaded and removed while the kernel runs.	**T**	No recompiling or rebooting is needed.
Windows personalities run as user-mode processes.	**T**	A microkernel-like feature of largely monolithic Windows.
In Darwin, Mach provides networking and the file system.	**F**	Mach: memory, CPU scheduling, IPC; BSD: networking, files.
iOS runs Mac OS X applications natively.	**F**	Different CPU architecture (ARM vs. Intel).
Android programs written with JNI are portable across hardware.	**F**	JNI accesses specific hardware; generally not portable.
Android uses glibc as its standard C library.	**F**	Android uses Google's Bionic C library.

X3 · Ch3 Processes · IPC · Interrupts

X3-01 Program vs process; loading a program

What is the difference between a program and a process? When does a program become a process, and can one program be associated with several processes?

- A **program is a passive entity** — an **executable file stored on disk**. A **process is an active entity** — a **program in execution**, with a **program counter** specifying the next instruction to fetch and execute and a set of **associated resources** (address space, registers, open files).
- A program **becomes a process when its executable file is loaded into main memory** (double-click an icon or enter a command in the shell): the OS (1) loads the **code and static data**, (2) allocates the **run-time stack** and fills in `argc` / `argv`, (3) allocates the **heap**, (4) sets up I/O — **three open file descriptors: stdin (0), stdout (1), stderr (2)**.
- **Yes — one program can be executed by multiple processes** (e.g., several copies of a web browser): the **text section is the same, but each has its own data, heap, stack and PC**, so they are **separate execution sequences**. A process can itself be an execution environment for other programs (e.g., the **JVM**).

X3-02 Memory layout: text, data, heap, stack

Describe the sections of a process in memory. In the program below, where are `x`, `y`, `values`, `i`, the five `int`s from `malloc()`, `argc` / `argv` and the code of `main` stored?

```
int x;
int y = 15;
int main(int argc, char *argv[]) {
    int *values;
    int i;
    values = (int *)malloc(sizeof(int) * 5);
    for (i = 0; i < 5; i++)
        values[i] = i;
    return 0;
}
```

Section	Contents
Text	the **program code**
Data	**global variables**, divided into **initialized data** and **uninitialized data**
Heap	**memory dynamically allocated during run time** (`malloc()` / `free()`)
Stack	**temporary data for function calls: function parameters, return addresses, local variables**

- A **separate section holds `argc` and `argv`** for `main()`. The **stack and heap grow toward each other**, and the OS must keep them from overlapping. The PC and registers form the **current activity**, not a memory section.
- `x` → **uninitialized data**; `y` → **initialized data**; `values`, `i` (locals) → **stack**; the five `int`s → **heap**; `argc` / `argv` → **argc/argv section**; code of `main` → **text**.

X3-03 Process states and transition events

Name the five process states. Give the transition caused by each event: (a) admitted, (b) selected by the CPU scheduler, (c) timer interrupt, (d) a disk read is issued, (e) it calls `wait()` for its child, (f) the disk read completes, (g) it calls `exit()`.

New (being created — allocating resources), **ready** (waiting to be assigned to a CPU core), **running** (instructions are being executed), **waiting** (waiting for some event to occur), **terminated** (finished — deallocating resources).

Event	Transition
(a) admitted	**new → ready**
(b) scheduler dispatch (CPU scheduling)	**ready → running**
(c) timer interrupt	**running → ready** (interrupt)
(d) I/O request	**running → waiting** (I/O or event wait)
(e) `wait()` for a child	**running → waiting** (event wait)
(f) I/O completion	**waiting → ready** (I/O or event completion)
(g) `exit()`	**running → terminated**

- **There is no waiting → running and no ready → waiting**: a process whose event completes **re-enters the ready queue** and must be dispatched; a ready process is not executing, so it cannot request I/O.
- On one core **at most one process is running**; any number can be ready or waiting.

X3-04 PCB contents; process table; Linux task_struct

What is a Process Control Block (PCB) and what does it contain? How does the OS keep track of all PCBs, e.g., in Linux?

The **PCB (task control block)** is a **kernel data structure** holding **all information associated with a process**:

1. **Process state** — new, ready, running, waiting, …
2. **Program counter** — location of the next instruction to execute
3. **CPU registers** — accumulators, index registers, **stack pointers**, general-purpose registers, condition codes
4. **CPU-scheduling information** — **priority, pointers to scheduling queues**, scheduling parameters
5. **Memory-management information** — memory allocated (page / segment tables)
6. **Accounting information** — CPU time used, clock time elapsed, time limits
7. **I/O status information** — I/O devices allocated, **list of open files**

- Each process has a unique **PID** (positive integer; **PID 1 = init / systemd**). The **process table (process list)**, indexed by PID, points to every PCB. **Linux** keeps a **circular doubly linked list (task list) of `struct task_struct`** (pid, state, time_slice, parent, children, files, mm).
- The PCB is what is **saved and restored in a context switch**.

X3-05 Threads within a process

How is a thread represented, and what do the threads of one process share? Why use several threads in a process?

- A **thread** is a **single unique execution context**: its own **program counter, registers, execution flags and stack**. It is executing when this context is **resident in the CPU registers**.
- Threads of a process **run within the same address space — they share code, data and open files**, while each keeps **its own thread state**.
- So a process can **perform more than one task at a time** (e.g., a **web browser** with one thread displaying images or text and another **retrieving data from the network**), and on **multicore systems its threads run in parallel**. A traditional single-threaded process is a **heavy-weight process**.

X3-06 I/O-bound vs CPU-bound; good process mix

Define I/O-bound and CPU-bound processes. Why should the long-term scheduler select a good process mix?

- **I/O-bound** — spends **more time doing I/O than computation**; **many short CPU bursts**. **CPU-bound** — **generates I/O requests infrequently**, mostly computing; **few very long CPU bursts**.
- A **good mix keeps both the CPU and the I/O devices busy**. If all processes are I/O-bound, the **ready queue is almost always empty** and the CPU idles; if all are CPU-bound, the **I/O wait queues are almost always empty** and the devices sit unused.
- Hence the long-term scheduler (mainframes) **strives for a good process mix** to maximize **CPU utilization**.

X3-07 Scheduling queues and the queueing diagram

Which scheduling queues does the OS maintain, and how does a process migrate among them during its lifetime?

- **Ready queue** — processes **in main memory, ready and waiting to execute**. **Wait queues** — processes **waiting for an event** such as I/O completion (one per device or event). The queues link **PCBs** (the PCB holds **pointers to scheduling queues**).
- A new process joins the ready queue until it is **dispatched**. While running, it leaves the CPU when it:
  1. **issues an I/O request** → I/O wait queue → back to ready when the I/O completes;
  2. **uses up its time slice** → straight back to the ready queue;
  3. **creates a child and waits for its termination** → wait queue until the child ends;
  4. **waits for an interrupt** → wait queue until the interrupt occurs.
- This repeats until it terminates. **The state of a process is determined by the queue it is in** (the running process is the exception), and **each queue can have its own scheduling policy**.

X3-08 Long-, short- and medium-term schedulers compared

Compare the long-term, short-term and medium-term schedulers. What is the degree of multiprogramming, and why must the short-term scheduler be fast?

	Long-term (job) scheduler	Short-term (CPU) scheduler	Medium-term scheduler
Role	**selects which processes are brought into memory** from the job queue (allocates memory)	**selects a process from the ready queue and allocates a CPU core**	**swaps a process out to disk and back in later**
Frequency	**very infrequent** (seconds, minutes) — may be slow	**very frequent** (milliseconds) — **must be fast**	when the workload is too heavy
Degree of multiprogramming	**dictates it**	no direct effect	**reduces it**
Where used	**mainframes / minicomputers, not PCs**	all systems	mainframes / minicomputers

- **Degree of multiprogramming = the number of processes currently residing in memory.**
- The short-term scheduler runs every few milliseconds: if choosing takes 1 ms and the process then runs 10 ms, **1/(10 + 1) ≈ 9 % of the CPU is wasted on scheduling**.

X3-09 Swapping and the medium-term scheduler

What is swapping? When and why does the medium-term scheduler swap a process out?

- **Swapping**: the **medium-term scheduler removes a partially executed process from memory, stores it temporarily on secondary storage (swap space)**, and later **brings it back into memory to continue execution** where it left off.
- **When**: the **degree of multiprogramming must be reduced** — **memory is over-committed / the workload is heavy** and execution becomes slow (or to improve the process mix).
- **Why**: it **frees memory and other resources for the remaining processes**, so they run faster; the swapped-out process resumes when resources become available.

X3-10 Context switch: saved state and kernel actions

What is a context switch? Describe the actions the kernel takes, and name events that cause one.

- A **context switch** occurs when the **CPU switches from one process (or thread) to another**: the OS must **save the state of the old process and load the saved state of the new process**. The **context is represented in the PCB** — **CPU registers, program counter, stack pointer**, process state, memory-management information.
- Kernel actions: (1) an **interrupt or system call** enters the kernel; (2) **save the running process's context in its PCB**, set its state (ready / waiting) and queue it; (3) the **short-term scheduler selects** the next process; (4) **restore that process's context from its PCB** and resume it in user mode. Architecture-specific work such as **flushing data and instruction caches** may also be needed.
- Causes: **timer interrupt (time slice expired)**; a **system call** that blocks (I/O request, `wait()`) or lets the scheduler pick another process (e.g., `fork()`); an **interrupt that readies a higher-priority process**; **process termination**.

X3-11 Context-switch overhead; multiple register sets

Why is context-switch time overhead, and what does it depend on? Some CPUs have multiple register sets: what happens if the new context is already in a register set, and if it is not?

- **Context-switch time is pure overhead — the system does no useful work while switching**; **the more complex the OS and the PCB, the longer the switch**.
- It is **highly dependent on the hardware**: **memory speed, the number of registers to copy, and special instructions** (e.g., a single instruction to load or store all registers).
- **Multiple register sets** (e.g., **SUN UltraSPARC**): if the new context is already loaded, the switch **only changes the pointer to the current register set** — very fast. If it is in memory and **all register sets are in use**, one set must be **saved to memory** and the new context **loaded from memory** into it — slower, like an ordinary switch.

X3-12 Context switch vs mode switch

Is a context switch the same as switching from user mode to kernel mode? Explain.

No.

- A **mode switch** flips the **mode bit** (user ↔ kernel) on a **system call, interrupt or trap**; the CPU saves the running process's state, runs kernel code and may **return to the same process** — e.g., `getpid()` returns at once and **no other process runs**.
- A **context switch** changes **which process runs**: the kernel **saves the old process's state in its PCB and loads another process's saved state**. It is **performed by the kernel (in kernel mode)**, typically after a **timer interrupt or a blocking system call**.
- So every context switch requires entering the kernel, but **a mode switch does not always lead to a context switch**.

X3-13 Interrupt-driven I/O and device controllers

Explain how the CPU and a device controller cooperate to perform interrupt-driven I/O.

- **I/O devices and the CPU execute concurrently and asynchronously. Each device controller is in charge of a particular device and has a local buffer.**
- The **device driver** starts the operation by loading the controller's registers; the **controller moves data between the device and its local buffer** while the **CPU continues other work**.
- When finished, the **device controller informs the CPU by causing an interrupt** (signal on the **interrupt-request line**): for input, **data is available in the local buffer**; for output, the **I/O operation is completed**.
- The CPU **handles the interrupt** — for slow devices (keyboard, mouse) the **CPU moves the data between the local buffer and main memory** — and then **returns to the interrupted instruction**.

X3-14 Interrupt handling and the interrupt vector

What is an interrupt? Describe how the CPU handles one, and explain why an interrupt vector is used.

- An **interrupt** is a **signal from hardware to the CPU, through an interrupt-request line, that an event requires CPU attention** (e.g., **I/O completion**). It **transfers control to an interrupt service routine (interrupt handler)**, which is **part of the kernel**.
- Handling: the CPU **senses the interrupt-request line after executing every instruction**; on a signal it **saves the state of the interrupted process** (PC, registers), **reads the interrupt number**, **uses it as an index into the interrupt vector** and **jumps to the handler** in kernel mode; afterwards it **restores the state and returns to the interrupted instruction**.
- **Interrupt vector** = a **table (array) holding the addresses of the interrupt handlers**, indexed by each interrupt's **unique number** → **fast dispatch to the proper handler**, with no generic routine needed to find the source first.
- **All modern operating systems are interrupt-driven** (hundreds of interrupts per second).

X3-15 Maskable vs nonmaskable interrupts; priority levels

Distinguish maskable and nonmaskable interrupts. What are interrupt priority levels for?

- Most CPUs have **two interrupt-request lines**:
    - **Nonmaskable** — **cannot be turned off**; reserved for events such as **unrecoverable memory errors and power failure**.
    - **Maskable** — **can be turned off by the CPU before executing critical instruction sequences that must not be interrupted**; used by **device controllers to request service**, which can be delayed.
- **Interrupt priority levels** let the CPU **defer low-priority interrupts without masking all interrupts**, and let a **high-priority interrupt preempt the execution of a low-priority interrupt** — interrupts are not all serviced the same way. A **trap (software interrupt) gets a relatively low priority** compared with device interrupts.

X3-16 Programmed I/O vs DMA; DMA transfer steps

Why is DMA used instead of programmed I/O? Describe how a DMA transfer is set up and completed.

- **Programmed I/O**: the **CPU runs special I/O instructions to move one byte at a time** between memory and a device — acceptable for **slow devices** (keyboard, mouse), but for **fast devices and large transfers** (disk) it **wastes CPU time** on data movement.
- **DMA**: the **DMA controller bypasses the CPU and transfers a whole block directly between the I/O device and memory**; the CPU/OS only **initializes** it, which **relieves the CPU** to run other work concurrently.
- Set-up: the OS **writes a DMA command block into memory** (**source and destination addresses, read or write mode, number of bytes**) and **writes its location to the DMA controller**; the controller uses **bus mastering** (grabs the bus, momentarily keeping the CPU out of main memory) and **interrupts the CPU once, when the transfer is complete** — not after every byte.
- Six steps (disk → buffer X): (1) the driver is told to transfer disk data to buffer X; (2) the driver tells the disk controller to transfer C bytes to X; (3) the disk controller initiates the DMA transfer; (4) it sends each byte to the DMA controller; (5) the DMA controller writes the bytes to X, **increasing the address and decreasing C until C = 0**; (6) **when C = 0, DMA interrupts the CPU** to signal completion.

X3-17 Dual-mode operation, mode bit, privileged instructions

What is dual-mode operation and why is it needed? How does the hardware support it, and what happens when a user program executes a privileged instruction?

- **Dual-mode operation** separates **user mode** (user code) from **kernel mode** (OS code); it **allows the OS to protect itself and other system components** — the **OS from errant users, and errant users from one another**.
- A **mode bit provided by hardware** shows the current mode: **user = 1, kernel = 0**. Some instructions are **privileged — executable only in kernel mode** (e.g., **I/O control, timer management, interrupt management**), and devices are accessed only in kernel mode, so user programs must request OS services through **system calls**.
- A **privileged instruction attempted in user mode is treated by the hardware as illegal and trapped to the OS**; it is not executed.
- **Control starts in kernel mode**; the mode is **set to user mode before control passes to a user application** and returns to kernel mode **via an interrupt, a trap or a system call**; **return from a system call resets user mode**.

X3-18 Which operations must be privileged

Which of these must be privileged, and why? (a) set the value of the timer (b) read the clock (c) clear memory (d) issue a trap instruction (e) disable interrupts (f) modify entries in the device-status table (g) access an I/O device.

- **Privileged: (a), (c), (e), (f), (g)** — set the timer, clear memory, disable interrupts, modify the device-status table, access I/O devices. A user program doing these could **monopolize the CPU** (timer, interrupts), **wipe out the OS or other processes** (clear memory), or **bypass the OS's control of devices**.
- **Not privileged: (b) read the clock** (harmless, read-only) and **(d) issue a trap instruction** — a trap is exactly **how a user program requests an OS service (system call)**, so it must work in user mode.

X3-19 Timer: OS always regains control

How does the OS ensure that a user program cannot keep the CPU forever (e.g., one stuck in an infinite loop)?

- With a **timer**: before giving control to a user program, the OS **sets the timer to interrupt after a specified period** (a counter decremented by the clock; an **interrupt occurs when it reaches 0**).
- The **timer interrupt transfers control back to the OS in kernel mode**, which may **treat it as a fatal error, give the program more time, or schedule another process** (time slice expired).
- **Instructions that modify the timer are privileged**, so a user program cannot turn it off — hence **a user program always returns control to the OS**.

X3-20 Three types of mode transfer

Compare the three ways control passes from user mode to kernel mode: system call, interrupt, and trap/exception.

Type	Nature	Examples
System call	**a user process requests a service from the OS** — like a function call, but **outside** the user process	`read()`, `write()`, `exit()`, `fork()`
Interrupt	**external, asynchronous event**, **independent of the user process**	**timer**, **I/O completion**
Trap / exception	**internal, synchronous event** caused by the running process	**protection violation (segmentation fault)**, **divide by zero**, privileged instruction in user mode

- All three **set the mode bit to kernel mode**, save the running process's state and run kernel code; the return resets user mode. A **trap is a software-generated interrupt**, and a **system call is carried out by executing a trap**.

X3-21 System-call path: printf() to write()

In terms of user mode and kernel mode, explain what happens when a program calls `printf("hi\n")`.

1. `printf()` **is a C library (API) function running in user mode**; it formats the text into a buffer and **invokes the** `write()` **system call** on fd 1 (buffering may delay this until a newline or flush).
2. The `write()` wrapper puts the **system-call number and parameters in registers** and executes a **trap** → the hardware sets the **mode bit to kernel mode (0)** and jumps to the kernel's system-call handler.
3. The kernel **indexes the system-call table** with the number, runs the **write service routine** (device driver) and stores the **return value** (bytes written, or −1).
4. **Return from the system call resets the mode bit to user mode (1)** and execution continues after the call. If the device is busy, the process may **block (running → waiting)** while another process runs.

X3-22 Beyond two modes: rings, VMM, machine switch

How can dual-mode operation be extended beyond two modes? What is a machine switch?

- **Intel processors have four protection rings: ring 0 = kernel mode, ring 3 = user mode**; rings 1 and 2 could run OS services but are **rarely used**. **ARMv8 systems have seven modes.**
- **CPUs that support virtualization have a separate mode for the virtual machine manager (VMM)**, with **more privileges than user processes but fewer than the kernel**.
- A **machine switch** is the VMM's counterpart of a context switch: it **saves the entire machine state of one guest OS — registers, PC and, unlike a context switch, any privileged hardware state** — **restores the new VM's machine state and jumps to its PC** (inside its kernel or in a user application).

X3-23 Process creation options; tree of processes

When a parent creates a child, what are the options for (a) sharing resources, (b) execution, and (c) the child's address space? Give UNIX examples.

- Processes form a **tree of processes**: a **parent creates children, which create their own children**; on Linux the root is **systemd (PID 1)**, and each process has a unique **pid**. The child needs CPU time, memory, files and I/O devices, and the parent may pass it **initialization data**.
- (a) **Resource sharing**: **parent and children share almost all resources —** `fork()` (the child gets copies of everything); **children share a subset of the parent's resources —** `clone()`; or **they share no resources**.
- (b) **Execution**: the **parent continues to execute concurrently with its children** (shell runs `a.out &`), or the **parent waits until some or all children have terminated** (shell runs `a.out` and waits).
- (c) **Address space**: the **child is a duplicate of the parent (same program and data)** — UNIX `fork()`; or the **child has a new program loaded into it** — `exec()` after `fork()`.

X3-24 fork() return values and why two

What does `fork()` return, and why must the parent and the child receive different values?

- `fork()` **takes no parameters**; each process gets **exactly one return value**:
    - **> 0 in the parent** — the **PID of the newly created child**;
    - **0 in the child**;
    - **−1 in the original process if it fails — no child is created** (`errno` is set).
- The **child duplicates the parent's entire address space, including the program counter**, so **both continue at the instruction right after** `fork()` **with identical code**; the **return value is the only way to tell which process is running**, so the two can take different branches. The parent also learns the **child's PID**, needed later to **wait for or terminate** that child.
- Which process runs first is **decided by the CPU scheduler**; their execution may interleave.

X3-25 fork() steps, copy-on-write, clone()

What steps does the kernel take to implement `fork()`? Why is this expensive, and how does Linux speed it up?

- Steps: (1) **create and initialize the PCB** in the kernel (unique PID); (2) **create a new address space** (allocate memory); (3) **initialize it with a copy of the entire contents of the parent's address space** — **time consuming**; (4) **inherit the parent's execution context** (e.g., **open files**); (5) **inform the CPU scheduler that the child is ready to run**.
- The copy is wasted when the child **calls** `exec()` **immediately** and discards it.
- **Copy-on-write (COW)**: **parent and child share a single copy of the pages**, and a page is **copied only when either process writes to it**. If `exec()` follows `fork()`, the pages **never need to be copied → greatly speeds up process creation**, while each process still behaves as if it had its own copy.
- Linux implements `fork()` with the more general `clone()` system call, whose **flags specify which resources parent and child share**.

X3-26 After fork(): copied, shared, different

After a successful `fork()`, what does the child get from the parent and what is different? Which of the stack, heap and shared-memory segments are shared?

- **Duplicated (the child gets its own copy)**: the **address space — text, data, heap, stack**, hence **global and local variables**; the **program counter and registers**; **open file descriptors** (referring to the same files and pipes); **current and root directory**, process resources, **resource limits**; customized **signal handlers**.
- **Different**: the **PID** (and parent PID), the **return value of** `fork()`, **running time** and **running state**.
- **Stack and heap are copied, not shared** — a change by one process is **invisible to the other**; of the three, **only shared-memory segments are shared**.
- Because descriptors are inherited, a **pipe created before** `fork()` connects parent and child.

X3-27 Output: child modifies a global array

What is printed at LINE X and LINE Y?

```
#define SIZE 5
int nums[SIZE] = {0, 1, 2, 3, 4};
int main() {
    int i;
    pid_t pid = fork();
    if (pid == 0) {
        for (i = 0; i < SIZE; i++) {
            nums[i] *= -i;
            printf("CHILD: %d ", nums[i]);  /* X */
        }
    } else if (pid > 0) {
        wait(NULL);
        for (i = 0; i < SIZE; i++)
            printf("PARENT: %d ", nums[i]); /* Y */
    }
    return 0;
}
```

- **LINE X (child):** `CHILD: 0 CHILD: -1 CHILD: -4 CHILD: -9 CHILD: -16`
- **LINE Y (parent):** `PARENT: 0 PARENT: 1 PARENT: 2 PARENT: 3 PARENT: 4`, printed **after** the child's output because of `wait(NULL)`.
- `fork()` **gives the child its own copy of the parent's data section**, so the child changes **only its own copy of** `nums`; the parent's array is unchanged. The same holds for the **heap**: after `int *h = malloc(...)`, both processes hold the **same address in** `h`, but `(*h)++` in the child does not change the parent's value.

X3-28 Output: fork() return value vs getpid()

The actual PIDs of the parent and the child are 2600 and 2603. What values are printed at lines A, B, C and D?

```
pid_t pid, pid1;
pid = fork();
if (pid == 0) {                             /* child */
    pid1 = getpid();
    printf("child: pid = %d", pid);         /* A */
    printf("child: pid1 = %d", pid1);       /* B */
} else if (pid > 0) {                       /* parent */
    pid1 = getpid();
    printf("parent: pid = %d", pid);        /* C */
    printf("parent: pid1 = %d", pid1);      /* D */
    wait(NULL);
}
```

- **A = 0** (`fork()` returns 0 in the child); **B = 2603** (`getpid()` gives the child's own PID); **C = 2603** (`fork()` returns the child's PID to the parent); **D = 2600** (the parent's own PID).
- Key: `getpid()` **always returns the caller's own PID** (`getppid()` its parent's), whereas `fork()`**'s return value is the child's PID only in the parent**.

X3-29 Output: unflushed printf buffer copied by fork()

What is printed (a) if every `printf` is flushed immediately, (b) if stdout is buffered? Why can `./a.out > out.txt` show `A` twice even with `printf("A\n")`?

```
int main() {
    printf("A");
    fork();
    printf("B\n");
    return 0;
}
```

- (a) **Flushed immediately:** `A` **once,** `B` **twice** → `AB` then `B`. `A` is printed before `fork()`, and **both processes execute** `printf("B\n")`.
- (b) **Buffered:** `AB` **is printed twice.** The unflushed `"A"` is still in the **stdio buffer inside the process's address space**, and `fork()` **copies the address space, buffer included**, so each process later flushes its own `"A"` followed by `"B\n"`.
- When stdout is a **file or pipe it is fully buffered**, so even `"A\n"` is still in the buffer at `fork()` and appears twice; a terminal is line-buffered and flushes at `\n`.
- Fix: **`fflush(stdout)` before `fork()`**.

X3-30 exec(): success, failure, execlp vs execvp

What does a successful `exec()` do to the calling process? Does it create a new process or change the PID? When is the statement after `exec()` executed? Contrast `execlp()` and `execvp()`.

- **`exec()` changes the program being run by the current process: it creates a new process image from an executable file, replacing the address space (text, data, heap, stack).** It **does not create a new process — the PID, PCB and open file descriptors stay the same** (so redirection set up before `exec()` still applies).
- **A successful `exec()` never returns**, so the code after it is **never executed**; **only if it fails** (e.g., program not found, no permission) does it **return −1**, and execution continues with the next statement.
- **`execlp("ls", "ls", "-l", NULL)`** passes the arguments as a **list** (count known at compile time); **`execvp(args[0], args)`** passes an **array (vector)** (count known only at run time, as for PA1's parsed commands). **p = search the PATH**. **`argv[0]` is the program name**, and the list/array **must end with `NULL`** because exec has no argument count and stops at the NULL — hence PA1's array of **9 entries for at most 8 arguments**.

X3-31 Output: exec in a loop succeeds/fails

How many processes are created and what is printed if (a) `execlp()` succeeds, (b) `execlp()` fails? What should the child do after a failed exec?

```
int main() {
    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();
        if (pid == 0)
            execlp("echo", "echo", "hi", NULL);
    }
    while (wait(NULL) > 0)
        ;
    printf("done\n");
    return 0;
}
```

- **(a) exec succeeds: 4 processes** (parent + 3 children). Each child **becomes `echo`**, prints `hi` and terminates — it **never returns to the loop**. The parent reaps all three and prints **`done` once, last**: `hi`, `hi`, `hi`, `done`.
- **(b) exec fails: `execlp()` returns −1 and the child continues the loop as a copy of the parent**, so every process forks in each remaining iteration → $2^3 = 8$ **processes**, **`done` printed 8 times**, no `hi`.
- So a child must **`exit()` after a failed exec** (e.g., `perror("exec"); exit(1);`). In PA1 a missing exit leaves an **extra shell process running** (redundant processes), so the **start and end PIDs no longer match**.

X3-32 Why a shell uses fork, exec, wait

Why does a shell create a child with `fork()` and call `exec()` in the child, instead of calling `exec()` directly? Why is it useful that UNIX separates the two calls?

- If the shell called `exec()` itself, **its own image would be replaced by the command**, and the **shell would be gone** when the command ended. So the shell **`fork()`s a child**, the **child `exec()`s the command** (overlays the new program), and the **parent `wait()`s for the child**, then reads the next command. For a **background command (`a.out &`) the shell does not wait**.
- **Most OSes create a process and load a program in one step; UNIX splits this into `fork()` and `exec()`.** The child can therefore **run code between the two calls to set up the new program's environment** — **I/O redirection with `dup2()`, connecting pipes, closing file descriptors** — without affecting the shell. **Copy-on-write** keeps the extra `fork()` cheap.

X3-33 PA1: cd and exit as built-ins

In PA1, why can `cd` not be executed with `exec*()`, and how is it implemented? Why is `exit` also a built-in?

- The **current working directory is a per-process attribute**. If the shell forked a child to change directory, **only the child's directory would change, and the change is lost when the child exits** — a child **cannot change its parent's directory**, so the shell would stay where it was.
- So `cd` is a **built-in command run inside the shell process itself, without `fork()`**, using `chdir(path)`: **0 on success, −1 on error** (print "Myshell cd command error"); it accepts absolute and relative paths, including `.` and `..`.
- **`exit` must terminate the shell process itself**, so it is likewise handled inside the shell, not in a child.

X3-34 PA1: why system() and popen() are banned

Why are `system()` and `popen()` forbidden in PA1? Why must pipes be used with `open/read/write/close` rather than `fopen/fread/fwrite/fclose`?

- `system()` **creates a shell (e.g., bash) that runs the command, and the parent is blocked until it finishes**; `popen()` also **has a shell run the command** concurrently and returns a `FILE*` for one-way communication with it. Using them **hands all process creation, redirection and piping to another shell**, defeating the purpose of PA1 (**process management and IPC through system calls**); they are detected by pattern matching and score **0 marks**.
- PA1 must use `fork`, `exec*`, `wait`, `pipe`, `dup/dup2`. `FILE*` **is a high-level struct defined only in the C standard library and cannot be used in pipe operations**; the low-level `open/read/write/close` **work on file descriptors**, which is what `pipe()` returns and `dup2()` manipulates.

X3-35 PA1: Ctrl-C (SIGINT) handling with signal()

How does PA1 customize Ctrl-C handling? What is a signal, and what does the handler receive?

- A **signal is a software-generated interrupt** that **notifies a process that an event has occurred**. **Ctrl-C** makes the OS send **SIGINT (signal number 2)**, whose **default handler terminates** the process; SIGINT is **asynchronous** (caused by an event outside the process).
- PA1 **overrides the default with a user-defined handler** registered at start-up; the **signal number is passed to the handler as its parameter**:

```
void handler(int sig) {          /* sig = 2 */
    printf("Myshell (pid=%d) terminates"
           " by Ctrl-C with signal: %d\n",
           getpid(), sig);
    exit(0);
}
/* in main(): */
signal(SIGINT, handler);
```

- **Child processes inherit the customized handler** through `fork()`. The PID printed must equal the start PID; otherwise redundant shell processes exist.

X3-36 Termination: exit, wait, abort, cascading

Describe how a process terminates normally and how its parent obtains the exit status. Why might a parent terminate a child, and what is cascading termination?

- After its last statement a process **asks the OS to delete it with** `exit()`, called **directly or indirectly** (the **C run-time library calls** `exit()` **when** `main()` **returns**). The OS **de-allocates and reclaims its resources: memory (program, data, heap, stack), open files, I/O buffers**.
- It may **return a status value (integer) to its waiting parent**: `pid = wait(&status);` returns the **PID of the terminated child** (so the parent knows which child ended) and stores its **exit status** (`exit(0)` = normal, non-zero = error); `wait()` returns −1 if there are no children.
- A parent may terminate a child (`abort()` on UNIX/Linux in the slides, `TerminateProcess()` on Windows; a process can also be killed with `kill()`) because the **child has exceeded its allocated resources**, the **task assigned to the child is no longer required**, or the **parent is exiting and the OS does not allow a child to continue**. The parent knows the **child's PID from** `fork()`.
- **Cascading termination**: some OSes **do not allow a child to exist if its parent has terminated — when a process terminates, all its children must also be terminated, initiated by the OS**. UNIX/Linux instead **re-parent orphans to init/systemd**.

X3-37 Zombie vs orphan; role of init/systemd

Compare zombie and orphan processes, and explain the role of `init` / `systemd` in process termination.

	Zombie	**Orphan**
Definition	**has terminated, but its parent has not yet called** `wait()`	**its parent terminated without calling** `wait()`
What remains	only its **PID, process-table entry and PCB** (holding the **exit status**); all other resources released	everything — it **may still be running**
Runnable?	**no** — it has no address space	**yes** (if cascading termination is not used)
Cleaned up by	the **parent's** `wait()`, which releases the PID, entry and PCB	**init/systemd**, its new parent

- **All processes pass through the zombie state, usually only briefly.** The entry is kept because the **process table holds the exit status** that the parent collects; if the child is already a zombie, `wait()` returns at once.
- **init/systemd (PID 1) is assigned as the new parent of orphans and periodically calls** `wait()`, collecting their exit status and releasing their PID and process-table entry; otherwise **parentless terminated processes would remain zombies forever, wasting system memory**.

X3-38 Zombie or orphan? Two scenarios

What happens to the child in each case? (A) The child calls `exit(0)` at once; the parent calls `sleep(30)` and then `wait(NULL)`. (B) The child calls `sleep(30)` and then prints; the parent returns from `main()` at once without `wait()`.

- **(A) The child is a zombie for about 30 seconds**: it has terminated, but its **PID, process-table entry and PCB (exit status) remain until the parent's** `wait(NULL)` reaps it; then they are released.
- **(B) The child becomes an orphan**: the parent ends first, so the **shell prompt reappears before the child's output**. The child **keeps running**, is **re-parented to init/systemd (PID 1)**, prints after the prompt, and when it exits **init/systemd's** `wait()` **reaps it**, so it does not remain a zombie.
- Correct practice: the parent calls `wait()` **for each child** before it exits.

X3-39 Independent vs cooperating processes

Distinguish independent and cooperating processes. Give reasons for allowing processes to cooperate.

- **Independent** processes cannot affect or be affected by other processes; **cooperating processes can affect or be affected by other processes** (e.g., they **share data**), so they need an **interprocess communication (IPC) mechanism** to exchange data.
- Reasons: **information sharing** — several applications need the same information (e.g., **copy and paste**); **computation speedup** — split a task into **parallel subtasks** (e.g., machine-learning jobs on multiple cores); **modularity** — divide system functions into separate processes or threads; **convenience** — a user can work on many tasks at once.

X3-40 Shared memory vs message passing

Compare the two IPC models, shared memory and message passing, giving an advantage of each.

	Shared memory	Message passing
How	a **region of memory is shared**; processes **read and write the shared region** — **implicit** communication	**explicit messages** exchanged with `send()` **/** `receive()`
Kernel role	**system calls only to set up the region**, then ordinary memory accesses	**kernel intervention (system call) for every message**
Speed	**faster**	slower for large data
Synchronization	**the processes must coordinate themselves** (e.g., producer–consumer)	provided by the IPC facility (e.g., blocking send/receive)
Suits	large data on one machine	**small amounts of data; distributed environments**

- Message passing lets processes **communicate and synchronize without sharing the address space** — **Mach** uses it as its primary IPC, and Windows provides a form of it. Both models are commonly used.

X3-41 Bounded buffer: empty, full, BUFFER_SIZE − 1

In the shared-memory bounded-buffer solution with `in` and `out`, when is the buffer empty or full, and why can it hold only `BUFFER_SIZE - 1` items? With `BUFFER_SIZE = 10`, `in = 3`, `out = 7`, how many items are in the buffer?

- **Unbounded buffer**: no practical limit on size (only the consumer may wait). **Bounded buffer**: fixed size — the consumer waits when it is empty, the **producer waits when it is full**.
- **Empty:** `in == out`. **Full:** `(in + 1) % BUFFER_SIZE == out`. The producer stores `buffer[in]` and advances `in = (in + 1) % BUFFER_SIZE`; the consumer takes `buffer[out]` and advances `out` the same way; each **busy-waits** ("do nothing") while the buffer is full or empty, which wastes CPU cycles.
- Since `in == out` **already means empty**, a completely full buffer would look identical, so **one slot is always left unused → at most `BUFFER_SIZE - 1` items**.
- Items = `(in - out + BUFFER_SIZE) % BUFFER_SIZE` = (3 − 7 + 10) % 10 = **6** (slots 7, 8, 9, 0, 1, 2).
- Using all slots needs a shared **`count`**, but then `count++` / `count--` in two processes must be **synchronized**, or a **race condition** occurs (Chapter 6).

X3-42 Message passing: link and design choices

What must processes P and Q do to communicate by message passing? List the design choices for implementing the link, and compare fixed- and variable-size messages.

- P and Q must **establish a communication link** between them, then **exchange messages with the primitives** `send(message)` **and** `receive(message)`.
- Design choices for the link: **direct or indirect communication** (naming); **synchronous (blocking) or asynchronous (non-blocking)** send/receive; **automatic or explicit buffering**.
- **Fixed-size messages** keep the **system-level implementation straightforward but make programming harder**; **variable-size messages need a more complex system implementation but make programming simpler**.

X3-43 Direct vs indirect communication; mailbox sharing

Compare direct and indirect communication. If P1, P2 and P3 share mailbox A, P1 sends a message, and both P2 and P3 call `receive()`, who gets it?

	Direct	Indirect (mailbox)
Naming	**processes name each other explicitly**: `send(P, msg)`, `receive(Q, msg)`	**send to / receive from a mailbox**: `send(A, msg)`, `receive(A, msg)`; each mailbox has a **unique id**
Link set-up	**automatic**; each needs only the other's **identity**	**only if both processes share a mailbox**
Processes per link	**exactly one pair**	**may be more than two**
Links per pair	**exactly one**	**several**, one per shared mailbox
Drawback	**limited modularity**: changing a process's identifier means re-examining all other processes	the OS must provide **create, send/receive and destroy** for mailboxes

- Who receives depends on the rule the system adopts: **(1) allow a link to be associated with at most two processes; (2) allow only one process at a time to execute `receive()`; or (3) let the system select the receiver arbitrarily and notify the sender who the receiver was.**

X3-44 Blocking vs non-blocking send/receive; rendezvous

Explain blocking and non-blocking send and receive. What is a rendezvous, and how does it simplify producer–consumer?

	Blocking (synchronous)	Non-blocking (asynchronous)
`send()`	the **sender is blocked until the message is received** by the receiving process or mailbox	the **sender sends and resumes** its operation
`receive()`	the **receiver blocks until a message is available**	the **receiver retrieves either a valid message or null**

- **When both `send()` and `receive()` are blocking, we have a rendezvous** between the sender and the receiver.
- Producer–consumer becomes **trivial**: the producer calls **`send(next_produced)`** and the consumer **`receive(next_consumed)`**; the blocking calls do the synchronization — **no shared buffer, `in` / `out` or busy waiting**.
- Blocking is simple but the sender may wait unnecessarily; non-blocking allows more concurrency — so systems usually offer both.

X3-45 Buffering: zero, bounded, unbounded capacity

Describe the three ways to implement the queue of messages on a link. When must the sender wait?

Capacity	Queue	Sender waits?
Zero capacity	**no messages are queued** on the link	**always — it must wait for the receiver (rendezvous)**
Bounded capacity	**finite length of n messages**	**only if the link is full**
Unbounded capacity	**infinite length**	**never**

- Messages reside in this **temporary queue whether communication is direct or indirect**. Zero capacity means **no buffering**; bounded and unbounded are **automatic buffering**.
- A **UNIX pipe behaves like bounded capacity**: it has a limited size, so a **writer blocks when the pipe is full**.

X3-46 Pipe design issues; ordinary vs named pipes

List the four issues to consider when implementing a pipe, and answer them for ordinary pipes and UNIX named pipes (FIFOs).

Issue	Ordinary pipe	Named pipe (UNIX FIFO)
Unidirectional or bidirectional?	**unidirectional** (one-way, producer → consumer)	**bidirectional**
Half or full duplex?	one direction only	**half-duplex** — two FIFOs for both directions at once
Relationship needed?	**parent–child**	**none**; several processes, typically **several writers**
Over a network?	**no — same machine**	**no — same machine**
Creation	`pipe(int fd[2])` : **fd[0] = read end, fd[1] = write end**	**`mkfifo()`** ; appears as a **typical file in the file system**
Lifetime	**ceases to exist when the processes finish communicating and terminate**	**continues to exist until explicitly deleted**

- An ordinary pipe **cannot be accessed from outside the process that creates it**; the parent creates it and the **child inherits it through `fork()`**. UNIX treats a pipe as a **special type of file** accessed with `read()` / `write()` .
- Windows: ordinary pipes are **anonymous pipes** (`ReadFile()` / `WriteFile()`); **Windows named pipes are full-duplex and work between the same or different machines** (`CreateNamedPipe()`).

X3-47 When to use ordinary vs named pipes

Give a situation in which an ordinary pipe is more suitable than a named pipe, and one in which a named pipe is more suitable.

- **Ordinary pipe**: simple **one-way communication between a parent and its child** on one machine — e.g., a shell running `ls | sort` , or a parent writing a file's contents into a pipe while its child **counts the characters**. No name is needed, and the pipe **disappears automatically** when they terminate.
- **Named pipe**: **unrelated processes** (no parent–child relationship) or **many writers** — e.g., several processes **write log messages to a named pipe** and a **logger server reads it** and appends them to the log file. The processes can start independently, and the FIFO **persists** until it is deleted.

X3-48 Why close unused pipe ends (EOF)

Why must every process close the pipe ends it does not use? In the lab's `ls | wc -l` program, what happens if the process that runs `wc` does not close `pfds[1]` ?

- **`read()` on a pipe returns 0 (end-of-file) only when every write end is closed in every process.** If any process — even the reader itself — **keeps a write end open**, the reader **blocks forever** once the data is used up, and the program hangs.
- In the lab the parent runs `close(0); dup(pfds[0]); close(pfds[1]); wait(0); execlp("wc", ...)` . Without **`close(pfds[1])`** , `wc` holds a write end itself, so **after `ls` exits, `wc` never sees EOF and never prints the count**.
- Rule from the slides: **if the parent writes and the child reads, the parent must close the read end and the child must close the write end**.
- Note: since that parent calls `wait(0)` before `wc` reads, an `ls` output larger than the **pipe capacity** would make `ls` **block on a full pipe**, and both would wait forever.

X3-49 What happens: blocking pipe and FIFO calls

What happens when: (a) a process reads an empty pipe whose write end is still open; (b) it reads after all write ends are closed; (c) a process writes to a full pipe; (d) a process opens a FIFO for writing while no process has it open for reading; (e) two processes read the same FIFO?

- (a) **`read()` blocks** until data is written — pipe **read/write are blocking I/O**.
- (b) **`read()` returns 0 = end-of-file** (after any remaining data has been read).
- (c) **`write()` blocks** until the reader removes data — a pipe has a **limited size** (lab: a 4 KB page; 64 KB on some systems).
- (d) **`open()` blocks until a reader opens the FIFO** — FIFOs block until **both ends are opened** (`O_NONBLOCK` makes FIFO access non-blocking).
- (e) **Data read from a FIFO is discarded**, so each byte goes to **only one reader** — a FIFO **cannot broadcast** a message. Writes of **at most `PIPE_BUF` bytes are atomic**; larger writes from several writers may interleave.

X3-50 Output: parent-to-child pipe program

List all possible outputs (assume each `printf` is flushed immediately).

```
int pfds[2]; char buf[30];
pipe(pfds);
pid_t pid = fork();
if (pid != 0) {
    printf("PARENT: writing to pipe\n");
    close(pfds[0]);
    write(pfds[1], "test", 5);
    wait(NULL);
    printf("PARENT: exiting\n");
} else {
    printf("CHILD: reading from pipe\n");
    close(pfds[1]);
    read(pfds[0], buf, 5);
    printf("CHILD: read \"%s\"\n", buf);
}
printf("world!\n");
```

Two outputs — only the first two lines can swap:

1. `PARENT: writing to pipe` / `CHILD: reading from pipe` / `CHILD: read "test"` / `world!` / `PARENT: exiting` / `world!`
2. `CHILD: reading from pipe` / `PARENT: writing to pipe` / `CHILD: read "test"` / `world!` / `PARENT: exiting` / `world!`

- The child's **`read()` blocks until the parent writes**, and the parent prints its line **before** `write()` , so `CHILD: read "test"` follows both first lines, then comes the child's `world!` .
- **`wait(NULL)`** puts `PARENT: exiting` and the parent's `world!` **after all of the child's output**. (5 bytes = `"test"` plus `'\0'` .)

X3-51 Two-way communication with ordinary pipes

How can a parent and child communicate in both directions using ordinary pipes? Why not use one pipe?

- An ordinary pipe is **unidirectional**, so **create two pipes before `fork()`** : `p2c` (parent → child) and `c2p` (child → parent).
- **Parent**: `close(p2c[0]); close(c2p[1]);` , then **writes to `p2c[1]`** and **reads from `c2p[0]`** . **Child**: `close(p2c[1]); close(c2p[0]);` , then **reads from `p2c[0]`** and **writes to `c2p[1]`** .
- With one pipe, a process that writes and then reads **may read back its own message**. (A UNIX FIFO is likewise **half-duplex**; two FIFOs are used for simultaneous two-way traffic.)
- Order the calls so that **one side writes first**; if both begin with a blocking `read()` , both wait forever.

X3-52 Shell pipelines and multi-level pipes

How does a shell implement `ls | grep x`? How is this extended to an n-segment pipeline such as PA1's `ls | sort -r | sort | ...`?

- The shell calls `pipe(fd)` **before forking**, then forks two children: **child 1 does `dup2(fd[1], 1)`** (stdout → write end) and **execs `ls`**; **child 2 does `dup2(fd[0], 0)`** (stdin ← read end) and **execs `grep x`** (it prints only the lines of `ls` output containing x). **Every process closes the pipe ends it does not use**, and the **shell closes both ends and waits for both children**. Neither program knows about the pipe — **pipes let separate tools be combined without being designed to work together**.
- **n segments**: in a **for-loop**, use **n − 1 pipes** and **n children**. Child i does `dup2(pipe[i-1][0], 0)` **if i > 0** and `dup2(pipe[i][1], 1)` **if i < n − 1**, **closes all pipe fds**, and **`execvp()`s its segment**. The parent **closes all pipe fds and waits for all n children** (PA1: at most 8 segments).

X3-53 dup vs dup2; I/O redirection

What do `dup()` and `dup2()` do? How is `wc -l < in.txt > out.txt` implemented, and why is the redirection done in the child after `fork()` and before `exec()`? What appears on the screen and in `log.txt` below?

```
int fd = open("log.txt", O_CREAT | O_WRONLY,
              S_IRUSR | S_IWUSR);
printf("A\n");
int saved = dup(1);
dup2(fd, 1);
printf("B\n"); fflush(stdout);
dup2(saved, 1);
printf("C\n");
```

- `dup(oldfd)` copies an open descriptor into the **lowest-numbered unused fd** (so `close(1); dup(fd);` makes fd 1 refer to the file). **`dup2(oldfd, newfd)`** copies it into **`newfd`, closing `newfd` first if it is open**. Both return the new fd (−1 on error); `dup2` is clearer when several fds are replaced.
- In the **child**: `in = open("in.txt", O_RDONLY); dup2(in, 0);` and `out = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644); dup2(out, 1);`, close `in` and `out`, then **`execvp("wc", args)`**. **File descriptors stay open across `exec()`**, so `wc` reads fd 0 and writes fd 1 without knowing they are files.
- Doing it **in the child** leaves the **shell's own stdin/stdout unchanged** (its prompts still reach the terminal).
- Program: **the screen shows `A` and `C`; `log.txt` contains** `B` — `dup(1)` saves the terminal, `dup2(fd, 1)` sends stdout to the file, `dup2(saved, 1)` restores it.

X3-54 Sockets, ports and the 4-tuple

What is a socket? How does client–server communication work, and how are two connections from the same client host to the same web server told apart?

- A **socket is an endpoint for communication**; two processes communicating over a network use **a pair of sockets**, one at each end. A socket is **identified by an IP address (host) and a port number (process or application)**.
- The **server is always running with a well-known address and port, listening** for requests; **only clients can initiate communication**. **Ports below 1024 are well known**, reserved for standard services (**SSH 22, FTP 21, HTTP 80**); a client picks an **unused port number above 1024**.
- **The 4-tuple (source IP address, source port, destination IP address, destination port) uniquely identifies a connection.** Two connections from host 146.86.5.20 to web server 161.25.19.8 port 80 use **different client ports** (e.g., 1625 and 1626), so their 4-tuples differ.

X3-55 Remote procedure calls (RPC)

What is a remote procedure call (RPC), and what is it commonly used for? How does it differ from using sockets directly?

- **RPC abstracts the procedure-call mechanism for use between systems with network connections**: a client calls a procedure on a remote host **as if it were a local call**. It is a **message-based communication scheme to provide remote services**.
- A **client-side stub** packs the procedure name and parameters into a **message** and sends it to an **RPC daemon** on the server, which executes the procedure and **sends the result back in a message**.
- Common use: **distributed file systems**, implemented as a **set of RPC daemons and clients**.
- **Sockets are low-level**: they exchange an **unstructured stream of bytes**, and the application must impose structure; RPC hides the networking behind a procedure call.

X3-TF True / False quick verdicts

Statement	T/F	Why (≤ 12 words)
A program is an active entity and a process is a passive one.	**F**	Reversed: a program is passive; a process is active.
Function parameters, return addresses and local variables are on the stack.	**T**	Heap holds `malloc()` memory; data section holds globals.
A process can move directly from waiting to running.	**F**	It goes waiting → ready, then is dispatched.
Issuing an I/O request moves a process from running to waiting.	**T**	It sits in a wait queue until the I/O completes.
On a single-core CPU, two processes can be running at the same instant.	**F**	At most one runs; the others interleave (concurrency).
The short-term scheduler runs infrequently, so it may be slow.	**F**	Invoked every few milliseconds; it must be fast.
The medium-term scheduler swaps processes out to disk.	**T**	Swapping reduces the degree of multiprogramming.
Context-switch time is useful work for the system.	**F**	Pure overhead: no useful work is done while switching.
A context switch means switching from user mode to kernel mode.	**F**	That is a mode switch; a context switch changes process.
Interrupts can only be generated by I/O devices.	**F**	Traps/exceptions are software-generated interrupts.
A trap is given higher priority than device interrupts.	**F**	Traps get relatively low priority versus device interrupts.
Device controllers request service on the nonmaskable interrupt line.	**F**	They use the maskable line; nonmaskable is for power/memory failure.

Statement	T/F	Why (≤ 12 words)
DMA needs one interrupt per byte transferred.	**F**	One interrupt when the whole block is done.
The mode bit is 1 in user mode and 0 in kernel mode.	**T**	Hardware-provided mode bit: user = 1, kernel = 0.
Reading the system clock must be a privileged instruction.	**F**	Harmless read; setting the timer is privileged.
A privileged instruction executed in user mode traps to the OS.	**T**	The hardware treats it as illegal.
A divide-by-zero error causes an asynchronous interrupt.	**F**	It is a synchronous trap (exception).
`fork()` returns 0 to the parent and the child's PID to the child.	**F**	Reversed: the child gets 0, the parent the child's PID.
A failed `fork()` returns −1 and creates no child.	**T**	The error is returned to the original process.
After `fork()`, the parent automatically waits for the child.	**F**	The parent must call `wait()` explicitly.
With copy-on-write, a child's writes become visible to the parent.	**F**	The written page is copied; the processes stay separate.
`exec()` creates a new process with a new PID.	**F**	It replaces the program image; the PID is unchanged.
Code after a successful `exec()` call is never executed.	**T**	`exec()` returns (−1) only on failure.
Open file descriptors stay open across `exec()`.	**T**	So redirection set up before `exec()` still applies.
A zombie keeps its PID and process-table entry until `wait()`.	**T**	The entry holds the exit status for the parent.
UNIX/Linux use cascading termination by default.	**F**	Orphans are re-parented to init/systemd instead.
Shared memory requires a system call for every access.	**F**	System calls are needed only to establish the region.
With zero-capacity buffering, the sender never waits.	**F**	It waits for the receiver; unbounded capacity never waits.
The `in`/`out` bounded buffer can hold `BUFFER_SIZE` items.	**F**	At most `BUFFER_SIZE - 1`, since `in == out` means empty.
An ordinary pipe can be used by any two processes.	**F**	Parent–child only; not accessible outside its creator.
A UNIX FIFO is full-duplex and works across machines.	**F**	Half-duplex, same machine only (unlike Windows named pipes).
Data read from a FIFO can be read again by another reader.	**F**	FIFO data is discarded once read; no broadcasting.
`read()` on a pipe returns 0 only after all write ends are closed.	**T**	Otherwise it blocks waiting for more data.
`cd` can be implemented by forking and calling `execvp("cd", ...)`.	**F**	It must be a built-in calling `chdir()` in the shell.
Only clients initiate client–server socket communication.	**T**	The server listens on a well-known port.

X4 · Ch4 Threads

X4-01 thread definition, thread vs process

What is a thread? Compare a thread with a process.

[4.6, 4.14–4.16]

A **thread is a basic unit of CPU utilization**: an independently scheduled **instance of execution** within a process, represented by a **thread ID, a program counter (PC), a register set and a stack**. A process is an **address space** (code, data, open files, other resources) **plus one or more threads**.

	Process	Thread
Role	**"passive" part: address space encapsulates protection**	**"active" component: encapsulates concurrency**
Memory	own address space, separate from other processes	**shares the address space** of its process
Creation	**heavyweight** (new address space, copy of program and data)	**lightweight** (new TCB and stack only)
Context switch	costly (address space changes)	**cheaper within a process** (address space stays)
Communication	needs **IPC** (shared memory / message passing)	directly through shared global data and heap
OS record	**PCB**	**TCB** (points to its PCB)

X4-02 shared vs private per-thread state

What do threads of the same process share, and what does each thread keep private? Why must each thread have its own stack and registers? [4.6, 4.16]

- **Shared by all threads of a process: code (text), data (global variables), heap, open files (file descriptors, network connections) and signals** (signal handlers), i.e., the address space and OS resources.
- **Private to each thread (kept in its TCB): thread ID, program counter, register set and stack**.
- **Why private:** each thread is an **independent flow of control** at a different instruction, so it needs its own **PC and registers**; its **stack** holds its own call frames (**parameters, temporary/local variables, saved PC**), which would overwrite each other if shared.
- Private does not mean protected: all stacks lie in the same address space, so a thread can still reach another thread's local variable through a pointer (e.g., main passes `&counter` to a thread).

X4-03 four benefits of multithreading, examples

State the four benefits of multithreaded programming and give an example of each. [4.7]

- **Responsiveness**: **may allow continued execution if part of a process (one thread) is blocked**, which matters for interactive applications. E.g., a word processor keeps responding to keystrokes while another thread does **background spelling and grammar checking**.
- **Resource sharing**: **threads share the resources of a process by default** (same address space), **easier than shared memory or message passing** between processes. E.g., a browser thread retrieves data from the network into a buffer that the display thread reads directly.
- **Economy**: **thread creation is much cheaper than process creation (less time and memory)**, and **context switching between threads of a process is faster** than between processes. E.g., a web server creates a thread, not a process, per client request.
- **Scalability**: **a process can take advantage of multicore architectures**, because its threads run in parallel on different cores. E.g., data mining, graphics or AI work split across 8 cores.

X4-04 thread creation and switching cheaper

Explain why creating a thread, and switching between two threads of the same process, are cheaper than doing the same for processes. [4.4, 4.14]

Threads of a process share its address space and resources, so only the thread-specific state has to be created or switched.

- **Creation:** a new process needs a **new address space and copies of program, data and resources** (PCB, memory, open files), so it is **heavyweight**. A new thread needs only **a new TCB and a stack**; the process-specific data structures are unchanged, so it is **lightweight**.
- **Context switch T1 → T2 (same process): save T1's register state and restore T2's**; **the address space remains the same, so the overhead is much smaller**. **Switching to a thread of a different process costs more**, because the address space must also be switched.

X4-05 disadvantages of threads vs processes

Give two disadvantages of using multiple threads instead of multiple processes. When would separate processes be preferable? [4.9, 4.16]

(1) There is no protection between threads, and (2) shared data needs synchronization.

- All threads run in **one address space**, and the address space is what **encapsulates protection**. A buggy thread can **corrupt shared data or crash the whole process**, e.g., an illegal memory access terminates the program and all its threads. Separate address spaces **keep a buggy program from thrashing the entire system**.
- Shared globals and heap invite **race conditions**: **if there is a data dependency, synchronization is required**, and **different paths of execution make testing and debugging difficult**.
- Prefer processes when tasks are **independent or untrusted** and **fault isolation** matters more than cheap sharing.

X4-06 multicore programming challenges

List and briefly explain the challenges of programming for multicore systems. [4.9]

Five challenges:

1. **Dividing tasks**: find **separate, concurrent tasks** in the application.
2. **Balance**: each task should perform an **"equal" amount of work**; a task that contributes little value may not be worth a separate core.
3. **Data splitting**: **data used by the tasks must be divided to run on separate cores**.
4. **Data dependency**: when one task uses data produced by another, **synchronization is required**.
5. **Testing and debugging**: **different paths of execution** (many possible interleavings) **make debugging difficult**.

X4-07 concurrency vs parallelism

Distinguish concurrency from parallelism. Can there be concurrency without parallelism? Parallelism without concurrency? [4.9–4.10]

- **Concurrency supports more than one task making progress**; on a **single processor core, the scheduler provides concurrency by multiplexing over time**.
- **Parallelism means a system can perform more than one task simultaneously**, which requires **multiple cores or processors**.
- **Concurrency without parallelism: yes**, e.g., 4 threads time-sliced on one core.
- **Parallelism without concurrency: no.** Tasks running simultaneously are all making progress, so every parallel system is also concurrent (not vice versa).
- E.g., 8 threads on 4 cores run in parallel across the cores and concurrently (interleaved) within each core.

X4-08 data vs task parallelism, classify examples

Explain data and task parallelism using an array of N integers on a dual-core system. Classify: (a) a blur filter applied to four image quadrants on four cores; (b) a video player with one thread decoding video, one decoding audio and one handling the UI. [4.11]

- **Data parallelism distributes subsets of the same data across multiple cores and performs the same operation on each core.** Thread A on core 0 sums elements 0 … N/2 − 1, thread B on core 1 sums N/2 … N − 1, and the two partial sums are added.
- **Task parallelism distributes threads (tasks) across cores, each thread performing a unique operation.** On the same array, thread A computes the **mean** on core 0 while thread B finds the **maximum** on core 1.
- They are **not mutually exclusive**: an application may use **both (hybrid)**.
- (a) **Data parallelism** (same filter, different subsets of the image). (b) **Task parallelism** (different operations).

X4-09 Amdahl's law statement, limit 1/S

State Amdahl's Law. What happens to the speedup as the number of cores N → ∞, and why does the serial portion have a "disproportionate effect"? [4.12–4.13]

$\text{speedup} \le 1/(S + (1 - S)/N)$, where **S = serial portion, 1 − S = parallel portion, N = number of processing cores**. It gives the **theoretical speedup in latency of a task at a fixed workload** gained by adding cores.

- **As N → ∞, (1 − S)/N → 0, so the speedup approaches 1/S**: the serial part takes the same time however many cores are added.
- **Disproportionate effect:** with only 10% serial (S = 0.1), 8 cores give $1/(0.1 + 0.9/8) = 1/0.2125 \approx 4.71$, not 8, and unlimited cores give at most $1/0.1 = 10$.

X4-10 Amdahl speedup, 60% parallel

An application is 60% parallel. Using Amdahl's Law, find the maximum speedup on 4 cores, on 16 cores, and with unlimited cores. [4.12]

S = serial portion = 1 − 0.6 = 0.4 (the 60% is the parallel part).

- N = 4: $1/(0.4 + 0.6/4) = 1/(0.4 + 0.15) = 1/0.55 \approx$ **1.82**
- N = 16: $1/(0.4 + 0.6/16) = 1/(0.4 + 0.0375) = 1/0.4375 \approx$ **2.29**
- N → ∞: $1/S = 1/0.4 =$ **2.5**

Four times as many cores raise the speedup only from 1.82 to 2.29, because the 40% serial part dominates.

X4-11 Amdahl serial fraction from measured speedup

A program runs 4 times faster on 8 cores than on 1 core. Assuming Amdahl's Law holds with equality, what fraction of it is serial? What is the maximum speedup with unlimited cores? [4.12]

$4 = 1/(S + (1 - S)/8)$ → $S + (1 - S)/8 = 0.25$ → multiply by 8: $8S + 1 - S = 2$ → $7S = 1$ → **S = 1/7 ≈ 0.143 (14.3% serial, 85.7% parallel)**. Maximum speedup as N → $\infty = 1/S =$ **7**.

Template: measured speedup X on N cores gives $S = (N/X - 1)/(N - 1)$. Check: $(8/4 - 1)/(8 - 1) = 1/7$.

X4-12 Amdahl cores needed for target speedup

An application is 10% serial. How many cores are needed for a speedup of at least 5? Can it ever reach a speedup of 12? [4.12]

$1/(0.1 + 0.9/N) \geq 5$ → $0.1 + 0.9/N \leq 0.2$ → $0.9/N \leq 0.1$ → **N ≥ 9 cores**. Check: $1/(0.1 + 0.9/9) = 1/0.2 = 5$, while 8 cores give only 4.71. **12 is impossible:** the **speedup approaches 1/S = 1/0.1 = 10 as N → ∞** and never exceeds it. Template: $N \geq (1 - S)/(1/X - S)$, valid only when the target X < 1/S.

X4-13 doubling cores vs halving serial part

A program has serial fraction S = 0.2 and runs on N = 4 cores. Which helps more: doubling the cores or halving the serial portion? Does the answer change if N = 8? [4.12]

- **N = 4** (now $1/(0.2 + 0.8/4) = 2.5$): doubling the cores gives $1/(0.2 + 0.8/8) = 1/0.3 \approx$ **3.33**; halving S gives $1/(0.1 + 0.9/4) = 1/0.325 \approx$ **3.08**. So **doubling the cores helps more**.
- **N = 8** (now $1/0.3 \approx 3.33$): doubling the cores gives $1/(0.2 + 0.8/16) = 1/0.25 =$ **4**; halving S gives $1/(0.1 + 0.9/8) = 1/0.2125 \approx$ **4.71**. So **halving the serial part helps more**.
- **Rule:** halving S beats doubling N exactly when **S > 1/N** (S·N > 1). As cores increase, the **serial portion dominates**, and only reducing S raises the ceiling **1/S** (here from 5 to 10).

X4-14 why real speedup is lower

Measured speedups on N cores are usually lower than Amdahl's Law predicts. Why? [4.9, 4.12]

Amdahl's Law is a theoretical upper bound (≤): it assumes the parallel part splits perfectly over N cores with zero overhead. In practice there is also:

- **Overhead** of creating, managing and context-switching threads.
- **Synchronization: data dependencies** force threads to wait for each other (locks, joins).
- **Imbalance:** tasks rarely perform an **"equal" amount of work**, so the slowest thread decides the finish time.
- **Contention** for shared memory, bus and caches, and other processes competing for the same cores.

X4-15 multithreading on a single core

Can a multithreaded program finish faster than its single-threaded version on a single-core CPU? Explain. [4.9, 4.14]

Yes if the threads perform I/O (or otherwise block); no for purely CPU-bound work.

- One core gives only **concurrency (multiplexing over time)**, not parallelism, so CPU-bound threads gain no speedup and add context-switch overhead.
- Threads **enable overlap of I/O with other activities within a single program: one thread runs on the CPU while another thread of the process is doing I/O**. The total time drops and the program stays **responsive**.
- This needs kernel threads (one-to-one or many-to-many). Under **many-to-one**, a blocking I/O call **blocks the entire process**, so there is no overlap.

X4-16 Thread Control Block contents

What does the OS keep in a Thread Control Block (TCB)? How is the TCB related to the PCB? [4.15]

Each thread has a TCB holding its private execution state, its scheduling and accounting information, and a pointer to its enclosing process (PCB).

- **Execution state:** CPU registers, program counter, **pointer to stack** (saved here while the thread is not running).
- **Scheduling info:** thread state, priority, CPU time. **Accounting info.**
- **Various pointers** that link the TCB into **scheduling queues**.
- **Pointer to enclosing process (PCB):** process-wide information (**process ID**, memory/address space, open files) stays in the **PCB** and is shared by all its threads, so the **PID is not in the TCB**.

X4-17 process state with multiple threads

A process has three threads: one running, one waiting for a disk read, one ready. What is the state of the process? Where are the threads' TCBs? [4.17–4.18]

Each thread has its own state (new, ready, running, waiting, terminated), recorded in its **TCB**; the threads are scheduled individually, so one thread of the process can be running while another waits and a third is ready.

- Running thread: executing on the CPU; it is **resident in the registers**.
- Ready thread: its TCB is in the **ready queue**.
- Waiting thread: its TCB is in the **disk unit's device queue**. **TCBs are organized into queues based on their states**, one queue per device/signal/condition, **each with its own scheduling policy**.

X4-18 user threads vs kernel threads

Distinguish user-level threads from kernel-level threads. [4.19–4.20]

User threads are created and managed by a user-level thread library and are unknown to the kernel; kernel threads are supported, managed and scheduled by the OS.

	User threads	Kernel threads
Managed by	**user-level threads library**	**operating system**
Known to kernel	**no, visible only to programmers**	yes; each has a **TCB**
Scheduling	**cannot be scheduled on a CPU directly**	**the only threads the OS schedules on CPUs**
Creation cost	lower	**more overhead** (kernel allocates resources)
Examples	Pthreads / Windows / Java thread APIs	Windows, Linux, macOS, iOS, Android

A mapping must exist between user threads and kernel threads (many-to-one, one-to-one, many-to-many).

X4-19 thread libraries, Pthreads specification

What is a thread library? Name the three primary thread libraries. Is Pthreads an implementation? [4.20]

A thread library provides the programmer with an API for creating and managing threads (user threads) in programs.

- Three primary libraries: **POSIX Pthreads** (POSIX API, UNIX/Linux), **Windows threads** (Win32 API), **Java threads** (Java API).
- **Pthreads is a specification, not an implementation:** the **POSIX standard (IEEE 1003.1c)** defines the API for thread creation and synchronization, and **different UNIX-like OSes implement it differently**.
- User threads are the **concurrent and modular entities** of a program that the OS maps onto kernel threads.

X4-20 pthread_create arguments, pthread_join purpose

Explain the arguments of `pthread_create()` and the purpose of `pthread_join()`. What can go wrong if `main()` never calls `pthread_join()`? [4.20, 4.33]

`pthread_create()` starts a new thread that runs `start_routine(arg)`; `pthread_join()` makes the caller wait until that thread terminates.

```
int pthread_create(pthread_t *tid,
                   const pthread_attr_t *attr,
                   void *(*start_routine)(void *),
                   void *arg);
```

- **tid** receives the new **thread identifier**; **attr** holds the thread **attributes** (stack size, scheduling info; **NULL or `pthread_attr_init()` = default**); **start_routine** is the function where the thread **begins execution**; **arg** is the **single `void *` argument passed to it**. Returns **0 on success**.
- **`pthread_join(tid, &ret)` blocks the caller until thread tid terminates** and collects its return value (**create/join strategy**: the parent thread waits for its child thread).
- Without a join, **main may return first, and returning from main calls `exit()`, which terminates the whole process including all its threads**. A worker may be killed before it finishes (e.g., `sum` is printed before it is computed).
- Compile and link with `gcc prog.c -lpthread`.

X4-21 passing arguments, returning thread results

A start routine takes only one `void *` argument. How do you pass several values to a thread, and how does the thread return a result to the thread that joins it? [4.20, 4.33]

Pack the values in a struct, pass its address as `arg`, and return a pointer with `return` or `pthread_exit()`; the joining thread receives it through `pthread_join()`.

```
struct args { int lo, hi; long sum; };
void *worker(void *p) {
    struct args *a = (struct args *)p;  /* cast back */
    a->sum = 0;
    for (int i = a->lo; i <= a->hi; i++) a->sum += i;
    return a;                          /* = pthread_exit(a) */
}
/* in main: */
struct args a = {1, 100, 0};  void *res;
pthread_create(&tid, NULL, worker, &a);
pthread_join(tid, &res);  /* res == &a; sum = 5050 */
```

- **Never return the address of the thread's local variable**: it lives on the thread's **private stack**, which is gone once the thread terminates. Return heap memory (`malloc`), a global, or a field of the argument struct.

X4-22 passing &i to threads bug

What is wrong with this code? What can it print, and how would you fix it? [4.20, 4.33]

```
void *show(void *arg) {
    printf("%d\n", *(int *)arg);  return NULL;
}
int main() {
    pthread_t t[3];  int i, j;
    for (i = 0; i < 3; i++)
        pthread_create(&t[i], NULL, show, &i);
    for (j = 0; j < 3; j++) pthread_join(t[j], NULL);
}
```

All three threads get the address of the same variable `i`, which main keeps changing. This is a race condition, so the output is unpredictable.

- Thread k reads `i` only when it gets scheduled, so it prints some value **from k to 3**, in any order: `0 1 2`, `1 2 3`, `2 3 3` and `3 3 3` are all possible.
- **Fix: give each thread its own copy:** `int id[3]; … id[i] = i; pthread_create(&t[i], NULL, show, &id[i]);` (or pass the value itself as `(void *)(long)i` and read it back with `(int)(long)arg`).

X4-23 pthread_exit vs return vs exit

In a multithreaded program, what is the difference between a thread calling `pthread_exit()`, returning from its start routine, and calling `exit()`? What happens when `main()` returns? [4.20, 4.33]

- **`pthread_exit(v)` or `return v` from the start routine terminates only the calling thread**; v goes to the thread that calls `pthread_join()` on it. The other threads keep running.
- **`exit()` called by any thread terminates the entire process, i.e., all of its threads. Returning from `main()` is the same as calling `exit()`.**
- So `main()` should **`pthread_join()` its workers** first, or end with `pthread_exit()`, which ends only the main thread; the process then ends when its last thread terminates.

X4-24 Pthreads trace, possible outputs

`f(arg)` prints the string `arg`. How many different outputs are possible? List them. [4.20, 4.33]

```
printf("M1\n");
pthread_create(&t1, NULL, f, "A");
pthread_create(&t2, NULL, f, "B");
printf("M2\n");
pthread_join(t1, NULL);  pthread_join(t2, NULL);
printf("M3\n");
```

6 possible outputs. M1 is always first (no thread exists yet) and **M3 is always last** (both joins return only after A and B are printed). **A, B and M2 can appear in any order**, because a created thread is only placed in the ready queue and **runs independently, depending on scheduling**: 3! = 6. M1, then one of {M2 A B, M2 B A, A M2 B, B M2 A, A B M2, B A M2}, then M3.

X4-25 compare four multithreading models

Compare the many-to-one, one-to-one, many-to-many and two-level models. [4.21–4.26]

They differ in how user threads are mapped to kernel threads, which decides blocking behaviour and parallelism:

Model	Mapping	One thread makes a blocking system call	Parallel on multicore?	Main drawback	Examples
Many-to-one	many user threads → **one** kernel thread	**entire process blocks**	**No: only one kernel thread can be active at a time**	no parallelism; blocking	Solaris Green Threads, GNU Portable Threads
One-to-one	**each** user thread → its own kernel thread	others keep running	**Yes**	**creating a user thread requires creating a kernel thread** (overhead); **threads per process may be restricted**	**Windows, Linux** (most OSes now)
Many-to-many	many user threads → **smaller or equal number** of kernel threads	others run on other kernel threads	**Yes**	**difficult to implement**	Windows ThreadFiber package
Two-level	many-to-many **+ a user thread may be bound to a kernel thread**	others run on other kernel threads	**Yes**	complex, like many-to-many	—

Concurrency: **many-to-one < many-to-many < one-to-one (maximum)**. **No model maps one user thread to several kernel threads.**

X4-26 blocking system call, many-to-one vs one-to-one

A process has 4 user threads on a 4-core machine. Thread T1 calls `read()` on a slow disk. What happens to T2–T4 under (a) the many-to-one model, (b) the one-to-one model? [4.22–4.23]

(a) **Many-to-one: T2–T4 are blocked too, because the entire process is blocked.** All four user threads are mapped to **one kernel thread**; T1's blocking system call blocks that kernel thread, and the OS has no other kernel thread of this process to schedule. Even before the call, **only one thread can run at a time, so 3 cores are idle for this process** (concurrency, no parallelism). (b) **One-to-one: T2–T4 keep running.** Each has its own kernel thread; only T1's kernel thread waits, and the others are scheduled **in parallel on the other cores**. This is why one-to-one **provides the maximum concurrency**.

X4-27 why most OSes use one-to-one

The one-to-one model has more overhead than many-to-one, yet most operating systems (Windows, Linux) use it. Why? [4.23, 4.26]

It gives maximum concurrency and true parallelism, and its cost (a limit on the number of kernel threads) matters less as the number of cores grows.

- A blocking thread does not block the others, and **multiple threads run in parallel on multiprocessor or multicore systems**.
- Drawback: **creating a user-level thread mandates creating a corresponding kernel thread**; kernel threads **consume resources (memory, I/O, etc.)**, so the number of threads per process may be **restricted**. But **with an increasing number of processing cores, limiting the number of kernel threads has become less important.**
- It is simpler than **many-to-many, which is most flexible but difficult to implement in practice**.

X4-28 many-to-many and two-level models

Explain the many-to-many and two-level models. What do they offer over one-to-one, and why is many-to-many rarely used? [4.24–4.26]

- **Many-to-many multiplexes many user-level threads onto a smaller or equal number of kernel threads, specific to a particular application or machine**: an application **may be allocated more kernel threads on an 8-core system than on a 4-core system**. The thread library decides which user thread runs on which kernel thread (**process-contention scope, PCS**).
- It offers: programmers can create **as many user threads as needed, not restricted by the number of kernel threads**; the **OS can create a sufficient number of kernel threads in advance (thread pool)**; a blocking call does not block the whole process; threads still run in parallel.
- **Two-level model: like many-to-many, but it also allows a user thread to be bound to a kernel thread**, so that thread always has a kernel thread of its own.
- Rarely used because it is **most flexible but difficult to implement in practice**, and with many cores one-to-one is good enough. Example: **Windows with the ThreadFiber package**.

X4-29 LWP, scheduler activations, upcalls

What is a lightweight process (LWP)? How do the kernel and the thread library cooperate under the many-to-many model?

- **An LWP is an intermediate data structure between user and kernel threads. To the user-thread library, an LWP appears to be a virtual processor on which the application can schedule a user thread to run** (many-to-one / many-to-many mapping; this choice is **PCS**).
- **Each LWP is attached to a kernel thread, and it is the kernel threads that the OS schedules on the CPUs. If a kernel thread blocks (e.g., waiting for I/O), the LWP blocks as well**, and so does the user thread running on it.
- **The OS coordinates with the thread library so that the number of kernel threads (LWPs) given to the process is dynamically adjusted for best performance.** The textbook calls this **scheduler activation**: the kernel tells the library about events (e.g., a thread is about to block) by an **upcall** and gives it a new LWP, on which the library runs another user thread.

X4-30 thread pool benefits

What is a thread pool? Why is serving requests from a thread pool better than creating a new thread for each request? [4.24]

A thread pool is a set of threads created in advance (at start-up) that wait in a pool for work. A request is handed to an idle pool thread, which **returns to the pool** when done instead of terminating. (The many-to-many model likewise lets the OS **create a sufficient number of kernel threads in advance**.)

- **Faster:** an existing thread services the request at once, with no **thread-creation delay** per request.
- **Bounded resources:** the pool **limits the number of threads that exist at any one time**, so a burst of requests cannot exhaust memory and CPU (each kernel thread **consumes resources**).
- **Flexibility: separating the task to be performed from the mechanics of creating the task** allows different strategies for running it (e.g., run after a delay or periodically).

X4-31 fork() and exec() in multithreaded process

If one thread of a multithreaded process calls `fork()`, should the child duplicate all threads or only the calling thread? What happens when a thread calls `exec()`? [4.28]

It depends on whether the child calls exec(); some UNIX systems provide two versions of fork() for this reason.

- **If exec() is called immediately after forking, duplicate only the calling thread. Duplicating all threads is unnecessary, as the program specified in exec() will replace the entire process.**
- **If the child does not call exec(), duplicate all threads**, so the child is a full copy of the parent. (Linux/POSIX `fork()` duplicates **only the calling thread**.)
- **exec() works as normal: it replaces the entire process, including all threads** (same PID). E.g., if a worker thread calls `execlp("echo", "echo", "X", NULL)` while main waits in `pthread_join()`, only `X` is printed and main never resumes.

X4-32 fork + pthread_create loop with join

How many processes are there in total, how many threads are created by `pthread_create()`, and how many times are `T` and `M` printed? (Assume each `printf` is output immediately.) [4.28]

```
void *work(void *arg) { printf("T\n"); return NULL; }
int main() {
    pthread_t tid;
    for (int i = 0; i < 2; i++) {
        fork();
        pthread_create(&tid, NULL, work, NULL);
        pthread_join(tid, NULL);
    }
    printf("M\n");
}
```

4 processes and 6 threads created; T is printed 6 times and M 4 times.

- i = 0: fork gives **2 processes**; each creates and joins one thread: 2 threads, 2 × T.
- i = 1: each process forks, giving **4 processes**; each creates and joins one thread: 4 threads, 4 × T.
- Threads created **= 2 + 4 = 6** (10 threads if each process's main thread is counted). Each of the 4 processes prints M once after the loop.
- Every worker is joined before the next `fork()`, so each fork happens in a single-threaded process and fork semantics do not matter.

X4-33 fork + pthread_create in a branch

Assume `fork()` duplicates only the calling thread (Linux). How many processes (including the original) are there, and how many threads are created by `pthread_create()`? [4.28]

```
pid_t pid = fork();
if (pid > 0) {
    pthread_create(&t1, NULL, work, NULL);
    fork();
}
pthread_create(&t2, NULL, work, NULL);
```

3 processes and 4 threads created.

- First fork: P0 (parent, pid > 0) and P1 (child, pid == 0).
- Only P0 enters the `if`: it creates thread t1, then forks P2. **P2 gets only a copy of the calling (main) thread**, not t1. P2 continues after that `fork()`, so it also reaches the last line.
- P0, P1 and P2 each run the last `pthread_create()`: 3 more threads.
- Processes = **3**; threads created = 1 + 3 = **4** (7 threads in total with the 3 main threads).

X4-34 thread shares, fork copies global

What is printed? [4.6]

```
int count = 0;
void *inc(void *arg) { count++; return NULL; }
int main() {
    pthread_t t;
    pthread_create(&t, NULL, inc, NULL);
    pthread_join(t, NULL);
    if (fork() == 0) {
        pthread_create(&t, NULL, inc, NULL);
        pthread_join(t, NULL);
        printf("child: %d\n", count);
        return 0;
    }
    wait(NULL);
    printf("parent: %d\n", count);
}
```

`child: 2`, then `parent: 1`.

- The first thread **shares the process's global data**, so its `count++` updates main's `count` to 1.
- `fork()` gives the child **its own copy** of the address space (count = 1); the child's thread increments the child's copy to **2**.
- The parent's copy stays **1**, and `wait(NULL)` makes the parent print last.
- **Key: threads share memory; processes have separate memory.**

X4-35 fork called inside a thread

Assume `fork()` duplicates only the calling thread (Linux). How many processes are there, and how many times are `A` and `B` printed? [4.28]

```
void *work(void *arg) {
    fork();
    printf("A\n");
    return NULL;
}
int main() {
    pthread_t t;
    pthread_create(&t, NULL, work, NULL);
    pthread_join(t, NULL);
    printf("B\n");
}
```

2 processes; A is printed 2 times and B once.

- The worker thread calls `fork()`, so **the child process contains only a copy of the calling (worker) thread**, not main.
- Parent: the worker prints A; main's `pthread_join()` returns and main prints B.
- Child: its only thread prints A and returns from `work`. As the last thread of the child, this ends the child process, and **there is no main thread in the child to print B**.
- (If fork() duplicated all threads, the child would also contain main, and B would be printed twice.)

X4-36 fork + pthread_create loop, formula

Assume `fork()` duplicates only the calling thread. For n = 3, how many processes exist after the loop, and how many threads are created by `pthread_create()`? Give formulas in n. [4.28]

```
for (int i = 0; i < n; i++) {
    fork();
    pthread_create(&tid[i], NULL, work, NULL);
}
```

n = 3: 8 processes and 14 threads created. In general: 2^n processes and $2+4+\cdots+2^n = 2^{n+1}-2$ threads.

- In each iteration, every process's **main thread** forks (2, 4, 8 processes), then each process creates one thread (+2, +4, +8 = 14).
- A child receives only a copy of the calling main thread, **not the worker threads already running in its parent**, so earlier workers are never duplicated.

X4-37 what is a signal, handling steps

What is a signal? Describe how a signal is handled. Can a program change how a signal is handled? [4.29–4.30]

A signal is used in UNIX systems to notify a process that a particular event has occurred. A **signal handler** processes it in this pattern:

1. The signal is **generated** by a particular event (e.g., division by 0, Ctrl-C, a timer expiring, process termination).
2. The signal is **delivered** to a process (or thread).
3. The signal is **handled by one of two signal handlers: default or user-defined.**

- **Every signal has a default handler that the kernel uses**: some signals are ignored, while others (e.g., an illegal memory access) are handled by **terminating the program**.
- **Yes: a user-defined signal handler can override the default handler**, e.g., `signal(SIGINT, handler);` (SIGKILL and SIGSTOP cannot be caught).

X4-38 synchronous vs asynchronous signals

Distinguish synchronous and asynchronous signals, with two examples of each. Why is the first kind called synchronous? [4.29]

- **Synchronous signals** are caused by an operation of the running process itself, e.g., an **illegal memory access** (segmentation fault) or a **division by 0**. They are **delivered to the same process that performed the operation that caused the signal**, which is why they are called **synchronous**.
- **Asynchronous signals** are **generated by an event external to the running process**, e.g., **terminating a process with Ctrl-C**, a **timer expiring**, or another process calling `kill()`.
- **A signal may be received either synchronously or asynchronously, depending on the source and the reason.**

X4-39 signal delivery in multithreaded process

Where should a signal be delivered in a multithreaded process? A process has threads T1–T3; T2 divides by zero, and later the user presses Ctrl-C. Where should each signal go? [4.30–4.31]

The delivery method depends on the type of signal. There are four options:

1. **Deliver the signal to the thread to which the signal applies.**
2. **Deliver the signal to every thread in the process.**
3. **Deliver the signal to certain threads in the process.**
4. **Assign a specific thread to receive all signals for the process.**

- T2's division by zero is **synchronous, so it is delivered to the thread causing the signal (T2) and not to the other threads**.
- **Ctrl-C terminates the process, so this asynchronous signal should be sent to all threads** (T1–T3).
- UNIX **`kill(pid, signal)`** delivers a signal to a **process**; Pthreads **`pthread_kill(tid, signal)`** delivers it to a **specified thread**. Hence "all signals go to all threads" is **false**.

X4-40 asynchronous vs deferred cancellation

What is thread cancellation? Compare asynchronous and deferred cancellation. Which is preferred, and why? [4.32]

Thread cancellation means terminating a thread before it has finished; the thread to be cancelled is the target thread.

- **Asynchronous cancellation terminates the target thread immediately**, possibly **in the middle of a critical update** to shared data, which leaves the data inconsistent; **the OS may not be able to reclaim all resources** (memory, locks, files).
- **Deferred cancellation: the target thread periodically checks whether it should terminate, allowing it an opportunity to terminate itself in an orderly fashion.** It is cancelled only at a safe **cancellation point** (e.g., blocking calls such as `read()` or `close()`, or `pthread_testcancel()`).
- **Deferred is preferred** (and is the **Pthreads default type**) because the thread can finish its update and release its resources first.

X4-41 pthread_cancel only requests cancellation

Thread A calls `pthread_cancel(tid)` and then `pthread_join(tid, NULL)`. Is the target terminated immediately? What if it is in a long computation loop with no function calls, or has cancellation disabled? [4.32–4.33]

No. Invoking `pthread_cancel()` only requests cancellation; the actual cancellation depends on the target thread's state and type.

- **The default type is deferred**, so the request stays pending until the target reaches a **cancellation point**. A pure computation loop has none, so the thread is not cancelled until it calls, e.g., `pthread_testcancel()` or a blocking call that is a cancellation point.
- **If the thread has cancellation disabled, the cancellation remains pending until the thread enables it.**
- **`pthread_join()` makes A wait until the target has actually terminated.** Only a target whose type is set to **asynchronous** can be terminated immediately.

X4-42 thread-local storage

What is thread-local storage (TLS)? How does it differ from local variables and from global data? Give an example. [4.36]

Thread-local storage is data that is unique to each thread: every thread has its own copy.

- vs **global / static data**: globals are **shared by all threads** of the process, while TLS gives each thread a **private copy** that, like static data, persists across function calls.
- vs **local variables**: locals live on the stack and exist only **during a single function invocation**, while TLS is visible **across all function invocations** of that thread.
- Example: each thread's own `errno`, or a **unique transaction ID** when each transaction is serviced by a separate thread. (Windows keeps TLS in the thread's **TEB**.)

X4-43 Windows thread components, ETHREAD KTHREAD TEB

List the components of a Windows thread and its primary data structures. Where does each data structure reside? [4.35–4.36]

Windows uses the one-to-one mapping: each user-level thread maps to an associated kernel thread. A thread has:

- a **thread ID**, a **register set** (processor state), a **program counter**, **separate user and kernel stacks**, and a **private data storage area** used by run-time libraries and DLLs. **The register set, stacks and private storage area are known as the context of the thread.**
- **ETHREAD** (executive thread block): pointer to the **process** the thread belongs to and to its **KTHREAD**; in **kernel space**.
- **KTHREAD** (kernel thread block): **scheduling and synchronization info, kernel-mode stack, pointer to the TEB**; in **kernel space**.
- **TEB** (thread environment block): **thread ID, user-mode stack, thread-local storage**; in **user space**.

X4-44 Linux tasks, task_struct vs Windows

How does Linux represent threads? Contrast Linux with Windows for a process that has two threads. [4.37]

Linux does not differentiate between threads and processes: each is a task. A thread is a process that may share certain resources (e.g., the address space) with other tasks; each thread has its own unique `task_struct` and appears as a normal process.

- **Windows** (explicit kernel support for threads): **one PCB** describes the shared resources (address space, open files) and **points to the two threads' TCBs**, each describing the resources that thread alone possesses.
- **Linux:** simply **two tasks with normal** `task_struct` **structures, set up to share resources**.

X4-45 clone() flags vs fork()

How does Linux create a thread with `clone()`? Explain the flags and how `clone()` differs from `fork()`. [4.38]

Threads are created like normal tasks, except that `clone()` **is passed flags naming the resources the child task shares with its parent:** `clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0);`

- **CLONE_VM**: share the **address space**; **CLONE_FS**: share **file-system information** (e.g., current directory); **CLONE_FILES**: share the **open files (file descriptors)**; **CLONE_SIGHAND**: share the **signal handlers**. With these flags, the child task behaves like a **thread**.
- **Instead of copying all data structures like** `fork()`**, the new task points to the data structures of the parent task, depending on the set of flags.**
- With no sharing flags, it behaves like a separate process: **a normal** `fork()` **can be implemented as** `clone(SIGCHLD, 0)` (SIGCHLD is the signal sent to the parent when the child terminates).

X4-TF True / False quick verdicts

Statement	T/F	Why (≤ 12 words)
Threads of a process share the heap, but each thread has its own stack.	T	Heap and globals shared; stack, PC, registers private.
Each thread has its own set of open file descriptors.	F	Open files are shared by all threads of the process.
The process identifier (PID) is stored in each thread's TCB.	F	PID is in the PCB; TCB points to the PCB.
Each thread has its own state (ready/running/waiting) kept in its TCB.	T	TCBs are organized into queues by state.
Most operating-system kernels are multithreaded.	T	E.g., Linux boot creates kernel threads for devices, memory.
Context switching between threads of one process is cheaper than between processes.	T	Address space stays the same; only registers saved and restored.
Multithreading can shorten run time on one core if the threads do I/O.	T	One thread computes while another waits for I/O.
Parallelism is possible on a single core through time multiplexing.	F	Time multiplexing gives concurrency only; parallelism needs multiple cores.
A multicore chip uses more power than several single-core chips.	F	Uses less power; on-chip communication is also faster.
Two threads summing the two halves of an array is task parallelism.	F	Same operation on different data subsets: data parallelism.
By Amdahl's Law, a 50%-parallel program can never exceed 2× speedup.	T	Limit is 1/S = 1/0.5 = 2.
A 90%-parallel program gets at most about 5.26× speedup on 10 cores.	T	1/(0.1 + 0.9/10) = 1/0.19 ≈ 5.26.
With enough cores, the Amdahl speedup can exceed 1/S.	F	Speedup approaches 1/S as N → ∞, never exceeds it.
The OS schedules user-level threads directly onto CPUs.	F	OS only manages and schedules kernel threads.
A many-to-one process on 8 cores uses at most one core at a time.	T	Only one kernel thread, so only one core at a time.
In the one-to-one model, the number of threads per process may be restricted.	T	Every user thread needs a kernel thread, which consumes resources.
In many-to-many, there must be as many kernel threads as user threads.	F	Smaller or equal number of kernel threads.
The two-level model allows a user thread to be bound to a kernel thread.	T	Many-to-many plus optional binding.
Pthreads is a specification, not an implementation.	T	POSIX standard API; each OS implements it differently.
`pthread_join()` terminates the specified thread.	F	It waits for that thread to terminate.
Returning from `main()` terminates all threads of the process.	T	Returning from main calls exit(), ending the whole process.
If exec() follows fork() immediately, duplicating all threads is unnecessary.	T	exec() replaces the entire process, including all threads.
A segmentation fault (illegal memory access) is an asynchronous signal.	F	Caused by the process's own operation: synchronous.

Statement	T/F	Why (≤ 12 words)
A signal sent by `kill()` from another process is asynchronous.	**T**	Generated by an event external to the running process.
`pthread_kill()` delivers a signal to a specific thread.	**T**	`kill()` targets a process; `pthread_kill()` a thread.
Asynchronous cancellation lets the target thread clean up before it terminates.	**F**	That is deferred; asynchronous terminates immediately.
A cancellation request to a thread with cancellation disabled is discarded.	**F**	It remains pending until the thread enables cancellation.
Thread-local storage is shared by all threads of a process.	**F**	TLS is unique to each thread.
Linux keeps separate PCB and TCB structures for a multithreaded process.	**F**	Each thread is a task with its own task_struct.
In Windows, the TEB resides in kernel space.	**F**	TEB is in user space; ETHREAD and KTHREAD in kernel.

X5 · Ch5 CPU Scheduling

X5-01 CPU–I/O burst cycle and burst distribution

Describe the CPU–I/O burst cycle. What does the histogram of CPU-burst times show, and why does it matter for scheduling?

- **Process execution consists of a cycle of CPU execution and I/O wait — a CPU burst followed by an I/O burst**, alternating until the process ends. While one process waits for I/O, the **CPU scheduler** gives the CPU to another ready process, so some process is running at all times → **maximize CPU utilization** (the goal of multiprogramming).
- Measured CPU bursts follow a **long-tail distribution: many short CPU bursts and a small number of long ones**. An **I/O-bound** program has **many short CPU bursts**; a **CPU-bound** program has **a few long CPU bursts**.
- This distribution is **important for designing the scheduling algorithm**, e.g., choosing the RR quantum (about 80% of bursts shorter than q) and favouring short bursts (SJF, MLFQ) to cut the average waiting time.

X5-02 Distinguish I/O-bound from CPU-bound processes

Why is it important for the scheduler to distinguish I/O-bound processes from CPU-bound processes?

Because they use the CPU very differently: an **I/O-bound process runs only short CPU bursts and seldom uses its full quantum**, while a **CPU-bound process runs long bursts and uses its whole quantum**.

- Giving **I/O-bound processes higher priority** lets them start their next I/O quickly, keeping **I/O devices busy** and giving **short response time** to interactive jobs; the CPU-bound process uses the CPU while they wait → better **CPU and device utilization**.
- Treating them alike (e.g., FCFS) makes I/O-bound jobs wait behind a CPU-bound one (**convoy effect**) and leaves the devices idle.
- MLFQ makes this distinction automatically: **CPU-bound jobs sink to lower queues, I/O-bound jobs stay near the top**.

X5-03 Four scheduling decision points, which preemptive

List the four situations in which CPU-scheduling decisions take place. Which of them are preemptive?

CPU scheduling may take place when a process:

1. **switches from running to waiting** (e.g., I/O request, `wait()`);
2. **terminates**;
3. **switches from running to ready** (e.g., an **interrupt — timer / quantum expiry**);
4. **switches from waiting to ready** (e.g., **I/O completion**), or a **new process arrives** in the ready queue (new → ready) with a higher priority.

Cases 1 and 2 are non-preemptive — the process **gives up the CPU voluntarily** and a new process must be selected. **Cases 3 and 4 are preemptive** — the CPU can be **taken away involuntarily**. A scheduler that acts only in cases 1 and 2 is non-preemptive (cooperative).

X5-04 Preemptive vs non-preemptive scheduling issues

Distinguish preemptive and non-preemptive scheduling. What problems does preemption introduce?

- **Non-preemptive**: once a process gets the CPU it **keeps it until it terminates or blocks** (releases it voluntarily). Simple, low overhead, but a long process can **monopolize the CPU** (poor response time, convoy effect). E.g., FCFS, SJF.
- **Preemptive**: the OS can **take the CPU away** on a **timer interrupt (quantum expiry)** or when a **higher-priority / shorter process becomes ready**. Better **response time**, no monopolization. E.g., RR, SRTF, preemptive priority, MLFQ.
- Problems of preemption: (1) **more context switches (overhead)**; (2) **access to shared data** — a process preempted while updating shared data can leave it **inconsistent (race condition)**, so synchronization is needed; (3) **preemption while in kernel mode** — kernel data structures may be left inconsistent; (4) **interrupts during crucial OS activities** — such code must be protected (e.g., interrupts disabled).

X5-05 Dispatcher and dispatch latency

What is the dispatcher and what does it do? What is dispatch latency, and why should it be small?

- The **dispatcher** is the module that **gives control of the CPU to the process selected by the CPU scheduler**. It involves: (1) **switching context** from one process to another; (2) **switching to user mode**; (3) **jumping to the proper location in the user program** to restart it.
- **Dispatch latency** is **the time it takes for the dispatcher to stop one process and start another running**. It is **pure overhead** paid on **every context switch** (typically hundreds per second, visible with `vmstat` on Linux), so the dispatcher must be **as fast as possible**.
- The **scheduler decides which process** runs (policy); the **dispatcher performs the switch** (mechanism).

X5-06 Scheduling criteria to maximize or minimize

Define the CPU-scheduling criteria and state whether each should be maximized or minimized.

Criterion	Definition	Goal
CPU utilization	fraction of time the CPU is busy	maximize
Throughput	number of processes completed per unit time	maximize
Turnaround time	time to execute a process = completion − arrival (waiting + CPU + I/O time)	minimize
Waiting time	total time spent waiting in the **ready queue** = turnaround − CPU burst	minimize
Response time	time from submission until the **first response** is produced (this course: end of first run − arrival)	minimize
Fairness	adequate CPU allocation to every process	ensure

The scheduling algorithm **does not change CPU-burst or I/O time; it only changes the waiting time in the ready queue**. Usually the **average** is optimized, and criteria can conflict (e.g., throughput favours short processes).

X5-07 Response time definition and computation

Define response time. How is it computed in this course for RR and for non-preemptive algorithms, and why is it important?

- **Response time is the amount of time from when a request is submitted until the first response is produced**; in this course **RT = completion of the first run on the CPU − arrival time**.
- **RR / MLFQ**: the first run ends at the **end of the first quantum** (or at completion if the burst < q). E.g., RR q = 4 with P1 24, P2 3, P3 3 (all at 0): RT = 4, 7, 10 → **avg 7** (FCFS: 24, 27, 30 → avg 27).
- **Non-preemptive (FCFS, SJF, non-preemptive priority)**: the first run is the whole burst, so **RT = turnaround time**.
- It matters for **interactive (time-sharing) systems**, where users need quick feedback; **RR and MLFQ give short response times**, while turnaround time matters more for batch jobs.

X5-08 Conflicting scheduling criteria

Explain how these pairs of criteria conflict: CPU utilization vs response time; average turnaround vs maximum waiting time; I/O device utilization vs CPU utilization.

Scheduling criteria **conflict**, so an algorithm improves some at the expense of others:

- **CPU utilization vs response time**: utilization is highest when **context switches are rare** (long quanta, little overhead), but short response time needs **frequent switching** (short quanta), which adds overhead.
- **Average turnaround vs maximum waiting time**: average turnaround is minimized by running the **shortest jobs first (SJF)**, but long jobs may then wait indefinitely (**starvation**), so the maximum waiting time grows.
- **I/O device utilization vs CPU utilization**: CPU utilization is maximized by letting **CPU-bound jobs run long without switching**; I/O utilization is maximized by dispatching **I/O-bound jobs as soon as they are ready** to start their next I/O, which means more context switches.
- In general, **performance and fairness are often at odds**: optimizing turnaround (SJF) sacrifices fairness; RR is fair but has a longer average turnaround.

X5-09 FCFS and the convoy effect

Describe FCFS scheduling. What is the convoy effect, and why is FCFS unsuitable for interactive systems?

- **FCFS** serves processes **in their order of arrival** to the ready queue (FIFO). It is **non-preemptive**: a process keeps the CPU until it **completes its current CPU burst**. It is simple and **starvation-free**.
- **Convoy effect**: **short processes get stuck behind a long process**, so the **average waiting time is long and depends purely on the arrival order** — bursts 24, 3, 3: order P1, P2, P3 → avg WT (0 + 24 + 27)/3 = **17**; order P2, P3, P1 → (6 + 0 + 3)/3 = **3**. With **one CPU-bound and many I/O-bound** processes, the I/O-bound ones wait behind the CPU-bound one → **low I/O device utilization** (and later an idle CPU).
- **Unsuitable for interactive systems**: without preemption, a process cannot be guaranteed **a share of the CPU at regular intervals**, so response time is poor.

X5-10 Round-robin mechanism and waiting bound

Explain how Round-Robin scheduling works. What does it guarantee with n processes and time quantum q?

- RR is **FCFS with preemption**: each process gets a small unit of CPU time, the **time quantum q (usually 10–100 ms)**. When the quantum expires, a **timer interrupt** preempts the process and it is **added to the tail of the ready queue**; if its (remaining) burst is **shorter than q**, it releases the CPU itself and the next process runs at once.
- With n ready processes, each gets **1/n of the CPU time in chunks of at most q**, and **no process waits more than (n − 1)q time units** for its next turn (context-switch time ignored).
- RR is **fair** (FIFO order, equal shares), gives **good response time** for interactive jobs and has **no starvation**; its average waiting time can be long.
- Conventions: a **new arrival at the same instant as a quantum expiry enters the ready queue first**; if the queue is empty when the quantum expires, the process simply continues.

X5-11 Choosing q from the waiting bound

Ten interactive processes share a CPU under RR, and each must wait at most 90 ms in the ready queue before running again. Find the largest quantum (a) ignoring context switches, (b) if every context switch costs 1 ms.

(a) RR bound: **no process waits more than (n − 1)q** → $(10-1)q \le 90$ → **q ≤ 10 ms**. (b) Between two turns of a process, the other n − 1 processes each run one quantum and **n context switches** occur → worst wait = $(n-1)q + ns$: $9q + 10 \times 1 \le 90$ → $q \le 80/9$ → **q ≤ 8.9 ms**.

- Trade-off: a smaller q gives a tighter bound (better response) but more switching overhead (efficiency q/(q + s)).

X5-12 Time quantum size effects, 80% rule

How does the size of the time quantum affect RR performance? State the rule of thumb for choosing it.

- **q too large** (larger than every CPU burst) → each burst finishes in its first quantum → **RR degenerates to FCFS**.
- **q too small** → too many **context switches**; **q must be large with respect to the context-switch time (usually < 10 μs), otherwise the overhead is too high**. In the limit (tiny q, zero switch cost) RR becomes **processor sharing**: each of n processes appears to have **its own processor running at 1/n the speed**.
- **Average turnaround time does not necessarily improve as q increases**; it improves when **most processes finish their CPU burst within a single quantum**. E.g., bursts 6, 3, 1, 7 (all at 0): avg TAT = 11.0 (q = 1), 10.75 (q = 3), 12.25 (q = 5), 10.5 (q ≥ 6).
- **Rule of thumb: about 80% of CPU bursts should be shorter than the time quantum q.**

X5-13 RR CPU efficiency with context-switch overhead

(a) RR with context-switch cost s = 0.1 ms: CPU efficiency for q = 10, 1 and 0.1 ms when all processes are CPU-bound? (b) Ten I/O-bound tasks (each computes 1 ms, then does a 10 ms I/O) and one CPU-bound task, s = 0.1 ms: CPU utilization for q = 1 ms and q = 10 ms?

(a) Each quantum of useful work q is followed by one switch s → **efficiency = q/(q + s)**:

- q = 10 ms: 10/10.1 = **99.0%**; q = 1 ms: 1/1.1 = **90.9%**; q = 0.1 ms: 0.1/0.2 = **50%** → **a quantum close to the switch time wastes the CPU**. (A process that blocks after a burst b < q gives b/(b + s).)

(b) Each 10 ms I/O finishes before the task's next turn, so the CPU always has a ready task.

- **q = 1 ms**: every task, I/O-bound or CPU-bound, runs 1 ms and then a 0.1 ms switch follows → U = 1/1.1 = **90.9%**.
- **q = 10 ms**: one round = 10 × (1 + 0.1) for the I/O-bound tasks + (10 + 0.1) for the CPU-bound task = 21.1 ms, containing 20 ms of useful work → U = 20/21.1 = **94.8%**.
- A larger quantum means fewer switches per unit of work → higher utilization, at the cost of response time.

X5-14 Utilization and throughput with switch cost

P1 (10 ms), P2 (4 ms), P3 (6 ms) arrive at time 0; RR with q = 4 ms; each context switch takes 1 ms. Draw the Gantt chart with the switches; find CPU utilization, throughput and average turnaround time, and compare with FCFS.

Assume no switch before the first process and a 1 ms switch (CS) whenever the CPU moves to a different process.

- RR: `0 P1 4 CS 5 P2 9 CS 10 P3 14 CS 15 P1 19 CS 20 P3 22 CS 23 P1 25` → 5 switches; 20 ms of useful work in 25 ms.
- **CPU utilization** = 20/25 = **80%**; **throughput** = 3 processes / 25 ms = **0.12 per ms (120 per second)**; completions P1 25, P2 9, P3 22 → **avg TAT** = (25 + 9 + 22)/3 = 56/3 = **18.67 ms**.
- FCFS: `0 P1 10 CS 11 P2 15 CS 16 P3 22` → 2 switches; U = 20/22 = **90.9%**; avg TAT = (10 + 15 + 22)/3 = 47/3 = **15.67 ms**.
- RR's extra context switches **lower CPU utilization** — the price paid for better response time.

X5-15 RR versus FCFS with equal jobs

Is RR always better than FCFS? Four jobs, each needing 6 ms, arrive at time 0. Compare FCFS and RR (q = 2) in average turnaround, waiting and response time.

No.

- FCFS: `0 P1 6 P2 12 P3 18 P4 24` → TAT 6, 12, 18, 24 → avg **15**; WT 0, 6, 12, 18 → avg **9**; RT = TAT → avg **15**.
- RR (q = 2): `0 P1 2 P2 4 P3 6 P4 8 P1 10 P2 12 P3 14 P4 16 P1 18 P2 20 P3 22 P4 24` → TAT 18, 20, 22, 24 → avg **21**; WT 12, 14, 16, 18 → avg **15**; RT 2, 4, 6, 8 → avg **5**.
- **When all jobs have the same length, longer than q, RR has worse turnaround and waiting time** — the jobs are interleaved and all finish near the end, while FCFS finishes them one by one — **but RR gives much better response time**. (Slide 5.15 case: 10 jobs × 100 s, q = 1 s → avg TAT 995.5 s under RR vs 550 s under FCFS.)

X5-16 SJF optimality and proof sketch

Describe SJF scheduling and explain why it is optimal.

- **SJF (shortest-next-CPU-burst)** associates each process with the **length of its next CPU burst** and runs the process with the **shortest next burst** (ties → FCFS). The idea: **get short jobs out of the system sooner** — a big gain for short jobs, a small cost for long ones.
- **SJF is optimal: it gives the minimum average waiting time for a given set of processes.**
- **Proof sketch**: **moving a short process before a long one decreases the waiting time of the short process more than it increases the waiting time of the long process**. Swapping an adjacent pair (long L before short S) changes the total waiting time by S − L < 0, so repeated swaps towards shortest-first can only reduce the average.
- E.g., bursts 6, 8, 7, 3 (all at 0): SJF `0 P4 3 P1 9 P3 16 P2 24` → avg WT (3 + 16 + 9 + 0)/4 = **7**, versus (0 + 6 + 14 + 21)/4 = 10.25 in FCFS order.

X5-17 SJF implementation difficulty and approximation

Why is SJF difficult to implement in practice? How can it be approximated, and what other drawbacks does it have?

- **The difficulty is knowing the length of the next CPU request** — the scheduler cannot see the future, so SJF/SRTF are hard to implement and are mainly used as a **benchmark** for practical algorithms.
- **Approximation**: **predict the next CPU burst from the lengths of previous bursts using exponential averaging**, $\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$, then pick the process with the **shortest predicted** next burst (a wrong prediction gives a sub-optimal schedule).
- SJF can be applied to an **entire program** (e.g., a run-time estimate supplied with a batch job) or to the **current CPU burst**.
- Other drawbacks: **starvation of long processes** if shorter ones keep arriving; **fairness cannot be enforced**.

X5-18 Exponential averaging formula and alpha

Explain the exponential averaging formula used to predict the next CPU burst. What happens when α = 0 and when α = 1?

- $\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$, $0 \le \alpha \le 1$: t_n **= actual length of the n-th CPU burst** (most recent information), τ_n **= the previous prediction** (past history), **α = the relative weight of recent vs past history** (commonly **α = 1/2**).
- **α = 0** → $\tau_{n+1} = \tau_n$: **recent history does not count**. **α = 1** → $\tau_{n+1} = t_n$: **only the actual last CPU burst counts**.
- Expanded: $\tau_{n+1} = \alpha t_n + (1-\alpha)\alpha t_{n-1} + \cdots + (1-\alpha)^j \alpha t_{n-j} + \cdots + (1-\alpha)^{n+1}\tau_0$ → the prediction depends on **all past bursts (with different weights) and the initial estimate** τ_0; **recent bursts weigh more**, and the effect of older ones **diminishes exponentially** (one more factor 1 − α per step). With α = 1/2, the weight of τ_0 in τ_{10} is $(1/2)^{10} = 1/1024$ — negligible.

X5-19 Exponential averaging calculation, alpha 0.5

With α = 0.5 and τ0 = 12 ms, the next four actual CPU bursts are 8, 6, 10 and 4 ms. Predict τ1 to τ4.

$\tau_{n+1} = 0.5\, t_n + 0.5\, \tau_n$:

- τ1 = 0.5(8) + 0.5(12) = 4 + 6 = **10 ms**
- τ2 = 0.5(6) + 0.5(10) = 3 + 5 = **8 ms**
- τ3 = 0.5(10) + 0.5(8) = 5 + 4 = **9 ms**
- τ4 = 0.5(4) + 0.5(9) = 2 + 4.5 = **6.5 ms**

Check with the expanded form (weights 1/2, 1/4, 1/8, 1/16 on bursts 4, 10, 6, 8 and 1/16 on τ0 = 12): τ4 = 2 + 2.5 + 0.75 + 0.5 + 0.75 = **6.5 ms**.

X5-20 Exponential averaging: alpha 0.25, solve alpha

(a) α = 0.25, τ0 = 16 ms, actual bursts 8, 2, 15 ms: find τ1, τ2, τ3. (b) A prediction of 10 ms is followed by an actual burst of 4 ms and the new prediction is 8.5 ms: find α. (c) Using the data of (a), what is τ3 when α = 0 and when α = 1?

(a) $\tau_{n+1} = 0.25\, t_n + 0.75\, \tau_n$: τ1 = 2 + 12 = **14 ms**; τ2 = 0.5 + 10.5 = **11 ms**; τ3 = 3.75 + 8.25 = **12 ms**. A **small α reacts slowly** — the 15 ms burst raises the guess only from 11 to 12. (b) $8.5 = 4\alpha + 10(1-\alpha) = 10 - 6\alpha$ → $\alpha = 1.5/6 =$ **0.25**. (c) **α = 0**: history is ignored → τ3 = τ0 = **16 ms**. **α = 1**: only the last burst counts → τ3 = t2 = **15 ms**.

X5-21 SRTF definition and comparison with SJF

Describe SRTF scheduling. When must the scheduler be invoked, and how does SRTF compare with SJF?

- **SRTF (shortest-remaining-time-first) is preemptive SJF**: when a process joins the ready queue (new → ready, or waiting → ready after I/O) with a **CPU burst shorter than the remaining time of the running process**, it **preempts** the running process (equal remaining time → no preemption).
- Hence the scheduler must run **at every arrival to the ready queue (scheduling condition 4)**, not only at completions.
- **SRTF is optimal among preemptive algorithms** for average waiting time and is **always at least as good as SJF**; if all processes arrive at the same time, SRTF = SJF; with equal bursts both behave like **FCFS**.
- Drawbacks: needs the (predicted) burst length, causes **more context switches**, and can **starve long processes** if short ones keep arriving.

X5-22 FCFS, SJF, SRTF, RR averages calculation

P1 (arrival 0, burst 6), P2 (2, 5), P3 (3, 2), P4 (5, 4). Draw Gantt charts for FCFS, non-preemptive SJF, SRTF and RR (q = 2); find the average waiting, turnaround and response times.

Algorithm	Gantt chart	WT P1–P4 → avg	TAT P1–P4 → avg	RT P1–P4 → avg
FCFS	`0 P1 6 P2 11 P3 13 P4 17`	0, 4, 8, 8 → **5**	6, 9, 10, 12 → **9.25**	= TAT → **9.25**
SJF	`0 P1 6 P3 8 P4 12 P2 17`	0, 10, 3, 3 → **4**	6, 15, 5, 7 → **8.25**	= TAT → **8.25**
SRTF	`0 P1 3 P3 5 P1 8 P4 12 P2 17`	2, 10, 0, 3 → **3.75**	8, 15, 2, 7 → **8**	3, 15, 2, 7 → **6.75**
RR q = 2	`0 P1 2 P2 4 P1 6 P3 8 P2 10 P4 12 P1 14 P2 15 P4 17`	8, 8, 3, 8 → **6.75**	14, 13, 5, 12 → **11**	2, 2, 5, 7 → **4**

- SJF: only P1 is ready at 0; at t = 6 pick P3 (2), then P4 (4), then P2 (5).
- SRTF: at t = 2, P2 (5) does not preempt P1 (4 left); at t = 3, P3 (2) preempts P1 (3 left); at t = 5, P1 (3) < P4 (4) < P2 (5).
- RR: at t = 2, P2 arrives as P1's quantum expires → queue [P2, P1] (new arrival first); then t = 4 [P1, P3, P2]; t = 6 [P3, P2, P4, P1].
- **SRTF gives the minimum average waiting time; RR gives the best response time but the worst waiting and turnaround times.**

X5-23 Priority scheduling and its relation to SJF

Describe priority scheduling (preemptive and non-preemptive). How is SJF related to it?

- Each process has a **priority number**; the CPU is allocated to the **process with the highest priority** (**smallest integer = highest priority** unless stated otherwise); **equal-priority processes are scheduled in FCFS order** (or RR).
- **Preemptive**: a newly arrived process with a **higher priority preempts** the running process. **Non-preemptive**: a higher-priority arrival is put at the head of the ready queue and waits until the running process finishes its burst.
- **SJF is a special case of priority scheduling where the priority is the inverse of the predicted next CPU burst** (shorter burst = higher priority); SRTF is its preemptive form.
- Main problem: **starvation (indefinite blocking)** of low-priority processes; solution: **aging**.

X5-24 Starvation and aging

What is starvation? Explain how aging prevents it, with a numerical example.

- **Starvation (indefinite blocking)**: a process is **ready to run but waits indefinitely for the CPU**, because there are **always higher-priority processes** (or, in SJF/SRTF, shorter jobs) ahead of it.
- **Aging**: **gradually increase the priority of a process the longer it waits**, so it eventually becomes high enough to run (implemented with a **timer for each process in the ready queue**).
- Example: priorities range from 127 (lowest) to 0 (highest); raise a waiting process's priority by 1 every second → even a priority-127 process reaches priority 0 within **127 s (about 2 minutes)**, so its waiting time is bounded.
- In MLFQ, aging is done by **moving long-waiting jobs to a higher queue** (e.g., periodically moving all jobs to the top queue).

X5-25 Preemptive vs non-preemptive priority calculation

P1 (arrival 0, burst 5, priority 3), P2 (1, 3, 1), P3 (2, 4, 4), P4 (3, 2, 2); smaller number = higher priority. Draw Gantt charts for preemptive and non-preemptive priority scheduling; find the average waiting and turnaround times.

Version	Gantt chart	WT P1–P4 → avg	TAT P1–P4 → avg	RT P1–P4 → avg
Preemptive	`0 P1 1 P2 4 P4 6 P1 10 P3 14`	5, 0, 8, 1 → 14/4 = **3.5**	10, 3, 12, 3 → 28/4 = **7**	1, 3, 12, 3 → 19/4 = **4.75**
Non-preemptive	`0 P1 5 P2 8 P4 10 P3 14`	0, 4, 8, 5 → 17/4 = **4.25**	5, 7, 12, 7 → 31/4 = **7.75**	= TAT → **7.75**

- Preemptive: at t = 1, P2 (priority 1) preempts P1 (3); at t = 3, P4 (2) cannot preempt P2 (1); at t = 4 the order is P4 (2), then P1 (3), then P3 (4).
- Non-preemptive: P1 runs to completion at 5; then by priority P2, P4, P3.

X5-26 Priority with RR, remaining quantum rule

Preemptive priority scheduling with RR (q = 2) among equal priorities, smaller number = higher priority: P1 (arrival 0, burst 5, priority 2), P2 (0, 4, 2), P3 (3, 2, 1), P4 (4, 3, 2). Draw the Gantt chart; find the average waiting time.

Rule (Lab 5, S23 official answer): a process **preempted by a higher-priority arrival returns to the head of its priority level and keeps its remaining quantum**; a process whose quantum expires goes to the tail.

- Gantt: `0 P1 2 P2 3 P3 5 P2 6 P1 8 P4 10 P2 12 P1 13 P4 14`
- t = 2: P1's quantum expires → [P2, P1]. t = 3: **P3 (priority 1) preempts P2 after 1 unit** → P2 goes back to the **head with 1 unit of quantum left**. t = 4: P4 joins → [P2, P1, P4]. t = 5: **P2 runs only 5–6**, then P1 6–8, P4 8–10, P2 10–12 (done), P1 12–13 (done), P4 13–14 (done).
- WT = completion − arrival − burst: P1 13 − 0 − 5 = 8; P2 12 − 0 − 4 = 8; P3 5 − 3 − 2 = 0; P4 14 − 4 − 3 = 7 → **avg WT = 23/4 = 5.75** (avg TAT = 37/4 = 9.25).
- Common mistake: giving P2 a fresh quantum at t = 5 (running 5–7) — that produces a wrong chart.

X5-27 Dynamic priority alpha beta gives FCFS or LIFO

A preemptive scheduler uses dynamic priorities (larger number = higher): a waiting process's priority changes at rate α, a running process's at rate β, and every process enters the ready queue with priority 0. Which algorithm results from (a) $\beta > \alpha > 0$, (b) $\alpha < \beta < 0$?

(a) **$\beta > \alpha > 0$ → FCFS.** Every priority rises from 0, so a new process (priority 0) is **lower than every process already in the system** and never preempts; the running process rises fastest ($\beta > \alpha$), so it is **never preempted**; among the waiting processes, the one that **has waited longest has the highest priority** → processes run in **arrival order**. (b) **$\alpha < \beta < 0$ → LIFO (last-in, first-out).** Every priority falls from 0, so a **new process (priority 0) is higher than all existing ones** and **preempts the running process immediately** → the most recently arrived process always runs first.

X5-28 Least-recent-CPU policy favours I/O-bound

A scheduler favours the processes that have used the least processor time in the recent past. Why does it favour I/O-bound processes, yet not permanently starve CPU-bound processes?

- It favours **I/O-bound processes** because they run only **short CPU bursts** before blocking for I/O, so their recent CPU usage is small and they get **high priority**.
- **CPU-bound processes do not starve permanently** because the I/O-bound processes **give up the CPU frequently** to do their I/O, and the CPU-bound process runs in those gaps. Also, usage is measured only over the **recent past**: while a CPU-bound process waits, its recent usage falls and its priority rises (an aging effect).

X5-29 Multilevel queue scheduling

Describe multilevel queue scheduling. How is the CPU shared among the queues?

- The ready queue is **partitioned into separate queues based on process type** (e.g., real-time, system, interactive, batch). The **priority is assigned statically**: a process **remains in the same queue for its whole runtime**.
- **Each queue can have its own scheduling algorithm** (e.g., RR for interactive/foreground, FCFS for batch/background).
- **Scheduling among the queues**: (1) **fixed-priority preemptive scheduling** — a lower queue runs only when all higher queues are empty, and an arrival in a higher queue preempts it → **low queues can starve**; or (2) **time slicing** — each queue gets a **certain share of CPU time** (e.g., 60%, 20%, 10%, 10%).
- Priority scheduling with RR is easily implemented as a multilevel queue (one RR queue per priority level). Low scheduling overhead, but **inflexible**.

X5-30 Multilevel queue versus MLFQ

Compare multilevel queue scheduling with multilevel feedback queue (MLFQ) scheduling.

Aspect	Multilevel queue	MLFQ
Queue membership	**permanent** (by process type)	process **can move between queues**
Priority	**static**	**dynamic**, based on observed CPU usage
Movement rules	none	**demote** after using a full quantum; **upgrade** by aging
Starvation	possible for low queues (fixed priority)	possible, but handled by **aging / periodic reshuffling**
Needs burst knowledge	no (classified by type)	no (learns from behaviour)
Flexibility / cost	inflexible, low overhead	flexible, but the **most complex** (many parameters)

MLFQ therefore **separates processes by their CPU-burst behaviour**: CPU-bound jobs drop to low queues, while I/O-bound and interactive jobs stay high.

X5-31 MLFQ defining parameters

What parameters define a multilevel feedback queue scheduler?

An MLFQ scheduler is defined by:

1. **the number of queues**;
2. **the scheduling algorithm for each queue** (e.g., RR with its own quantum per queue, FCFS in the last queue);
3. **the method used to determine when to upgrade a process** to a higher-priority queue (e.g., aging);
4. **the method used to determine when to demote a process** to a lower-priority queue (e.g., after it uses a full quantum);
5. **the method used to determine which queue a process will enter when it needs service** (e.g., every new job enters Q0).

Because a process can move between queues, **aging can be implemented this way**, which gives MLFQ its **flexibility**.

X5-32 MLFQ rules: demotion, preemption, head of queue

MLFQ with Q0 (RR, q = 8 ms), Q1 (RR, q = 16 ms), Q2 (FCFS). (a) Where does a new job go, and when is it demoted? (b) A Q1 job that has used 5 ms of its quantum is preempted by a new arrival: where does it go, with how much quantum? (c) Can a Q1 job preempt a Q2 job? (d) Which jobs never reach Q2?

(a) A new job **enters the tail of Q0** (RR, FCFS order) and **preempts a running Q1 or Q2 job**. A job that **uses its full quantum without finishing is demoted to the tail of the next queue** (Q0 → Q1 → Q2). A running Q0 job is never preempted by an arrival (the arrival waits in Q0). (b) It returns to the **head of Q1** and later resumes with its **remaining quantum, 16 − 5 = 11 ms**; it is **not demoted** and does **not get a fresh quantum** (a job never receives more than 16 ms in Q1). A preempted Q2 job returns to the head of Q2 with its remaining burst. (c) **No.** No job arrives directly at Q1; a job enters Q1 only by using a full Q0 quantum, i.e., while a Q0 job is running. A Q2 job runs only when Q0 and Q1 are empty, and only a new arrival to Q0 can preempt it. (d) Jobs with a CPU burst **≤ 8 + 16 = 24 ms** finish within Q0 and Q1. This is how MLFQ **approximates SRTF: CPU-bound jobs drop like a rock; short-running I/O-bound jobs stay near the top**.

X5-33 MLFQ disadvantages and remedies

What are the disadvantages of MLFQ, and how can they be handled?

- **Starvation**: with fixed priority among queues, long CPU-bound jobs in the lowest queue may **never run if new or short jobs keep arriving** → handled by **aging / reshuffling jobs among the queues periodically**, e.g., **after some period, move all jobs to the top queue**.
- **Complexity**: it is the **most general but most complex** scheduler — the number of queues, each queue's quantum and algorithm, and the upgrade/demote/entry rules must all be **chosen and tuned** for the workload.
- **Changing behaviour**: a job demoted during a CPU-bound phase stays at low priority even if it later becomes interactive, unless an **upgrade (aging) rule** moves it back up.
- (Its advantages: no prior knowledge of burst times, good response time for interactive jobs, SJF/SRTF-like performance, and fair progress for CPU-bound jobs.)

X5-34 MLFQ Gantt calculation, remaining quantum

MLFQ: Q0 RR (q = 4), Q1 RR (q = 8), Q2 FCFS; new jobs enter Q0 and preempt Q1/Q2; a preempted job returns to the head of its queue with its remaining quantum. P1 (arrival 0, burst 14), P2 (3, 5), P3 (9, 3), P4 (15, 2). Draw the Gantt chart; find the average waiting, turnaround and response times.

- Gantt: `0 P1 4 P2 8 P1 9 P3 12 P1 15 P4 17 P1 21 P2 22 P1 24`
- Queues: P1@Q0 0–4, P2@Q0 4–8, P1@Q1 8–9, P3@Q0 9–12, P1@Q1 12–15, P4@Q0 15–17, P1@Q1 17–21, P2@Q1 21–22, P1@Q2 22–24.
- t = 4: P1 used its Q0 quantum → Q1. t = 8: P2 → Q1 behind P1; P1 runs from Q1. t = 9: P3 preempts P1 (1 of 8 used) → P1 to the **head of Q1 with 7 left**. t = 15: P4 preempts P1 (**4 left**). t = 17–21: P1 uses its last 4 units (1 + 3 + 4 = 8) with 2 ms of burst left → **demoted to Q2**; P2 finishes in Q1 at 22; P1 finishes in Q2 at 24.

P	Arrival	Burst	Completion	TAT	WT	RT
P1	0	14	24	24	10	4
P2	3	5	22	19	14	5

P	Arrival	Burst	Completion	TAT	WT	RT
P3	9	3	12	3	0	3
P4	15	2	17	2	0	2

- **Avg WT = 24/4 = 6; avg TAT = 48/4 = 12; avg RT = 14/4 = 3.5.**
- Resetting P1's quantum after each preemption (the common mistake) would wrongly let P1 finish in Q1.

X5-35 Algorithm property comparison table

Compare FCFS, SJF, SRTF, RR, priority, multilevel queue and MLFQ: preemptive? need the next-burst length? can starve? main property.

Algorithm	Preemptive?	Needs next-burst length?	Starvation?	Key property
FCFS	No (must be non-preemptive)	No	No	simplest; convoy effect
SJF	No	Yes (predicted)	Yes (long jobs)	minimum avg WT among non-preemptive
SRTF	Yes (on arrival)	Yes (predicted)	Yes (long jobs)	minimum avg WT (preemptive)
RR	Yes (timer)	No	No	fair, good response; depends on q
Priority	Either	No	Yes → aging	SJF is a special case
Multilevel queue	Yes (fixed priority among queues)	No	Yes (low queues)	permanent queues, low overhead
MLFQ	Yes	No	Possible → aging / reshuffle	approximates SJF/SRTF, good response

X5-36 Which algorithm: quick judgement answers

Which scheduling algorithm (a) minimizes average waiting time, (b) suits interactive systems best, (c) is free from starvation, (d) is hard to implement, (e) must be non-preemptive, (f) is used in BSD Unix, Solaris and Windows?

Question	Answer	Reason
(a) Minimum average waiting (and turnaround) time	**SJF** (non-preemptive), **SRTF** (preemptive)	provably optimal: short jobs first
(b) Best for interactive systems / response time	**RR** or **MLFQ**	short quanta; MLFQ keeps interactive jobs in the top queues
(c) Free from starvation	**FCFS and RR**	every process reaches the head of the queue
(d) Difficult to implement	**SJF / SRTF**	the next CPU-burst length is unknown
(e) Must be non-preemptive	**FCFS**	each burst runs to completion
(f) Used in BSD Unix, Solaris, Windows	**MLFQ**	no prior burst knowledge; adapts to behaviour
Periodic real-time, static priority	**RM**	shorter period = higher priority
Real-time up to 100% utilization	**EDF**	dynamic, deadline-based priority

X5-37 Equivalences between scheduling algorithms

State the relation between: (a) RR and FCFS; (b) SJF and priority; (c) priority and FCFS; (d) MLFQ and FCFS; (e) SJF/SRTF and FCFS; (f) RR and SJF.

- (a) **RR with a quantum larger than every CPU burst = FCFS**; RR with a very small quantum ≈ **processor sharing**.
- (b) **SJF = priority scheduling with priority = inverse of the predicted next CPU burst**; SRTF = preemptive priority on the remaining time.
- (c) **FCFS = priority scheduling with priority = arrival time** (the job that has been in the system longest has the highest priority); priority scheduling with all-equal priorities is FCFS.
- (d) **The lowest queue of an MLFQ is FCFS**; MLFQ as a whole **approximates SRTF/SJF**.
- (e) **SJF/SRTF = FCFS when all CPU bursts are equal**; SRTF = SJF when all processes arrive at the same time.
- (f) **RR and SJF: no direct relation.**
- Also: a multilevel queue with one RR queue per priority level = priority scheduling with RR.

X5-38 How FCFS, RR, MLFQ treat short jobs

Explain how much FCFS, RR and MLFQ discriminate in favour of short processes.

- **FCFS — discriminates against short jobs**: a short job that arrives after long jobs must wait for all of them (**convoy effect**).
- **RR — treats all jobs equally**: each job gets the same quantum in turn, so a **short job finishes after only a few quanta** and leaves the system sooner, but it gets no extra priority.
- **MLFQ — strongly favours short jobs**: short (interactive, I/O-bound) jobs **finish in the high-priority top queues**, while long CPU-bound jobs are **demoted to lower queues** — similar to **SJF/SRTF** without knowing the burst lengths.

X5-39 PCS versus SCS and LWP

Distinguish process-contention scope (PCS) from system-contention scope (SCS). What is an LWP?

On systems with both thread levels, **the OS schedules kernel-level threads**; user-level threads are managed by the **thread library** (the OS is unaware of them).

	PCS	SCS
Who schedules	**thread library**	**kernel** (CPU scheduler)
What	user-level threads onto an available **LWP** (kernel thread)	**kernel threads onto a CPU**
Competition among	**threads of the same process**	**all threads in the system**
Models	many-to-one, many-to-many	all; **one-to-one systems (Windows, Linux, Solaris) use only SCS**
Policy	usually **priority set by the programmer** (the library does not adjust it); preempts for a higher-priority user thread	the OS CPU-scheduling algorithm

An **LWP (lightweight process)** is an **intermediate data structure between user and kernel threads**; it **appears as a virtual processor** on which the thread library schedules user threads to run, and each LWP is **attached to a kernel thread**.

X5-40 AMP versus SMP

Compare asymmetric multiprocessing (AMP) and symmetric multiprocessing (SMP) for CPU scheduling.

	AMP	SMP
Scheduling	**all scheduling decisions, I/O processing and system activities are handled by one processor — the master server**	**each processor is self-scheduling**
Kernel data	**only the master accesses kernel data structures** → no data sharing needed	all processors access shared kernel data → must **synchronize**
Other processors	execute **only user code**	run both kernel and user code
Ready queue	kept by the master	**one common ready queue** or a **private queue per processor**
Drawback	**the master can become a bottleneck**	more complex: two processors must not pick the same thread, and no process may be lost
Use	simple designs	**all modern OSes** (Windows, Linux, macOS, Android, iOS)

X5-41 Common versus per-processor ready queues

In SMP, what race condition can a single common ready queue cause? Compare common and per-processor ready queues in load balancing, processor affinity and locking.

With a common queue, **two processors may choose to schedule the same thread** (or a thread may be lost) unless the queue is **locked — a race condition**.

	Common ready queue	Per-processor (private) ready queues
Load balancing	**automatic** — no CPU is idle while the queue is non-empty	needs **push / pull migration**
Processor affinity	hard to keep — a thread may run on a different CPU each time (cold cache)	**natural — affinity "for free"** (warm cache)
Locking	the shared queue must be locked → contention	little sharing, no global-queue contention

X5-42 Memory stall and chip multithreading

What is a memory stall, and how do multithreaded multicore processors (CMT) exploit it? How many logical CPUs does the OS see on a quad-core processor with 2 hardware threads per core?

- Multicore = **multiple processor cores on the same chip**: **faster and consumes less power**, but **complicates scheduling**.
- **Memory stall**: when a processor accesses memory it **spends a significant amount of time waiting for the data**, because modern processors are **much faster than memory**, especially on a **cache miss**.
- **Chip multithreading (CMT; Intel: hyperthreading)** gives each core **multiple hardware threads**, each with its own **architectural state (PC, register set)**, so each appears to the OS as a **logical CPU**. **If one hardware thread stalls on memory, the core switches to another hardware thread**, so useful work continues during the stall.
- **Logical CPUs = cores × hardware threads per core**: 4 × 2 = **8** (dual-core dual-threaded = 4; UltraSPARC T3: 16 × 8 = 128).

X5-43 Two levels of multicore thread scheduling

Describe the two levels of scheduling on a multithreaded multicore processor, and the full mapping from user threads to cores.

1. **OS level**: the operating system decides **which software (kernel) thread runs on each logical CPU (hardware thread)** — this is the **CPU scheduling** of this chapter, using any algorithm.
2. **Core level**: each **core decides which of its hardware threads runs on the physical core**, e.g., simple **round-robin** (UltraSPARC T3).

- Full mapping: **user thread → LWP / kernel thread** (thread library, **PCS**) → **kernel (software) thread → hardware thread / logical CPU** (OS, **SCS**) → **hardware thread → physical core** (core hardware, typically RR).

X5-44 Processor affinity: soft, hard, NUMA

What is processor affinity? Distinguish soft and hard affinity, and explain how NUMA affects it.

- **Processor affinity**: a thread has **affinity for the processor it runs on**, because that processor's **cache holds the thread's recently accessed memory**. Migration means **invalidating and repopulating caches** (high cost), so most SMP systems **try to avoid migrating** processes; **per-processor ready queues provide affinity for free**.
- **Soft affinity**: the OS **attempts to keep a process on the same processor but does not guarantee it** — the process may **migrate during load balancing**.
- **Hard affinity**: a process **specifies the subset of processors on which it may run** (e.g., Linux `sched_setaffinity()`); many systems, such as Linux, provide both.
- **NUMA**: a CPU accesses **its local memory faster** than remote memory, so a thread should run on a CPU near its memory → **affinity to both the processor and the memory**.

X5-45 Load balancing: push, pull, affinity conflict

Explain push and pull migration. Why does load balancing conflict with processor affinity?

- **Load balancing keeps the workload evenly distributed** across processors; it is needed when **each processor has its own private ready queue** (a common queue balances itself).
- **Push migration**: a **specific task periodically checks the load on each processor** and, if it finds an imbalance, **pushes tasks from overloaded CPUs to idle or less-busy CPUs**.
- **Pull migration**: an **idle processor pulls a waiting task from a busy processor**.
- They are **not mutually exclusive** — often **both are implemented** (e.g., Linux CFS).
- **Conflict**: a migrated thread **loses its cache contents** (and on NUMA its local memory), so **load balancing often counteracts the benefits of processor affinity** — a natural tension between balancing the load and minimizing memory access time (e.g., migrate only when the imbalance exceeds a threshold).

X5-46 Hard versus soft real-time

Distinguish hard and soft real-time systems. What must a real-time CPU scheduler support?

- **Hard real-time**: stricter — a **task must be serviced by its deadline; service after the deadline has expired is the same as no service at all** (e.g., airbag or flight control).
- **Soft real-time**: **no guarantee as to when a critical real-time process will be scheduled**; it **guarantees only that the real-time process will be given preference over non-critical processes** (e.g., video streaming).
- Real-time scheduling demands **performance guarantees — predictability**. The scheduler **must support a priority-based algorithm with preemption**, so a ready high-priority real-time task gets the CPU at once; but **a preemptive priority-based scheduler alone guarantees only soft real-time**. Hard real-time also needs deadline guarantees (**admission control**, RM/EDF analysis).

X5-47 Event, interrupt and dispatch latency

Define event latency, interrupt latency and dispatch latency in a real-time system. (Textbook topic; the slides define only dispatch latency, slide 5.7.)

- **Event latency**: the time from **when an event occurs to when it is serviced**. Two latencies affect real-time performance:
- **Interrupt latency**: the time from **the arrival of an interrupt at the CPU to the start of the routine that services it** — the CPU finishes the current instruction, determines the interrupt type and **saves the state of the current process**. For hard real-time it must be **bounded** (interrupts may be disabled only briefly).
- **Dispatch latency**: the time for the **dispatcher to stop one process and start another**. **Preemptive kernels** keep it low; its **conflict phase** consists of (1) **preemption of any process running in the kernel** and (2) **release by low-priority processes of resources needed by a high-priority process**.

X5-48 Periodic tasks, utilization, admission control

Describe the characteristics of a periodic real-time task. What is the CPU utilization of a task set, and what is admission control?

- A periodic task requires the CPU at **constant intervals (periods)**: **processing time t, deadline d, period p, with 0 ≤ t ≤ d ≤ p**; its **rate is 1/p**. One job is released every p time units and must finish by its deadline (often d = p, i.e., by the start of the next period).
- **CPU utilization of task i** = t_i/p_i; total $U = \sum_i t_i/p_i$. E.g., (t, p) = (20, 50) and (35, 100): U = 0.40 + 0.35 = **75%**. If **U > 1**, the CPU is overloaded and **some deadlines must be missed**.
- **Admission control**: a process **announces its deadline requirements** to the scheduler, which **admits it only if it can guarantee that the process will complete on time** and otherwise rejects it.

X5-49 Rate-monotonic scheduling and missed deadlines

Explain rate-monotonic (RM) scheduling. Why can RM miss a deadline even when U < 1?

- **RM schedules periodic tasks using a static priority policy with preemption**: **priority = inverse of the period — shorter period (higher rate) = higher priority**, because tasks that need the CPU more often get higher priority. Priorities never change.
- **RM is optimal among static-priority algorithms**: if a task set **cannot be scheduled by RM, no other static-priority algorithm can schedule it**. It handles only **periodic tasks with a constant processing time**.
- **Deadlines can be missed with U < 1**, because the static priority ignores how close each deadline is. E.g., P1 (t = 25, p = 50), P2 (t = 35, p = 80): U = 0.50 + 0.44 = 94%; P1 preempts P2 at 50: `0 P1 25 P2 50 P1 75 P2 85` → **P2 misses its deadline 80, finishing at 85**.
- RM is guaranteed only if $U \le N(2^{1/N} - 1)$ (0.828 for N = 2; tends to ln 2 ≈ 0.693 for large N) — a sufficient, not necessary, condition.

X5-50 Earliest-deadline-first scheduling

Explain EDF scheduling and its advantages over RM. Why can it not reach 100% CPU utilization in practice?

- **EDF assigns priorities dynamically according to the deadline: the earlier the (absolute) deadline, the higher the priority**; priorities are re-evaluated whenever a job becomes runnable; it is preemptive.
- **EDF does not require processes to be periodic, nor a constant amount of CPU time per burst — the only requirement is that a process announces its deadline when it becomes runnable.**
- **Theoretically optimal**: it can schedule processes so that **each meets its deadline with CPU utilization up to 100%** (U ≤ 1 when deadline = period). E.g., P1 (25, 50), P2 (35, 80): `0 P1 25 P2 60 P1 85 P2 100 P1 125 P2 145 idle 150` — at t = 50, P2's deadline 80 is earlier than P1's 100, so P2 continues → **no deadline missed** (RM misses 80).
- **In practice 100% is impossible** because of the cost of **context switching and interrupt handling**.

X5-51 RM versus EDF comparison

Compare rate-monotonic and earliest-deadline-first scheduling.

	Rate-monotonic (RM)	**Earliest-deadline-first (EDF)**
Priority	**static** — inverse of period (shorter = higher)	**dynamic** — earlier absolute deadline = higher
Task model	**periodic, constant processing time**	periodic or not, burst may vary; must **announce its deadline**
Preemption	yes	yes
Optimality	**optimal among static-priority algorithms**	**theoretically optimal** overall
Schedulable if (d = p)	$U \le N(2^{1/N} - 1)$ guarantees it (sufficient only)	$U \le 1$ (necessary and sufficient)
Cost	priorities fixed once: simple, predictable	priorities recomputed at each release: more overhead
Example P1 (25, 50), P2 (35, 80)	**P2 misses deadline 80**	**all deadlines met**

X5-52 Schedulability test with utilization bound

Using CPU utilization (deadline = period), decide whether each task set (t, p) is schedulable by RM and by EDF: (a) (1, 4), (2, 6), (1, 10); (b) (2, 5), (4, 7); (c) (2, 4), (4, 8); (d) (3, 5), (3, 6).

RM bound $N(2^{1/N} - 1)$: N = 1: 1.000; N = 2: 0.828; N = 3: 0.780; N = 4: 0.757; N = 5: 0.743; large N: 0.693. **EDF: schedulable iff U ≤ 1.**

- (a) U = 1/4 + 2/6 + 1/10 = 0.250 + 0.333 + 0.100 = **0.683 ≤ 0.780** → **RM guaranteed**; EDF also schedulable.
- (b) U = 2/5 + 4/7 = 0.400 + 0.571 = **0.971 > 0.828** → RM **not guaranteed — draw the timeline** (RM actually misses at t = 7; see the next entry); **EDF schedulable** (U ≤ 1).
- (c) U = 2/4 + 4/8 = **1.00 > 0.828**, yet RM meets every deadline: `0 T1 2 T2 4 T1 6 T2 8`, repeating → the RM bound is **sufficient, not necessary**; EDF also schedulable (U = 1).
- (d) U = 3/5 + 3/6 = **1.10 > 1** → **overloaded: neither RM nor EDF (nor any algorithm) can meet all deadlines**.

X5-53 RM and EDF timelines, missed deadlines

Soft real-time: P1 (t = 2, period = deadline = 5) and P2 (t = 4, period = deadline = 7), both released at 0. Draw the RM and EDF schedules for 0–14 and count the missed deadlines.

U = 2/5 + 4/7 = 0.971. Releases and deadlines: P1 at 0, 5, 10 (deadlines 5, 10, 15); P2 at 0, 7 (deadlines 7, 14).

- **RM** (P1 has higher priority, period 5 < 7): `0 P1 2 P2 5 P1 7 P2 10 P1 12 P2 14`. At t = 5, P1 preempts P2 (P2 has 1 unit left); at t = 7, P2's first job is unfinished → **misses deadline 7**; it keeps running (soft real-time) and finishes at 8; P2's second job runs 8–10 and 12–14 → meets deadline 14. **RM: 1 missed deadline.**
- **EDF**: `0 P1 2 P2 6 P1 8 P2 12 P1 14`. At t = 5, P2's deadline 7 < P1's new deadline 10 → **P2 continues** and finishes at 6; at t = 10, P2's deadline 14 < P1's 15 → P2 continues to 12. **EDF: 0 missed deadlines.**
- EDF wins because its **dynamic priorities follow the deadlines**, whereas RM's static priority ran P1 (deadline 10) ahead of P2 (deadline 7).

X5-54 Algorithm evaluation methods

How are CPU-scheduling algorithms evaluated? Compare deterministic modeling, queueing models, simulation and implementation.

First **define the criteria** and their relative importance, e.g., **maximize CPU utilization under the constraint that the maximum response time is 300 ms**, or **maximize throughput such that turnaround time is (on average) linearly proportional to total execution time**. Then evaluate:

Method	How	Advantages	Disadvantages
Deterministic modeling	takes a **particular predetermined workload** and computes each algorithm's performance (Gantt chart)	**simple, fast, exact numbers**	**requires exact input; results apply only to that workload**
Queueing models	CPU and devices as a **network of servers with queues**; arrival and service **distributions** (often exponential) give utilization, average queue length, waiting time (Little's formula)	handles stochastic workloads mathematically	**only a few classes of algorithms and distributions** are tractable; assumptions are often **unrealistic**
Simulation	**programs a model of the system**; a clock advances the state; driven by **random numbers from distributions** or by **trace files** of a real system	**more accurate** (traces give the real order of events)	still limited accuracy; can be expensive (computation, storage, coding)
Implementation	codes the scheduler into a **real OS** and measures it on real workloads	**most accurate**	**high cost and high risk** (new bugs); the environment keeps changing

E.g., deterministic: P1–P5 = 10, 29, 3, 7, 12 ms at t = 0 → avg WT FCFS 28, non-preemptive SJF 13, RR (q = 10) 23 → SJF is best for this workload. Flexible systems also let system managers **tune the scheduling parameters**, and APIs (Java, POSIX, Windows) can change a process's or thread's priority.

X5-55 Little's formula calculations

State Little's formula and its assumptions. (a) n = 14, λ = 7/s: find W. (b) λ = 20/s, W = 0.5 s: find n. (c) n = 6, W = 300 ms: find λ. (d) 120 processes arrive per minute and each waits 250 ms on average: find n.

- **Little's formula:** $n = \lambda \times W$, where **n = average queue length**, **λ = average arrival rate into the queue**, **W = average waiting time in the queue**. It holds in **steady state** (processes leaving the queue = processes arriving; the arrival rate must be smaller than the service rate) and is **valid for any scheduling algorithm and arrival distribution**.
- (a) W = n/λ = 14/7 = **2 s**.
- (b) n = λW = 20 × 0.5 = **10 processes**.
- (c) λ = n/W = 6/0.3 = **20 processes/s**.
- (d) Convert units first: λ = 120/60 = 2 per s; n = 2 × 0.25 = **0.5 process** on average.

X5-TF True / False quick verdicts

Statement	T/F	Why (≤ 12 words)
Scheduling when a process switches from running to waiting is non-preemptive.	**T**	The process gives up the CPU voluntarily (condition 1).
A process moving from waiting to ready can trigger preemption.	**T**	Condition 4: it may have a higher priority.
The dispatcher decides which process runs next.	**F**	The scheduler selects; the dispatcher performs the context switch.
CPU utilization and throughput measure the same thing.	**F**	Utilization = busy fraction; throughput = completions per unit time.

Statement	T/F	Why (≤ 12 words)
A better scheduling algorithm shortens a process's CPU burst.	**F**	It only changes time spent waiting in the ready queue.
Under this course's definition, FCFS response time equals turnaround time.	**T**	Non-preemptive: the first run ends when the burst completes.
FCFS can suffer the convoy effect but never starves a process.	**T**	Short jobs wait behind long ones; the queue always advances.
In RR, a process whose quantum expires rejoins the head of the ready queue.	**F**	It is added to the tail of the ready queue.
A larger time quantum always lowers RR's average turnaround time.	**F**	Not monotonic; a very large q degenerates to FCFS.
RR with a quantum larger than every CPU burst behaves like FCFS.	**T**	Every burst finishes within its first quantum.
About 80% of CPU bursts should be shorter than the time quantum.	**T**	Rule of thumb for choosing q (slide 5.16).
A very small quantum always improves RR performance.	**F**	Switch overhead dominates; efficiency q/(q + s) falls.
SJF gives the minimum average waiting time for a given set of processes.	**T**	Running shorter jobs first always lowers total waiting time.
SRTF can give a larger average waiting time than SJF.	**F**	SRTF is always at least as good as SJF.
SRTF must invoke the scheduler whenever a process joins the ready queue.	**T**	A shorter arrival may preempt the running process.
In exponential averaging, α = 0 makes the prediction equal the last burst.	**F**	α = 0 keeps old prediction; α = 1 uses last burst.
Exponential averaging gives older bursts exponentially smaller weights.	**T**	Each older term is multiplied by another factor (1 − α).
Aging lowers the priority of processes that have run for a long time.	**F**	Aging raises the priority of processes that wait long.
Non-preemptive priority scheduling cannot cause starvation.	**F**	Higher-priority arrivals are always chosen first at each completion.
In a multilevel queue, fixed-priority scheduling among queues can starve low queues.	**T**	Lower queues run only when all higher queues are empty.
In MLFQ, a job demoted for using its full quantum joins the head of the lower queue.	**F**	Demoted jobs join the tail; only preempted jobs return to head.
In MLFQ, a Q1 job preempted by a new arrival keeps its remaining quantum.	**T**	It returns to the head of Q1 without a fresh quantum.
In MLFQ, a Q1 job can preempt a running Q2 job.	**F**	No job enters Q1 while a Q2 job is running.
Under many-to-many, the kernel chooses which user thread runs on an LWP.	**F**	The thread library does: process-contention scope (PCS).
In AMP, every processor accesses the kernel data structures.	**F**	Only the master does; others execute user code only.
With one common ready queue, explicit load balancing is unnecessary.	**T**	An idle CPU simply takes the next thread from the queue.
Soft affinity guarantees that a process never migrates.	**F**	It only attempts; migration may occur during load balancing.
A dual-core CPU with 2 hardware threads per core shows 4 logical CPUs.	**T**	2 cores × 2 hardware threads = 4 logical processors.
A preemptive priority-based scheduler guarantees hard real-time deadlines.	**F**	It guarantees only soft real-time functionality.
RM is optimal among all real-time scheduling algorithms.	**F**	Optimal only among static-priority algorithms; EDF is optimal overall.
If U ≤ 1, RM meets every deadline.	**F**	RM is guaranteed only if $U \le N(2^{1/N} - 1)$.
Exceeding the RM utilization bound means some deadline will be missed.	**F**	The bound is sufficient, not necessary; draw the timeline.
In theory, EDF meets all deadlines up to 100% utilization when deadline = period.	**T**	Optimal; switching and interrupts prevent 100% in practice.
Little's formula holds for any scheduling algorithm in steady state.	**T**	Valid for any algorithm and arrival distribution.
Deterministic-modeling results hold for any workload.	**F**	They apply only to the exact workload that was modelled.

X6 · Ch6 Synchronization

X6-01 Race condition: counter++ / counter-- interleaving

Define a race condition. Using `counter++` (producer) and `counter--` (consumer) with counter = 5, show at the machine level how the result can be wrong. [6.7, 6.9]

A **race condition** occurs when several processes **access and manipulate shared data concurrently** and the **outcome depends on the particular order** in which the accesses take place. `counter++` and `counter--` are **not atomic**: each becomes **three machine instructions**, and the two sequences can be **interleaved**:

```
S0: producer  register1 = counter          /* register1 =
5 */
S1: producer  register1 = register1 + 1    /* register1 =
6 */
S2: consumer  register2 = counter          /* register2 =
5 */
S3: consumer  register2 = register2 - 1    /* register2 =
4 */
S4: producer  counter = register1          /* counter = 6
*/
S5: consumer  counter = register2          /* counter = 4
*/
```

The correct value is 5, but the result is **4** (or **6** if S4 and S5 are swapped): one update is **lost** because both processes read the old value before either stored its result. Fix: make the update a **critical section** executed under **mutual exclusion**.

X6-02 All possible final values of counter

counter = 5. The producer executes `counter++` once and the consumer executes `counter--` once, concurrently and without synchronization. Give all possible final values of counter.

4, 5 or 6.

- **5**: the two updates do not overlap (one stores before the other loads), which is the correct result.
- **4**: both load 5; the producer stores 6, then the consumer stores 4, so **the increment is lost**.
- **6**: both load 5; the consumer stores 4, then the producer stores 6, so **the decrement is lost**.

No other value is possible: each process stores (the value it loaded) ± 1, and it loads either 5 or the other process's result.

X6-03 Unprotected counter in a loop: lost updates

In Lab 6 Q3, two threads each execute `++counter` 5,000 times (counter = 0). If the `pthread_mutex_lock()` / `pthread_mutex_unlock()` calls are removed, is the final value still 10,000?

No. The result is non-deterministic: any value from 2 to 10,000 is possible (10,000 only if no update is lost).

- `++counter` is load, add, store; if the other thread stores in between, this thread's store **overwrites (loses)** the other's increments: a **race condition**.
- Worst case 2: T1 loads 0 and is preempted; T2 does 4,999 increments; T1 stores 1; T2 loads 1 and is preempted; T1 does its other 4,999 increments; T2 stores 2.
- With the mutex, `++counter` is a **critical section** executed under mutual exclusion, so the result is always **10,000**.

X6-04 Race condition inside the kernel

Give an example of a race condition in the kernel. Which kernel data structures must be protected? [6.8, 6.12]

- Processes **P0 and P1 call `fork()` at the same time**. Both read the kernel variable `next_available_pid` before either one updates it, so **the same pid could be assigned to two different processes**, unless updates of the variable are mutually exclusive.
- Other kernel data structures subject to race conditions: the **list of open files** (two processes open files simultaneously), structures for **memory allocation**, the **process list / process table**, and **interrupt handling**.
- How the kernel protects them depends on whether it is **preemptive or non-preemptive**.

X6-05 Critical section; entry, exit, remainder sections

What is a critical section? Describe the general structure of a process in the critical-section problem. [6.9–6.10]

A **critical section** is a segment of code in which a process **accesses or updates shared data** (changing shared variables, updating a table, writing a file), so a race condition may occur. The **critical-section problem** is to design a **protocol** such that **when one process is executing in its critical section, no other process is executing in its critical section**.

```
do {
    entry section       /* request permission to enter */
        critical section
    exit section        /* give up permission, let others
in */
        remainder section
} while (true);
```

- **Entry section**: requests permission (e.g., `acquire()`, `wait(mutex)`).
- **Exit section**: releases it (e.g., `release()`, `signal(mutex)`); the **remainder section** is the rest of the code.

X6-06 Three requirements with violation examples

State the three requirements a solution to the critical-section problem must satisfy, and give one way each can be violated. [6.11]

1. **Mutual exclusion**: if process P_i is executing in its critical section, **no other process can be executing in its critical section**. *Violated by:* `counter++` without a lock, or a lock whose test and set are not atomic.
2. **Progress**: if no process is in its critical section and some processes wish to enter, **the selection of the process that enters next cannot be postponed indefinitely** (a process in its remainder section must not block the others). *Violated by:* strict alternation with `turn`, or a deadlock in the entry section.
3. **Bounded waiting**: **a bound must exist on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter and before that request is granted** (no starvation). *Violated by:* the simple test_and_set lock.

Mutual exclusion is a **safety** property (no wrong results); progress and bounded waiting are **liveness** properties. Assumption: each process executes at a **nonzero speed**, with **no assumption about the relative speed** of the processes.

X6-07 Preemptive vs non-preemptive kernels

Compare preemptive and non-preemptive kernels with respect to race conditions. Why might a preemptive kernel still be preferred? [6.12]

- **Preemptive kernel**: allows a process to be **preempted while it is running in kernel mode**, so it is **not free from race conditions** on kernel data, and it is increasingly difficult to design on **SMP** architectures.
- **Non-preemptive kernel**: a kernel-mode process **runs until it exits kernel mode, blocks, or voluntarily yields the CPU**, so on a **single processor** only one process is active in the kernel at a time: it is **essentially free of race conditions in kernel mode** (on a multiprocessor, processes on different CPUs can still be in the kernel at the same time).
- A preemptive kernel is still preferred because it is **more responsive** (no kernel-mode process can run for an arbitrarily long time while others wait) and **more suitable for real-time** processes.

X6-08 Disabling interrupts: uniprocessor vs multiprocessor

Can the critical-section problem be solved by disabling interrupts? Answer for uniprocessor and multiprocessor systems. [6.13]

- **Uniprocessor: yes.** With interrupts disabled while the shared data is modified, the current code **executes without being preempted or interrupted**, so its instructions run in order without interleaving.
- **Multiprocessor / SMP: no, it is generally inefficient.** Disabling interrupts on one CPU **does not stop the other CPUs** from entering their critical sections; disabling them on **all** CPUs requires a message to every processor, which is **time-consuming, delays entry into each critical section and decreases system efficiency** (it also disturbs a system clock that is updated by timer interrupts).
- Disabling interrupts is also a **privileged** operation and is unsafe to give to user programs, so systems use **atomic hardware instructions** (test_and_set, compare_and_swap) and the locks built on them.

X6-09 Peterson's solution: code and assumption (Lab 6)

Write Peterson's solution for two processes P_i and P_j. Explain the shared variables and the assumption it relies on.

```
int turn;                          /* shared: whose turn
it is      */
boolean flag[2] = {false, false};  /* shared: flag[i] =
Pi wants in */

do {                               /* process Pi, j = 1 -
i */
    flag[i] = true;                /* I want to enter ...
*/
    turn = j;                      /* ... but you may go
first    */
    while (flag[j] && turn == j)
        ;                          /* busy wait */
    /* critical section */
    flag[i] = false;               /* I have left */
    /* remainder section */
} while (true);
```

- **`flag[i] == true`** means P_i is ready (wants) to enter its critical section; **`turn`** decides which process enters when both want to.
- It is a **software-only** solution, **restricted to two processes**, and it assumes that **load and store instructions are atomic** (a variable is never read half-written); otherwise the correctness proof does not hold.

X6-10 Peterson satisfies all three requirements

Prove that Peterson's solution satisfies mutual exclusion, progress and bounded waiting.

- **Mutual exclusion**: P_i enters only if `flag[j] == false` or `turn == i`. If both try, `flag[0] == flag[1] == true`, and `turn` is **either 0 or 1, not both**, so only one process can pass its while loop (the one that wrote `turn` last waits).
- **Progress**: P_i is stuck only while `flag[j] == true && turn == j`. If P_j does not want to enter, `flag[j] == false` and P_i **enters immediately**; if both want to enter, `turn` admits one of them at once.
- **Bounded waiting**: when P_j leaves, it sets `flag[j] = false`; if it wants to re-enter, it must set `turn = i`, and P_i does not change `turn` while waiting, so P_i **enters after at most one entry by** P_j.

X6-11 Peterson on modern hardware: memory barriers

Why is Peterson's solution not guaranteed to work on modern computer architectures? How can it be fixed, and what are its other drawbacks?

- To improve performance, **processors and/or compilers may reorder read and write operations that have no dependencies**. Peterson's entry section writes `flag[i]` and `turn` and then reads `flag[j]`, all different variables. If the two writes are reordered, or P_i's read happens before its own write becomes visible to P_j, **both processes can be in their critical sections at the same time**.
- Fix: a **memory barrier (memory fence)**, an instruction that forces all earlier loads and stores to complete and become visible to other processors before any later load or store. Alternatively, use **sequentially consistent atomic operations**; the Lab 6 code uses `atomic_store_explicit(&flag[i], true, memory_order_seq_cst)`.
- Other drawbacks: it is **complex even for two processes**, each thread needs **different code**, and it uses **busy waiting** (wastes CPU cycles). This is why the OS provides **mutex locks, semaphores and condition variables**.

X6-12 Atomic hardware instructions

What does "atomic" mean? Why are atomic hardware instructions useful for synchronization, and why can't every operation simply be made atomic? [6.14]

- **Atomic = non-interruptible**: the instruction (possibly several machine-level steps, e.g., read and write a word) **executes as one unit that cannot be interrupted**, so **no race condition can occur inside it**. If two such instructions run **simultaneously on different cores**, they are **executed sequentially in some order (serialization)**.
- They are the **building blocks** of more sophisticated tools (spinlocks, mutex locks, semaphores, atomic variables). The two common ones are **`test_and_set()`** (test a memory word and set it) and **`compare_and_swap()`** (compare a word with an expected value and, if equal, swap in a new value).
- Not everything can be atomic: hardware instructions work on **a single word / simple variable**. A **complex data structure** (a linked list, a 1000 × 1000 matrix) cannot be updated by one atomic instruction, so **locks and semaphores** built on these instructions protect larger critical sections.

X6-13 test_and_set(): definition, lock, bounded waiting

Give the definition of `test_and_set()`, use it to implement mutual exclusion, and state which requirement the solution does not satisfy. [6.15–6.16]

```
boolean test_and_set(boolean *target) {  /* executed
atomically */
    boolean rv = *target;
    *target = TRUE;
    return rv;                           /* returns the
OLD value */
}
boolean lock = FALSE;                    /* shared */
do {
    while (test_and_set(&lock))
        ;                                /* spin: lock
was held */
    /* critical section */
    lock = FALSE;
    /* remainder section */
} while (true);
```

- **Mutual exclusion holds**: the first caller gets FALSE (lock free) and enters; the same atomic call sets `lock = TRUE`, so every other caller gets TRUE and spins until `lock = FALSE`. **Progress** also holds.
- **Bounded waiting is NOT satisfied**: when the lock is released, **whichever spinning process executes `test_and_set()` first wins**. There is no order, so an unlucky process can lose every time (starvation). The **waiting[]-array version** fixes this.

X6-14 compare_and_swap(): definition and lock

Give the definition of `compare_and_swap()` and use it to implement a lock. Why is the loop condition `!= 0`? [6.17–6.18]

```
int compare_and_swap(int *value, int expected, int
new_value) {
    int temp = *value;                   /* executed
atomically */
    if (*value == expected)
        *value = new_value;
    return temp;                         /* always the
OLD value */
}
int lock = 0;                            /* 0 = free, 1 =
held */
do {
    while (compare_and_swap(&lock, 0, 1) != 0)
        ;                                /* spin */
    /* critical section */
    lock = 0;
    /* remainder section */
} while (true);
```

- `compare_and_swap(&lock, 0, 1)` sets lock to 1 **only if it is 0** and returns the **old** value. A return of **0** means the lock was free and **this process has just taken it**, so it enters; a return of **1** means the lock is held (and unchanged), so it keeps spinning. Hence `!= 0`.
- It satisfies **mutual exclusion and progress**, but **not bounded waiting** (the same race as test_and_set).
- Wrong versions: `== 0` (inverted test) and `compare_and_swap(&lock, 1, 0)` (never sets the lock) both let two processes in.

X6-15 Bounded-waiting TAS: waiting[], key, lock

In the algorithm below, explain the roles of `waiting[]`, `key` and `lock`, and why the exiting process sets `waiting[j] = false` instead of `lock = false`. [6.19]

```
/* shared: boolean waiting[n] (all false), lock = false;
local: key, j */
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;
    /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = false;        /* no one is waiting, so
release the lock */
    else
        waiting[j] = false;  /* unblock process j */
    /* remainder section */
} while (true);
```

- `waiting[i] == true`: P_i **wants to enter** its critical section and is waiting in its entry section.
- `key` (local) holds the value returned by `test_and_set(&lock)`; `key == false` means P_i found the lock free and **acquired it itself**. `lock` is TRUE while some process owns the critical section.
- P_i leaves the entry loop when `key == false` (it took the free lock) **or** `waiting[i] == false` (another process **handed the lock to it**).
- On exit, `waiting[j] = false` means P_i **releases the lock by handing it directly to the next waiting P_j, while `lock` stays TRUE**, so no other process can grab it with test_and_set in between. Because the scan is **cyclic** (i+1, …, n−1, 0, …, i−1), every waiting process gets in **within n − 1 turns**. `lock = false` is executed only when **nobody is waiting**.

X6-16 Bounded-waiting TAS: sketch proof

Give a sketch proof that the bounded-waiting `test_and_set()` algorithm satisfies mutual exclusion, progress and bounded waiting. [6.20]

- **Mutual exclusion**: P_i enters only if `waiting[i] == false` or `key == false`. `key` becomes false only by executing `test_and_set()`, and **only the first process to execute it finds key == false**; all others must wait. `waiting[i]` becomes false only when another process **leaves its critical section**, and **only one** `waiting[j]` is set to false per exit. So at most one process is in its critical section.
- **Progress**: a process exiting its critical section **either sets `lock = false` or sets `waiting[j] = false`**; both allow a waiting process to proceed.
- **Bounded waiting**: the exiting process scans `waiting[]` in **cyclic order {i+1, i+2, …, n−1, 0, 1, …, i−1}** and designates the **first** waiting process as the next to enter, so any waiting process enters **within n − 1 turns**.

X6-17 Broken variants of the bounded-waiting algorithm

In the bounded-waiting TAS algorithm, which requirement fails if (a) the whole exit section is just `lock = false;`, or (b) the exit scan always starts from index 0 instead of i + 1?

- (a) **Bounded waiting fails** (mutual exclusion and progress still hold). The lock is never handed over, so the algorithm becomes the **simple test_and_set spinlock**: whichever process executes test_and_set first after the release wins, and a process can be overtaken indefinitely.
- (b) **Bounded waiting fails** for n ≥ 3 (mutual exclusion and progress still hold). The exiting process always prefers **low-index** waiters, e.g., P_0 and P_1 can keep handing the lock to each other while P_2 waits forever.
- Correct exit: start at `j = (i + 1) % n`, scan **cyclically**, and execute `lock = false` **when j == i** (nobody waiting). If the lock is never released when nobody waits, progress fails.

X6-18 Atomic variables: increment() with CAS

What is an atomic variable? Implement `increment()` on an atomic integer using `compare_and_swap()`, and explain why a loop is needed. [6.21]

An **atomic variable** provides **atomic (uninterruptible) updates on basic data types such as integers and Booleans**; e.g., `increment(&sequence)` increments `sequence` **without interruption**. It is built on compare_and_swap:

```
void increment(atomic_int *v) {
    int temp;
    do {
        temp = *v;                       /* read the
current value */
    } while (temp != compare_and_swap(v, temp, temp +
1));
}
```

- CAS writes `temp + 1` **only if `*v` still equals `temp`**, i.e., no other thread changed `v` since it was read. Because CAS returns the old value, the update succeeded exactly when the return value equals `temp`.
- If another thread changed `v` in between, nothing is written and the loop **retries with the new value**, so **no update is lost** and no lock is needed. Atomic variables protect only a **single variable**; larger critical sections still need locks.

X6-19 Mutex lock: why acquire() must be atomic

Give the mutex-lock `acquire()` / `release()` code. Why must `acquire()` be atomic? Show what can go wrong otherwise.

[6.22–6.23]

```
acquire() {                      release() {
    while (!available)               available = true;
        ;   /* busy wait */      }
    available = false;
}
do {
    acquire lock;     critical section;
    release lock;     remainder section;
} while (true);
```

- A process **acquires the lock before entering** its critical section and **releases it when it exits**; the Boolean `available` indicates whether the lock is free.
- If `acquire()` were not atomic, its **test** and **set** could interleave: T1 sees `available == true`, T2 also sees `available == true`, both set it to false and **both enter the critical section**, violating mutual exclusion. So acquire() and release() are implemented with **hardware atomic instructions** (e.g., compare_and_swap).
- The waiting process loops in `while (!available)`, i.e., **busy waiting**, so this lock is called a **spinlock**.

X6-20 Spinlocks: when to use, not on uniprocessors

When are spinlocks preferable to blocking (sleeping) locks? Why is a spinlock a poor choice on a single-processor system?

[6.22]

- Spinlocks are preferable on **multiprocessor (multicore) systems when locks are held for short times**: a waiting thread **spins on one processor** while the lock holder runs its short critical section **on another processor** and releases the lock soon. **No context switch is required**, and a context switch may take considerable time (rule of thumb: spin if the lock is held for less than the time of two context switches).
- On a **single processor**, the spinning thread occupies the only CPU, so **the lock holder cannot run to release the lock** until the spinner is preempted, and the spinner **wastes its whole time slice**. It should **block (sleep)** instead.
- Likewise, if the wait may be **long**, a thread should block rather than spin, because **busy waiting wastes CPU cycles**.

X6-21 Semaphore: definition, wait/signal, P and V

Define a semaphore and its two operations (busy-waiting definition). What do P and V stand for?

[6.24]

A **semaphore S** is an **integer variable** (a **generalized lock**, introduced by **Dijkstra**, and the main synchronization primitive of the original UNIX) that, **apart from initialization, is accessed only through two standard atomic operations: `wait()` and `signal()`**, originally **P()** (*proberen*, "to test") and **V()** (*verhogen*, "to increment").

```
wait(S) {                    signal(S) {
    while (S <= 0)               S++;
        ;  /* busy wait */   }
    S--;
}
```

- `wait(S)`: while S ≤ 0 the process waits; then it **decrements S** (takes a permit). `signal(S)`: **increments S** (returns a permit).
- Both must be executed **atomically**: no two processes may execute wait() or signal() on the same semaphore at the same time (**serialization**).

X6-22 Why wait() and signal() must be atomic

Why must `wait()` and `signal()` be executed atomically? How is this guaranteed on uniprocessor and multiprocessor systems?

- Otherwise two processes could **both test `S > 0` (e.g., S = 1) before either decrements it**, so **both pass `wait()`** and enter the critical section (mutual exclusion violated); S can even end at −1, or an update of S can be lost. Hence **no two processes may execute wait()/signal() on the same semaphore at the same time**; this is itself a critical-section problem.
- **Uniprocessor**: **disable interrupts** while wait() and signal() execute, so their instructions cannot be interleaved.
- **Multiprocessor (SMP)**: disabling interrupts on every core is difficult and slow, so wait() and signal() are protected with **atomic hardware instructions: compare_and_swap() or spinlocks**. Busy waiting is not eliminated entirely, but it is limited to these **very short** code sections instead of the application's critical section.

X6-23 Counting vs binary semaphore; semaphore vs mutex

Distinguish counting and binary semaphores, and explain how a semaphore differs from a mutex lock. [6.25]

- **Counting semaphore**: its integer value ranges over an **unrestricted domain**. It controls access to a resource with a **finite number of instances** and is **initialized to the number of resources available** (`wait()` takes an instance, `signal()` returns one).
- **Binary semaphore**: its value is **only 0 or 1**. Initialized to **1**, it **behaves like a mutex lock**, but it can also be used in other ways, e.g., initialized to **0** to **enforce an execution order** (S1 before S2).
- **Semaphore vs mutex lock**: a mutex lock has a **binary value** (available or not) and is used only for **mutual exclusion**; a semaphore has an **integer value** (a generalized lock) that can also **count resources** and **signal events / enforce order** between processes. (A Pthreads mutex should be unlocked by the thread that locked it, whereas any thread may `sem_post()` a semaphore.)

X6-24 Mutual exclusion with a semaphore: initial value

Show how a semaphore provides mutual exclusion among n processes. What happens if it is initialized to 0 or to 2 instead of 1?

```
semaphore mutex = 1;          /* shared */
do {
    wait(mutex);
        /* critical section */
    signal(mutex);
        /* remainder section */
} while (true);
```

- **Initialized to 1**: the first `wait()` takes the only permit (mutex = 0) and all others **block** until `signal(mutex)`, so **at most one process is in its critical section**.
- **Initialized to 0**: there is no permit, so **no process can ever enter**; all of them block forever (deadlock).
- **Initialized to 2**: **two processes can be in their critical sections at the same time**, violating mutual exclusion.

X6-25 Ordering S1 before S2 (synch = 0)

Processes P1 and P2 run concurrently. Use a semaphore to guarantee that statement S1 (in P1) executes before statement S2 (in P2). Why must it be initialized to 0? [6.25]

```
semaphore synch = 0;
P1:   S1;  signal(synch);
P2:   wait(synch);  S2;
```

- If P2 reaches `wait(synch)` first, synch is 0, so **P2 blocks** until P1 finishes S1 and calls `signal(synch)`. If P1 runs first, synch becomes 1 and P2 passes `wait()` immediately, but S1 is already done. **Either way S2 starts only after S1 completes.**
- If synch were initialized to **1**, P2 could pass `wait()` before S1 runs, so the order would not be enforced.
- General rule: **"X before Y" needs a semaphore initialized to 0, with `signal()` right after X and `wait()` right before Y.**

X6-26 Counting semaphore: at most k users

A server has 3 identical database connections, so at most 3 threads may use them at the same time. Write the synchronization with a semaphore. What happens if 5 threads request a connection together?

```
semaphore conn = 3;     /* number of available instances
*/
wait(conn);             /* take a connection (block if
none left) */
    /* use the connection */
signal(conn);           /* return it, waking one waiter
if any */
```

- A **counting semaphore initialized to k (the number of instances)** lets **at most k** threads pass `wait()` until someone calls `signal()`.
- With 5 requests, **3 proceed and 2 block**; in the waiting-queue implementation conn = **−2** (2 waiting). Each `signal()` wakes one waiter.
- It limits the number of users to k but gives **no mutual exclusion among them**; shared data that they update still needs a separate **mutex initialized to 1**.

X6-27 Rendezvous and barrier with semaphores

Threads A and B: a1 must execute before b2, and b1 before a2 (a rendezvous). Give a semaphore solution, then extend it to a barrier for n threads.

```
semaphore aArrived = 0, bArrived = 0;
A:   a1;  signal(aArrived);  wait(bArrived);  a2;
B:   b1;  signal(bArrived);  wait(aArrived);  b2;
```

- Each thread **signals its own arrival first, then waits for the other**, so neither second statement starts before both first statements finish. If **both wait before signalling**, each waits for a signal that is never sent: **deadlock**.
- **Barrier for n threads** (no thread passes until all n have arrived):

```
int count = 0;  semaphore mutex = 1, barrier = 0;
wait(mutex);
count++;
if (count == n) signal(barrier);   /* the last to arrive
opens it */
signal(mutex);
wait(barrier);
signal(barrier);                   /* turnstile: let the
next one through */
/* critical point: all n threads have arrived */
```

X6-28 Semaphore without busy waiting: block() and wakeup()

How can a semaphore be implemented without busy waiting? Give the code and explain `block()` and `wakeup()`. [6.26–6.27]

Each semaphore has an integer **value** and a **waiting queue** (a list of processes):

```
typedef struct { int value; struct process *list; }
semaphore;

wait(semaphore *S) {
    S->value--;
    if (S->value < 0) { add this process to S->list;
block(); }
}
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) { remove a process P from S->list;
wakeup(P); }
}
```

- `block()` suspends the calling process and places it on the semaphore's **waiting queue** (running → waiting) instead of spinning, so **no CPU time is wasted**. `wakeup(P)` removes P from the waiting queue and places it on the **ready queue**; `signal()` calls it only if value ≤ 0 after the increment, i.e., someone is waiting.
- The decrement/increment is done **before** the value is tested (unlike the busy-waiting version), so the value **can become negative**, and **its magnitude is the number of processes waiting** on the semaphore.
- Better than busy waiting when waits can be **long**; the spinning version is better only for **very short** waits on multiprocessors (no context switch).

X6-29 Tracing a semaphore value with a waiting queue

A semaphore S (waiting-queue implementation) is initialized to 2. Five processes call `wait(S)`, and then another process calls `signal(S)` twice. What is the value of S, and how many processes are still blocked?

S = −1, and one process is still blocked.

- Five waits: $2 - 5 = -3$, so the first **two** processes pass and **three** are blocked in the queue ($|-3| = 3$).
- Two signals: $-3 + 2 = -1$; each signal finds the value ≤ 0 and **wakes one** blocked process (moves it to the ready queue), so **one** remains blocked ($|-1| = 1$).
- Rule: S ≥ 0 means S permits are free and nobody waits; S < 0 means no permit is free and $|S|$ processes are waiting.

X6-30 Deadlock with two semaphores

S and Q are semaphores initialized to 1. Show how P0 and P1 below can deadlock, and how to prevent it. [6.28]

```
P0: wait(S);  wait(Q);  ...  signal(S);  signal(Q);
P1: wait(Q);  wait(S);  ...  signal(Q);  signal(S);
```

- **Deadlock**: two or more processes are **waiting indefinitely for an event that can be caused only by one of the waiting processes**.
- P0 executes `wait(S)` and then P1 executes `wait(Q)`. When P0 executes `wait(Q)`, it must wait until P1 executes `signal(Q)`; but P1 is waiting in `wait(S)` until P0 executes `signal(S)`. Since **these signal() operations can never be executed, P0 and P1 are deadlocked**. It happens only under some interleavings, so it is **extremely difficult to debug**.
- Prevention: make **every process acquire the semaphores in the same order** (e.g., both execute `wait(S); wait(Q);`), so a circular wait cannot form.

X6-31 Starvation (indefinite blocking) vs deadlock

What is starvation in the context of semaphores, and how does it differ from deadlock? [6.28]

- **Starvation (indefinite blocking)**: a process **may never be removed from the semaphore queue in which it is suspended**, e.g., if processes are removed from the queue in **LIFO (last-in, first-out) order or by priority**, newer or higher-priority processes can always be chosen ahead of it.
- Remedy: remove waiting processes in **FIFO order**, so every waiting process is eventually woken.
- **Deadlock**: a set of processes all wait for each other and **none can proceed**. **Starvation**: the system keeps running and **other processes make progress**, but one process waits forever. Both are **liveness** failures, and a deadlock-free system can still suffer starvation.

X6-32 Verdict table for classic CS protocols

For each entry/exit protocol (two processes with j = 1 − i, unless noted), state whether mutual exclusion (ME), progress and bounded waiting (BW) hold.

Protocol (entry … exit)	ME	Progress	BW	Reason
`while (turn != i);` … `turn = j;`	Yes	**No**	Yes[1]	strict alternation: a process in its remainder section blocks the other
`while (flag[j]); flag[i] = true;` … `flag[i] = false;`	**No**	Yes	**No**	both test before either sets its flag
`flag[i] = true; while (flag[j]);` … `flag[i] = false;`	Yes	**No**	Yes[1]	both set their flags, then both wait forever (deadlock)
Peterson: `flag[i] = true; turn = j; while (flag[j] && turn == j);` … `flag[i] = false;`	Yes	Yes	Yes	`turn` breaks the tie; at most one overtake
Peterson with `turn = j;` and `flag[i] = true;` swapped	**No**	Yes	**No**	both can slip through
`while (test_and_set(&lock));` … `lock = false;`	Yes	Yes	**No**	any spinning process may win; starvation
`while (compare_and_swap(&lock, 0, 1) != 0);` … `lock = 0;`	Yes	Yes	**No**	same race as test_and_set
Bounded-waiting TAS (`waiting[]`, cyclic scan), n processes	Yes	Yes	Yes	lock handed to the next waiter: ≤ n − 1 turns
`acquire()` whose test and set are not atomic	**No**	Yes	**No**	both see `available == true`

[1] Yes by the slide definition (the number of entries by others is bounded: at most one with strict alternation, none when each process sets its flag before testing), but the waiting time is still unbounded because progress fails.

X6-33 Strict alternation using only turn

Shared `int turn = 0;` Process P_i (j = 1 − i): `do { while (turn != i); /* CS */ turn = j; /* remainder */ } while (true);` Does this solve the critical-section problem?

No. It satisfies mutual exclusion but violates progress.

- **Mutual exclusion: yes.** P_i enters only when `turn == i`, and `turn` has a single value, so only one process can be in its critical section.
- **Progress: no.** It forces **strict alternation**: if `turn == 1` and P_1 stays in its **remainder section** (does not want to enter), P_0 **cannot enter although the critical section is free**; the decision is postponed indefinitely by a process that is not even competing.
- **Bounded waiting: yes by the definition.** After P_i requests, P_j can enter **at most once** before `turn` passes to P_i; but P_i still waits forever if P_j never returns (the progress failure).

X6-34 Flag checked before it is set

Shared `boolean flag[2] = {false, false};` Process P_i: `while (flag[j]); flag[i] = true; /* CS */ flag[i] = false;` Which requirements hold?

Mutual exclusion is violated, so it is not a solution.

- Interleaving: P_0 tests `flag[1]` (false) and passes; before P_0 sets `flag[0]`, P_1 tests `flag[0]` (false) and passes; both set their flags and **both enter the critical section**. The **test and the set are not one atomic step**.
- **Progress holds**: a process waits only while the other's flag is up, i.e., while the other is in its critical section, and the flag is cleared on exit.
- **Bounded waiting fails**: P_i raises its flag only after passing the test, so its request is invisible; P_j can leave and re-enter any number of times while P_i keeps seeing `flag[j] == true`.

X6-35 Flag set before checking: deadlock

Shared `boolean flag[2] = {false, false};` Process P_i: `flag[i] = true; while (flag[j]); /* CS */ flag[i] = false;` Which requirements hold?

Mutual exclusion holds, but progress is violated: the processes can deadlock.

- **Mutual exclusion**: each process raises its flag **before** testing the other's, so if both are interested, the one that raises its flag later sees the other's flag and waits; they cannot both enter.
- **Progress: no.** P_0 sets `flag[0] = true`, then P_1 sets `flag[1] = true`; now **each waits for the other's flag to become false, forever** (deadlock), although the critical section is empty.
- **Bounded waiting**: neither process can overtake one whose flag is up, but after a deadlock the waiting time is unbounded. Peterson's `turn` variable breaks exactly this tie.

X6-36 Peterson variants that break mutual exclusion

In Peterson's solution, what happens if (a) `flag[i] = true;` and `turn = j;` are swapped, or (b) a process sets `turn = i;` instead of `turn = j;`?

Both versions violate mutual exclusion (progress still holds; bounded waiting is lost).

- (a) `turn = j; flag[i] = true;`: P_0 sets `turn = 1`; P_1 sets `turn = 0`, sets `flag[1] = true`, finds `flag[0] == false` and **enters**; P_0 sets `flag[0] = true`, finds `turn == 0` (not 1) and **enters**. Both are in their critical sections. This is exactly what **instruction reordering** can do to the correct code, hence **memory barriers**.
- (b) `turn = i`: P_0 sets `flag[0] = true; turn = 0;`, finds `flag[1] == false` and **enters**; P_1 sets `flag[1] = true; turn = 1;`, finds `turn == 1` (not 0) and **enters**. Both are in their critical sections.
- Correct rule: **raise your own flag first, then give the turn to the other process** (`turn = j`); whoever writes `turn` last waits.

X6-37 Semaphores for a precedence graph

Six concurrent processes P1–P6 execute statements S1–S6 with precedence S1→S2, S1→S3, S2→S4, S3→S4, S3→S5, S4→S6, S5→S6. Enforce it with semaphores and give their initial values.

Rule: for each edge X → Y use a semaphore **initialized to 0**; X calls `signal()` **after** it finishes, and Y calls `wait()` **before** it starts. Compactly, use **one semaphore per statement that has predecessors**, waited on **once per incoming edge**:

```
semaphore s2 = 0, s3 = 0, s4 = 0, s5 = 0, s6 = 0;
P1:                         S1;  signal(s2); signal(s3);
P2:  wait(s2);              S2;  signal(s4);
P3:  wait(s3);              S3;  signal(s4); signal(s5);
P4:  wait(s4); wait(s4);    S4;  signal(s6);   /* after
S2 and S3 */
P5:  wait(s5);              S5;  signal(s6);
P6:  wait(s6); wait(s6);    S6;                /* after
S4 and S5 */
```

- All semaphores start at **0**, so no statement can start before every predecessor has signalled; S1 needs no wait.
- The possible execution orders are exactly the topological orders **123456, 123546, 132456, 132546, 135246**, and no execution deadlocks.
- Equivalent per-edge version: 7 semaphores (e12, e13, e24, e34, e35, e46, e56), e.g., P4: `wait(e24); wait(e34); S4; signal(e46);`.

X6-38 Strict alternation of threads with semaphores

Thread A prints "ping" and thread B prints "pong" in infinite loops. Make the output strictly alternate, starting with ping. Generalize to k threads in a fixed cyclic order.

```
semaphore ping = 1, pong = 0;
A:  while (true) { wait(ping);  print("ping");
signal(pong); }
B:  while (true) { wait(pong);  print("pong");
signal(ping); }
```

- **Exactly one permit circulates**: ping = 1 lets A go first; each thread waits on its own semaphore and signals the **next** thread's, so the output is always ping pong ping pong ....
- **k threads T1 → T2 → ... → Tk → T1**: one semaphore per thread, **the first initialized to 1 and all others to 0**; each Ti executes `wait(own semaphore); critical section; signal(next thread's semaphore);`, and Tk signals T1's semaphore.
- If two semaphores started at 1, two threads could run together; if all started at 0, the threads would deadlock.

X6-39 Fill in the blanks: TAS, CAS, increment

Fill in the blanks (`lock` is initially false / 0):

```
/* (a) */  while (__1__) ;          /* critical section
*/   __2__;
/* (b) */  while (compare_and_swap(&lock, __3__, __4__)
__5__ 0) ;
           /* critical section */   __6__;
/* (c) */  void increment(atomic_int *v) {
               int temp;
               do { temp = __7__; } while (temp !=
__8__);
           }
```

- **1** `test_and_set(&lock)` · **2** `lock = false`
- **3** `0` · **4** `1` · **5** `!=` · **6** `lock = 0`
- **7** `*v` · **8** `compare_and_swap(v, temp, temp + 1)`
- Reason: both instructions **return the old value**. A process has the lock exactly when the old value was **free (false / 0)**, so it spins **while** the returned value says "held"; release is an ordinary store of the free value.

X6-40 Spinlock from a new instruction: fetch_and_add

`fetch_and_add(int *x, int a)` atomically adds a to `*x` and returns the old value of `*x`. Use it to build a spinlock that also satisfies bounded waiting.

A **ticket lock**: each process takes a ticket and waits until its number is served.

```
int next_ticket = 0, now_serving = 0;          /* shared
*/
void acquire() {
    int my = fetch_and_add(&next_ticket, 1);   /* unique
ticket */
    while (now_serving != my)
        ;                                      /* spin */
}
void release() {
    fetch_and_add(&now_serving, 1);            /* serve
the next ticket */
}
```

- **Mutual exclusion**: `fetch_and_add()` is atomic, so tickets are **unique**, and only the process whose ticket equals `now_serving` may enter. **Progress**: each release serves the next ticket, and a newcomer with no one waiting gets in at once.
- **Bounded waiting**: processes enter in **ticket (FIFO) order**, so a waiting process is overtaken **at most n − 1 times**.
- General recipe for any atomic instruction: find the return value that means "I got the lock", spin while the instruction does not return it, and release by storing the free value (e.g., `while (swap(locked, 1) == 1) ;` then `locked = 0;`).

X6-41 Possible values of c: worked example

Shared `int a = 3, b = 5, c = 0;` Thread 1: `if (a > 0) c = a + b; else c = b - a;` Thread 2: `a = -1; b = 10;` Reads and writes are atomic, each thread's order is preserved, and expressions read their operands left to right. Give all possible final values of c.

c ∈ {4, 6, 8, 9, 11, 13}. Method: split Thread 1 into atomic steps (**read a for the condition**, then the branch's reads in order, then **write c**). Thread 2's writes keep their order (`a = -1` before `b = 10`) but may fall into any gap. Branch on what each read sees:

Branch	Where Thread 2's writes fall	Thread 1 reads	c
then (condition saw a = 3)	both after Thread 1	a = 3, b = 5	3 + 5 = **8**
then	`a = -1` before the 2nd read of a; `b = 10` after the read of b	a = −1, b = 5	−1 + 5 = **4**
then	both before the 2nd read of a	a = −1, b = 10	−1 + 10 = **9**
then	both between the 2nd read of a and the read of b	a = 3, b = 10	3 + 10 = **13**
else (condition saw a = −1)	`b = 10` after the read of b	b = 5, a = −1	5 − (−1) = **6**
else	`b = 10` before the read of b	b = 10, a = −1	10 − (−1) = **11**

Check: in the else branch a is already −1, so only b varies (2 values); in the then branch all four pairs a ∈ {3, −1}, b ∈ {5, 10} are reachable (4 values); 6 values in total.

X6-42 Can both threads read 0?

Shared `x = 0, y = 0;` T1: `x = 1; r1 = y;` T2: `y = 1; r2 = x;` Which results (r1, r2) are possible? Can both be 0?

(0, 1), (1, 0) and (1, 1) are possible; (0, 0) is impossible when each thread's order is preserved.

- (0, 1): T1 runs entirely before T2. (1, 0): T2 runs entirely before T1. (1, 1): both writes happen before both reads (e.g., `x = 1; y = 1; r1 = y; r2 = x;`).
- (0, 0) would need `r1 = y` before `y = 1` **and** `r2 = x` before `x = 1`; with program order this forms the cycle x = 1 → r1 = y → y = 1 → r2 = x → x = 1, which cannot happen.
- On modern CPUs that **reorder** a write after a later independent read, (0, 0) **can** occur: the same effect that breaks Peterson's solution. A **memory barrier** between the write and the read prevents it.

X6-43 Possible outputs with two semaphores

Semaphores `s = 0, t = 0`. P1: `print("A"); signal(s); wait(t); print("C");` P2: `print("B"); wait(s); signal(t); print("D");` List all possible outputs.

ABCD, ABDC, BACD, BADC: four outputs, and the program never deadlocks.

- Program order: **A before C** and **B before D**.
- P2 passes `wait(s)` only after P1's `signal(s)`, which follows A, so **A comes before D**. P1 passes `wait(t)` only after P2's `signal(t)`, which follows B, so **B comes before C**.
- Hence A and B (in either order) both precede C and D (in either order): 2 × 2 = 4 outputs.
- Method: write the program-order constraints, add a "signal before the matching wait" constraint for each semaphore, list every order that satisfies them, and check whether some process can block forever.

X6-44 Counting permits: what gets printed

Semaphores `A = 2, B = 0`. P1: `while (true) { wait(A); print("x"); signal(B); }` P2: `while (true) { wait(B); wait(B); print("y"); signal(A); }` What is printed?

Always exactly `xxyx`; then both processes block forever (deadlock).

- A's 2 initial permits let P1 print **x x** (A = 0, B = 2).
- P2 needs **two** B permits per y: it prints **y** (B = 0) and returns **one** A permit, so P1 prints one more **x** (A = 0, B = 1).
- Now P2 takes the single B permit and blocks on its second `wait(B)`, while P1 blocks on `wait(A)`; nobody can signal, so the processes are deadlocked. In total, 3 x's and 1 y are printed.
- Method: **count permits (tokens)**; every print consumes permits, so trace the semaphore values until every process is blocked.

X6-45 Misusing wait() and signal() around a CS

`mutex` is a semaphore initialized to 1. What goes wrong if a process (a) swaps the calls: `signal(mutex); CS; wait(mutex);` (b) writes `wait(mutex); CS; wait(mutex);` (c) omits `wait(mutex)`, (d) omits `signal(mutex)`, or (e) calls `signal(mutex)` twice?

- (a) **Mutual exclusion violated**: `signal()` first raises mutex to 2, so **several processes may be executing in their critical sections simultaneously**.
- (b) **Deadlock**: the process **permanently blocks on the second `wait()`** (mutex is 0), and every other process then blocks on its first `wait()`.
- (c) **Mutual exclusion violated**: the process enters its critical section without permission.
- (d) The lock is never released, so **all other processes block forever** (no progress).
- (e) mutex becomes 2, so later **two processes can be inside together** (mutual exclusion violated).
- Rule: **too many signals or too few waits violate mutual exclusion; too many waits or too few signals make processes block forever.**

X6-46 Bounded buffer: wrong order of wait()

In the bounded-buffer producer, what happens if `wait(empty); wait(mutex);` is changed to `wait(mutex); wait(empty);`? Is swapping `signal(mutex); signal(full);` harmful too?

- **Deadlock is possible.** When the buffer is **full** (empty = 0), the producer acquires `mutex` and then **blocks on `wait(empty)` while holding `mutex`**; the consumer then blocks on `wait(mutex)`, so it can never remove an item and call `signal(empty)`. Both wait forever. (Symmetrically, a consumer doing `wait(mutex); wait(full);` on an empty buffer deadlocks.)
- Rule: **wait on the counting semaphore (empty / full) first, then on mutex**; never block while holding the mutex.
- Swapping the two **signal()** calls is **harmless**: `signal()` never blocks, so correctness is not affected.

X6-47 Bounded-buffer solution (Ch7 preview / Lab 6)

Give a semaphore solution to the bounded-buffer (producer–consumer) problem with n buffers, and justify each initial value.

```
semaphore mutex = 1;   /* mutual exclusion for the buffer pool */
semaphore empty = n;   /* number of empty buffers */
semaphore full  = 0;   /* number of full buffers (items) */

/* producer */
while (true) {
    /* produce an item in next_produced */
    wait(empty);
    wait(mutex);
    /* add next_produced to the buffer */
    signal(mutex);
    signal(full);
}
/* consumer */
while (true) {
    wait(full);
    wait(mutex);
    /* remove an item from the buffer to next_consumed */
    signal(mutex);
    signal(empty);
    /* consume the item in next_consumed */
}
```

- **mutex = 1**: only one process accesses the buffer pool at a time.
- **empty = n**: all n buffers are empty at the start; the producer **blocks when the buffer is full** (empty = 0).
- **full = 0**: there are no items at the start; the consumer **blocks when the buffer is empty** (full = 0).

X6-48 First readers–writers solution (Ch7 preview / Lab 6)

Give the semaphore solution to the first readers–writers problem. Why can writers starve? If a writer is writing and n readers arrive, where do the readers wait?

```
semaphore rw_mutex = 1;  /* used by writers and the first/last reader */
semaphore mutex = 1;     /* protects read_count */
int read_count = 0;      /* readers reading or waiting to read */

/* writer */
wait(rw_mutex);
    /* writing is performed */
signal(rw_mutex);

/* reader */
wait(mutex);
read_count++;
if (read_count == 1) wait(rw_mutex);    /* first reader locks out writers */
signal(mutex);
    /* reading is performed */
wait(mutex);
read_count--;
if (read_count == 0) signal(rw_mutex);  /* last reader lets writers in */
signal(mutex);
```

- **Readers share** the object (only the first reader waits on `rw_mutex`); a **writer has exclusive access** (no other writer and no reader).
- **No reader is kept waiting unless a writer has already gained access**, so readers have priority: a **continuous stream of readers can starve writers** (and delay updates of the object).
- If a writer is writing and n readers arrive, the **first reader waits on `rw_mutex`** (while holding `mutex`), and the other **n − 1 readers wait on `mutex`**.

X6-49 Dining philosophers: deadlock and remedies (Ch7 preview)

Each chopstick is a semaphore initialized to 1, and philosopher i executes `wait(chopstick[i]); wait(chopstick[(i + 1) % 5]); eat; signal(chopstick[i]); signal(chopstick[(i + 1) % 5]);`. How can this deadlock, and how can deadlock be prevented?

- **Deadlock**: if all five philosophers become hungry at the same time and **each picks up its left chopstick**, all chopsticks are taken and **each waits forever for its right chopstick**.
- Remedies: (1) allow **at most four philosophers** at the table at the same time; (2) let a philosopher pick up the chopsticks **only if both are available** (picking them up in a critical section); (3) use an **asymmetric** solution: an **odd** philosopher picks up the left chopstick first, an **even** one the right chopstick first.
- A deadlock-free solution does **not** necessarily prevent **starvation**.

X6-50 POSIX condition variables (Lab 6)

Why does `pthread_cond_wait()` take a mutex as its second argument, and why is it called inside a `while` loop?

```
/* waiting thread */                    /* signalling
thread */
pthread_mutex_lock(&mutex);
pthread_mutex_lock(&mutex);
while (!ready)                          ready = 1;
    pthread_cond_wait(&cond, &mutex);
pthread_cond_signal(&cond);
pthread_mutex_unlock(&mutex);
pthread_mutex_unlock(&mutex);
```

- A condition variable is always used with a **mutex that protects the condition** (`ready`), and the mutex must be **held** when calling wait or signal.
- `pthread_cond_wait()` **atomically releases the mutex and puts the caller to sleep**; otherwise no other thread could acquire the mutex to change `ready` and signal. **Before returning, it re-acquires the mutex.**
- The **while** loop re-checks the condition after waking, because the condition may have changed again before the mutex was re-acquired (or the wakeup was spurious). The condition must live in a shared variable because `pthread_cond_signal()` with no waiting thread has **no effect** (unlike `sem_post()`, which is remembered). The waiting thread **sleeps instead of busy waiting**.

X6-TF True / False quick verdicts

Statement	T/F	Why (≤ 12 words)
A race condition requires two processes to access shared data at exactly the same instant.	**F**	Concurrent (interleaved) access is enough; not necessarily simultaneous.
`counter++` can compile into several machine instructions that may be interleaved.	**T**	Load, add, store; another thread can run in between.
Progress allows a process in its remainder section to block others from entering.	**F**	Only processes wishing to enter take part in the decision.
Bounded waiting bounds how many times others may enter before a waiting process.	**T**	Definition of bounded waiting **[6.11]**.
A correct CS solution may assume all processes run at the same speed.	**F**	Only nonzero speed; no assumption about relative speed.
Mutual exclusion is a safety property; progress and bounded waiting are liveness properties.	**T**	No wrong results vs. no pointless blocking or starvation.
A non-preemptive kernel is essentially free of race conditions in kernel mode on one CPU.	**T**	Runs until it exits kernel mode, blocks, or yields.
A preemptive kernel is free from race conditions on kernel data.	**F**	A kernel-mode process can be preempted mid-update.
Disabling interrupts is an efficient CS solution on multiprocessor systems.	**F**	Must message all CPUs; slow, and other CPUs keep running.
Peterson's solution works for any number of processes.	**F**	Restricted to two processes.
Peterson's solution assumes that load and store instructions are atomic.	**T**	Otherwise the correctness proof does not hold.
Peterson's solution may fail on CPUs that reorder independent reads and writes.	**T**	Needs memory barriers or sequentially consistent atomics.
In Peterson's solution, a waiting process enters after at most one entry by the other.	**T**	Bounded waiting with bound 1.
`test_and_set()` returns the new value of its target.	**F**	Returns the old value and sets the target TRUE.
Two `test_and_set()` calls on different cores may interleave their steps.	**F**	Atomic: executed sequentially in some arbitrary order.
`compare_and_swap()` always stores `new_value` into `*value`.	**F**	Only if `*value == expected`; always returns the old value.
In the bounded-waiting TAS algorithm, any waiting process enters within n − 1 turns.	**T**	The exit section scans `waiting[]` in cyclic order.
Executing `waiting[j] = false` in the bounded-waiting algorithm sets `lock` to FALSE.	**F**	Hands the lock directly to Pj; `lock` stays TRUE.
An atomic variable can make an update of a whole linked list atomic.	**F**	Only single basic variables such as integers and Booleans.
Calls to a mutex lock's `acquire()` and `release()` must be atomic.	**T**	Otherwise two threads might both acquire the lock.
A spinlock is a good choice on a single-processor system.	**F**	The lock holder cannot run while the waiter spins.
Apart from initialization, a semaphore is accessed only through `wait()` and `signal()`.	**T**	Two atomic operations **[6.24]**.
P() corresponds to `signal()` and V() to `wait()`.	**F**	P (proberen) = wait; V (verhogen) = signal.
A binary semaphore is exactly equivalent to a mutex lock.	**F**	It can also enforce ordering when initialized to 0.
A counting semaphore initialized to k lets at most k processes pass `wait()` before any `signal()`.	**T**	Initial value = number of available instances.
Under the busy-waiting definition, a semaphore's value can become negative.	**F**	Only the waiting-queue implementation goes negative.
`wakeup(P)` moves P from the semaphore's waiting queue to the ready queue.	**T**	P runs later, when the scheduler selects it.
`signal()` on a semaphore wakes up all processes waiting on it.	**F**	It removes at most one process from the queue.
Removing processes from a semaphore queue in LIFO order can cause starvation.	**T**	An early waiter may never be removed **[6.28]**.
A deadlock-free solution is always starvation-free.	**F**	One process can starve while others keep progressing.
The bounded-buffer semaphores start as mutex = 1, empty = n, full = 0.	**T**	n empty slots and no items initially.
In the first readers–writers solution, writers may starve.	**T**	Readers have priority while readers keep arriving.
`pthread_cond_wait()` releases the mutex while the caller sleeps.	**T**	It re-acquires the mutex before returning.
Strict alternation using only `turn` satisfies progress.	**F**	A process in its remainder section blocks the other.

XI · Cross-chapter traces（綜合「發生咩事」題）

XI-01 read() system call: user → kernel → DMA → interrupt → return

Describe what happens, step by step, from the moment a process calls `read(fd, buf, n)` on a disk file until it gets the data.

1. The program calls the **C library wrapper** `read()` (API). The wrapper puts the **system-call number and parameters in registers** and executes a **trap** (software interrupt) → **mode bit set to kernel mode**.
2. The kernel's system-call handler looks up the **system-call table**, checks the parameters and calls the file system / **device driver**. If the block is not already in memory, the driver programs the **disk controller for a DMA transfer** (source, destination address in memory, count).
3. The process **cannot continue** → state **running** → **waiting**, placed on the device's **wait queue**; the **CPU scheduler** picks another ready process (**context switch**: save the caller's state in its **PCB**, load the next one).
4. The **DMA controller transfers the whole block directly into memory** without the CPU (CPU only sets up the transfer), then raises **one interrupt** when the block is done.
5. The CPU saves the current context and jumps (via the **interrupt vector**) to the **interrupt service routine**, which marks the I/O complete and moves the process **waiting** → **ready**.
6. When the scheduler **dispatches** the process again, the kernel copies the data into `buf`, sets the **return value = number of bytes read** (0 = EOF, −1 = error) and **returns to user mode** (mode bit = user).

XI-02 Timer interrupt ends a time quantum (RR)

What happens when the time quantum of the running process expires under round-robin scheduling?

1. The **timer** (loaded by the OS with a **privileged instruction**) counts down and raises a **timer interrupt** at the end of the quantum.
2. The CPU switches to **kernel mode** and jumps to the timer **interrupt handler**.
3. The OS **saves the running process's context (PC, registers, state) in its PCB**, changes its state **running** → **ready** and puts it at the **tail of the ready queue**.
4. The **short-term (CPU) scheduler** selects the next process (RR: the **head** of the ready queue).
5. The **dispatcher** restores that process's context from its PCB, **switches to user mode** and **jumps to the saved PC**. The time for steps 3–5 is **dispatch latency / context-switch overhead**: no useful work is done, which is why q must be large relative to the switch time.

XI-03 How a shell runs `ls -l > out.txt` and `ls | wc -l`

Explain which system calls a shell uses to execute `ls -l > out.txt` and `ls | wc -l`.

- **`ls -l > out.txt`**: the shell parses the line → **`fork()`** → in the **child**: `fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644)`, **`dup2(fd, 1)`** (stdout now refers to the file), `close(fd)`, **`execvp("ls", argv)`** (the redirection survives exec) → the **parent (shell)** calls **`waitpid()`** and then prints the next prompt. The shell's own stdout is untouched because only the child redirected.
- **`ls | wc -l`**: the shell calls **`pipe(fd)`** **before forking**; child 1 does `dup2(fd[1], 1)` and execs `ls`; child 2 does `dup2(fd[0], 0)` and execs `wc -l`. **Every process closes the pipe ends it does not use**. The shell closes **both** ends (otherwise `wc` never gets EOF) and **waits for both** children.
- **Built-ins** (`cd`, `exit`) run **inside the shell process itself** without fork: `cd` uses **`chdir()`**, because a child's directory change would be lost when the child exits.

XI-04 Life cycle of a process through all states

Describe the life cycle of a process using the process states, naming the event that causes each transition.

- **new**: created by `fork()`; the OS allocates a **PCB** and memory.
- **new** → **ready**: **admitted** (long-term scheduler), placed in the **ready queue**.
- **ready** → **running**: **dispatched** by the short-term scheduler.
- **running** → **ready**: **interrupt** / **time quantum expires** / preempted by a higher-priority process.
- **running** → **waiting**: requests **I/O** or waits for an **event** (e.g. `wait()`, `read()` on an empty pipe).
- **waiting** → **ready**: **I/O completion** (interrupt) or the event occurs. A process never goes waiting → running directly.
- **running** → **terminated**: calls **`exit()`** (or is killed). It stays a **zombie** (PCB + exit status kept) until the parent calls **`wait()`**, then its PCB is removed. Only **one process is running per core** at any instant.

XI-05 From source file to running process (compiler, linker, loader, exec)

Explain how a C source file `prog.c` becomes a running process.

1. **Compiler**: `gcc -c prog.c` → **relocatable object file** `prog.o` (machine code with unresolved addresses/symbols).
2. **Linker**: combines object files and **libraries** into one **executable** (e.g. ELF); static libraries are copied in, **dynamically linked libraries (DLLs / shared libraries)** are only referenced and linked at **load or run time** (saves memory and disk; one shared copy).
3. **Launch**: typing `./prog` makes the shell **`fork()`** a child that calls **`exec()`**. The **loader** brings the executable into memory (text, data, heap, stack), **relocates** addresses and loads needed DLLs.
4. The OS sets up the **PCB**, puts the process in the **ready queue**, and the scheduler dispatches it; execution starts at the program's **entry point**, which calls `main()`.

XI-06 What happens when you press Ctrl-C

What happens when the user presses Ctrl-C while a program runs in the shell, and how can a program change this?

- The terminal driver sends the signal **SIGINT (signal number 2)** to the **foreground process**. A signal is a software notification of an event; it is **asynchronous** (generated outside the receiving process).
- **Default action** for SIGINT: **terminate** the process.
- A program can install its own **signal handler** with **`signal(SIGINT, handler)`** (PA1 prints a message and exits), or ignore it with `signal(SIGINT, SIG_IGN)`. **SIGKILL (9) and SIGSTOP cannot be caught or ignored.**
- In a multithreaded process the signal is delivered to one thread (or as the OS chooses: to the thread it applies to, every thread, certain threads, or a designated thread).

XC · Copy-ready C programs（Lab / PA coding，已編譯測試）

XC-01 fork + wait + exit status

Write a program in which the child prints its own PID and its parent's PID, and the parent waits for the child and prints the child's exit status.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid < 0) {                          /* fork failed */
        perror("fork");
        exit(1);
    } else if (pid == 0) {                  /* child: fork() returned 0 */
        printf("child: pid=%d parent=%d\n", getpid(), getppid());
        exit(7);                            /* status passed to the parent */
    } else {                                /* parent: fork() returned the child's PID */
        int status;
        waitpid(pid, &status, 0);           /* or wait(&status) */
        if (WIFEXITED(status))
            printf("parent: child %d exited with %d\n", pid, WEXITSTATUS(status));
    }
    return 0;
}
```

- Parent output is always after the child's (it **waits**). Without `wait`, the finished child stays a **zombie** until reaped.

XC-02 N children (fan) + reap all

The parent creates N child processes; child i prints i and terminates; the parent waits for all children before printing "done".

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#define N 4

int main(void) {
    for (int i = 0; i < N; i++) {
        pid_t pid = fork();
        if (pid == 0) {                     /* child i */
            printf("child %d: pid=%d\n", i, getpid());
            exit(0);                        /* MUST exit: otherwise the child continues the loop and forks too */
        }
    }
    while (wait(NULL) > 0)                  /* reap every child (no zombies); wait returns -1 when none left */
        ;
    printf("parent: done\n");
    return 0;
}
```

- Exactly **N + 1** processes. Children print in **any order** (scheduler decides); "done" is always last.

XC-03 Chain of processes

Create a chain P0 → P1 → ... → PN in which each process creates exactly one child and waits for it.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#define N 3

int main(void) {
    for (int i = 0; i < N; i++) {
        pid_t pid = fork();
        if (pid > 0) {                      /* parent: wait for its only child, then stop forking */
            wait(NULL);
            break;
        }                                   /* child: continue the loop and create the next process */
    }
    printf("pid=%d parent=%d\n", getpid(), getppid());
    return 0;
}
```

- **N + 1** processes; the deepest prints first, each parent prints after its child ends (reverse order of creation).

XC-04 fork + execvp a command line

The parent reads a command line (e.g. "ls -l -a"), runs it in a child with `execvp`, and waits for it.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    char line[] = "ls -l -a";
    char *argv[9];                          /* at most 8 arguments + NULL terminator */
    int argc = 0;
    for (char *tok = strtok(line, " "); tok != NULL && argc < 8; tok = strtok(NULL, " "))
        argv[argc++] = tok;
    argv[argc] = NULL;                      /* execvp needs a NULL-terminated array */

    pid_t pid = fork();
    if (pid == 0) {
        execvp(argv[0], argv);              /* replaces the child's image; returns only on error */
        perror("execvp");
        exit(1);
    }
    waitpid(pid, NULL, 0);
    printf("command finished\n");
    return 0;
}
```

- Code after a **successful exec never runs** (the process image is replaced; PID stays the same).
- `execlp("ls", "ls", "-l", "-a", (char *)NULL)` is the list form of the same call.

XC-05 Pipe: parent sends a string to child

Use an ordinary pipe so that the parent sends the string "Greetings" to the child, which prints it.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int fd[2];                              /* fd[0] = read end, fd[1] = write end */
    char buf[100];
    if (pipe(fd) == -1) { perror("pipe"); exit(1); }   /* create BEFORE fork so both share it */
    pid_t pid = fork();
    if (pid == 0) {                         /* child = reader */
        close(fd[1]);                       /* close unused write end */
        ssize_t n = read(fd[0], buf, sizeof(buf) - 1);
        buf[n > 0 ? n : 0] = '\0';
        printf("child read: %s\n", buf);
        close(fd[0]);
        exit(0);
    } else {                                /* parent = writer */
        close(fd[0]);                       /* close unused read end */
        const char *msg = "Greetings";
        write(fd[1], msg, strlen(msg));
        close(fd[1]);                       /* signals EOF to the reader */
        wait(NULL);
    }
    return 0;
}
```

XC-06 Pipe: child computes, parent receives

The child computes 1 + 2 + … + 100 and sends the result to the parent through a pipe; the parent prints it.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int fd[2];
    pipe(fd);
    if (fork() == 0) {                      /* child: writer */
        close(fd[0]);
        int sum = 0;
        for (int i = 1; i <= 100; i++) sum += i;
        write(fd[1], &sum, sizeof(sum));    /* pass the ADDRESS and the SIZE */
        close(fd[1]);
        exit(0);
    }
    close(fd[1]);                           /* parent: reader */
    int result;
    read(fd[0], &result, sizeof(result));   /* blocks until data arrives */
    close(fd[0]);
    wait(NULL);
    printf("sum = %d\n", result);           /* 5050 */
    return 0;
}
```

- A global variable changed in the child is **not** seen by the parent (separate address spaces). That's why IPC (a pipe) is needed.

XC-07 Two pipes: two-way communication

Parent sends "Parent" to the child and the child replies "Child", using two ordinary pipes.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int p2c[2], c2p[2];                     /* parent->child, child->parent */
    char buf[20] = {0};
    pipe(p2c);
    pipe(c2p);
    if (fork() == 0) {                      /* child */
        close(p2c[1]); close(c2p[0]);       /* keep: read p2c, write c2p */
        read(p2c[0], buf, sizeof(buf) - 1);
        printf("child received: %s\n", buf);
        write(c2p[1], "Child", 6);          /* 6 bytes include the '\0' */
        close(p2c[0]); close(c2p[1]);
        exit(0);
    }
    close(p2c[0]); close(c2p[1]);           /* parent keeps: write p2c, read c2p */
    write(p2c[1], "Parent", 7);
    read(c2p[0], buf, sizeof(buf) - 1);
    printf("parent received: %s\n", buf);
    close(p2c[1]); close(c2p[0]);
    wait(NULL);
    return 0;
}
```

- One ordinary pipe is **unidirectional** → two-way needs **two pipes** (a single pipe could read back its own message).

XC-08 Redirection: cmd < in > out

Implement `sort < in.txt > out.txt`: run `sort` in a child with stdin from in.txt and stdout to out.txt.

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    if (fork() == 0) {
        int in  = open("in.txt", O_RDONLY);
        int out = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);   /* 0644: rw-r--r-- */
        if (in < 0 || out < 0) { perror("open"); exit(1); }
        dup2(in, STDIN_FILENO);             /* fd 0 now refers to in.txt   */
        dup2(out, STDOUT_FILENO);           /* fd 1 now refers to out.txt  */
        close(in); close(out);              /* originals no longer needed  */
        execlp("sort", "sort", (char *)NULL);
        perror("execlp");
        exit(1);
    }
    wait(NULL);
    return 0;
}
```

- Equivalent old style: `close(1); dup(out);` (dup returns the **lowest unused fd** = 1). Redirect **in the child, before exec**; the shell's own stdin/stdout stay unchanged.

XC-09 Pipeline: ls | wc -l

Implement the shell command `ls | wc -l` with `pipe`, `fork`, `dup2` and `exec`.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int fd[2];
    pipe(fd);
    if (fork() == 0) {                      /* child 1: ls writes into the pipe */
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]); close(fd[1]);
        execlp("ls", "ls", (char *)NULL);
        exit(1);
    }
    if (fork() == 0) {                      /* child 2: wc reads from the pipe */
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]); close(fd[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        exit(1);
    }
    close(fd[0]); close(fd[1]);             /* parent MUST close both ends, else wc never sees EOF */
    wait(NULL);
    wait(NULL);
    return 0;
}
```

- Every process **closes the pipe ends it does not use**; an open write end anywhere keeps the reader blocked (no EOF).

XC-10 Multi-level pipe (PA1 Feature 5)

Write `run_pipeline(cmds, n)` that runs `cmd0 | cmd1 | … | cmd(n-1)` (n ≤ 8), where `cmds[i]` is a NULL-terminated argv array.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void run_pipeline(char **cmds[], int n) {
    int fds[8][2];                          /* n-1 pipes */
    for (int i = 0; i < n - 1; i++) pipe(fds[i]);
    for (int i = 0; i < n; i++) {
        if (fork() == 0) {
            if (i > 0)     dup2(fds[i - 1][0], STDIN_FILENO);   /* read the previous pipe */
            if (i < n - 1) dup2(fds[i][1], STDOUT_FILENO);      /* write the next pipe    */
            for (int k = 0; k < n - 1; k++) { close(fds[k][0]); close(fds[k][1]); }
            execvp(cmds[i][0], cmds[i]);
            perror("execvp");
            exit(1);
        }
    }
    for (int k = 0; k < n - 1; k++) { close(fds[k][0]); close(fds[k][1]); }   /* parent closes all */
    for (int i = 0; i < n; i++) wait(NULL);                                   /* then waits for all */
}

int main(void) {
    char *c0[] = {"printf", "b\\na\\nc\\n", NULL};
    char *c1[] = {"sort", NULL};
    char *c2[] = {"head", "-n", "2", NULL};
    char **cmds[] = {c0, c1, c2};
    run_pipeline(cmds, 3);                  /* prints a, b */
    return 0;
}
```

- n segments → **n − 1 pipes** and **n children**; the first child keeps its stdin, the last keeps its stdout.

XC-11 Mini shell loop: built-in cd, exit, Ctrl-C

Sketch the main loop of a simple shell (PA1): prompt, read a line, handle `exit` and `cd` as built-ins, run other commands in a child, and print a message on Ctrl-C.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

void on_sigint(int sig) {                   /* SIGINT = 2 */
    printf("\nshell (pid=%d) terminated by Ctrl-C, signal %d\n", getpid(), sig);
    exit(0);
}

int main(void) {
    char line[1024], *argv[9];
    signal(SIGINT, on_sigint);              /* register the handler */
    while (1) {
        printf("$$ 3511> ");
        fflush(stdout);
        if (fgets(line, sizeof(line), stdin) == NULL) break;   /* EOF */
        line[strcspn(line, "\n")] = '\0';
        int argc = 0;
        for (char *t = strtok(line, " "); t != NULL && argc < 8; t = strtok(NULL, " "))
            argv[argc++] = t;
        argv[argc] = NULL;
        if (argc == 0) continue;
        if (strcmp(argv[0], "exit") == 0) break;
        if (strcmp(argv[0], "cd") == 0) {   /* built-in: must change the SHELL's own directory */
            if (argc < 2 || chdir(argv[1]) != 0) perror("cd");
            continue;
        }
        pid_t pid = fork();
        if (pid == 0) {
            execvp(argv[0], argv);
            perror("execvp");
            exit(1);
        }
        waitpid(pid, NULL, 0);              /* foreground: wait before the next prompt */
    }
    return 0;
}
```

- `cd` in a child would change only the **child's** working directory, which disappears when the child exits. So it must be built in, using `chdir()`.
- `system()` / `popen()` are not allowed (they start another shell to do the work).

XC-12 Named pipe (FIFO) writer / reader

Two unrelated processes communicate through a named pipe "/tmp/myfifo": the writer sends a string, the reader prints it.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <unistd.h>
#define FIFO "/tmp/myfifo"

void writer(void) {                         /* program 1 */
    int fd = open(FIFO, O_WRONLY);          /* blocks until a reader opens the FIFO */
    const char *msg = "hello via FIFO";
    write(fd, msg, strlen(msg) + 1);
    close(fd);
}

void reader(void) {                         /* program 2 */
    char buf[100];
    int fd = open(FIFO, O_RDONLY);          /* blocks until a writer opens the FIFO */
    read(fd, buf, sizeof(buf));
    printf("reader got: %s\n", buf);
    close(fd);
}

int main(void) {                            /* demo in one file; normally two separate programs */
    if (access(FIFO, F_OK) == -1) mkfifo(FIFO, 0666);   /* create the FIFO file once */
    if (fork() == 0) { writer(); exit(0); }
    reader();
    wait(NULL);
    unlink(FIFO);                           /* remove the FIFO when finished */
    return 0;
}
```

- A FIFO **appears as a file** in the file system, needs **no parent–child relationship**, and **exists until deleted** (`unlink`). An ordinary pipe exists only while its processes use it.

XC-13 Pthreads: partial sums with N threads

Use N threads to sum an array: thread i sums its own slice, and `main` joins all threads and adds the partial sums.

```
#include <stdio.h>
#include <pthread.h>
#define N 4
#define SIZE 1000

int a[SIZE];
long partial[N];                            /* one slot per thread → no shared write → no race */

void *worker(void *arg) {
    int id = *(int *)arg;
    long s = 0;
    for (int i = id * (SIZE / N); i < (id + 1) * (SIZE / N); i++) s += a[i];
    partial[id] = s;
    return NULL;                            /* or pthread_exit(NULL) */
}

int main(void) {
    pthread_t tid[N];
    int ids[N];
    for (int i = 0; i < SIZE; i++) a[i] = i + 1;
    for (int i = 0; i < N; i++) {
        ids[i] = i;                         /* pass &ids[i], NOT &i (i changes while threads start) */
        pthread_create(&tid[i], NULL, worker, &ids[i]);
    }
    long total = 0;
    for (int i = 0; i < N; i++) {
        pthread_join(tid[i], NULL);         /* wait for thread i */
        total += partial[i];
    }
    printf("total = %ld\n", total);         /* 500500 */
    return 0;
}
```

- Compile: `gcc sum.c -o sum -pthread` (or `-lpthread`). Signature of a thread function: `void *f(void *arg)`.

XC-14 Thread returns a value to main

Create a thread that computes the square of a number passed as its argument and returns the result to `main` through `pthread_join`.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void *square(void *arg) {
    int x = *(int *)arg;
    int *res = malloc(sizeof(int));         /* heap: survives after the thread ends (not a local!) */
    *res = x * x;
    pthread_exit(res);                      /* same as: return res; */
}

int main(void) {
    pthread_t t;
    int x = 7;
    int *res;
    pthread_create(&t, NULL, square, &x);
    pthread_join(t, (void **)&res);         /* receives the thread's return value */
    printf("%d squared = %d\n", x, *res);  /* 49 */
    free(res);
    return 0;
}
```

XC-15 Mutex-protected shared counter

Two threads each increment a shared counter 100000 times. Make the final value always 200000.

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;   /* or pthread_mutex_init(&lock, NULL) */

void *add(void *arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);          /* entry section */
        counter++;                          /* critical section */
        pthread_mutex_unlock(&lock);        /* exit section */
    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, add, NULL);
    pthread_create(&t2, NULL, add, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("counter = %d\n", counter);      /* always 200000 */
    pthread_mutex_destroy(&lock);
    return 0;
}
```

- Without the mutex, `counter++` (load, add, store) **races**, so the result can be anything from 2 to 200000; in practice it is usually less than 200000.

XC-16 Semaphores: force the order T1 → T2 → T3

Three threads each print their name. Use POSIX semaphores so the output is always T1, T2, T3.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t s12, s23;                             /* both start at 0 = "not yet allowed" */

void *t1(void *a) { printf("T1\n"); sem_post(&s12); return NULL; }
void *t2(void *a) { sem_wait(&s12); printf("T2\n"); sem_post(&s23); return NULL; }
void *t3(void *a) { sem_wait(&s23); printf("T3\n"); return NULL; }

int main(void) {
    pthread_t a, b, c;
    sem_init(&s12, 0, 0);                   /* pshared = 0: shared by threads of this process */
    sem_init(&s23, 0, 0);
    pthread_create(&c, NULL, t3, NULL);     /* creation order does not matter */
    pthread_create(&b, NULL, t2, NULL);
    pthread_create(&a, NULL, t1, NULL);
    pthread_join(a, NULL); pthread_join(b, NULL); pthread_join(c, NULL);
    sem_destroy(&s12); sem_destroy(&s23);
    return 0;
}
```

- Rule: for "A before B", use a semaphore initialised to **0**; **A signals (`sem_post`) after its work; B waits (`sem_wait`) before its work.**

XC-17 Bounded buffer (producer–consumer) with semaphores

Implement a producer and a consumer sharing a bounded buffer of size 5, using `empty` / `full` counting semaphores and a mutex.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#define BUF 5
#define ITEMS 20

int buffer[BUF], in = 0, out = 0;
sem_t empty, full;                          /* empty = free slots (init BUF), full = items (init 0) */
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&empty);                   /* wait for a free slot   */
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        in = (in + 1) % BUF;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);                    /* one more item          */
    }
    return NULL;
}

void *consumer(void *arg) {
    int sum = 0;
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&full);                    /* wait for an item       */
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        out = (out + 1) % BUF;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);                   /* one more free slot     */
        sum += item;
    }
    printf("consumed sum = %d\n", sum);     /* 210 */
    return NULL;
}

int main(void) {
    pthread_t p, c;
    sem_init(&empty, 0, BUF);
    sem_init(&full, 0, 0);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty); sem_destroy(&full);
    return 0;
}
```

- **Order matters:** `wait(empty)` / `wait(full)` must come **before** locking the mutex. Locking first can **deadlock** (holding the mutex while blocked on a full/empty buffer).

XC-18 First readers–writers (semaphores)

Give a solution where many readers may read together but a writer needs exclusive access (readers have priority).

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t rw_mutex;                             /* init 1: writers' exclusive access / held by the reader group */
sem_t mutex;                                /* init 1: protects read_count */
int read_count = 0, data = 0;

void *writer(void *arg) {
    sem_wait(&rw_mutex);
    data++;                                 /* writing */
    sem_post(&rw_mutex);
    return NULL;
}

void *reader(void *arg) {
    sem_wait(&mutex);
    read_count++;
    if (read_count == 1) sem_wait(&rw_mutex);   /* first reader locks out writers */
    sem_post(&mutex);
    printf("read %d\n", data);              /* reading (many readers at once) */

    sem_wait(&mutex);
    read_count--;
    if (read_count == 0) sem_post(&rw_mutex);   /* last reader lets writers in */
    sem_post(&mutex);
    return NULL;
}

int main(void) {
    pthread_t r[3], w;
    sem_init(&rw_mutex, 0, 1);
    sem_init(&mutex, 0, 1);
    pthread_create(&w, NULL, writer, NULL);
    for (int i = 0; i < 3; i++) pthread_create(&r[i], NULL, reader, NULL);
    pthread_join(w, NULL);
    for (int i = 0; i < 3; i++) pthread_join(r[i], NULL);
    return 0;
}
```

- Writers may **starve** (readers keep arriving). Lab 6 wrote the same logic with two `pthread_mutex_t` (`mutex`, `rw_mutex`).

XC-19 Zombie and orphan demos

Write short programs that create (a) a zombie process and (b) an orphan process, and explain what happens.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    /* (a) zombie: child exits, parent is alive but has not called wait() */
    if (fork() == 0) exit(0);
    sleep(1);                               /* during this time the child is a zombie (ps shows <defunct>) */

    /* (b) orphan: parent exits first, child is still running */
    if (fork() == 0) {
        sleep(2);
        printf("orphan: my parent is now %d\n", getppid());   /* re-parented to init / systemd (or a subreaper) */
        exit(0);
    }
    return 0;                               /* parent ends; the zombie from (a) is reaped by init */
}
```

- **Zombie** = terminated, but its PCB / exit status is kept until the parent calls `wait()`. **Orphan** = parent ended without waiting, so **init (systemd) adopts it** and reaps it.

1 Ch1 Introduction 引言

📌 **考試範圍提示**：Ch1 主要出 **MC（2–4 題）** 同 **short question（定義題）**。最常考：**caching / locality / average access time**、**von Neumann**、**storage hierarchy**、**SMP / multicore / NUMA**、**multiprogramming vs timesharing**、**virtualization / cloud**。 ⚠️ 留意：今個 term（Fall 2026）嘅 slides 將 **interrupt / DMA / dual-mode** 放咗喺 **Ch3 slides 3.21–3.31**（舊 paper 寫 "1.40" 係舊版 slide number）。呢部分喺 **Ch3 notes §0**，但 MC 一樣會考。

1. What is an Operating System?（OS 係乜？） [1.20–1.24]

1.1 Computer system 四個組成部分 [1.20]

Component	解釋
Users	人、機器、其他電腦 / devices
Application programs	定義點樣用 system resources 去解決問題（editors, compilers, web browsers, database, games）
Operating system	**controls and coordinates use of computing resources among various applications and users**
Hardware	basic computing resources：**CPU, memory, I/O devices**

1.2 OS 嘅定義 [1.21, 1.23]

- **OS is a system software that acts as an intermediary between users (or application software) and computer hardware.**（OS 係用戶/應用程式同硬件之間嘅中介）
- OS **hides the complicated details of the hardware** 並提供 **simple and easy-to-use interface**（例如你處理 photos、web pages、files，而唔係 bytes 同 blocks）→ 呢個叫 **abstraction**。
- **There is no universally accepted definition of OS.** "Everything a vendor ships when you order an operating system" 只係一個 approximation。
- 兩個角色（必背）：
    - **OS is a resource allocator** — manages all resources (hardware and software); **decides between conflicting requests for efficient and fair resource use**.
    - **OS is a control program** — **controls execution of programs to prevent errors and improper use of the computer**.
- 一句總結：**"OS manages and controls hardware and helps to facilitate programs to run on computers."**

1.3 OS 嘅兩大目標（Chapter summary）

1. **User convenience** — provides an environment for users to execute programs in a **convenient, safe, protected and efficient** manner.
2. **Resource allocation** — allocates resources (hardware: CPU, memory; software: signals, locks) **in a fair and efficient manner**.

📝 **HW1 MC3 例子**：「幾個 job 喺 shared computer 等運行，OS 決定邊個 job 用 CPU、memory 點分配」→ **OS acts as a resource allocator**。

1.4 唔同環境嘅 OS [1.22]

環境	重點
Shared computers（mainframe / minicomputer）	要令所有 user 滿意 → **performance vs. fairness**
Individual systems（workstations）	dedicated resources，重 **performance** 多過 fairness
Mobile devices	**resource constrained**；touch screen、voice（Siri）；optimized for **usability and battery life**
Embedded systems	好少或無 UI；OS **pre-installed in ROM**（冷氣機、汽車、太空船）
Real-time systems	**strict time constraints**

1.5 常見名詞 [1.24]

- **Kernel** — **"the one program running at all times on the computer"**（核心功能）。
- **Middleware** — a set of software frameworks providing **additional services to application developers**（databases, multimedia, graphics）；mobile OS（iOS, Android）好常見。
- **System programs** — ships with the OS but **not part of the kernel**（例如 compilers）。
- **Application programs** — 同 OS 無關嘅程式（App Store 啲 app）。
- **OS = always-running kernel + middleware frameworks + system programs.**

2. Computer System Organization（電腦系統組織） [1.25–1.27]

- **One or more CPU cores** 同 **device controllers** 經 **common bus** 連接，**share memory**。[1.25]
- **CPUs and devices execute concurrently**，**compete for memory cycles through the shared bus**。

- 每個 **device controller** 管一類 device，有自己嘅 **local buffer**；OS 用 **device driver** 同 controller 溝通（device driver = kernel module that controls a device）。
- [1.27] 圖：processor + cache ↔ bridge/memory controller ↔ memory；**PCI bus** 連 graphics controller、SCSI controller、IDE disk controller、expansion bus interface（keyboard, parallel/serial port）。

2.1 von Neumann Architecture [1.26] ⭐（MC 常客）

- **Stored-program architecture：instructions and data are stored in the same memory**（同一個 memory space，用 address 存取）。
- **CPU** contains an **ALU (arithmetic logic unit)** and **processor registers** — **PC (Program Counter)**, **AC (Accumulator)**, **MAR (Memory Address Register)**, **MDR (Memory Data Register)**。
- **Control unit** — **IR (Instruction Register)** and **PC**。
- **Memory** stores data and instructions（along with caches）；**external mass storage** = secondary storage；**input and output mechanisms**。
- **Instruction execution steps（instruction–execution cycle）**：
  1. **Fetch instruction**（from memory/cache → 放入 IR）
  2. **Decode instruction**
  3. **Fetch data**（operands → data registers）
  4. **Execute instruction**
  5. **Write back**（if any，結果寫返 memory/cache）
- **Programs and data must be brought into memory before execution.**
- **MMU (memory management unit) sees only a stream of memory addresses** — 佢唔知 address 點產生、亦唔知係 instruction 定 data。

> ⚠️ **陷阱：**
> - "Instructions and data are stored in **separate** physical memories" ✗（嗰個係 Harvard architecture）。
> - "PC stored in external mass storage" ✗；"CPU has no ALU" ✗。
> - Sp21 MC：「processing unit contains a/an ___ and processor registers」→ **ALU**（唔係 "all of the mentioned"，因為 IR/PC 屬 control unit）。

---

3. Storage（儲存） [1.28–1.35]

3.1 單位 [1.28]

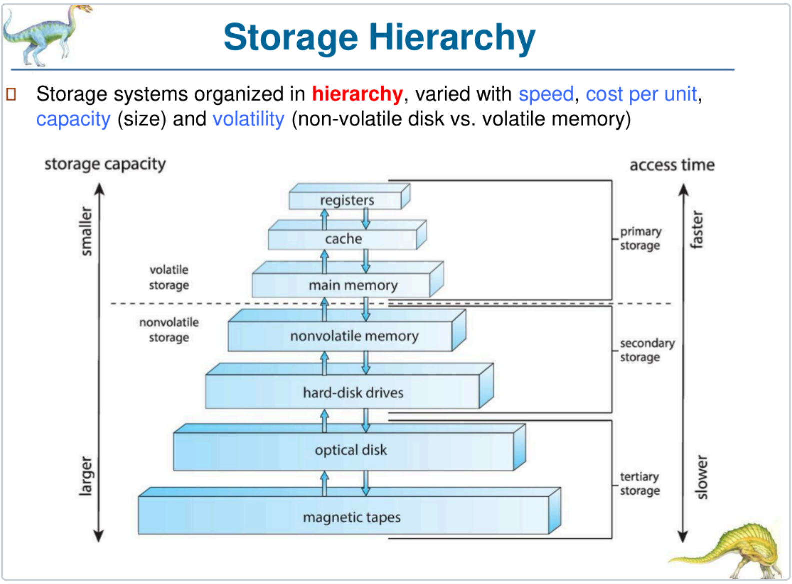
- **bit**（0/1）；**byte = 8 bits**（大部分電腦最細嘅 convenient chunk，冇 move 1 bit 嘅 instruction，但有 move 1 byte）。
- **word** = 一個 architecture 嘅 **native unit of data**（64-bit registers → 64-bit (8-byte) word）。
- **KB = 2^{10} = 1,024 bytes；MB = 1024^2 bytes；GB = 1024^3；TB = 1024^4；PB = 1024^5**。
- 廠商通常 round off（MB ≈ 1 million bytes）；**networking measured in bits**（Mb/s, Gb/s）。

3.2 Storage Hierarchy [1.29] ⭐

按 **speed, cost per unit, capacity (size), volatility** 分層：

```
          registers          ┐
          cache              │ primary storage   (volatile)
          main memory        ┘
- - - - - - - - - - - - - - - - - - - - - - - - - - - -
          nonvolatile memory (NVM) ┐ secondary storage (nonvolatile)
          hard-disk drives (HDD)   ┘
          optical disk       ┐ tertiary storage
          magnetic tapes     ┘
 上面：smaller, faster, more expensive per byte
 下面：larger, slower, cheaper per byte
```

- 越近 CPU → **越快、越細、每 byte 越貴**；越落 → **cost per byte 減、access time 增、capacity 增**。



Storage Hierarchy

- Storage systems organized in **hierarchy**, varied with speed, cost per unit, capacity (size) and volatility (non-volatile disk vs. volatile memory)

Storage hierarchy **[1.29]**

3.3 Main Memory [1.30]

- **Main memory is the only large storage media that the CPU can access directly.**（CPU 係 **indirectly** 存取 secondary storage）
- **Volatile**；通常係 **DRAM (Dynamic Random-Access Memory)**。
- **load / store** instructions to specific addresses；**byte addressable — each address refers to one byte**。⭐（S26 MC3 答案）
- 開機第一個程式 **bootstrap program** 存喺 **EEPROM**（firmware）。

3.4 Secondary Storage [1.31]

- **Extension of main memory providing large non-volatile storage capacity**。
- 最常見：**HDDs** 同 **NVM devices**（例如 SSD）。
- **Mechanical**（HDD, optical disk, magnetic tape）vs **Electrical**（flash, SSD, FRAM, NRAM → 統稱 **NVM**）。
- **Mechanical: larger and less expensive per byte. Electrical: more costly, smaller, more reliable, and faster.**

3.5 Characteristics of storage [1.34]（睇表記「Managed by」）

Level	1 registers	2 cache	3 main memory	4 SSD	5 magnetic disk
Typical size	< 1 KB	< 16 MB	< 64 GB	< 1 TB	< 10 TB
Technology	custom memory, multiple ports CMOS	on/off-chip CMOS SRAM	CMOS SRAM（slide 原文）	flash memory	magnetic disk
Access time (ns)	0.25–0.5	0.5–25	80–250	25,000–50,000	5,000,000
Bandwidth (MB/s)	20,000–100,000	5,000–10,000	1,000–5,000	500	20–150
Managed by	**compiler**	**hardware**	**operating system**	**OS**	**OS**
Backed by	cache	main memory	disk	disk	disk or tape

- **Movement between levels of storage hierarchy can be explicit or implicit.**（例如 cache ↔ memory 由 hardware 隱式處理；memory ↔ disk 由 OS 顯式處理）

3.6 Range of Timescales [1.35]（Jeff Dean "Numbers Everyone Should Know"）

L1 cache 0.5 ns；branch mispredict 5 ns；L2 cache 7 ns；mutex lock/unlock 25 ns；**main memory reference 100 ns**；compress 1KB 3 µs；send 2KB over 1 Gbps 20 µs；read 1MB from memory 250 µs；round trip in datacenter 500 µs；**disk seek 10 ms**；read 1MB from disk 20 ms；packet CA→Netherlands→CA 150 ms。

4. Caching ⭐⭐（必考 short question）[1.32–1.33]

4.1 Fundamental idea

> 📝 **Model answer**：*The fundamental idea of caching is to temporarily copy a subset of information from a slower but larger storage to a faster but smaller storage, so that frequently used data can be accessed more quickly.*（make frequently used case faster, less frequent case less dominant）

- Caching **performed at many levels**：memory、address translation、file blocks、file names、file directories、network routes...

- 存取時先 check cache：
  - **Hit** — information is in the cache → used directly (fast)
  - **Miss** — not in cache → **data copied from slower storage to cache** and used there
- **Cache is usually much smaller than the storage being cached.**
- **Cache management：cache size** 同 **replacement policy**。
- **Major criterion: cache hit ratio**（percentage of content found in cache）。

4.2 Average access time（公式必背）

> 🧮 **Average Access Time（AMAT）**
>
> $$\text{AMAT} = (\text{Hit Ratio} \times \text{Hit Time}) + (\text{Miss Ratio} \times \text{Miss Time})$$

- $\text{Miss Ratio} = 1 - \text{Hit Ratio}$。
- **HW1 例**：hit ratio 90%、hit time 2 ns、miss time 50 ns → $0.9 \times 2 + 0.1 \times 50 = 1.8 + 5 =$ **6.8 ns**。
- 如果題目畀 **miss penalty**（額外時間）而唔係 miss time：$T_{\text{miss}} = T_{\text{hit}} + \text{penalty}$ → $\text{AMAT} = T_{\text{hit}} + (1-h) \times \text{penalty}$（兩條公式結果一樣）。

> 📝 **S26 Q2.2 Model answer（high hit ratio 夠唔夠？）**：*No. Cache performance is not judged only by cache hit ratio. Although hit ratio is a major criterion, the overall performance should be evaluated using the average access time, which depends on both the hit time and the miss time (miss penalty). A cache with a high hit ratio but a long hit time or a huge miss penalty can still perform poorly.*

4.3 Locality — why caching works ⭐

- **Temporal locality (locality in time)** — **recently accessed items are likely to be accessed again** in the near future.
  - 例：loop counter `i`、loop 入面重複執行嘅 instructions、成日 call 嘅 function。
- **Spatial locality (locality in space)** — **items near (contiguous to) the recently accessed items are likely to be accessed shortly**（data 同 program 都係）。
  - 例：順序讀 array `a[0], a[1], a[2]…`；順序執行 instructions。
- **Without access locality（e.g. all items accessed with equal probability）, cache would never work!**
- F20 MC1：**"Locality is essential to __" → Cache**。

---

5. Processor Systems（處理器系統） [1.36–1.41]

5.1 Single-Processor Systems [1.36]

- 一個 CPU，**single processing core**；core 執行 instructions，有 registers 存 data。
- 仲有 **special-purpose processors**（device-specific：disk controller、GPU）→ 跑 **limited instruction set，do not execute instructions from user processes**。

5.2 Multiprocessor Systems [1.37] ⭐

- 而家由 mobile 到 server 都係 multiprocessor 主導。
- **Speed-up ratio with N processors is less than N** — 因為 **overhead, e.g., contention for shared resources (bus or memory)**。
- **三大優點（必背）**：
  1. **Increased throughput** — more computing capability
  2. **Economy of scale** — share other devices（I/O devices, power supplies, housings, peripherals）→ 平過多部單 CPU 機
  3. **Increased reliability** — **graceful degradation or fault tolerance**
- 缺點（summary）：hardware design 同 software（**parallel programming**）都複雜好多。
- **兩類**：
  - **Asymmetric multiprocessing (AMP)** — **master–slave**：**master** assigns specific tasks to slaves and **handles I/O**。（Ch5：master 可以變 **bottleneck**）
  - **Symmetric multiprocessing (SMP)** — **each processor performs all tasks, including OS functions and user processes**；I/O 可以由任何 processor 處理。

5.3 SMP [1.38]

- **Each CPU has its own set of registers and a private (local) cache; all processors share physical memory through the system bus.**

5.4 Multicore [1.39] ⭐

- **Multiple computing cores reside on a single physical chip.**
- 優點：**faster on-chip communication than between-chip communication**；**uses significantly less power**（mobile/laptop 重要）。

> 📝 **F19 Q2.5**：*Multicore SMP systems are faster and consume less power than systems in which each CPU has its own physical chip (on-chip communication is faster).*

5.5 NUMA (Non-Uniform Memory Access) [1.40] ⭐

- 問題：加更多 CPU 未必 scale，因為 **contention for the system bus → bottleneck**。
- 解決：**each CPU (or group of CPUs) has its own local memory accessed via a small, fast local bus**；CPUs connected by a **shared system interconnect**；**all CPUs share one physical memory address space**。
- **Drawback: increased latency when a CPU must access remote memory across the system interconnect** → 影響 **scheduling and memory management**（Ch5 processor affinity）。

> ⚠️ HW1 MC4：「In NUMA, every CPU accesses every part of memory with exactly the same latency」→ **False**（Non-**uniform** 就係唔同 latency）。

5.6 名詞定義 [1.41]（MC 易混）

Term	Definition
CPU	The hardware that executes instructions
Processor	A physical chip that contains one or more CPUs
Core	The basic computation unit of the CPU（executes instructions + registers）
Multicore	Multiple computing cores on a single physical processor chip
Multiprocessor	Including multiple processors

6. Multiprogramming & Timesharing ⭐ [1.42]

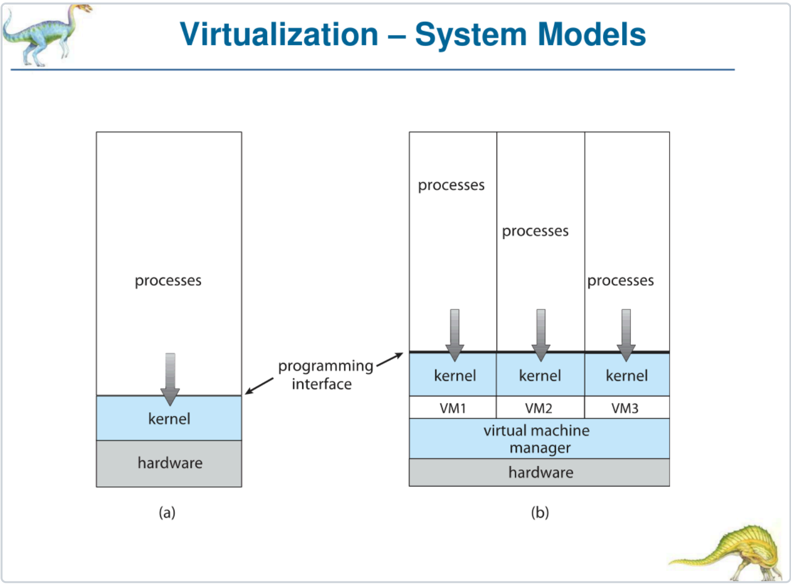
- 所有 modern OS 都有兩個特徵：
  1. **Multiprogramming (batch system)** — needed for **efficiency**。
     - 以前一次 load 一個 program；**single program cannot keep CPU or I/O devices busy at all times**。
     - **Multiprogramming organizes jobs so that the CPU always has one to execute.**（一個 job 等 I/O → CPU 轉做另一個 job）
     - Mainframe：jobs submitted remotely and queued → **job scheduling** 揀邊啲 load 入 memory。
  2. **Timesharing (multitasking)** — **logical extension of multiprogramming in which the CPU switches jobs so frequently that users can interact with each job while it is running → interactive computing**。
     - **Response time should be under a certain threshold**（通常 < 1 秒）。
- 相關概念：
  - A program in execution → **process**
  - Several processes ready at the same time → **CPU scheduling**
  - Processes don't fit in memory → **swapping** moves them in and out
  - **Virtual memory** allows execution of processes **not completely in memory**

> 📝 **HW1 Q2.2 Model answer**：*Multiprogramming keeps several jobs in memory at the same time. When the running job has to wait (e.g., for I/O), the OS switches the CPU to another job, so the CPU always has something to execute and CPU utilization increases. Time sharing (multitasking) extends multiprogramming by switching the CPU among jobs so frequently (using CPU scheduling and timer interrupts) that each user gets a fast response time and can interact with the program while it is running — this supports interactive computing.*

7. Virtualization ⭐ [1.43–1.46]

- **Virtualization abstracts the hardware of a single computer into multiple different execution environments**，令每個 user/program 以為自己有部 **"private computer"**。
- 創造 **virtual machine (VM)**，OS 同 applications 可以喺上面跑。
- **It allows an operating system to run as an application within another operating system.** ⭐（F20 MC2 答案：**Virtualization**，唔係 Emulation）
- **Components** [1.44]：
  - **Host** — underlying hardware system
  - **Virtual machine manager (VMM) or hypervisor** — **creates and manages virtual machines** by providing an interface **identical to the host**
  - **Guest OS** — process provided with a **virtual copy of the host**（通常係一個 OS）
- **Motivations** [1.43]：
  - **Server consolidation** — 將多個 OS 合併到較少 hardware 上
  - Desktop 上跑一個 OS（Linux/macOS）但仍用到另一個平台（Windows）嘅 native apps
  - **Developers can run many OS types/versions on one machine** → easier testing and debugging

- [1.45] 圖：(a) processes → kernel → hardware；(b) 每個 VM 有自己 kernel，VM1/VM2/VM3 → **VMM** → hardware。



Virtualization – System Models

Virtualization system models [1.45]

- **History** [1.46]：IBM mainframes (1972)；VMware 喺 Intel x86 跑多個 Windows guest；**programs in a VM show only minor performance decreases**（多咗幾層 software）；**VMM is in complete control of system resources**；1990s 後期 Intel CPU 夠快；Xen、VMware。
- **Emulation vs Virtualization**（課本背景）：**Emulation** = 模擬 **different** CPU architecture（source CPU ≠ target CPU，慢）；**Virtualization** = guest OS **natively compiled for the CPU**。

> ⚠️ HW1 MC8：「An application always runs **faster** inside a VM」→ **False**（只會 minor performance decrease）。

8. Cloud Computing [1.47–1.48]

- **Delivers computing, storage, and apps as a service over a network.**
- **Logical extension of virtualization**（用 virtualization 作基礎）；例：Amazon EC2（pay based on usage）。
- **Types**：
  - **Public cloud** — available via Internet to anyone willing to pay
  - **Private cloud** — run by a company for its own use
  - **Hybrid cloud** — both public and private components
- **Service models**（MC 易考 match）：
  - **SaaS (Software as a Service)** — applications via Internet（e.g. word processor）
  - **PaaS (Platform as a Service)** — **software stack** ready for application use（e.g. **database server**）
  - **IaaS (Infrastructure as a Service)** — **servers or storage** over Internet（e.g. storage for backup）
  - **MaaS** — Machine learning as a Service

9. I/O Subsystem（Chapter 1 summary 提及）

- I/O 硬件三元素：**buses, device controllers, devices**。
- 搬 data（device ↔ main memory）：**CPU in programmed I/O** 或者 **offloaded to a DMA controller**。
- **Device driver** = kernel module that controls a device。
- **Kernel I/O subsystem services**：**I/O scheduling, buffering, caching, spooling, device reservation, error handling**。
  - **Buffering** — store data temporarily while it is being transferred；**Caching** — faster storage holding a **copy** of data（Lab 2）。⭐ **A buffer may hold the only existing copy of a data item, whereas a cache only holds a copy of an item that resides elsewhere.**
  - **Spooling** — hold output for a device that **cannot accept interleaved data**（e.g. **printer**），coordinating concurrent output。
- I/O hardware（Lab 2）：**port**（connection point）、**bus**（PCI/PCIe；expansion bus for slow devices）、**controller (host adapter)**；device registers：**data-in, data-out, status, control**。
- （Interrupt、DMA、dual-mode 詳細見 **Ch3 notes §0**（slides 3.21–3.31））

10. 常見陷阱 & MC 速查（Traps）

敘述	真/假	原因
Main memory is byte addressable	✓	each address refers to one byte [1.30]

敘述	真/假	原因
Main memory is nonvolatile	✗	DRAM 係 **volatile**
Registers are slower than main memory	✗	registers 最快
HDD is smaller than volatile memory	✗	HDD 大好多
CPU can directly access secondary storage	✗	只可 directly access main memory
Instructions and data stored in same memory (von Neumann)	✓	stored-program
Speed-up with N processors = N	✗	**< N** due to overhead/contention
Multicore uses less power than multiple single-core chips	✓	[1.39]
NUMA: same latency for all memory	✗	remote memory slower
SMP: each processor has private cache, shares memory	✓	[1.38]
AMP: master handles I/O	✓	[1.37]
Timesharing is an extension of multiprogramming	✓	[1.42]
VM always faster than native	✗	minor slowdown
Virtualization lets an OS run as an app within another OS	✓	[1.43]
Cache works even with uniform random access	✗	needs locality
High hit ratio alone guarantees performance	✗	also hit time & miss time
Cache is managed by OS	✗	**hardware**（[1.34]）；registers by **compiler**

11. Past-paper 題目（Ch1 相關）

1. **S26 MC3**：storage hierarchy 正確嘅係？→ **D. Main memory is byte addressable**。
2. **S26 MC4**：von Neumann 核心特徵？→ **B. Instructions and data stored in the same memory space, accessed by address**。
3. **S26 Q2.2**：caching 基本概念 + 高 hit ratio 夠唔夠？→ 見 §4。
4. **F19 Q1.2**：temporal vs spatial locality？→ 見 §4.3。
5. **F20 MC1**：Locality is essential to → **Cache**。
6. **F20 MC2**：___ allows a guest OS to run as an application on a native OS → **Virtualization**。
7. **Sp21 MC1**：von Neumann processing unit contains a/an ___ and processor registers → **ALU**。
8. **HW1 MC3** resource allocator；**MC4** NUMA same latency → false；**MC8** VM always faster → false；**Q2.1** AMAT = 6.8 ns；**Q2.2** multiprogramming vs timesharing。

12. 自我測驗（Self-test）

1. 寫出 average access time 公式；hit ratio 95%、hit time 1 ns、miss time 100 ns？（答：0.95 + 5 = **5.95 ns**）
2. 點解 N 個 processors speedup < N？（contention for shared resources e.g. bus/memory, overhead）
3. Multiprocessor 三個優點？（throughput, economy of scale, reliability/graceful degradation）
4. NUMA 嘅 drawback？（remote memory access latency）
5. VMM 又叫咩？做咩？（hypervisor; creates and manages VMs, interface identical to host）
6. Cache 由邊個 manage？Main memory 呢？（hardware；OS）

2 Ch2 OS Structures 作業系統結構

📌 **考試範圍提示**：Ch2 每年都有 **3–5 題 MC** + **1–2 題 short question**。必考：**API vs system call（點解要分開）**、**printf() → write()**、**parameter passing 三種方法**、**linker vs loader**、**monolithic / layered / microkernel / LKM / hybrid** 比較（優缺點）、**policy vs mechanism**、**OS services**。⚠️ **Dual-mode / interrupt / trap** 今個 term 放咗喺 **Ch3 slides（3.x）**，見 Ch3 notes §0。

---

1. Operating System Services（OS 服務） [2.4–2.7]

OS provides an environment for **execution of programs** and **other services to programs and users**。分兩組：

1.1 對 user 有用嘅服務（helpful to the user）

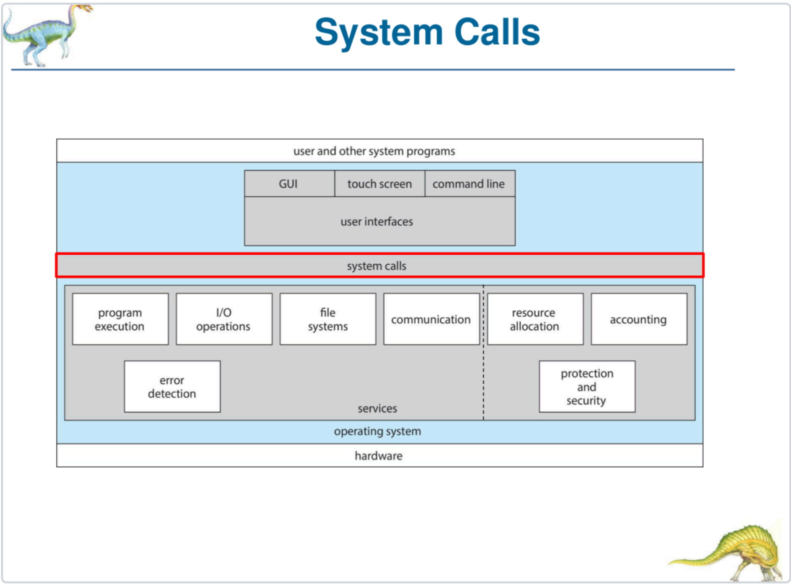
Service	解釋
User interface (UI)	**CLI (command-line)**, **GUI**, **touch-screen**
Program execution	load a program into memory and run it; end execution **normally or abnormally (indicating error)**
I/O operations	running program may require I/O（file 或 I/O device）
File-system manipulation	read/write/create/delete/search files & directories, list info, **permission management**
Communications	processes exchange information（同一部機或 network）via **shared memory** or **message passing (packets moved by the OS)**
Error detection	errors **may occur in CPU and memory, in I/O devices, in user programs**；OS takes appropriate actions；**debugging facilities**

1.2 確保系統本身有效運作（efficient operation via resource sharing）

Service	解釋
Resource allocation	多 user/job 同時跑 → 分配 **CPU cycles, memory, file storage, I/O devices**
Logging (accounting)	**keep track of which users use how much and what kinds of computer resources**
Protection and security	**Protection**: ensure all access to system resources is controlled；**Security**: defend from outsiders → **user authentication**, defend external I/O devices from invalid access

⚠️ **S26 MC5 陷阱**：「**Errors only occur in memory and I/O devices**」→ **False**（CPU、memory、I/O devices、user programs 都可以出錯）。✓「Logging service should keep track of computer resources that each user uses」→ True。

- Summary 補充：OS services 兩大類 —（1）**enforce protection among concurrent processes**（process 只可以存取自己 address space；唔可以直接存取 hardware）；(2）**provide functionalities not directly supported by hardware**（memory management、file systems）。
- [2.7/2.13 圖] user & system programs → user interfaces (GUI/touch/CLI) → **system calls** → services (program execution, I/O, file systems, communication, resource allocation, accounting, error detection, protection & security) → OS → hardware。



System Calls

user and other system programs
GUI
touch screen
command line
user interfaces
system calls
program execution
I/O operations
file systems
communication
resource allocation
accounting
error detection
protection and security
services
operating system
hardware

OS services and the system-call layer [2.13]

---

2. User Interfaces [2.8–2.12]

- **CLI / command interpreter** — allows **direct command entry**。
    - **Sometimes implemented in kernel, sometimes by system programs.**
    - 有多個 flavor → **shells**（C shell, **Bourne-Again shell (bash)**, Korn shell）。
    - **Shell 嘅主要功能：(1) fetches a command from user, (2) interprets it, (3) executes it.** ⭐（PA1 myshell 就係做呢樣）
- **GUI** — user-friendly **desktop metaphor**；icons；mouse；**invented at Xerox PARC (early 1970s)**，first widely used in **Apple Macintosh (1984)**。Windows = GUI + CLI "command" shell；Mac OS X = **"Aqua" GUI** + UNIX kernel；Unix/Linux = CLI + optional GUI（KDE, GNOME）。
- **Touchscreen** — gestures, virtual keyboard, voice command（Siri）。

---

3. System Calls ⭐ ⭐ [2.13–2.26]

3.1 定義

- **System calls provide the programming interface to the services provided by the OS.**（Sp21 MC：「__ provide an interface to the services provided by an OS」→ **System calls**）
- 通常以 **high-level language (C/C++) functions** 提供；**certain low-level tasks written in assembly** for accessing hardware。
- 例：`cp in.txt out.txt` 涉及**一連串 system calls**（acquire input file name、open input file、create output file、loop read/write、close、write completion message、terminate normally）。

3.2 API (Application Programming Interface) [2.15]

- **API specifies a set of functions available to application programmers, including the parameters passed to each function and the return values it may expect.**
- **The functions that make up an API invoke the actual system calls on behalf of the programmer.**
    - 例：Windows `CreateProcess()` invokes `NTCreateProcess()` system call。
- Programmer 經 OS 提供嘅 **library** 用 API；UNIX/Linux C → **libc**。
- **三大 API：Win32 API**（Windows）、**POSIX API**（UNIX, Linux, Mac OS X）、**Java API**（JVM）。
- [2.16] `ssize_t read(int fd, void *buf, size_t count)` — `fd` file descriptor；`buf` buffer；`count` max bytes；**return 0 = end of file；−1 = error**。

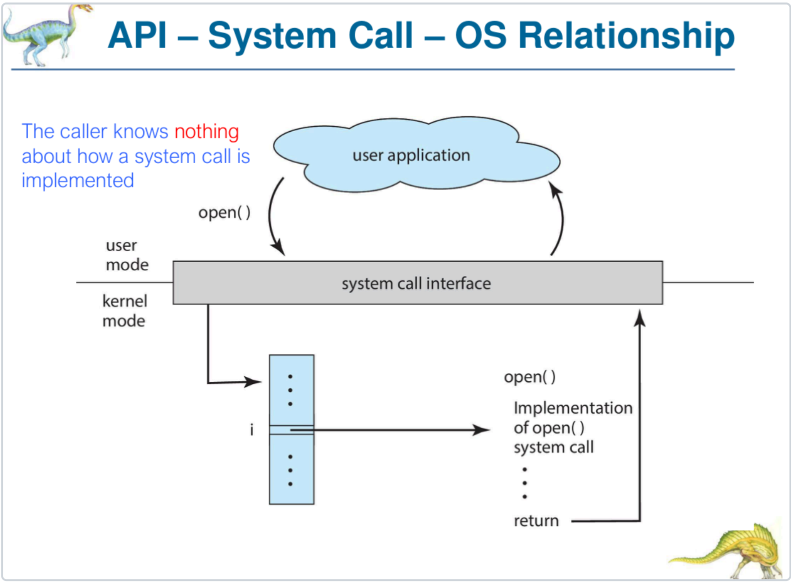
3.3 點解用 API 而唔直接 call system call？⭐（F19 Q1.3、S26 MC6）[2.17]

> 📝 **Model answer**：*(1)* ***Program portability*** *— a program using an API can compile and run on any system that supports the same API, even though the implementation of system calls varies from machine to machine. (2)* ***Hide the complex details of system calls*** *— actual system calls are often more detailed and difficult to work with; the caller only needs to obey the API and understand what the OS will do, not how the system call is implemented.*

- S26 MC6 答案：**APIs provide a more convenient programming interface**。
- ✗「APIs always execute faster than system calls」、✗「APIs can bypass kernel protection」、✗「APIs eliminate the need for linkers and loaders」。
- F20 MC4：「Programmers interface with ___ to obtain services from the OS」→ **APIs**（best answer）。

3.4 System Call Implementation [2.18–2.19]

- 大部分語言嘅 **run-time environment (RTE)**（a set of functions built into libraries）提供 **system-call interface**，作為連接 OS system calls 嘅 **link**。
- **Each system call is associated with an identity number**；system-call interface **maintains a table indexed by these numbers**。
- System-call interface 做三件事：**intercepts function calls in the API → invokes the necessary system call within the OS → returns the status of the system call and return value(s)**。
- **The caller knows nothing about how a system call is implemented.**（只需要 obey API）


API – System Call – OS Relationship

The caller knows nothing about how a system call is implemented

API – system call – OS relationship [2.19]

- [2.19 圖]：user application 呼叫 `open()` → (user mode) **system call interface** → (kernel mode) 用 index *i* 查 table → implementation of `open()` → return。

> **HW1 Q2.4：user program call read() 點樣 user→kernel→user？（三步）**
>
> 1. *The program calls the `read()` API in libc (user mode); the library/system-call interface puts the system-call number and parameters in registers (or a block/stack) and executes a **trap (system-call) instruction**.*
> 2. *The trap switches the CPU to **kernel mode** (mode bit 1→0); the kernel uses the system-call number to index the **system-call table** and runs the `read()` service routine (which may block waiting for the device).*
> 3. *When finished, the kernel puts the return value/status in a register, executes a **return-from-trap** which resets the mode to **user mode**, and control returns to the instruction after the call.*
>
> - HW1 MC6 答案：**The API and run-time environment use the system-call interface to invoke the corresponding kernel service, and control later returns to the application.**

3.5 System Call Parameter Passing ⭐ [2.20–2.21]

三種方法（F20 MC5：**All of the above**）：

1. **Registers** — simplest；但可能 parameters 多過 registers。
2. **Block (table) method** — parameters stored in a **block/table in memory**, **address of the block passed in a register**（Linux & Solaris 用）。
3. **Stack method** — parameters **pushed onto the stack by the program and popped off by the OS**。

- **Block and stack methods do not limit the number or length of parameters.**

3.6 Types of System Calls（六類）⭐ [2.22–2.25]

類別	例子	UNIX 例	Windows 例
Process control	create/terminate process, end/abort, load/execute, get/set attributes, wait for time, wait/signal event, allocate/free memory, dump memory, debugger, **locks**	`fork()`, `exit()`, `wait()`	`CreateProcess()`, `ExitProcess()`, `WaitForSingleObject()`
File management	create/delete, open/close, read/write/reposition, get/set attributes	`open()`, `read()`, `write()`, `close()`	`CreateFile()`, `ReadFile()`, `WriteFile()`, `CloseHandle()`
Device management	request/release device, read/write, get/set attributes, attach/detach	`ioctl()`, `read()`, `write()`	`SetConsoleMode()`, `ReadConsole()`, `WriteConsole()`
Information maintenance	get/set time/date, system data, attributes	`getpid()`, `alarm()`, `sleep()`	`GetCurrentProcessID()`, `SetTimer()`, `Sleep()`
Communications	create/delete connection, send/receive messages (**message passing**), **shared-memory** create/access regions	`pipe()`, `shm_open()`, `mmap()`	`CreatePipe()`, `CreateFileMapping()`, `MapViewOfFile()`
Protection	control access, get/set permissions, allow/deny user access	`chmod()`, `umask()`, `chown()`	`SetFileSecurity()`, …

3.7 Standard C Library Example ⭐（S26 MC8）[2.26]

- **C program invoking `printf()` library call, which calls the `write()` system call.**
- ✓ **"printf() invokes write() system call to get service from OS"**
- ✗ 「printf() and write() are both system calls」（printf 係 library function）
- ✗ 「program enters kernel mode immediately when printf() is called」（printf 喺 user mode 處理 formatting/buffering，到 write() 先 trap 入 kernel）
- ✗ 「write() invokes printf()」

---

4. System Programs（系統程式）[2.27–2.30]

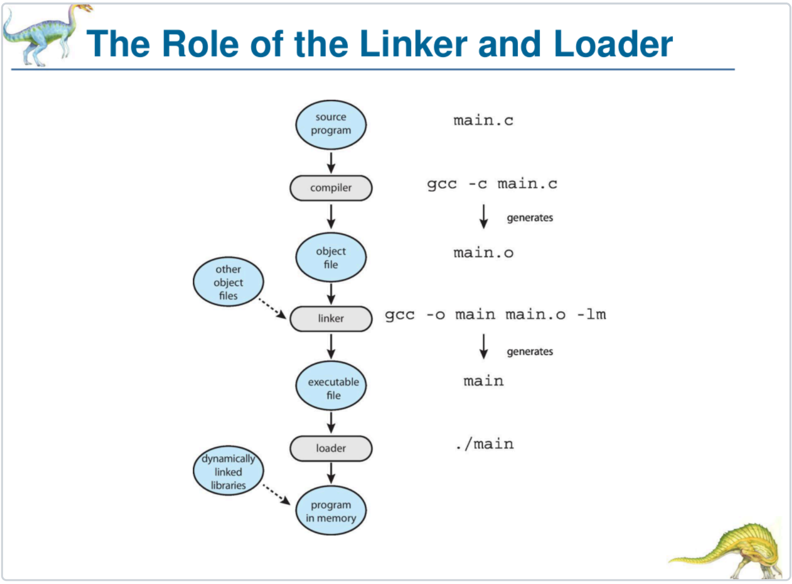
- **System services / system programs / system utilities provide a convenient environment for program development and execution.**
- 有啲只係 **user interfaces to system calls**；有啲複雜得多。
- **The view of the OS seen by most users is defined by the application and system programs, not the actual system calls.**（GUI 同 CLI 用同一套 system calls，但 user 睇落好唔同）
- **類別**：
  - **File management** — create, delete, copy, rename, print, list files/directories
  - **Status information** — date, time, available memory, disk space, number of users；performance, logging, debugging info
  - **File modification** — text editors；search/transform file contents
  - **Programming-language support** — compilers, assemblers, debuggers, interpreters
  - **Program loading and execution** — absolute loaders, relocatable loaders, linkage editors, overlay loaders, debugging systems
  - **Communications** — virtual connections among processes/users/systems（message, web, email, remote login, file transfer）
  - **Background services** — launch at boot；有啲跑到系統關機 → **services, subsystems, daemons**（disk checking, process scheduling, error logging）
- **Application programs** — **not part of the OS**（browsers, word processors, spreadsheets, databases, games）。

---

5. Linkers and Loaders ⭐ ⭐（F19 Q1.1、F20 Q2.2 都考過）[2.31–2.32]

```
main.c --(compiler: gcc -c main.c)--> main.o (relocatable object file)
main.o + other object files + libraries --(linker: gcc -o main main.o -lm)--> main (executable file)
main --(loader: ./main)--> program in memory  (+ dynamically linked libraries)
```

- **Source code is compiled into relocatable object files**（designed to be loaded into any physical memory location）。
- **Linker combines these relocatable object files into a single binary executable file**（along with libraries if needed, e.g. `-lm` math library）。
- **Programs reside on secondary storage as binary executables, which must be brought into memory by the loader to be executed.**
- **Relocation** assigns final addresses to program parts and adjusts code & data to match those addresses。
- Modern OS **don't link libraries statically**；而係用 **dynamically linked libraries (DLLs in Windows)** — **loaded when needed, shared by all programs using the same version (loaded only once)** → 慳 memory，避免 link 用唔到嘅 library。
- Object files / executables 有 **standard formats**（machine code + symbol table；Linux = **ELF**）→ OS 知點 load 同 start。



```
main.c

gcc -c main.c

main.o

gcc -o main main.o -lm

main

./main
```

Role of the linker and loader [2.32]

> 📝 **Model answer**：*The **linker** combines several relocatable object files (generated by the compiler), together with any needed libraries, into a single binary executable file stored on disk. The **loader** loads the binary executable file into memory (possibly together with dynamically linked libraries) so that it can be run on a CPU.*

6. OS Design and Implementation [2.33–2.37]

6.1 Design goals [2.33]

- Design 受 **choice of hardware, type of system**（desktop, mobile, distributed, real-time）影響；先定 **goals and specifications**。
- **User goals**：convenient to use, easy to learn and use, reliable, safe, fast。
- **System goals**：easy to design, implement, maintain; flexible, reliable, error-free, efficient。

6.2 Policy vs Mechanism ⭐ [2.34]

- **Policy: What will be done? Mechanism: How to do it?**
- 例：**A timer is a mechanism for ensuring CPU protection, but deciding how long the timer is set for a particular user is a policy decision.**
- **Separation of policy from mechanism allows flexibility if policy decisions are changed later**（同一個 mechanism 可以支持「I/O-intensive 優先」或者相反嘅 policy）。
- 總結：**goals → determine policies → implemented through mechanisms**。

6.3 Implementation [2.35]

- 早期用 **assembly**；而家多數用 **C/C++**；最底層 kernel 可能仍用 assembly。Android：system libraries C/C++，frameworks Java。
- **Advantages of high-level languages**：**easier to port to other hardware**；code written faster, more compact, easier to understand and debug；compiler 進步 → recompile 就變好。
- **Disadvantages**：reduced speed、increased storage requirements（今日唔係大問題）。

6.4 Examples [2.36–2.37]

- **MS-DOS**：**single-tasking**；開機 invoke shell；**no process created**；single memory space；load program overwriting all but the kernel；program exit → shell reloaded。
- **FreeBSD**（Unix variant）：**multitasking**；user login → shell；**shell executes `fork()` to create a new process, `exec()` to load a new program into the process**；shell **waits for the process to terminate** or continues with user commands（background）；需要 CPU scheduling、process coordination、memory management。

7. Operating System Structure ⭐⭐⭐（必考比較題）[2.38–2.53]

7.0 一覽表

Structure	核心概念	優點	缺點	例子
Simple structure	無 well-defined structure	most functionality in least space	interfaces/levels 唔分明；**app 可直接 access I/O**；no dual mode/no hardware protection	**MS-DOS**

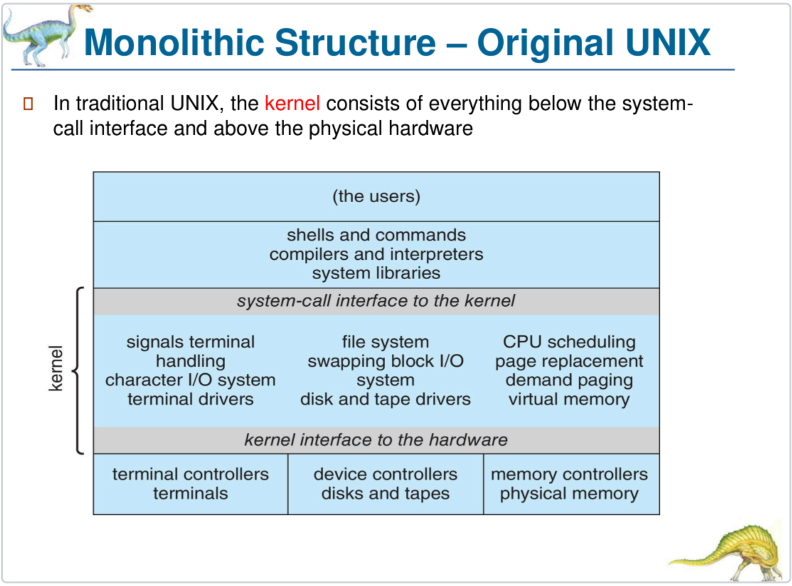
Structure	核心概念	優點	缺點	例子
Monolithic	**all kernel functionality in a single, static binary file running in a single address space**	**performance**：system-call interface overhead 好少，kernel 內部溝通快	**difficult to implement, debug, maintain/modify；tightly coupled**	**Original UNIX**, Linux, Windows（largely）
Layered	layer 0 = hardware ... layer N = user interface；每層只用下層服務	**simplicity of construction and debugging；information hiding**	**難定義 layers；performance overhead**（traverse 多層）	TCP/IP networks；(pure layered OS 好少)
Microkernel	**remove nonessential components from kernel, implement them as user-level programs in separate address spaces；靠 message passing**	**easier to extend、easier to port、more secure and reliable**（less code in kernel mode）	**performance suffers** due to **increased system-function overhead / user–kernel communication / message copying**	**Mach**, Darwin（macOS/iOS 部分），Windows NT（layered microkernel）
Loadable kernel modules (LKMs)	core kernel + **dynamically loaded modules（boot time or run time）**	唔使 recompile/reboot；**any module can call any other module；no message passing → more efficient than microkernel**	（仍喺 kernel mode，module 出錯可影響 kernel）	**Linux**（device drivers, file systems），Solaris
Hybrid	混合多種結構	performance + security + usability	複雜	**Linux**（monolithic + modular），**Windows**（monolithic + microkernel-like **personalities** + LKMs），**macOS/iOS**（Darwin = Mach + BSD）

7.1 Simple Structure — MS-DOS [2.39]

- **Written to provide the most functionality in the least space**；not carefully divided into modules。
- **Application programs can access I/O directly**（interfaces/levels 唔分開）。
- Written for **Intel 8088 — no dual mode and no hardware protection**。

7.2 Monolithic Structure — Original UNIX ⭐ [2.40–2.42]

- **Traditional UNIX kernel = everything below the system-call interface and above the physical hardware.**



Monolithic Structure – Original UNIX

- In traditional UNIX, the kernel consists of everything below the system-call interface and above the physical hardware

(the users)
shells and commands
compilers and interpreters
system libraries
system-call interface to the kernel
signals terminal handling
character I/O system
terminal drivers
file system
swapping block I/O system
disk and tape drivers
CPU scheduling
page replacement
demand paging
virtual memory
kernel interface to the hardware
terminal controllers
terminals
device controllers
disks and tapes
memory controllers
physical memory
kernel

Original UNIX: monolithic structure [2.40]

- **Place all functionality of the kernel into a single, static binary file that runs in a single address space.**
- **UNIX = two separable parts: kernel + system programs**；kernel 再分成 interfaces 同 device drivers。
- **Drawback**：enormous functionalities combined into one level → **difficult to implement, debug, and maintain**。
- **Advantage**：**distinct performance advantage — very little overhead in the system-call interface, and communication within the kernel is fast.** → 所以 UNIX、Linux、Windows 仍然用。
- Sp21 MC：Original UNIX 採用 → **Monolithic structure**。
- **Linux** [2.42]：based on UNIX；**monolithic — runs entirely in kernel mode in a single address space**；apps 用 **glibc** 同 system call interface 溝通；**modular design 可以 run time 改 kernel（LKM）**。

7.3 Modular Design [2.43]

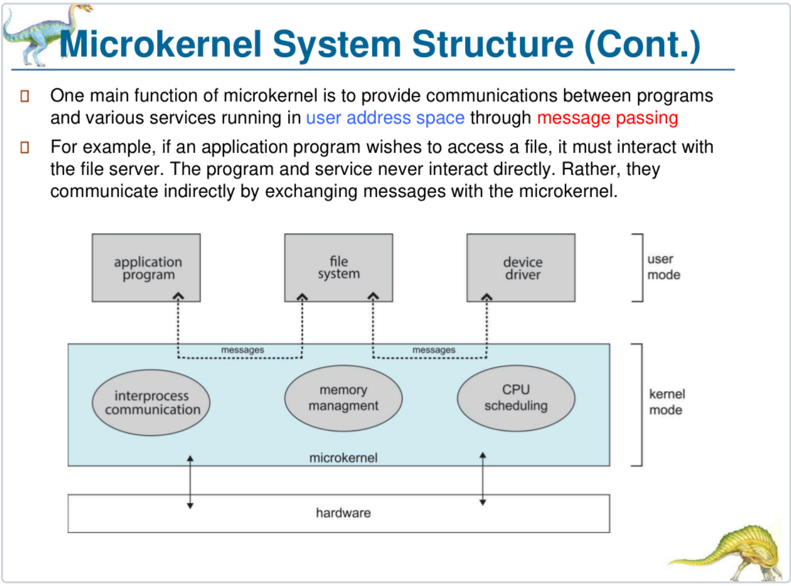
- Monolithic = **tightly coupled system**（改一部分影響其他部分）。
- **Loosely coupled system**：分成細 components，每個有 specific/limited 功能 → **changes in one component affect only that component** → 更自由靈活。
- **Layered approach 係 modular design 嘅其中一種。**

7.4 Layered Approach ⭐ [2.44–2.45]

- **OS divided into layers, each built on top of lower layers. Bottom layer (layer 0) = hardware; highest (layer N) = user interface.**
- 每層有 data structures + functions，可被上層調用、可調用下層。
- **Information hiding**：上層唔需要知下層點 implement，只需知 **what** these operations do（interface）；每層隱藏自己嘅 data structures、operations、hardware。
- **Main advantage：simplicity of construction and debugging**（由最底層開始逐層 debug）。
- 成功例子：**computer networks（TCP/IP）**、web applications。
- **Few OSes use a pure layered approach because：**
  1. **Significant challenges in appropriately defining layers and their functionalities**
  2. **Overall performance affected by the overhead of traversing multiple layers to obtain a service**
- Trend：**fewer layers with more functionality**，modularized code。
- S26 MC9 ✓：**The interfaces between adjacent layers are usually well defined**。✗ any layer may directly access any other layer；✗ eliminates all performance overhead；✗ pure layered is dominant。

7.5 Microkernel ⭐ ⭐ [2.46–2.48]

- Kernel 變大難管理 → **removing all nonessential components from the kernel and implementing them as system or user-level programs in separate address spaces** → **much smaller kernel**。
- **Mach**（CMU, mid-1980s）；best-known：**Darwin**（Mac OS X/iOS，包含 Mach microkernel）。
- 微核心通常提供：**minimal process and memory management + communication facility (IPC)**。
- **Main function：provide communication between client programs and services running in user space through message passing.** 例：app 要讀 file → 同 **file server** 經 microkernel **exchange messages**，兩者**唔直接 interact**。



Microkernel System Structure (Cont.)

- One main function of microkernel is to provide communications between programs and various services running in user address space through message passing
- For example, if an application program wishes to access a file, it must interact with the file server. The program and service never interact directly. Rather, they communicate indirectly by exchanging messages with the microkernel.

Microkernel: services in user mode talk via message passing [2.47]

- **Advantages：**
  - **Easier to extend** — new services added to **user space without modifying the kernel**
  - Kernel 要改時改動較少（kernel 細）
  - **Easier to port to new architectures**
  - **More secure and reliable** — less code in kernel mode；**if a service fails, the rest of the OS remains untouched**
- **Drawbacks：**
  - **Performance can suffer due to increased system-function overhead (user space ↔ kernel space communication)**
  - Two user-level services communicate → **messages must be copied between services in separate address spaces**
  - Windows NT（layered microkernel）比 Windows 95 慢 → Windows XP 變返 more monolithic。

> 📝 **F20 Q2.1 Model answer**：*Main idea: a microkernel removes all nonessential components from the kernel and implements them as system or user-level programs in separate address spaces; the kernel keeps only minimal process/memory management and IPC (message passing). Advantage: easier to extend/port, more secure and reliable since less code runs in kernel mode. Disadvantage: performance suffers due to increased system-function overhead — message passing / copying between user space and kernel space.*

> ⚠️ **S26 MC7**：「microkernel typically has better performance than monolithic」→ **False**。（「runs memory management in user space」亦被接受為 false，因為 minimal memory management 喺 kernel 入面。）HW1 MC5 ✓「microkernel keeps only essential functions in the kernel and provides other services outside the kernel through message passing」。

7.6 Loadable Kernel Modules (LKMs) — Modular Approach ⭐⭐ [2.49–2.51]

- **Best current methodology for OS design.**
- **Kernel has a set of core components and links in additional services via modules, either at boot time or during run time.**
- 好處：**dynamic linking is preferable to adding features directly to the kernel, which would require recompiling the entire kernel every time**。例：kernel 內有 CPU scheduling + memory management，**file systems 經 loadable modules 加入**。
- **Similar to layered**：each kernel section has **well-defined, protected interfaces**；**but more flexible — any module can call any other module**（無上下層限制）。
- **Similar to microkernel**：primary module 只有 core functions + 知點 load/communicate 其他 modules；**but more efficient — modules do not need message passing to communicate**。
- **Linux uses LKMs primarily for device drivers and file systems**；可以 boot/run time **insert** 或 **remove**。例：插 USB，kernel 冇 driver → dynamic load。
- **→ dynamic and modular kernel while keeping the performance benefits of a monolithic system**。
- [2.51] Solaris：core kernel + scheduling classes, file systems, loadable system calls, executable formats, STREAMS modules, misc modules, device & bus drivers。
- F20 MC6：「core components in kernel + adds services via modules during run time」→ **LKMs**。

> 📝 **F19 Q1.4 Model answer（LKM vs layered）**：*Similarity: like a layered design, OS functionality is divided into modules, each with a well-defined (protected) interface. Difference: LKM is much more flexible — any module can call any other module directly, with no higher/lower layer restriction (and modules can be loaded/removed at run time).*

7.7 Hybrid Systems [2.52–2.59]

- **Very few OSes adopt a single strictly defined structure** → **hybrid systems** combine approaches for **performance, security, usability**。
- **Linux**：**monolithic**（single address space → efficient）+ **modular**（LKM）。
- **Windows**：**largely monolithic**，但保留 microkernel 特色 — **separate subsystems (personalities) running as user-mode processes**；亦支援 dynamically loadable kernel modules。
- **macOS / iOS** [2.54–2.57]：kernel environment **Darwin = Mach microkernel + BSD UNIX kernel**（**XNU kernel**，hybrid）。
    - **Two system-call interfaces**：**Mach system calls (traps)** + **BSD system calls (POSIX)**。
    - **Mach** → memory management, CPU scheduling, IPC；**BSD** → POSIX API, networking, file system, security。
    - **I/O kit** for device drivers；dynamically loadable modules = **kernel extensions (kexts)**。
    - Mac：**Aqua** UI（mouse/trackpad）、**Cocoa**（Objective-C API）、Core frameworks（QuickTime, OpenGL）。
    - iOS：**Springboard** UI（touch）、**Cocoa Touch**、Media services、Core services（cloud, databases）；**ARM**（vs Intel）；不能原生跑 Mac apps。
- **Android** [2.58–2.59]：Open Handset Alliance（Google）；**open-source**（iOS closed-source）；layered；**Android API for Java** → 跑喺 **Android RunTime (ART)**（VM optimized for mobile）；**JNI (Java Native Interface)** bypass VM 存取 hardware（**not portable**）；libraries：webkit, SQLite, SSL；**HAL (hardware abstraction layer)** → consistent view；**Bionic** C library（唔係 glibc）；modified Linux kernel（power management）。
- **iOS vs Android**（summary）：相同 — 都基於現有 kernel、software stack、developer frameworks；唔同 — iOS closed / Android open；Objective-C vs Java；**Android uses a VM, iOS executes natively**。

7.8 Summary in OS Design [2.53]（原句，MC 用）

- **Monolithic**: no structure; all functionality in a single static binary in a single address space; difficult to modify; **primary benefit is efficiency**.
- **Layered**: discrete layers (bottom = hardware interface, top = user interface); **not ideal for OS due to performance problems and difficulty in defining layers**.
- **Microkernel**: minimal kernel; most services run as user-level applications; **communication via message passing**.
- **LKM**: services through modules **loaded and removed during runtime**.
- **Many contemporary OSes are hybrid: monolithic kernel + modules.**

8. History of OS [2.60]

- 硬件貴、人平：**Hand programmed (1945–55) → Batch processing (1955–65) → Multiprogramming (1965–80)**
- 硬件平、人貴：**Time sharing → Personal computer → Distributed OS → Real-time OS**

9. 補充：Protection & Security（Chapter 2 summary）

- **Protection** — controlling access of users/processes to resources。**Access right** = permission to perform operations on an object；**Domain** = a set of access rights；process 喺 domain 入面執行。
- **Security** — defend system against internal/external attacks（DoS, worms, viruses）。

10. 常見陷阱 & MC 速查

敘述	真/假
printf() is a system call	✗（library function，calls write()）
API always faster than system calls	✗
API can bypass kernel protection	✗
Parameters can be passed by registers, block/table, stack	✓（all of the above）
Block & stack methods limit number of parameters	✗（**do not** limit）
Shell may be implemented in kernel or as system program	✓
Linker loads program into memory	✗（loader 做）
DLLs are loaded once and shared	✓
Microkernel has better performance than monolithic	✗
Microkernel: services communicate by message passing	✓
Microkernel easier to port and extend	✓
Layered: any layer may call any other layer	✗（LKM 先可以 any module calls any module）
Layered: main advantage = simplicity of construction & debugging	✓
LKM requires recompiling entire kernel	✗
LKM more efficient than microkernel (no message passing)	✓
Monolithic: fast but hard to maintain	✓
Original UNIX = monolithic	✓
Linux = monolithic + modular (hybrid)	✓
Windows NT = layered microkernel	✓
Policy = how; mechanism = what	✗（**policy = what，mechanism = how**）
Darwin = Mach + BSD	✓
Android apps run on ART VM; iOS runs natively	✓

11. Past-paper 題目（Ch2）

1. **S26 MC5** false about OS services → **"Errors only occur in memory and I/O devices"**
2. **S26 MC6** advantage of API → **more convenient programming interface**
3. **S26 MC7** false about microkernel → **better performance than monolithic**（C 亦接受）
4. **S26 MC8** printf/write → **printf() invokes write() system call**
5. **S26 MC9** layered → **interfaces between adjacent layers are well defined**
6. **F19 Q1.1 / F20 Q2.2** linker vs loader（§5）
7. **F19 Q1.3** 點解分開 API 同 system call（§3.3）
8. **F19 Q1.4** LKM vs layered（§7.6）
9. **F20 MC4** APIs；**MC5** all of the above；**MC6** LKMs；**Q2.1** microkernel（§7.5）
10. **Sp21 MC**：two modes → **user mode and kernel mode**；interface to OS services → **system calls**；original UNIX → **monolithic**
11. **HW1 MC5** microkernel true statement；**MC6** read() path；**MC7** privileged instruction in user mode → **trap to OS**；**Q2.4** read() user→kernel→user

12. 自我測驗

1. 寫出 system call 六大類別。
2. 點解 block/stack parameter passing 優於 registers？（no limit on number/length）
3. LKM 同 microkernel 有咩相似同唔同？（both: core kernel small; LKM no message passing → efficient）

4. 舉例說明 policy vs mechanism。（timer = mechanism; how long = policy）
5. FreeBSD shell 點樣 run 一個 program？（fork → exec → wait）

3 Ch3 Processes 進程・fork・IPC・interrupt

📌 **考試範圍提示**：Ch3 係**分數最多**嘅章節之一。必考：

1. **fork() 計數 / process tree / possible outputs**（每年 2–3 題，8–13 分一題）
2. **fork + exec + wait 成功/失敗情況**、**pipe / dup 填空**（S26 Q3.3）
3. MC：**process state**、**PCB**、**context switch**、**schedulers**、**zombie/orphan**、**pipes（ordinary vs named）**、**interrupt**、**DMA**、**dual mode** ⚠️ 今個 term **interrupt / DMA / dual-mode** 喺 Ch3 slides（3.21–3.31）。

0. Computer-System Operation：I/O、Interrupt、DMA、Dual Mode [3.21–3.31]

0.1 I/O 基本運作 [3.21]

- **I/O devices and the CPU execute concurrently and asynchronously.**
- **Each device controller is in charge of a particular device type and has a local buffer.**
- **Device controller moves data between the peripheral devices and its local buffer storage.**（I/O = device ↔ controller local buffer）
- **CPU moves data from/to main memory to/from local buffers**（typically for slow devices: keyboard, mouse）。
- **Device controller informs the CPU that it has finished an operation by causing an interrupt.**
  - Input device：data available in local buffer；Output device：I/O operation completed。

0.2 Interrupts ⭐⭐ [3.22–3.24]

- **An I/O device may trigger an interrupt by sending a signal to the CPU**（經 **interrupt-request line**）。CPU **handles the interrupt and then returns to the interrupted instruction**。
- **Interrupts are widely used to handle asynchronous events — device controllers and hardware faults.**
- **Interrupt transfers control to an interrupt service routine (ISR) or interrupt handler — part of kernel code.**
- **Interrupt vector**（summary）：a table/array holding the **addresses of the interrupt handlers**；每個 interrupt 有 unique **interrupt number** 作為 index。
- 運作（summary）：CPU **每執行完一條 instruction 就 sense interrupt-request line** → 有 signal → 讀 interrupt number → 用佢做 index 入 interrupt vector → jump 去 handler。
- **兩條 interrupt request lines**（summary）：
  - **Nonmaskable interrupt** — reserved for events like **unrecoverable memory errors, power failure**
  - **Maskable interrupt** — **can be turned off by the CPU** before executing critical instruction sequences；device controllers 用（可以 delay）
- **Interrupt priority levels** ⭐ — **a high-priority interrupt can preempt the execution of a low-priority interrupt**；CPU 可以 defer low-priority interrupts 而唔使 mask 全部。
  - Lab 2 補充：**a trap (software interrupt) is given a relatively LOW interrupt priority compared with device interrupts**；**multi-CPU systems can process interrupts concurrently**。
- **A trap or exception is a software-generated interrupt caused either by an error (e.g., arithmetic error / divide by zero) or a user request (a system call).** ⭐
- **All modern operating systems are interrupt-driven**；每秒幾百個 interrupts。

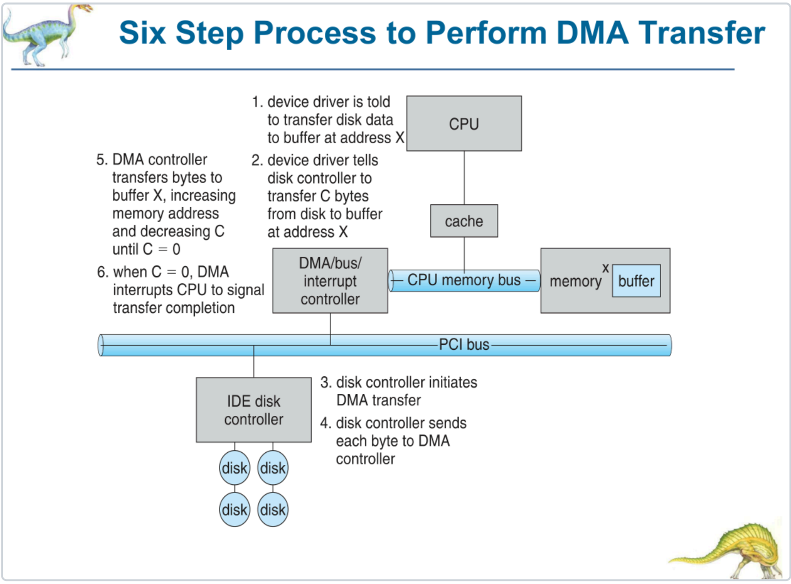
⚠️ **MC 陷阱**：

- S26 MC1 ✗「**All interrupts are serviced in exactly the same way, with no difference in priority**」（有 priority levels）
- HW1 MC2 ✗「**Interrupts can only be generated by I/O devices**」（software trap/exception 都係）；✓ hardware interrupts asynchronous；✓ trap caused by error or system call；✓ different priority levels
- F20 MC3 ✗「**Interrupt allows software to access hardware including the CPU**」（嗰個係 **system call** 嘅作用）
- Sp21 MC2：直接 trigger interrupt 嘅 event → **I/O job completion**

0.3 DMA (Direct Memory Access) ⭐⭐ [3.25–3.26]

- **Programmed I/O** — **CPU runs special I/O instructions to move one byte at a time between memory and slow devices**（keyboard, mouse）。
- **DMA** — for **fast devices and large amounts of data**：**DMA controller bypasses the CPU to transfer data between I/O device and memory directly**；CPU/OS 只係 **initialize** DMA controller。→ **relieves the CPU from slow data movement**。
- 步驟：OS writes **DMA command block** into memory（**source and destination addresses, read or write mode, number of bytes**）→ writes location of command block to DMA controller → **bus mastering**（DMA controller grabs bus from CPU — **momentarily prevents the CPU from accessing main memory**, Lab 2）→ **when done, sends an interrupt to the CPU to signal completion**。
- **Six-step DMA transfer** [3.26]：
  1. Device driver is told to transfer disk data to buffer at address X
  2. Device driver tells disk controller to transfer C bytes from disk to buffer at address X

3. Disk controller initiates DMA transfer
4. Disk controller sends each byte to DMA controller
5. DMA controller transfers bytes to buffer X, increasing memory address and decreasing C until C = 0
6. **When C = 0, DMA interrupts CPU to signal transfer completion**


Six Step Process to Perform DMA Transfer

1. device driver is told to transfer disk data to buffer at address X
CPU
2. device driver tells disk controller to transfer C bytes from disk to buffer at address X
5. DMA controller transfers bytes to buffer X, increasing memory address and decreasing C until C = 0
cache
6. when C = 0, DMA interrupts CPU to signal transfer completion
DMA/bus/ interrupt controller
CPU memory bus
memory
x
buffer
PCI bus
IDE disk controller
3. disk controller initiates DMA transfer
4. disk controller sends each byte to DMA controller
disk
disk
disk
disk

Six-step DMA transfer **[3.26]**

⚠️ **MC 陷阱：**

- HW1 MC1 ✗「**DMA controller sends an interrupt after every byte**」→ 只係 **block 完成後 interrupt 一次**
- S26 MC2 ✓「**DMA allows data transfer between device and memory with less CPU involvement**」
- ✗「DMA eliminates interrupts completely」；✗「CPU and DMA can't execute concurrently」；✗「DMA for slow devices, CPU for fast devices」（相反）

0.4 Dual-mode Operation ⭐ ⭐ [3.27–3.31]

- **Dual-mode operation allows the OS to protect itself and other system components.**
- **User mode** vs **kernel mode**（亦叫 supervisor / system / privileged mode）— distinguish execution of OS code and user code。
- **Provides the basic means of protecting the OS from errant users, and errant users from one another.**
- **Mode bit provided by hardware：user mode = 1，kernel mode = 0.** ⭐
- **Privileged instructions — can only be executed in kernel mode**，例：**I/O control, timer management, interrupt management**（summary：set timer value, clear memory, disable interrupts, modify device-status table, access I/O devices）。
- **If a privileged instruction is attempted in user mode, the hardware treats it as illegal and traps it to the OS.**（HW1 MC7 答案 A）
- **Beyond two modes：Intel — four protection rings（ring 0 = kernel, ring 3 = user；rings 1, 2 rarely used）；ARMv8 — seven modes；CPUs supporting virtualization have a separate VMM mode（more privileges than user, fewer than kernel）**。VMM 做 **machine switch**（save entire machine state 包括 privileged hardware state）。
- **Initial control in kernel mode → set to user mode when giving control to user app → back to kernel via interrupt, trap, or system call.**
- **Timer** — ensures a user program always returns control to the OS（防止 infinite loop 霸住 CPU）；設定 timer 係 privileged。
- **System call**：user mode → kernel mode to fulfil request；**return from system call resets it to user mode**。

0.5 Three Types of Mode Transfer（user → kernel） ⭐ [3.31]

Type	性質	例子
System call	user process **requests a service** from OS；"like a function call but outside the user process"	`exit()`, `write()`, `read()`
Interrupt	**external, asynchronous** events；**independent of user process**	timer, I/O completion
Trap / Exception	**internal, synchronous** events in the process	**protection violation (segmentation fault)**, **divide by zero**

1. Process Concept（進程概念） [3.4–3.8]

1.1 Four Fundamental OS Concepts [3.4]

1. **Process** — **an instance of an executing program**；consists of **an address space and one or more threads of control**。

2. **Thread** — **a single unique execution context**；fully describes program state — **program counter, registers, execution flags, stack**。
3. **Address space** — program 喺自己嘅 address space 執行，**distinct from other programs'**（logical/virtual address，Ch9–10）。
4. **Dual-mode operation** — basic protection（見 §0.4）。

1.2 Process 嘅組成部分 [3.5] ⭐

- **Text section** — the program code
- **Current activity** — **program counter**, processor registers
- **Stack** — **temporary data when invoking functions: function parameters, return addresses, local variables** ⭐（Sp21 MC 答案：stack）
- **Data section** — **global variables**
- **Heap** — **memory dynamically allocated during run time**（`malloc()` / `free()`）
- **Program = passive entity**（executable file on disk）；**Process = active entity**（有 PC 指住下一條 instruction + associated resources）。
- **A program becomes a process when an executable file is loaded into memory**（double-click icon / shell command）。
- **One program can be executed by multiple processes.**
- A process can be an execution environment for other programs（例：**JVM** 執行 Java program）。

1.3 Loading a program into memory [3.7]

1. **Load code and static data**（initialized variables）into the address space
2. **Allocate the runtime stack**；fill in `main()` parameters **argc, argv**
3. **Allocate heap**（`malloc()` / `free()`）
4. **Other initialization (I/O)** — UNIX：每個 process 預設有 **three open file descriptors: stdin (0), stdout (1), stderr (2)** ⭐

1.4 Memory Layout of a C Program [3.8]

```
high address ┌───────────────────┐
             │ argc, argv        │ (separate section for main() params)
             ├───────────────────┤
             │ stack   ↓         │ local vars, params, return addr
             │                   │
             │ heap    ↑         │ malloc()
             ├───────────────────┤
             │ uninitialized data│ (bss)  ┐ global data
             │ initialized data  │        ┘
             ├───────────────────┤
low address  │ text (code)       │
             └───────────────────┘
```

- **Global data section is divided into (1) initialized data and (2) uninitialized data.**

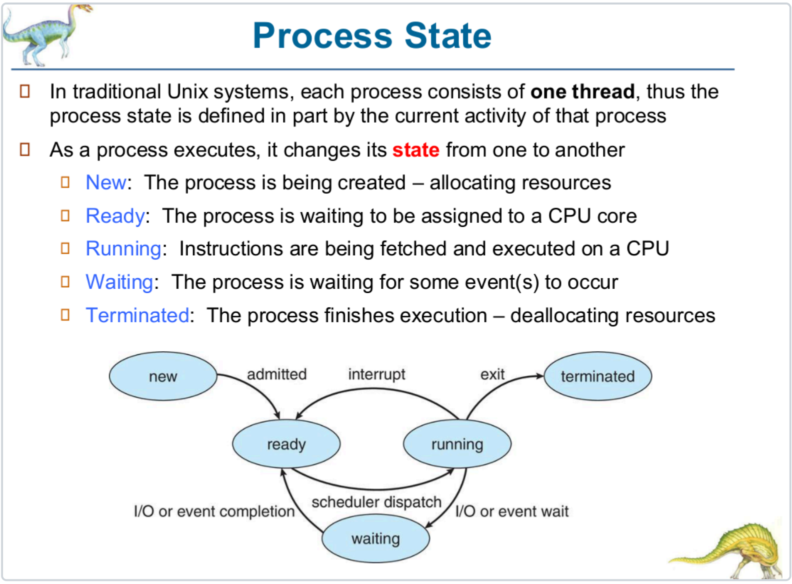
2. Process State（進程狀態） ⭐ ⭐ [3.9]

State	意思
New	process is being created — **allocating resources**
Ready	**waiting to be assigned to a CPU core**
Running	**instructions are being executed** on a CPU
Waiting	**waiting for some event(s)** to occur（I/O completion, signal）
Terminated	finished execution — **deallocating resources**

```
new --admitted--> ready <--interrupt-- running --exit--> terminated
                    |  ----scheduler dispatch---->  |
                    ^                               v
                    +--I/O or event completion-- waiting <--I/O or event wait--+
```

轉換（F19 Q2.1 答案）：

- New → Ready：**admitted**
- Ready → Running：**scheduler dispatch**（CPU scheduling）
- Running → Ready：**interrupt**（e.g. timer / time slice expired / preempted）
- Running → Waiting：**I/O or event wait**
- Waiting → Ready：**I/O or event completion**
- Running → Terminated：**exit**



Process State

- In traditional Unix systems, each process consists of **one thread**, thus the process state is defined in part by the current activity of that process
- As a process executes, it changes its **state** from one to another
  - New: The process is being created – allocating resources
  - Ready: The process is waiting to be assigned to a CPU core
  - Running: Instructions are being fetched and executed on a CPU
  - Waiting: The process is waiting for some event(s) to occur
  - Terminated: The process finishes execution – deallocating resources

Process state diagram **[3.9]**

> 📝 **HW1 Q2.3**：admitted → dispatched → disk I/O → I/O completes → timer interrupt → dispatched → exits **New** → **Ready** → **Running** → **Waiting** → **Ready** → **Running** → **Ready** → **Running** → **Terminated** 離開 Running 時 OS 喺 **PCB** 記錄：**process state, program counter, CPU registers**（context），以及更新 scheduling info、accounting info（CPU time used）、I/O status（e.g. pending I/O request）。

> ⚠️ 注意：**Waiting → Running 係冇嘅**（要先返 Ready）；**Ready → Waiting 亦冇**。 S26 MC14：令 process 離開 running state 嘅 event → **It issues an I/O request**（call function / 加 local variable / read cache 都唔會）。 Sp21 MC6：might force a process to leave the CPU → **all of the above**（fork system call、I/O request、interrupt）。

3. Process Control Block (PCB) ⭐ [3.10, 3.13]

PCB（又叫 task control block） = information associated with each process（kernel data structure）：

Field	內容
Process state	running, waiting, etc.
Program counter	location of the next instruction to execute
CPU registers	accumulators, index registers, **stack pointers**, general-purpose registers, condition codes
CPU-scheduling information	**priorities, pointers to scheduling queues**, scheduling parameters
Memory-management information	memory allocated（page tables / segment tables）
Accounting information	CPU used, clock time elapsed, time limits
I/O status information	I/O devices allocated, **list of open files**

- 每個 process 有 unique **PID (process identifier)**（UNIX/Linux 係 positive integer；**PID 1 = init / systemd**，root process）。
- **Process table / process list** — kernel data structure keeping track of all processes（indexed by PID, pointing to PCBs）。
- **Linux** [3.13]：**task list = circular doubly linked list** of `struct task_struct`（process descriptor = PCB）：`pid_t pid; long state; unsigned int time_slice; struct task_struct *parent; struct list_head children; struct files_struct *files; struct mm_struct *mm;`

4. Threads（簡介，詳見 Ch4） [3.11–3.12]

- **A thread is represented by a program counter, registers, execution flags, and a stack — part of a process.**
- 一個 thread 喺 CPU 執行 = 佢嘅 state 喺 CPU **hardware registers**。
- **Multiple threads of a process run within the same address space (share code, data, files); each thread has its own thread state.**（multicore 上可以 parallel）
- 例：web browser 一個 thread display、一個 thread 由 network 拎 data。
- 傳統 process（single thread） = **heavy-weight process**。

5. Process Scheduling（進程排程） ⭐ [3.14–3.18]

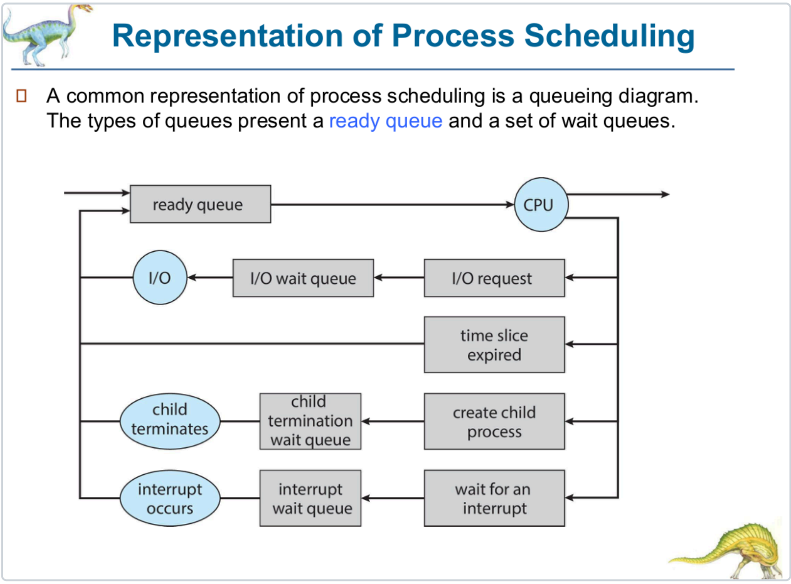
5.1 基本

- **Objective of multiprogramming：have some process running at all times to maximize CPU utilization.**
- **Process scheduler** — selects an available process (in memory) for execution on a CPU core。

- **Single CPU core → never more than one process running at a time.**
- **Degree of multiprogramming = number of processes currently residing in memory.** ⭐ （Sp21 MC）
- **I/O-bound process** — more time doing I/O；**many short CPU bursts**。
- **CPU-bound process** — infrequent I/O；**few very long CPU bursts**。

5.2 Scheduling Queues [3.15–3.16]

- **Ready queue** — processes in main memory, ready and waiting to execute
- **Wait queues** — processes waiting for an event（e.g. I/O completion）
- **Processes migrate among the various queues during their lifetime.**
- **Queueing diagram** [3.16]：ready queue → CPU → 出口有：**I/O request**（→ I/O wait queue → I/O → ready）、**time slice expired**（→ ready）、**create child process**（→ child termination wait queue）、**wait for an interrupt**（→ interrupt wait queue → interrupt occurs → ready）。



Representation of Process Scheduling

- A common representation of process scheduling is a queueing diagram. The types of queues present a ready queue and a set of wait queues.

Queueing diagram [3.16]

- Summary：**the state of a process can be determined by which queue it resides on**（running 啯個例外）。

5.3 Schedulers ⭐ ⭐ [3.17–3.18]

Scheduler	又叫	做咩	頻率
Long-term scheduler	**job scheduler**	**selects which processes should be brought into memory**（from job queue, after resource allocation）；**dictates the degree of multiprogramming**；aims for a **good process mix** of I/O-bound and CPU-bound；mainframes/minicomputers 用，**PC 冇**	**very infrequent**（seconds, minutes）→ may be slow
Short-term scheduler	**CPU scheduler**	**selects a process from the ready queue to execute next and allocates a CPU core**	**very frequent**（milliseconds）→ **must be fast**
Medium-term scheduler	—	**reduces degree of multiprogramming** — **removes a process from memory, stores it on disk, brings it back later = swapping**（swap space）	需要時

- F20 MC10：「__ swaps a process out of memory and temporarily stores it on disk」→ **medium-term scheduler**。
- Sp21 MC：「selects from the ready queue to execute on the CPU」→ **short-term scheduler**。

6. Context Switch（上下文切換） ⭐ ⭐ [3.19–3.20]

- **When the CPU switches to another process, the OS must save the state of the old process and load the saved state of the new process via a context switch.**
- **Context of a process is represented in the PCB**（or thread）：registers, program counter, stack pointer。
- **Context-switch time is pure overhead — the system does no useful work while switching.**
- **The more complex the OS and the PCB, the longer the context switch.**
- 速度取決於 **memory speed, number of registers to copy, special instructions (load/store all registers)** → **highly dependent on hardware**。
- **Some processors provide multiple sets of registers（SUN UltraSPARC）→ context switch = changing a pointer to another register set.**

⚠️ S26 MC11 ✓「**During context switch, OS saves the state of one process/thread and restores another**」。✗「Context switch means switching from user mode to kernel mode」（嗰個係 **mode switch**，唔係 context switch）✗「requires same hardware on all architectures」；✗「shorter if OS is more complex」

7. Process Creation（建立進程）⭐⭐⭐ [3.32–3.46]

7.1 概念

- **Parent process creates child processes → tree of processes.** 每個 process 有 unique **pid**。
- Linux tree [3.33]：`systemd (pid 1)` 有三個 children：`logind (8415)`、`python (2808)`、`sshd (3028)`；`logind` → `bash (8416)` → `ps (9298)`、`vim (9204)`；`sshd (3028)` → `sshd (3610)` → `tcsh (4005)`。
- Child 需要 resources（CPU time, memory, files, I/O devices）；parent 可傳 initialization data。
- **Resource sharing 三種選擇** [3.34]：
  1. **Parent and children share (almost) all resources** — `fork()`
  2. **Children share a subset of parent's resources** — `clone()`（Ch4）
  3. **Parent and child share no resources**
- **Execution 兩種選擇**：
  1. **Parent continues to execute concurrently with its children**（例：`a.out &`）
  2. **Parent waits until some or all of its children have terminated**（例：shell 等 `a.out` 完成）
- **Address space 兩種選擇** [3.35]：
  1. **Child is a duplicate of the parent (same program and data)** — UNIX/Linux `fork()`
  2. **Child has a new program loaded into it** — `exec()`

7.2 fork() ⭐⭐⭐ [3.35–3.39]

- **`fork()` creates a new process that duplicates the entire address space of the parent. Both processes continue execution at the instruction after `fork()`.**
- **Return value**（each process receives exactly one return value）[3.36–3.37]：

Return value	喺邊個 process	意思
> 0	**parent**	值 = **PID of the newly created child**
= 0	**child**	新建嘅 child
< 0（**−1**）	**parent (original)**	**fork failed**，**no child created**，errno set

- **All state of the parent is duplicated in the child**：memory (program, data), **file descriptors**…；每個 process 有**自己一份** program 同 data（time consuming）。
- **Steps to implement fork()** [3.38]：
  1. **Create and initialize the PCB** in the kernel
  2. **Create a new address space**（allocate memory）
  3. **Initialize the address space with a copy of the parent's entire address space**（time consuming）
  4. **Inherit the execution context of the parent**（e.g. open files）
  5. **Inform the CPU scheduler that the child is ready to run**
- **Parent vs Child after fork()** [3.39]：
  - **Duplicated**：address space, **global & local variables**, current working directory, root directory, process resources, resource limits, **program counter (PC)**…
  - **Different**：**PID**, **return value from fork()**, running time, running state
- ⚠️ **Child 改 variable 唔會影響 parent**（各自一份 copy）— HW1 MC9 ✗「If the child changes a global variable, the parent's copy changes automatically」。
- ⚠️ **Execution order of parent and child depends on the scheduler** unless synchronization（`wait()`）is used。
- F20 MC7 ✗「**The parent process waits until the completion of the child process**」（fork 本身唔會等，要 `wait()`）。
- Sp21 MC ✗「**The child process can call wait() in order to finish earlier than the parent**」。

7.3 Copy-on-Write (COW) & clone() [3.46]

- **Process creation in UNIX is unique — separated into two system calls: `fork()` and `exec()`.**（其他 OS：create new address space + read executable + start 一次過）
- **Linux implements fork() with copy-on-write (COW) pages** — parent and child **share a single copy**；**copying is delayed until a page is written**。若 `fork()` 後即刻 `exec()`，pages **never need to be copied** → **greatly speeds up process creation**。
- **Linux also implements fork() via `clone()`**（more general；flags 指定 share 邊啲 resources）。

7.4 exec / wait / exit / signal [3.41–3.43]

- `exec()` / `execlp()` — **changes the program being run by the current process；creates a new process image from a regular executable file（replaces the address space；成功就唔會 return**，fail 先 return −1 並繼續執行下一行）。
- `wait()` — wait for a (child) process to finish；`pid = wait(&status);` 返回 terminated child 嘅 PID，`status` 存 exit status。
- `exit()` — terminate the current process and free all its resources。
- `signal()` — send a notification to another process（slide 3.41 原文）。
  - ⚠️ slides 前後唔一致：slide 4.31 話真正 **deliver** signal 畀 process 嘅 UNIX function 係 **`kill(pid, signal)`**；而實際 UNIX 嘅 `signal(SIGINT, handler)` 係**裝 handler**（PA1 就係咁用）。MC 照題目用字對返邊張 slide 作答。
- 經典 code [3.42]：

```
pid = fork();
if (pid < 0) { fprintf(stderr, "Fork Failed"); exit(-1); }   /* error */
else if (pid == 0) { execlp("/bin/ls", "ls", NULL); }        /* child */
else { wait(NULL); printf("Child Complete"); exit(0); }      /* parent */
```

- **Implementing a shell** [3.43]：loop `readAndParseCmdLine` → `fork()` → child `exec(prog, args)`（NOT REACHED after）→ parent `wait(child_pid)`。
- F19 Q2.4：shell 要 start new process 必須用嘅兩個 system calls → **fork()**（clones the current process）+ **exec()**（overlays a new program onto the calling process）。

8. Process Termination（終止）⭐ ⭐ [3.47–3.50]

- Process 執行完最後一句 → `exit()` 要求 OS 刪除佢；可以經 `wait()` 回傳 **status value** 畀 parent。
- **Resources (memory, open files, I/O buffers) are deallocated and reclaimed by the OS.**
- **Parent can terminate children**：`abort()`（UNIX/Linux）、`TerminateProcess()`（Windows）；parent 需要知 child PID（由 fork() 返回）。理由：child **exceeded allocated resources**；**task no longer required**；**parent is exiting**（有啲 OS 唔准 child 冇 parent）。
- **Cascading termination** — if a process terminates, **all its children must also be terminated**（initiated by OS）。
- Normal termination：`exit()` 直接或間接被 call（C runtime library 預設 call `exit()`）。

8.1 Zombie ⭐ ⭐

- **A process that has terminated, but whose parent has not yet called `wait()`, is a zombie process.**
- 佢嘅 **entry in the process table, PID, and PCB still exist**（因為 process table entry 存住 **exit status**）；其他 resources 已釋放；**not runnable**（冇 address space）。
- **All processes transition into the zombie state before termination, but generally only briefly.**
- **Once the parent calls `wait()`, the zombie's PID, process-table entry and PCB are released.**
- S26 MC13 ✓「**A zombie process has terminated but its parent has not yet collected its status**」；✗ still executing user code；✗ created when exec() fails；✗ owns no PID。

8.2 Orphan ⭐

- **If a parent terminates without invoking `wait()`, its child becomes an orphan**（如果冇 cascading，可能仍 runnable，最終變 zombie）。
- **`init` / `systemd` is assigned as the new parent and periodically calls `wait()`** → 收集 exit status，釋放 PID 同 process-table entry。

9. fork() 解題大法 ⭐ ⭐ ⭐（考試必用）

9.1 基本規則

1. **每 call 一次 fork()，「每個執行到嗰句嘅 process」都會變兩個**：parent（return child PID > 0 → **true**）、child（return 0 → **false**）。
2. **Child 由 fork() 下一句開始行**，有 parent 所有 variables 嘅 **copy**（包括 loop counter `i`）。
3. **n 句連續 fork()（無條件）** → 2^n **processes**。
4. **Short-circuit evaluation** ⭐：
   - `A && B`：A **false (0 → child)** 就**唔會**執行 B；A **true (parent)** 先執行 B。
   - `A || B`：A **true (parent)** 就**唔會**執行 B；A **false (child)** 先執行 B。
5. **printf 次數 = 執行到該句 printf 嘅 process 數目**（每個 printf 位置分開計）。
6. `return` / `exit()` / `break` 會令 process 停止繼續 loop 或者唔再執行後面。
7. `exec()` 成功 → 之後嘅 code **唔會執行**；失敗 → return −1 繼續。

9.2 常見 pattern 公式（全部已用真 C 程式驗證 ✓）

Code（每個 iteration，所有 process 都繼續 loop）	每 iteration 每個 process 變成	n 次後 processes
`fork();`	2	2^n
`fork(); fork();` [3.44]	4	4^n（slide：10 次 → 4^{10}）
`if (fork()) fork();` [3.45]	3（parent 再 fork 一次）	3^n（slide：3^{10}）
`if (fork() == 0) fork();`（Sp21 MC8）	3（child 再 fork）	3^n（n=3 → **27**）
`if (fork() && fork()) ...` / `fork() || fork()`	3	3^n

9.3 有 break / return 嘅 loop（追蹤「仲 active」嘅 process）

S26 Q3.1：

```
for (int i = 0; i < n; i++) {
    if (fork()) fork();    // parent: fork 多一次
    else break;             // 第一個 child: break（停 loop）
}
```

- 每個 active process 每 iteration：第一個 fork 產生 child（→ else → **break**，停）；parent 再 fork 一次，**新 child（第二個）繼續 loop**（佢喺 if branch 入面，return 0 但已經喺 if 入面）。
- 所以 1 個 active → 3 個 processes，其中 **2 個繼續 active**、1 個 stopped。
- $A_k = 2^k$（iteration k 開始前 active 數）；每 iteration 新增 2^k 個 break。
- **Total processes** $= \underbrace{2^n}_{\text{active}} + \underbrace{(2^n - 1)}_{\text{stopped}} = 2^{n+1} - 1$；**break 次數** $= 2^n - 1$。
- n = 2：**7 processes，3 breaks** ✓；n = 3：15 processes，7 breaks ✓。
- Process tree（n=2）：P0 → P1 (break), P2 → {P5 (break), P6}, P3 (break), P4...（P0 第 0 輪產生 P1(break)、P2；第 1 輪產生 P3(break)、P4；P2 第 1 輪產生 P5(break)、P6）

```
P0
├── P1   (break at i=0)
├── P2   (continues)
│   ├── P5 (break at i=1)
│   └── P6
├── P3   (break at i=1)
└── P4
```

S26 MC10：

```
for (i = 0; i < 3; i++) {
    if (fork() == 0) { fork(); printf("X "); return 1; }
}
printf("Y ");
```

- 只有 original parent 繼續 loop；每 iteration 產生 1 個 child，child 再 fork → 2 個 process 各印 "X " 後 return。
- 3 iterations → 3 × 2 = 6 個 process 印 X；original 印 1 個 Y。**Total = 7 processes, X = 6, Y = 1 → 答案 C** ✓

9.4 Short-circuit 題

Sp21 MC7： `if (fork() || fork()) { fork(); }`

- P0：fork1 → P0 得 >0（true）→ **跳過第二個 fork** → 入 if → fork → 2 個
- C1（fork1 得 0）→ 執行 fork2：C1 得 >0（true）→ 入 if → fork → 2 個；C2 得 0 → false → 唔入 if → 1 個
- **Total = 2 + 2 + 1 = 5** ✓

F19 Q2.7： `if (fork() == fork() || fork() != fork()) fork(); printf("Fork!\n");`

- `fork() == fork()`：4 個 processes，返回值組合 (x,y)：P0 (c1,c2) 唔等；C2 (c1,0) 唔等；C1 (0,c3) 唔等；**C3 (0,0) 相等 → true**（只有收到兩次 0 嗰個 grandchild）。
- C3：|| 短路 → 執行 body fork() → **2 個** process 印。
- 其餘 3 個 process 執行 `fork() != fork()` → 每個變 4 個 = 12；每組 4 個入面 3 個 true（(c,c')、(0,c)、(c,0)），1 個 false (0,0)。
- 9 個 true → fork → **18**；3 個 false → **3**。
- **Total = 2 + 18 + 3 = 23 lines** ✓

9.5 Nested / combined

Sp21 Q1.1：

```
pid1 = fork(); pid2 = fork();
if (pid1 > 0) if (pid2 == 0) fork();
```

- 兩個 fork 後 4 個 processes；只有「pid1 > 0（P0 或 P0 嘅第二個 child）且 pid2 == 0」嘅 **P0 第二個 fork 生出嘅 child** 符合 → 再 fork 1 次 → **5 processes** ✓

Sp21 Q1.2：

```
if (fork() == 0) { for (int i = 0; i < 5; i++) if (fork() == 0) fork(); }
fork();
```

- 第一個 fork：child 入 loop；loop 5 次，每次每個 process 變 3 個 → $3^5 = 243$（包括 child 自己）。
- 加 original parent = 243 + 1 = 244；最後 fork 全部 ×2 → $(3^5 + 1) \times 2 = 488$ ✓

F20 Q3.2：

```
if (!fork()) { for (int i = 0; i < n; i++) if (fork()) fork(); }
fork(); printf("a");
```

- 同上：child branch 變 3^n，加 parent → $3^n + 1$，最後 fork ×2 → **"a" 數** $= 2(3^n + 1)$ ✓（n=1 → 8；n=2 → 20）

F19 Q2.6（loop 入面 parent 同 child 都 printf）：

```
for (i = 0; i < n; ++i) { if (fork()) printf("Fork!\n"); else printf("Fork!\n"); }
```

- 第 i 次 iteration（由 1 數）有 2^i 個 process 各印一行 → $2 + 4 + \cdots + 2^n = \sum_{i=1}^{n} 2^i = 2^{n+1} - 2$ ✓

9.6 Possible output orderings（輸出次序題）⭐

方法：

1. 畫 process tree，寫低每個 process 印咩、依咩次序（**同一 process 內次序固定**）。
2. 加 **wait()** 約束：parent `wait()` 之後嘅 print 一定喺 child **全部** print 之後。
3. Fork 之前印嘅一定最先（假設 flushed）。
4. 列出所有符合約束嘅 **interleavings**（topological orders）。

F20 Q3.1：

```
if (fork() == 0) {              // child C
    if (fork() > 0) { printf("1"); wait(NULL); printf("2"); }  // C
    else printf("3");           // grandchild G
} else printf("4");             // parent P
```

- 約束：C 印 "1" 然後等 G（印 "3"）→ 再印 "2"。即 **"2" 喺 "1" 同 "3" 之後**；"4" 隨意。
- "2" 最後：1、3、4 任意排 = 3! = 6 種：**1342, 1432, 3142, 4132, 3412, 4312**
- "4" 喺 "2" 之後：**1324, 3124**
- **Total = 8**（unbuffered）。若 buffered（"1" 同 "2" 同一 buffer 一齊 flush）→ 只剩 **4312, 3412, 3124**（3 種）。

Sp21 Q1.3：`printf("A\n"); if (fork()==0) printf("B\n"); else { printf("C\n"); ___A___ } printf("D\n");`

- (a) A 空白：**ABDCD, ABCDD, ACBDD, ACDBD**（4 種；A 一定最先；child 印 B,D；parent 印 C,D）
- (b) A = `wait(NULL)`：parent 嘅 D 一定最後 → **ABDCD, ABCDD, ACBDD**
- (c) A = `printf("E\n")`：parent 印 C,E,D；child 印 B,D → 7 種：**ABDCED, ABCDED, ABCEDD, ACBDED, ACBEDD, ACEBDD, ACEDBD**

HW1 Q3.1（x 值）：

```
int x = 1; pid_t c1 = fork();
if (c1 == 0) { x += 2; printf("C1: %d\n", x); return 0; }   // 3
x *= 2; pid_t c2 = fork();
if (c2 == 0) { x += 3; printf("C2: %d\n", x); return 0; }   // 2+3 = 5
wait(NULL); wait(NULL); printf("P: %d\n", x);                // 2
```

- **C1: 3, C2: 5, P: 2** — 因為 **each process has its own copy of x**（fork 時複製）。
- Possible outputs：`C1:3 → C2:5 → P:2` 或 `C2:5 → C1:3 → P:2`（P 一定最後，因為 wait 兩次）。
- 若將第一個 `wait(NULL)` 移到 `x *= 2` 前：parent 先等 C1 完，先 fork C2 → **保證 C1: 3, C2: 5, P: 2**；**數值不變**（wait 唔改 x）。

HW1 Q3.2（exec）：

```
printf("A\n"); pid_t p = fork();
if (p == 0) { pid_t q = fork();
    if (q == 0) { execlp("./worker","worker",(char*)NULL); printf("X\n"); return 1; }
    wait(NULL); printf("C\n"); return 0; }
printf("P\n"); wait(NULL); printf("D\n");
```

- **3 processes**：parent → child → grandchild（**grandchild exec worker**，印 W）。
- **X 唔會印**：execlp 成功 → address space 被取代，**never returns**。
- 約束：A 最先；W < C < D；P < D。→ **A P W C D**、**A W P C D**、**A W C P D**（3 種）。

9.7 fork + exec 成功/失敗情況（S26 Q3.2）⭐

```
int ret = fork();
if (ret == 0) { printf("Hello1\n"); exec("some_executable"); printf("Hello2\n"); }
else if (ret > 0) { wait(NULL); printf("Hello3\n"); }
else { printf("Hello4\n"); }
```

Case	Output
fork 成功、exec 成功	`Hello1`, `Hello3`（Hello2 唔印：image replaced）
fork 成功、exec 失敗	`Hello1`, `Hello2`, `Hello3`（exec 返回 −1，繼續）
fork 失敗	`Hello4`（only parent, ret < 0）

- 因為有 `wait(NULL)`，Hello3 一定喺 child 嘅輸出之後。

9.8 更多歷屆 fork 題（Lab 3 review 提供：F22 / F24 / S25，全部驗證 ✓）

題目	Code（重點）	答案
Fall 2022 Q3	`for(i<n){ if (fork() == fork()) printf(...); }`	processes 4^n；printed lines $4^0 + 4^1 + \cdots + 4^{n-1} = \frac{4^n - 1}{3}$
Fall 2024 P3.1	`for(i<M){ pid = fork(); if(!pid) for(j<N) fork(); }`	**child** 行 line Y；M=N=1 → 3；一般 $(2^N + 1)^M$
Spring 2025 MC12	`if (fork() || fork()) { if (fork() && !fork())` `fork(); }`	**9**（外層 2 入 if × 每個變 4 + 1 入 else）
Lab 3 Ex3	`for(i<n){ if(fork()){fork();fork();} else fork(); }`	6^n（parent 變 4 + child 變 2）

- 詳細解釋：**Labs notes L3.7**。Fall 2022 嘅 lines：每輪只有收到兩個 0 嘅 grandchild 印，第 k 輪有 4^{k-1} 個這樣嘅 process。

9.9 Buffering 陷阱

- `printf` 冇 `\n` 或者 output 去 file/pipe → **stdout buffer 未 flush 就 fork → buffer 被複製 → 重複印**。
- 考試通常講明「stdout is flushed after each printf」或「printf not buffered」。如果冇講，可以寫一句 assumption。

---

10. Interprocess Communication (IPC) ⭐ ⭐ [3.51–3.64]

10.1 Independent vs Cooperating

- **Cooperating processes can affect or be affected by other processes**（e.g. share data）。
- **Reasons for cooperation**：**information sharing**（copy & paste）、**computation speedup**（parallel subtasks）、**modularity**（+ convenience）。
- 需要 **IPC mechanism** 交換 data。**兩大模型**：
  1. **Shared memory** — a region of memory is shared；processes read/write the shared region（**implicit** communication；快，但要自己 synchronize）
  2. **Message passing** — **explicit** messages exchanged（**useful in distributed environment**；Mach OS 主要用；唔需要 share address space）

10.2 Producer–Consumer Problem（Bounded Buffer, Shared Memory） [3.53–3.56]

- **Unbounded buffer**：no practical limit；**bounded buffer**：fixed size。

```
#define BUFFER_SIZE 10
item buffer[BUFFER_SIZE]; int in = 0, out = 0;
/* Producer */
while (true) {
    while (((in + 1) % BUFFER_SIZE) == out) ;   /* buffer full → do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
/* Consumer */
while (true) {
    while (in == out) ;                          /* buffer empty → do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
}
```

- ⭐ **This solution only allows at most `BUFFER_SIZE - 1` items in the buffer at the same time**（因為 `in == out` 表示 empty，要留一格分辨 full）。

10.3 Message Passing [3.57–3.64]

- **Mechanism for processes to communicate and synchronize without sharing the address space.**
- 兩個 operations：`send(message)`、`receive(message)`；message size **fixed or variable**。
- P、Q 要通訊：**establish a communication link** → exchange via send/receive。

- Implementation 選擇：**direct or indirect**；**synchronous or asynchronous**；**automatic or explicit buffering**。

Direct communication [3.59]：

- **Processes must name each other explicitly**：`send(P, message)`、`receive(Q, message)`。
- Link **established automatically**；**associated with exactly one pair**；**between each pair exactly one link**。
- **Disadvantage：limited modularity** — 改一個 process 嘅 identifier 要檢查所有其他 processes。

Indirect communication [3.60–3.61]：

- **Messages sent to and received from mailboxes (ports)**；每個 mailbox 有 unique id；**share a mailbox 先可以通訊**。
- **A link may be associated with more than two processes**；**each pair may have several links**（each = one mailbox）。
- Operations：create mailbox、send/receive through mailbox、destroy mailbox；`send(A, msg)`、`receive(A, msg)`。
- **Mailbox sharing problem**（P1 send，P2 P3 都 receive，邊個收？）解決：
  1. **Allow a link to be associated with at most two processes**
  2. **Allow only one process at a time to execute** `receive()`
  3. **Allow the system to select the receiver arbitrarily; sender is notified who the receiver was**

Synchronization (blocking vs non-blocking) [3.62–3.63] ⭐：

	Blocking (synchronous)	Non-blocking (asynchronous)
Send	sender **blocked until the message is received** (by receiver or mailbox)	sender sends and **resumes**
Receive	receiver **blocks until a message is available**	receiver gets **a valid message or null**

- **Both send and receive blocking → rendezvous**；producer–consumer 變得 trivial（`send(next_produced)` / `receive(next_consumed)`）。

Buffering（message queue） [3.64] ⭐：

1. **Zero capacity** — no messages queued；**sender must wait for receiver (rendezvous)**
2. **Bounded capacity** — finite length *n*；**sender must wait if the link is full**
3. **Unbounded capacity** — infinite；**sender never waits**

11. Pipes ⭐⭐⭐ [3.65–3.69]

11.1 設計問題（4 個）

1. **Unidirectional or bidirectional?**
2. If bidirectional：**half-duplex**（一次一個方向）or **full-duplex**（同時雙向）？
3. **Must there be a relationship (parent–child)?**
4. **Can pipes be used over a network, or only on the same machine?**

11.2 Ordinary Pipes vs Named Pipes（必背表）

	Ordinary pipe	Named pipe
方向	**Unidirectional**（one-way）	**Bidirectional**（UNIX FIFO：**half-duplex**；Windows：**full-duplex**）
關係	**Requires parent–child relationship**	**No parent–child relationship required**
存取	**Cannot be accessed from outside the process that creates it**	**Several processes can use it**（multiple writers）
生命週期	**Ceases to exist after processes finish communicating and terminate**	**Continues to exist after processes finish, until explicitly deleted**
機器	Same machine	UNIX：same machine；**Windows：same or different machines**
UNIX API	`pipe(int fd[2])`：**fd[0] = read end，fd[1] = write end**	`mkfifo()`；**appear as typical files in the file system**（FIFOs）
Windows	**anonymous pipes**（`ReadFile()` / `WriteFile()`）	`CreateNamedPipe()`

- **UNIX treats a pipe as a special type of file → standard `read()` and `write()`；child inherits the pipe from the parent.**
- **Parent calls `pipe()` first, then `fork()`** → child inherits both ends。
- **If the parent writes and the child reads, the parent must close the read end, and the child must close the write end.** ⭐
- `ls | grep x`：shell 用 pipe 將 ls 嘅 output 接到 grep 嘅 input（tools 唔使特別設計都可以組合）。

> ⚠️ **MC 陷阱**：S26 MC12 ✗「**ordinary pipes can be accessed by arbitrary processes**」；Sp21 MC5 ✗「**named pipes cease to exist after the communicating processes have finished**」；HW1 MC10 同樣 ✗。F20 MC9：「multiple processes, no parent-child relationship, independent from network」→ **named pipes**（socket/RPC 需要 network）。

11.3 Pipe example（Parent → Child） [3.68]

```
int pfds[2]; char buf[30];
pipe(pfds);                       /* 1. create pipe BEFORE fork */
pid_t pid = fork();               /* 2. child inherits pfds[0], pfds[1] */
if (pid != 0) {                   /* parent = writer */
    close(pfds[0]);               /* close unused READ end */
    write(pfds[1], "test", 5);
    wait(NULL);
} else {                          /* child = reader */
    close(pfds[1]);               /* close unused WRITE end */
    read(pfds[0], buf, 5);
    printf("CHILD: read \"%s\"\n", buf);
}
```

> 📝 **HW1 Q2.5 Model answer**：*(1) The parent calls `pipe(fd)` **before** `fork()` so that both processes share the same pipe (fd[0] read end, fd[1] write end). (2) Call `fork()`; the child inherits both file descriptors. (3) Parent (writer) closes fd[0] and writes data with `write(fd[1], ...)`, then closes fd[1]. (4) Child (reader) closes fd[1] and reads with `read(fd[0], ...)`, then closes fd[0]. Data flows parent → child (one-way).* **HW1 Q3.3(c)**：如果 parent 同 child **各自喺 fork() 之後 call pipe()** → 會建立**兩條獨立 pipe**，佢哋**唔共享** file descriptors → **cannot communicate**。 **HW1 Q3.3(b)**：`Child: 12` 然後 `Parent: done` — 因為 child 嘅 `read()` **blocks until data is written**，而 parent `wait(NULL)` 等 child 結束先印。

11.4 dup / dup2 & I/O redirection（Lab 3 / PA1） ⭐⭐

- **File descriptor table**：0 = stdin，1 = stdout，2 = stderr。
- `dup(oldfd)` — 複製 oldfd 到 **lowest-numbered unused fd**，返回新 fd。
- `dup2(oldfd, newfd)` — 令 newfd 指向 oldfd 嘅 file（如 newfd 已開會先 close）。
- **Input redirection 經典 pattern**：`close(0); dup(fin);` ⟹ fd 0 變成 file → `scanf` 由 file 讀。等價 `dup2(fin, 0);`
- **Output redirection**：`close(1); dup(fout);` 或 `dup2(fout, 1);`
- **Pipe 接 stdout → stdin**（`ls | wc`）：writer child：`dup2(fd[1], 1); close(fd[0]); close(fd[1]); execlp("ls",...)`；reader child：`dup2(fd[0], 0); close(fd[1]); close(fd[0]); execlp("wc",...)`；**parent 要 close 兩端**（否則 reader 永遠收唔到 EOF）。

S26 Q3.3（填空）：child 處理 1.txt、parent 處理 2.txt，數 even numbers，child 經 pipe 傳結果：

```
int fin = open(file_name[i], O_RDONLY);
close(0);                          // Blank 1
dup(fin);                          // Blank 2  → stdin now = file
scanf("%d", &length);
for (j = 0; j < length; j++) {
    scanf("%d", &number);
    if (number % 2 == 0) cnt++;    // Blank 3
}
if (pid == 0) { close(fd[0]); write(fd[1], &cnt, sizeof(int)); exit(0); }  // 4.1 fd[1], 4.2 &cnt
waitpid(pid, &status, 0); close(fd[1]);
read(fd[0], &child_cnt, sizeof(int));                                    // 5.1 fd[0], 5.2 &child_cnt
```

12. Client–Server Communication：Sockets & RPC ⭐ [3.70–3.72]

- **A socket is defined as an endpoint for communication.** A pair of processes over a network uses **a pair of sockets**。
- **A socket is identified by an IP address and a port number.**
- **Server** always running，**listens to a well-known port**；接受 request → 建立 socket connection。
- **Well-known ports**（< 1024）：**SSH 22、FTP 21、HTTP 80**。
- **Only clients can initiate communication.** Client 揀一個 **port number > 1024**（未用緊）。
- IP：IPv4 32-bit（128.32.244.172）、IPv6 128-bit。
- ⭐ **The 4-tuple (source IP address, source port number, destination IP address, destination port number) uniquely determines a pair of communication.**（F19 Q2.2 答案）
- **RPC (Remote Procedure Call)** — **procedure-call mechanism between systems with network connections**；**message-based**；常用於 **distributed file systems**（RPC daemons + clients）。

13. 常見陷阱 & MC 速查

敘述	真/假
Program = active, process = passive	✗（program passive；process active）
Stack holds function params, return addresses, local variables	✓
Heap holds global variables	✗（data section；heap = dynamic allocation）

敘述	真/假
Ready → Waiting transition exists	✗
Long-term scheduler controls degree of multiprogramming	✓
Short-term scheduler invoked infrequently	✗（very frequently, ms）
Medium-term scheduler does swapping	✓
Context switch = user→kernel mode switch	✗
Context-switch time is useful work	✗（pure overhead）
fork() returns 0 to parent	✗（0 to child；child PID to parent）
fork() fails → returns −1 to parent, no child	✓
After fork, child shares parent's variables	✗（copy；COW 只係 implementation 優化）
Parent automatically waits for child after fork	✗（需要 wait()）
exec() returns on success	✗（只有 failure 返回 −1）
Zombie still runs user code	✗
Zombie keeps PID & process-table entry	✓
Orphans are adopted by init/systemd	✓
Ordinary pipe: unidirectional, parent-child	✓
Named pipe ceases after processes end	✗
Named pipe (UNIX FIFO) full-duplex	✗（half-duplex；Windows named pipe 先 full-duplex）
Blocking send/receive = synchronous	✓
Zero-capacity buffer: sender never waits	✗（unbounded 先 never waits）
Bounded-buffer shared-memory solution holds BUFFER_SIZE items	✗（BUFFER_SIZE − 1）
Mode bit 1 = kernel	✗（1 = user，0 = kernel）
Privileged instruction in user mode → trap to OS	✓
DMA interrupts after every byte	✗
Trap = software-generated interrupt	✓
Interrupts all have same priority	✗

14. Past-paper 題目（Ch3）

- **S26**：MC1 interrupts（C）、MC2 DMA（B）、MC10 fork（C：7, X=6, Y=1）、MC11 context switch（B）、MC12 ordinary pipes false（B）、MC13 zombie（B）、MC14 leave running（A）、Q3.1 fork tree（7 / 3 / $2^{n+1}-1$）、Q3.2 fork+exec cases、Q3.3 pipe/dup 填空
- **F19**：Q2.1 state transitions、Q2.2 socket 4-tuple、Q2.4 fork+exec、Q2.6（$2^{n+1}-2$）、Q2.7（23）
- **F20**：MC3 interrupt、MC7 fork、MC8 address space（all of above）、MC9 named pipes、MC10 medium-term、Q3.1（8 outputs）、Q3.2（$2(3^n+1)$）
- **Sp21**：MC2 interrupt、MC degree of multiprogramming、MC own address space etc.（all）、MC stack、MC short-term、MC fork incorrect（B）、MC pipes not true（C）、MC leave CPU（all）、MC7（5）、MC8（27）、Q1.1（5）、Q1.2（488）、Q1.3 outputs
- **HW1**：MC1 DMA、MC2 interrupts、MC7 privileged、MC9 fork、MC10 pipes、Q2.3 states、Q2.5 pipe steps、Q3.1–3.3

15. 自我測驗

1. `for (i=0;i<4;i++) fork();` 有幾多個 processes？（16）
2. `fork(); fork() && fork(); fork();` 有幾多？（2 × 3 × 2 = **12**：第一個 fork → 2；`fork() && fork()` 每個變 3；最後 ×2）
3. Zombie 同 orphan 分別？
4. 點解 bounded buffer 只可以放 BUFFER_SIZE − 1 個 item？
5. Named pipe 喺 UNIX 叫咩？點建立？（FIFO；mkfifo()）
6. 寫出三種 mode transfer。（system call, interrupt, trap/exception）

4 Ch4 Threads 線程與並行

📌 **考試範圍提示**：Ch4 通常 **3–4 題 MC** + 有機會 1 題 short question。必考： **thread 有乜自己私有 / 乜同其他 thread 共享**、**benefits（4 個）**、**concurrency vs parallelism**、**data vs task parallelism**、**Amdahl's Law 計算**、**multithreading models（many-to-one / one-to-one / many-to-many）**、**signals（sync vs async）**、**thread cancellation（asynchronous vs deferred）**、**fork()/exec() semantics**、**Linux clone()**、**heavyweight vs lightweight**。

1. Motivation & Overview（動機） [4.4–4.6]

1.1 點解要 multithreading？

- 好多 applications 本身係 multithreaded：
  - **Web browser**：一個 thread display images/text，另一個 thread 由 network retrieve data。
  - **Word processor**：display graphics、respond to keystrokes、background spelling & grammar checking。
- **Most OS kernels are multithreaded**（Linux boot 時建立多個 kernel threads：managing devices、memory management、interrupt handling）。
- **Take advantage of processing capabilities on multicore systems**（data mining、graphics、AI）。
- ⭐ **Process creation is heavy-weight while thread creation is light-weight** — threads of the same process **share code, data and other resources**。
- 其他例子 [4.5]：embedded systems（elevators, planes）、OS kernels、**database servers**、**network servers**（file/web server、airline reservation）、parallel programming。

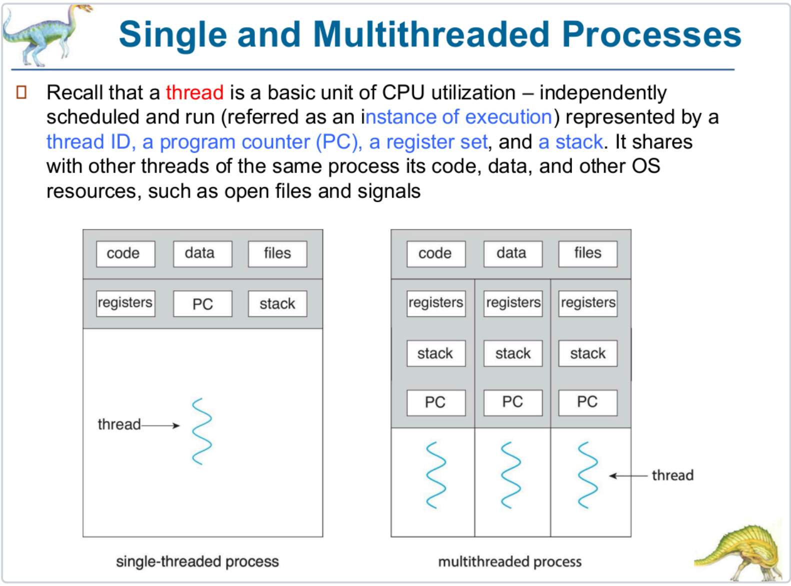
1.2 Thread 定義 ⭐ ⭐ [4.6]

- **A thread is a basic unit of CPU utilization** — independently scheduled and run（**an instance of execution**）。
- **每個 thread 私有（per-thread）：thread ID, program counter (PC), register set, stack**
- **同一 process 嘅 threads 共享（shared）：code (text), data (global variables), heap, open files, signals**（及其他 OS resources）

```
single-threaded process         multithreaded process

|  code  |  data  | files |      |  code  |  data  | files |    ← shared
|        |        |       |      |        |        |       |
|  regs  |   PC   | stack |      |  regs  |  regs  |  regs |    ← private
                                 | stack  | stack  | stack |      (per thread)
                                 |   PC   |   PC   |   PC  |
```

📝 **F19 Q2.3**：unique to each thread → **program counter, registers, stack, thread ID (thread state)**；shared → **code, data, files (heap)**。 ⚠️ **F20 MC11**：NOT shared by threads in the same process → **Stack**。



Single and Multithreaded Processes

□ Recall that a thread is a basic unit of CPU utilization – independently scheduled and run (referred as an instance of execution) represented by a thread ID, a program counter (PC), a register set, and a stack. It shares with other threads of the same process its code, data, and other OS resources, such as open files and signals

Single- vs multithreaded process [4.6]

2. Benefits（四大好處）⭐ ⭐ [4.7]

Benefit	解釋
Responsiveness	**may allow continued execution if part of a process (one thread) is blocked** — 對 interactive / UI applications 特別重要
Resource sharing	**threads share the resources of a process by default** — 比 shared memory / message passing 容易（同一 address space）
Economy	**thread creation is much cheaper than process creation**（less time & memory）；**context switching between threads is faster** than between processes
Scalability	process **can take advantage of multicore architectures**（threads run in parallel）

> 📝 HW2 MC1：app 喺一個 thread blocked 時仍然 respond user → **Responsiveness**。

2.1 Heavyweight vs Lightweight（F20 Q2.3）⭐

> 📝 **Model answer**：*A process is called a heavyweight process because creating it requires allocating a new address space and duplicating the program, data and other resources (PCB, memory, open files) — time- and resource-consuming. A thread is lightweight because it only contains an execution unit (thread ID, PC, registers, stack, TCB) and shares the code, data and open files of its process; creating a thread needs no duplication of program and data, and context switching between threads of the same process is cheaper since the address space stays the same.*

---

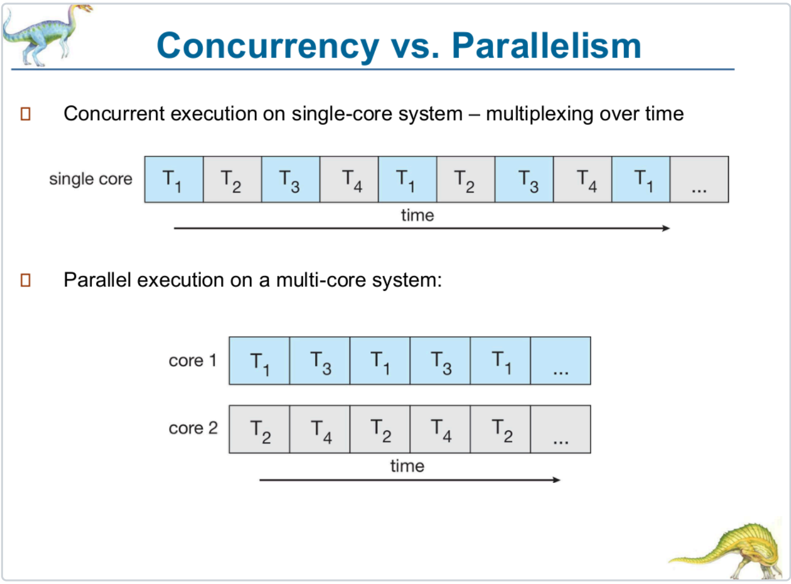
3. Multicore Programming（多核心編程）⭐ [4.8–4.11]

3.1 五大挑戰 [4.9]

1. **Dividing tasks** — how to divide into separate, concurrent tasks
2. **Balance** — each task performs an "equal" amount of work（value）
3. **Data splitting** — data must be divided to run on separate cores
4. **Data dependency** — **if there is a data dependency, synchronization is required**
5. **Testing and debugging** — **different execution paths make debugging difficult**

3.2 Concurrency vs Parallelism ⭐ ⭐ [4.9–4.10]

- **Parallelism** — **a system can perform more than one task simultaneously**（需要 multicore / multiprocessor）。
- **Concurrency** — **supports more than one task making progress；on a single core, the scheduler provides concurrency by multiplexing over time**。
- **Single-core：only concurrency is possible；multicore：parallelism possible.**
- **Concurrency without parallelism 係可以嘅；parallelism 一定 concurrent。**



Concurrency vs. Parallelism

- Concurrent execution on single-core system – multiplexing over time
- Parallel execution on a multi-core system:

single core
T_1 T_2 T_3 T_4 T_1 T_2 T_3 T_4 T_1 ...
time
core 1
T_1 T_3 T_1 T_3 T_1 ...
core 2
T_2 T_4 T_2 T_4 T_2 ...
time

Concurrency vs parallelism [4.10]

```
Single core (concurrency):   T1 T2 T3 T4 T1 T2 T3 T4 T1 ...   (time →)
Two cores (parallelism):     core 0: T1 T3 T1 T3 ...
                             core 1: T2 T4 T2 T4 ...
```

⚠️ **S26 MC16**：✓「**Concurrency can be supported on a single processor core through time multiplexing by the scheduler**」。✗「Concurrency means performing more than one task at exactly the same time」（嗰個係 parallelism）✗「Parallelism means tasks only make progress over time on a single core」✗「Testing and debugging become easier in multicore systems」 **Sp21 MC**：multiple tasks make progress on a single processor → **concurrency**。

3.3 Data vs Task Parallelism ⭐ [4.11]

- **Data parallelism — distributes subsets of the same data across multiple cores, same operation on each core**（e.g. 分開 sum 一個 array 嘅兩半；distributed ML）。
- **Task parallelism — distributes threads (tasks) across cores, each thread performing a unique operation**。
- **Not mutually exclusive** — an application may use both（hybrid）。

📝 HW2 MC2：「Threads on different cores perform the **same operation on different subsets of an array**」→ **data parallelism**。「One thread handles input, another does spell-check」→ task parallelism。

---

4. Amdahl's Law ⭐⭐⭐（計數題必考）[4.12–4.13]

🧮 **Amdahl's Law**

$$\text{speedup} \le \frac{1}{S + \dfrac{1-S}{N}}, \qquad \lim_{N\to\infty} \text{speedup} = \frac{1}{S}$$

- **S = serial portion，$1 - S$ = parallel portion，N = number of processing cores**。
- **As $N \to \infty$, speedup → $1/S$.**
- **Serial portion has a disproportionate effect on performance gained by adding cores.**
- Amdahl's Law identifies **theoretical speedup in latency at a fixed workload**。

4.1 例子（全部計算過）

題目	S	N	計算	Speedup
Slide：75% parallel	0.25	2	$\frac{1}{0.25+0.75/2} = \frac{1}{0.625}$	**1.6**
Sp21 MC：30% serial	0.3	7	$\frac{1}{0.3+0.7/7} = \frac{1}{0.4}$	**2.5**
HW2 Q3.1(a)：20% serial	0.2	4	$\frac{1}{0.2+0.8/4} = \frac{1}{0.4}$	**2.5**
HW2 Q3.1(b)	0.2	∞	$\frac{1}{S} = \frac{1}{0.2}$	**5**
S26 MC17 A：81% parallel	0.19	3	$\frac{1}{0.19+0.81/3} = \frac{1}{0.46}$	2.17
S26 MC17 B：81% parallel	0.19	5	$\frac{1}{0.19+0.81/5} = \frac{1}{0.352}$	2.84
S26 MC17 C：61% parallel	0.39	3	$\frac{1}{0.39+0.61/3} = \frac{1}{0.5933}$	**1.69 ✓（1.6–1.7）**
S26 MC17 D：61% parallel	0.39	5	$\frac{1}{0.39+0.61/5} = \frac{1}{0.512}$	1.95

4.2 解題 tips

- 留意題目畀 **parallel % 定 serial %**（S = serial！）。
- 反向題：要 speedup = X with N cores → 解 S：$S = \dfrac{1/X - 1/N}{1 - 1/N}$。
- 例：想 4 cores 有 3× speedup → $S = \dfrac{1/3 - 1/4}{1 - 1/4} = \dfrac{1/12}{3/4} = \dfrac{1}{9}$ ≈ **11.1% serial**。
- Speedup 上限永遠 $\le N$ 同 $\le 1/S$。

---

5. Multithreaded Process & TCB ⭐ [4.14–4.18]

5.1 Multithreaded process [4.14]

- **More than one instance of execution**，每個有自己 PC。
- **Threads share the same address space → can access the same data.**
- **Each thread has its own private set of registers.**
- **Context switch between two threads (T1 → T2) of the same process**：save T1's register state, restore T2's；**address space remains the same → overhead is much smaller**。**Switching to a thread of a different process → more overhead.**
- **Overlap of I/O with other activities within a single program**（一個 thread 用 CPU，另一個做 I/O）。

5.2 Thread Control Block (TCB) ⭐ [4.15]

- **Thread = single unique execution context = lightweight process**：PC, registers, execution flags, stack。
- Thread 喺 processor 執行 = **resident in the registers；PC register holds the address of the executing instruction**。
- **Each thread has a TCB**：
  - **Execution state**：CPU registers, program counter, **pointer to stack**
  - **Scheduling info**：state, priority, CPU time
  - **Accounting info**
  - **Various pointers**（implementing scheduling queues）
  - **Pointer to enclosing process (PCB)**
- ⚠️ Sp21 MC：**NOT in TCB → Process identifier**（PID 喺 PCB；TCB 只有 pointer to enclosing process）。

5.3 Shared vs Private state [4.16] ⭐

- **Threads encapsulate concurrency — "active" component；address space encapsulates protection — "passive" part.**
- **Shared by all threads in a process**：**contents of memory (global variables, heap)；I/O state (file descriptors, network connections)**
- **Private to each thread (kept in TCB)**：**CPU registers (including PC)；execution stack (parameters, temporary variables, saved PC)**
- **If a process has multiple threads, there is no "process state", only the state of each thread**（summary）。

5.4 Thread lifecycle [4.17–4.18]

- **new → ready → running → waiting → terminated**（同 process 一樣）。
- **"Active" threads are represented by their TCBs; TCBs are organized into queues by state**（ready queue、各 device queues：tape unit、disk unit、ethernet…，每個 queue 有自己 scheduling policy）。

> 📝 **HW2 Q2.1**：同一 process 兩個 threads 可唔可以有唔同 PC / register 值？→ **Yes** — 每個 thread 係獨立 flow of execution，有自己 PC、register set、stack；唔 running 時存喺 **TCB (Thread Control Block)**。

6. User Threads vs Kernel Threads ⭐ ⭐ [4.19–4.20]

- **User threads** — independently executable entities within a program, **created and managed by a user-level thread library；visible only to programmers and unknown to the kernel → cannot be scheduled on a CPU directly**。
- **Kernel threads** — **supported and managed by the OS；can be scheduled and executed on a CPU**（Windows, Linux, macOS, iOS, Android 都支援）。
- ⭐ **OS only manages and schedules kernel threads.**（F19 Q3.1：OS 排程嘅係 **kernel-level threads**）
- **Three primary thread libraries**：**POSIX Pthreads**、**Windows threads**、**Java threads**（對應 POSIX API、Win32 API、Java API）。
- F20 MC12：provides API for creating and managing threads → **Thread library (e.g. POSIX Pthreads)**。
- If a program defines no user threads, there is no kernel thread to schedule for it（user threads = concurrent & modular entities OS 可利用）。
- **TCB 只同 kernel thread 關聯；kernel thread is the only thing the CPU scheduler schedules**（summary）。

7. Multithreading Models ⭐ ⭐ ⭐ [4.21–4.26]

Model	Mapping	優點	缺點	例子
Many-to-One	many user threads → **1 kernel thread**	thread management 喺 user space（efficient）	**one thread blocking (blocking system call) blocks the entire process；cannot run in parallel on multicore**（only one kernel thread active at a time）	**Solaris Green Threads, GNU Portable Threads**（few systems now）
One-to-One	each user thread → **its own kernel thread**	**maximum concurrency**（another thread runs when one blocks）；**parallel on multicore**	**creating a user thread requires creating a kernel thread → overhead；number of threads per process may be restricted**	**Windows, Linux**（**most common now**）
Many-to-Many	many user threads → **smaller or equal number** of kernel threads	flexible；OS 可預先建立足夠 kernel threads（**thread pool**）；user threads 數目唔受 kernel threads 限制	**difficult to implement** in practice	Windows ThreadFiber package（otherwise uncommon）
Two-level	many-to-many + **allows a user thread to be bound to a kernel thread**	更靈活	複雜	—

- **Many-to-one 同 many-to-many 要決定邊個 user thread map 去 kernel thread → process-contention scope (PCS)**（Ch5）。

- **Concurrency 比較**：many-to-one < many-to-many < one-to-one。
- **Most OSes now use the one-to-one model**（cores 多咗，限制 kernel threads 數目冇咁重要）[4.26]。
- **LWP (lightweight process)**（summary）：user 同 kernel threads 之間嘅 intermediate data structure；**對 thread library 嚟講 LWP 好似一個 virtual processor**，library 將 user thread schedule 落 LWP；**每個 LWP attached to a kernel thread**；kernel thread block → LWP 都 block。

> ⚠️ **MC 陷阱：**
> - F20 MC13 ✗「**A user thread can be mapped to more than one kernel thread**」（冇 one-to-many）；✓ kernel threads scheduled by OS；✓ users manage user threads；✓ one user thread → at most one kernel thread。
> - Sp21 MC ✗「**The many-to-many model is easy to implement in practice**」。✓ many-to-one = many user → single kernel；✓ one-to-one more concurrency than many-to-one；✓ most OS now one-to-one。
> - **HW2 Q2.2**：many-to-one 喺 4 cores 可唔可以 parallel？→ **No**，只有一個 kernel thread，OS 只 schedule kernel threads，一次只可以喺一個 core 上跑（只有 concurrency 冇 parallelism）；一個 thread blocking system call 仲會 block 成個 process。

8. Threading Issues ⭐⭐ [4.27–4.33]

8.1 fork() and exec() semantics [4.28]

- **Does `fork()` called by a thread duplicate only the calling thread or all threads?**
- **Some UNIX systems have two versions of fork().**
- **If `exec()` is called immediately after forking → duplicating only the calling thread is appropriate**（exec 會取代成個 process）。
- **`exec()` works as normal — replaces the entire process including all threads.**

8.2 Signal Handling ⭐⭐ [4.29–4.31]

- **Signals notify a process that a particular event has occurred.**
- **Synchronous signals — illegal memory access, division by 0 → delivered to the same process that performed the operation causing the signal**。
- **Asynchronous signals — generated by an event external to the running process：terminating with 、timer expires**（`kill`）。
- 處理：有啲 ignore，有啲（illegal memory access）terminate program。
- **Signal handling pattern**：(1) **generated** by an event → (2) **delivered** to a process → (3) **handled** by a **default** or **user-defined** handler。
- **Every signal has a default handler (kernel); a user-defined handler can override the default.**
- **Multithreaded process 要 deliver 去邊？四個選擇：**
  1. **To the thread to which the signal applies**
  2. **To every thread in the process**
  3. **To certain threads in the process**
  4. **Assign a specific thread to receive all signals for the process**
- **Synchronous signals → delivered to the thread causing the signal**（not others）。
- **Some asynchronous signals (e.g. terminate process, Ctrl-C) → sent to all threads.**
- UNIX：`kill(pid_t pid, int signal)`（deliver to a process）；Pthreads：`pthread_kill(pthread_t tid, int signal)`（deliver to a specific thread）。
- ⚠️ 留意：slide 3.41 將 `signal()` 寫成「send a notification to another process」，但實際 **deliver** 用 `kill()`（4.31）；`signal(SIGINT, handler)` 係**安裝 user-defined handler**（PA1）。MC 照題目用字揀。

> ⚠️ **S26 MC18** ✗「**All signals should be delivered to all threads in a process**」。✓ signals can be received sync or async；✓ users can define custom handlers；✓ different signals can have different handlers。📝 HW2 MC3：**division by zero → synchronous；timer expiration → asynchronous**。📝 **HW2 Q2.3 (Ctrl+C)**：*Asynchronous — Ctrl+C makes the terminal send SIGINT, generated by an event external to the running process. Yes, the program can handle it differently: register a user-defined handler with `signal(SIGINT, handler)` (or `sigaction`), which overrides the default handler (terminate). (SIGKILL/SIGSTOP cannot be caught.)* — PA1 myshell 就係用 `signal(SIGINT, handle_sigint)`。

8.3 Thread Cancellation ⭐ [4.32–4.33]

- **Terminating a thread before it has finished；the thread to be cancelled = target thread.**
- **Asynchronous cancellation — terminates the target thread immediately**（可能喺 critical update 中途 → OS 未必可以 reclaim 所有 resources）。
- **Deferred cancellation — the target thread periodically checks whether it should terminate, allowing it to terminate itself in an orderly fashion**（**preferred**）。
- **Invoking cancellation only requests it; actual cancellation depends on thread state and type.**
- **Pthreads：if cancellation is disabled, it remains pending until enabled；default type is deferred.**

- Code [4.33]：

```
#include <pthread.h>
pthread_t tid;
pthread_create(&tid, 0, worker, NULL);   /* worker = start routine */
pthread_cancel(tid);                     /* request cancellation */
pthread_join(tid, NULL);                 /* wait for thread to terminate */
```

> 📝 HW2 MC4：deferred cancellation → **target thread periodically checks whether it should terminate**。

8.4 Thread-Local Storage（課本補充，Sp21 MC 考過）

- **Thread-local storage (TLS) is data that is unique to each thread**（類似 static data 但每個 thread 一份；同 local variables 唔同，local 只喺一個 function call 內存在）。

---

9. OS Examples [4.34–4.38]

9.1 Windows Threads

- Windows application = separate process，每個 process 可有多個 threads；**Windows uses the one-to-one mapping**。
- Thread 組成：**thread ID, register set, program counter, separate user and kernel stacks, private data storage area**（run-time libraries / DLLs 用）。
- **Register set + stacks + private storage area = context of the thread.**
- 數據結構：
  - **ETHREAD (executive thread block)** — pointer to process & to KTHREAD（kernel space）
  - **KTHREAD (kernel thread block)** — scheduling & synchronization info, **kernel-mode stack**, pointer to TEB（kernel space）
  - **TEB (thread environment block)** — thread id, **user-mode stack**, **thread-local storage**（**user space**）

9.2 Linux Threads ⭐ [4.37–4.38]

- **Linux does not differentiate between threads and processes — both are called tasks.**
- **A thread in Linux is a process that may share certain resources with other threads**；每個 thread 有自己 `task_struct`。
- 對比 Windows：Windows 一個 PCB（shared resources）指向兩個 TCB；**Linux 就係兩個 normal task_struct，set up to share resources**。
- `clone()` 用 flags 指定 share 咩：

```
clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0);
```

- CLONE_VM：share **address space**；CLONE_FS：share file-system info；CLONE_FILES：share **open files**；CLONE_SIGHAND：share **signal handlers**。
- **Instead of copying all data structures like fork(), the new task points to the parent's data structures depending on the flags.**
- **A normal `fork()` can be implemented as `clone(SIGCHLD, 0)`**（乜都唔 share → 等於 process）。

---

10. Pthreads API 速查（Lab 4 / Ch6 都用）

```
#include <pthread.h>
int pthread_create(pthread_t *tid, const pthread_attr_t *attr, void *(*start)(void *), void *arg);
int pthread_join(pthread_t tid, void **retval);     /* wait for thread tid */
void pthread_exit(void *retval);
int pthread_cancel(pthread_t tid);
pthread_t pthread_self(void);
/* mutex */
pthread_mutex_t m; pthread_mutex_init(&m, NULL);
pthread_mutex_lock(&m); /* critical section */ pthread_mutex_unlock(&m);
/* semaphore */
#include <semaphore.h>
sem_t s; sem_init(&s, 0 /*shared between threads*/, 1 /*initial value*/);
sem_wait(&s); /* P / wait: decrement or block */ sem_post(&s); /* V / signal: increment */
```

- 編譯：`gcc prog.c -lpthread`（或 `-pthread`）。
- 典型 pattern：

```
void *worker(void *arg) { int id = *(int *)arg; ...; return NULL; }
int main() {
    pthread_t t[4]; int ids[4];
    for (int i = 0; i < 4; i++) { ids[i] = i; pthread_create(&t[i], NULL, worker, &ids[i]); }
    for (int i = 0; i < 4; i++) pthread_join(t[i], NULL);
}
```

- ⚠️ 傳 `&i`（loop variable）畀多個 thread 會 race（i 會變）→ 用 `ids[i]` array。

11. 常見陷阱 & MC 速查

敘述	真/假
Each thread has its own address space and file descriptors	✗（share）
Threads of the same process cannot run concurrently on multiple cores	✗（one-to-one 可以 parallel）
Thread creation is usually faster than process creation	✓（S26 MC15）
Multithreaded process can't use multicore	✗
Stack is shared among threads	✗
Heap / global data shared	✓
Concurrency possible on single core	✓
Parallelism possible on single core	✗
Data parallelism = same operation on different data subsets	✓
As $N \to \infty$, Amdahl speedup → $1/S$	✓
Speedup can exceed N	✗
OS schedules user threads directly	✗（kernel threads）
Many-to-one allows parallelism on multicore	✗
Many-to-one: one blocking call blocks whole process	✓
One-to-one: maximum concurrency, but overhead of kernel threads	✓
Many-to-many easy to implement	✗
User thread can map to multiple kernel threads	✗
Most OS use one-to-one now	✓
Division by zero → asynchronous signal	✗（synchronous）
Ctrl-C → asynchronous	✓
All signals delivered to all threads	✗
User-defined handler can override default	✓
Asynchronous cancellation = target checks periodically	✗（嗰個係 deferred）
Pthreads default cancellation type = deferred	✓
Linux distinguishes threads and processes	✗（both tasks）
fork() = clone(SIGCHLD, 0)	✓
PID is in TCB	✗
Windows uses one-to-one	✓

12. Past-paper 題目（Ch4）

- **S26 MC15**：true about multithreaded process → **C. Thread creation is usually faster than process creation**
- **S26 MC16**：concurrency/parallelism → **C**
- **S26 MC17**：Amdahl 1.6–1.7 → **C (61%, 3 cores) = 1.69**
- **S26 MC18**：false about signals → **C. All signals delivered to all threads**
- **F19 Q2.3**：unique vs shared（§1.2）；**Q3.1**：OS schedules kernel threads；user-thread → kernel-thread/LWP scheduling = **PCS (process-contention scope)**，within the same process, based on priority set by programmer；CPU scheduling (SCS) 喺全系統 kernel threads 之間揀
- **F20 MC11** stack；**MC12** thread library；**MC13** user thread → multiple kernel threads（false）；**Q2.3** heavyweight vs lightweight
- **Sp21 MC**：Amdahl 30% serial 7 cores → **2.5**；TCB 唔包括 **process identifier**；single processor 多 task progress → **concurrency**；many-to-many easy to implement（false）；TLS → **unique to each thread**
- **HW2**：MC1 responsiveness、MC2 data parallelism、MC3 sync/async、MC4 deferred、Q2.1 TCB、Q2.2 many-to-one、Q2.3 Ctrl+C、Q3.1 Amdahl（2.5；5）

13. 自我測驗

1. 列出 multithreading 四個 benefits。
2. 50% serial 嘅 program 用 4 cores speedup？（$\frac{1}{0.5+0.5/4} = \frac{1}{0.625}$ = **1.6**）；無限 cores？（$1/S$ = **2**）
3. Many-to-one model 兩大缺點？
4. 點解 exec() 緊接 fork() 時只複製 calling thread 就夠？

5. Deferred vs asynchronous cancellation？邊個 preferred？點解？
6. Linux 點樣建立 thread？（clone() with CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND）

5 Ch5 CPU Scheduling CPU 排程

📌 **考試範圍提示：Ch5 係分數最重嘅一章**（S26：27/100；F19：40/100；F20：27+；Sp21：26）。必考：

1. **Gantt chart + average waiting / turnaround / response time**：FCFS、SJF、SRTF、RR、Priority（± RR）、**MLFQ**（每年都有！）、自創 policy（LUF、dynamic priority）
2. **RM / EDF** real-time Gantt + missed deadlines
3. MC / short：preemptive vs non-preemptive、convoy effect、starvation & aging、MLFQ 優點、RR vs FCFS、exponential averaging、Little's formula、PCS vs SCS、processor affinity、load balancing、hard vs soft real-time ⚠️ **Response time 本科定義 = 第一次運行「完結」時間 − arrival time**（slide 5.12），唔係課本「第一次開始」！

1. Basic Concepts（基本概念） [5.4–5.7]

1.1 CPU–I/O Burst Cycle

- **Objective of multiprogramming：have a process running at all times → maximize CPU utilization.**
- **Process execution consists of cycles of CPU execution and I/O wait — CPU burst and I/O burst.**
- [5.5] **Histogram of CPU-burst times：many short CPU bursts, few long ones (long-tail distribution)。I/O-bound → many short CPU bursts；CPU-bound → a few long CPU bursts.** 呢個分佈對設計 scheduling algorithm 好重要。
- CPU idle 時，OS 由 ready queue 揀一個 process（除非 ready queue empty）→ 由 **CPU scheduler**（**process scheduler / short-term scheduler**）揀。

1.2 When does scheduling happen? ⭐⭐ [5.6]

CPU scheduling 可能喺四種情況發生：

1. **Running** → **waiting**（e.g. I/O request, `wait()`）
2. **Process terminates**
3. **Running** → **ready**（e.g. **interrupt / timer**）
4. **Waiting** → **ready**（I/O completion）或者 **new** → **ready**（新 process 到達，priority 更高）

- **Scheduling under 1 and 2 is non-preemptive (cooperative)** — process **gives up the CPU voluntarily**。
- **Scheduling under 3 and 4 is preemptive.**
- Preemptive 要考慮：**access to shared data (race condition, Ch6)**、**preemption while in kernel mode**、**interrupts during crucial OS activities**。

> 📝 Sp21 MC：CPU scheduling decisions may take place in → **all of the above**（running→waiting、terminates、running→ready）。📝 HW2 MC5：preemptive 例子 → **time quantum expires, running process moved back to the ready queue**（A terminate、B I/O、D wait() 都係自願 → non-preemptive）。

1.3 Dispatcher [5.7]

- **Dispatcher module gives control of the CPU to the process selected by the CPU scheduler：**
  1. **switching context** from one process to another
  2. **switching to user mode**
  3. **jumping to the proper location in the user program** to restart it
- **Dispatch latency — time for the dispatcher to stop one process and start another（overhead）**。應該越快越好。
- `vmstat`（Linux）可睇 context switches 數目（每秒幾百次）。

2. Scheduling Metrics（排程指標） ⭐⭐⭐ [5.8, 5.12]

Metric	定義	目標
CPU utilization	**fraction of the time that the CPU is busy**	max
Throughput	**number of processes completed per unit time**（favors short processes）	max
Turnaround time (TAT)	**time to execute a particular process = completion time − arrival time**	min
Waiting time (WT)	**total time spent waiting in the ready queue = TAT − CPU burst**	min
Response time (RT)	**time from submission until the first response is produced**	min
Fairness	adequate allocation for different jobs/processes	—

2.1 公式（本科 convention，必背）⭐

> 🧮 **Metrics（本科 convention）**
>
> $$\begin{aligned}\text{Turnaround} &= \text{Completion} - \text{Arrival}\\ \text{Waiting} &= \text{Turnaround} - \text{CPU burst}\\ \text{Response} &= (\text{end of the FIRST run}) - \text{Arrival}\end{aligned}$$

- **Response time 嘅「first run」完結** = 第一次上 CPU 嘅嗰段（RR / MLFQ 就係 **end of first quantum**，或者 burst 細過 quantum 就係完成時間）[5.12]。
    - Slide 5.13 例：RR q=4，P1(24), P2(3), P3(3) 全部 t=0 到 → **RT：P1 = 4, P2 = 7, P3 = 10，avg = 7**。
    - S26 MLFQ 官方：P1 response = 4 − 0 = 4（P1 跑 0–4）。
    - **非 preemptive (FCFS/SJF) 嘅 RT = TAT**（第一次 run 就跑完）。
- Waiting time 另一個計法（**gap method**）：將 Gantt chart 上 process 唔喺 CPU 嘅時間（由 arrival 起）加埋。兩個方法結果一樣：
    - S26 P2：gap = (4−2)+(21−8)+(40−29)+(57−45)+(64−58) = 44 ✓；formula = 71 − 2 − 25 = 44 ✓。
- S26 MC19 ✓「**Response time is the amount of time from when a request is submitted until the first response is produced**」。✗ WT = arrival 到完成（嗰個係 TAT）；✗ TAT = 等 ready queue 時間（嗰個係 WT）；✗ throughput = CPU busy fraction（嗰個係 utilization）。

3. FCFS — First-Come, First-Served ⭐ [5.9–5.10]

- 按 **arrival order** 排；**non-preemptive**（跑完 current CPU burst 先放）。
- 例：P1 = 24, P2 = 3, P3 = 3（全部 t=0）
    - 次序 P1, P2, P3：`0 P1 24 P2 27 P3 30` → **WT = 0, 24, 27 → avg 17；TAT avg** $= \frac{24+27+30}{3} = 27$
    - 次序 P2, P3, P1：`0 P2 3 P3 6 P1 30` → **WT = 6, 0, 3 → avg 3；TAT avg = 13**
- ⭐ **Convoy effect — short processes stuck behind a long process**，令 average waiting time 好長，完全取決於 arrival order。One CPU-bound + many I/O-bound → **low I/O device utilization**。（例：銀行排隊存張支票，前面有人開戶）

> 📝 HW2 MC6：convoy effect → **several short processes wait behind a long process that arrived first**。 📝 **F19 Q3.2**：FCFS 問題 = **convoy effect**（short process stuck behind a long process）。

4. RR — Round Robin ⭐⭐⭐ [5.11–5.16]

- **FCFS + preemption**：每個 process 得 **time quantum q**（通常 **10–100 ms**）；用完 → **preempted and added to the end (tail) of the ready queue**。
- **n processes → each gets 1/n of CPU time in chunks of at most q；no process waits more than (n − 1)q time units** ⭐（ignoring context switch）。
- **Timer interrupts every quantum**；如果 burst（或 remaining）< q，process 自己完成就放 CPU。
- 例 [5.13]：q = 4，P1 24, P2 3, P3 3 → `0 P1 4 P2 7 P3 10 P1 14 P1 18 P1 22 P1 26 P1 30`
    - **WT：P1 = 6, P2 = 4, P3 = 7 → avg 5.67；TAT avg** $= \frac{30+7+10}{3} = 15.67$；**RT = 4, 7, 10 → avg 7**
- **RR：average waiting time can be long, but more "fair"；better for short jobs than FCFS；better average response time → good for interactive jobs.**

4.1 Time quantum 大細 [5.14, 5.16] ⭐

- **q large → RR degenerates to FCFS.**（Sp21 MC 答案：**FCFS**）
- **q small → many context switches; q must be large w.r.t. context-switch time (< 10 μs), otherwise overhead too high.**（q 極細 = **processor sharing**：好似 n 個 processor 每個 1/n 速度）
- **Average turnaround time does not necessarily improve as q increases.**
- **TAT improves if most processes finish their CPU burst within a single quantum.**
- **Rule of thumb：80% of CPU bursts should be shorter than q.**

4.2 FCFS vs RR ⭐（F19 Q3.2 考過）[5.15]

- **Is RR always better than FCFS? No!** 例：**10 jobs start at the same time, each needs 100 s, q = 1 s → average turnaround time much worse under RR**（全部差唔多 1000 s 先完；FCFS 平均 550 s），**but average response time much better**。
- 一般而言 **no clear winner between FCFS and RR** in TAT/WT — 取決於 burst times、arrival pattern、quantum size。

> 📝 **F19 Q3.2 Model answer**：*RR performs worse than FCFS in turnaround time when all processes have the same (equal) CPU burst times that are longer than the time quantum — the jobs are interleaved and all finish near the end, whereas FCFS finishes them one by one.*

4.3 RR 做 Gantt chart 規則（考試用）

1. 用 **queue**（寫低每個時刻 queue 內容！）。
2. **新到達 process 加入 queue tail；quantum 用完嘅 process 都去 tail**。
3. ⚠️ **同一時刻**「新 process 到達」同「running process quantum 用完」→ convention：**新到達嗰個先入 queue，然後先放被 preempt 嗰個**（寫低 assumption）。
4. Process burst 完 → 即刻揀下一個（唔使等 quantum 完）。
5. Queue 空而 running process quantum 用完 → 佢繼續跑（再得一個 quantum）。

5. SJF / SRTF — Shortest Job First ⭐⭐⭐ [5.17–5.24]

5.1 SJF（non-preemptive）

- **Associate each process with the length of its next CPU burst; schedule the shortest.**（更準確叫 **shortest-next-CPU-burst**）
- ⭐ **SJF is optimal — gives the minimum average waiting time for a given set of processes.**
- **Difficulty：knowing the length of the next CPU request.**（所以 practical 難 implement；F20 MC15 答案 **SJF difficult to implement**）
- **Sketch proof：moving a short process before a long one decreases the waiting time of the short process more than it increases the waiting time of the long process → average waiting time decreases.**
- 例 [5.18]：P1 = 6, P2 = 8, P3 = 7, P4 = 3。Slide 個 chart **當全部 t = 0 到**：`0 P4 3 P1 9 P3 16 P2 24` → **avg WT** $= \frac{3+16+9+0}{4} = 7$（slide 答案）。
    - ⚠️ 如果真係跟表入面 arrival（0, 2, 4, 5）：t=0 只有 P1 → `0 P1 6 P4 9 P3 16 P2 24` → WT = 0, 14, 5, 1 → avg 5。考試**一定要睇清楚有冇 arrival time**。

5.2 SRTF — Shortest-Remaining-Time-First（preemptive SJF）⭐

- **New arrival with burst shorter than the remaining time of the running process → preempts it.**
- Scheduling 喺 **every arrival (condition 4)** 都要 invoke。
- 例 [5.23]：P1(0,8), P2(1,4), P3(2,9), P4(3,5) → `0 P1 1 P2 5 P4 10 P1 17 P3 26`
    - **avg WT** $= \frac{(10-1)+(1-1)+(17-2)+(5-3)}{4} = \frac{26}{4} = 6.5$

5.3 比較 [5.24] ⭐

- **SJF/SRTF are the best for minimizing average waiting time (and average turnaround time).**
- **Provably optimal：SJF among non-preemptive, SRTF among preemptive. SRTF is always at least as good as SJF.**
- **SJF/SRTF = FCFS if all processes have the same CPU burst.**
- **Can lead to starvation of long processes if shorter processes keep arriving；fairness cannot be enforced.**
- F20 MC14：best (min) average waiting time → **SJF**（**SRTF** 亦接受）。

5.4 Exponential Averaging（預測下一個 CPU burst）⭐⭐ [5.19–5.22]

> 🧮 **Exponential averaging**
>
> $$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\,\tau_n, \qquad 0 \le \alpha \le 1$$

- t_n **= actual length of the n-th CPU burst；**τ_{n+1} **= predicted value for the next burst**；通常 $\alpha = 1/2$。
- $\alpha = 0 \Rightarrow \tau_{n+1} = \tau_n$ **（recent history does not count）**；$\alpha = 1 \Rightarrow \tau_{n+1} = t_n$ **（only the last burst counts）**。
- 展開 → **all past values count but recent ones weigh more；past effect diminishes exponentially**：

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\alpha\, t_{n-1} + \cdots + (1-\alpha)^j \alpha\, t_{n-j} + \cdots + (1-\alpha)^{n+1}\tau_0$$

- 例：$\alpha = 0.5$，10 rounds 後 $(1/2)^{10} = 1/1024$ → 可忽略。
- 預測取決於 **(1) all previous actual bursts with different weights, (2) the initial estimate** τ_0。
- Slide 5.20 例（$\alpha = 1/2, \tau_0 = 10$）：

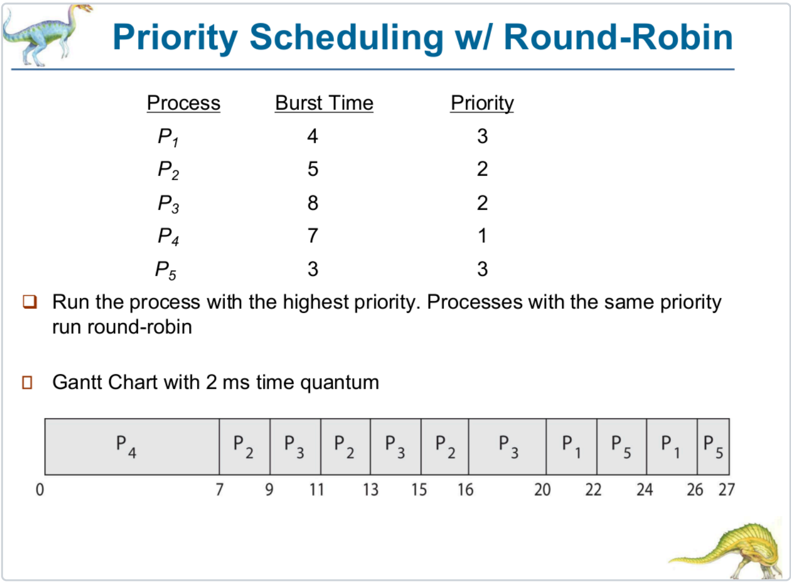
i	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
CPU burst t_i	6	4	6	4	13	13	13	…
guess τ_i	10	8	6	6	5	9	11	12

- **HW2 MC9**：$\tau_n = 10,\ t_n = 6,\ \alpha = 0.25 \Rightarrow \tau_{n+1} = 0.25 \times 6 + 0.75 \times 10 = 1.5 + 7.5 =$ **9 ms**。

6. Priority Scheduling ⭐⭐ [5.25–5.27]

- **Each process has a priority number；CPU allocated to the highest priority（本科：smallest integer = highest priority，除非題目另講）**。

- **Preemptive**（higher-priority arrival preempts）或 **non-preemptive**。
- **Equal priority → FCFS order**（或者 RR）。
- **SJF is a special case of priority scheduling where priority = inverse of predicted next CPU burst.**
- ⭐ **Problem：Starvation (indefinite blocking) — low-priority processes may never execute.**
- ⭐ **Solution：Aging — gradually increase the priority of a process as it waits**（需要每個 process 喺 ready queue 有 timer）。
- Sp21 MC：main problem of priority scheduling → **some processes may never execute (starvation)**；HW2 MC7：aging → **address starvation by gradually increasing the priority of a waiting process**。
- 例 [5.26]（all t=0；P1 10/3, P2 1/1, P3 2/4, P4 1/5, P5 5/2）→ `0 P2 1 P5 6 P1 16 P3 18 P4 19`，**avg WT** $= \frac{6+0+16+18+1}{5} = 8.2$
- **Priority + RR** [5.27]：**run the highest priority；same priority → round-robin**。例 q = 2（burst/priority：P1 4/3, P2 5/2, P3 8/2, P4 7/1, P5 3/3，全部 t=0）→ `0 P4 7 P2 9 P3 11 P2 13 P3 15 P2 16 P3 20 P1 22 P5 24 P1 26 P5 27`
  - P4（priority 1）先跑完；P2、P3（priority 2）RR；P2 喺 16 完咗之後 P3 獨自跑到 20；最後 P1、P5（priority 3）RR。



Priority Scheduling w/ Round-Robin

Process	Burst Time	Priority
P_1	4	3
P_2	5	2
P_3	8	2
P_4	7	1
P_5	3	3

❑ Run the process with the highest priority. Processes with the same priority run round-robin

❑ Gantt Chart with 2 ms time quantum

P_4 P_2 P_3 P_2 P_3 P_2 P_3 P_1 P_5 P_1 P_5
0 7 9 11 13 15 16 20 22 24 26 27

Priority scheduling with round-robin, q = 2 [5.27]

⚠️ **Priority + RR 有唔同 arrival time（Lab 5：HW / exam 常見）：被高 priority 新 process preempt 嘅 process 要保留佢剩餘嘅 quantum**，唔好 reset（lab 話呢個係 common mistake）。規則同 MLFQ 一樣。

- **Spring 2023 midterm 官方 Gantt（q = 2）**：`0 P1 1 P2 4 P1 5 P3 7 P1 8 P4 10 P1 11 P3 13 P5 15 P1 16 P3 18 P5 20 P3 22 P5 24` — P1 喺 0–1 被 P2 preempt，4–5 只跑剩低嘅 1 unit 就輪到 P3；8–10 P4 preempt 後 P1 10–11 又用返剩低 1 unit。（由 Gantt 推返嘅 table：P1 (0, 5, pr 2)、P2 (1, 3, pr 1)、P3 (2, 8, pr 2)、P4 (8, 2, pr 1)、P5 (9, 6, pr 2)，模擬器重現完全一致 ✓）

7. Multilevel Queue（多級佇列）⭐ [5.28–5.29]

- **Priority scheduling combined with RR；priority assigned statically；a process remains in the same queue for its whole runtime.**
- **Partition processes into queues based on process type**（例：real-time → system → interactive → batch）。
- **Each queue can have its own scheduling algorithm.**
- **Scheduling among queues**：(1) **fixed-priority preemptive scheduling**（高 queue 空先輪到低 queue → 可能 starvation）或 (2) **time-slice**（each queue gets a certain % of CPU time，e.g. 60%, 20%, 10%, 10%）。
- Sp21 MC：Multilevel queue（固定）有 starvation（free from starvation 嘅只有 **FCFS 同 RR**）。

8. MLFQ — Multilevel Feedback Queue ⭐⭐⭐（每年必考 Gantt）[5.30–5.35]

- **A process can move between queues；aging can be implemented this way → flexibility.**
- **MLFQ 由以下 parameters 定義**：
  1. **number of queues**
  2. **scheduling algorithm for each queue**
  3. **method to upgrade a process**（to higher queue）
  4. **method to demote a process**（to lower queue）
  5. **method to determine which queue a process enters when it needs service**

8.1 Slide example 規則（考試照用）[5.31] ⭐

- Q0 – RR（slide 5.31: q = 8；例子 5.32–5.34 用 **q = 4**）；Q1 – RR（q = 16 / **8**）；Q2 – **FCFS**
- **A new job always enters Q0 (RR, FCFS order)；it also preempts jobs from Q1 or Q2 currently running.**
- **Gets q_0 in Q0；if not finished → moved to Q1. In Q1 gets q_1 additional；if still not complete → preempted and moved to Q2.**
- ⭐ **If a job from Q1 or Q2 is preempted by a new job from Q0, it joins the HEAD of Q1 or Q2, with the remaining quantum (Q1) or remaining CPU burst (Q2).**（**唔會**攞一個全新 quantum；**唔會**因為被 preempt 而 demote）
- **Q1 job 只會喺 Q0 空先跑；Q2 job 只會喺 Q0 和 Q1 都空先跑。**
- **Q1 job 永遠唔會 preempt Q2 job**（因為冇 process 直接 arrive 去 Q1）— summary 重點。
- **A job in Q0 is never preempted**（Q0 內部只有 RR）。
- **Burst** $\le q_0 + q_1$ **（e.g. 4 + 8 = 12）嘅 job 永遠唔會落 Q2。**
- **This approximates SRTF：CPU-bound jobs drop like a rock；short-running I/O-bound jobs stay near the top.**

8.2 Lecture example [5.32–5.34]（Q0 q=4, Q1 q=8, Q2 FCFS）

Process	Arrival	Burst
P1	0	10
P2	2	15
P3	5	2
P4	12	14
P5	18	6

```
0 P1 4 P2 8 P3 10 P1 12 P4 16 P1 18 P5 22 P1 24 P2 32 P4 40 P5 42 P2 45 P4 47
 Q0   Q0   Q0   Q1   Q0   Q1   Q0   Q1   Q1   Q1   Q1   Q2   Q2
```

- t=4 P1 用完 Q0 quantum → Q1；t=8 P2 → Q1；t=10 P3 完；P1 喺 Q1 跑；t=12 P4 到 → preempt P1（P1 返 Q1 head，剩 6 quantum）；t=16 P4 → Q1 tail；P1 繼續（剩 quantum 6）；t=18 P5 到 → preempt P1；t=22 P5 → Q1；P1 跑到 24 完成（Q1 總共用 2+2+2 = 6 ≤ 8）；P2 24–32 用完 Q1 quantum → Q2；P4 32–40 → Q2；P5 40–42 完成；P2 42–45 完；P4 45–47 完。
- **WT：P1 = 14, P2 = 28, P3 = 3, P4 = 21, P5 = 18 → avg** $= \frac{84}{5} = 16.8$ ✓

8.3 MLFQ advantages ⭐ ⭐（F19 Q3.3、F20 Q2.5 必背）[5.35]

- Used in **BSD Unix, Solaris, Windows NT and later**。

1. **It does not need prior knowledge of the next CPU burst time**（SJF/SRTF 需要）
2. **It handles interactive jobs well — better response time than RR**（short jobs 喺 top queue 完成）
3. **It produces similar performance to SJF/SRTF**
4. **It is "fair" — makes progress on CPU-bound jobs**

- **Starvation** 可以用 **reshuffling jobs periodically**（e.g. 一段時間後將所有 jobs 搬返 top queue）解決。

> 📝 **F20 Q2.5 Model answer**：*(1) MLFQ resembles the performance of SJF/SRTF without requiring prior knowledge of the next CPU burst time; (2) like RR, it gives good response time to interactive (short, I/O-bound) jobs since they finish in the top queues; (3) it is fair — all processes, long or short, make progress (and starvation can be avoided by periodically moving jobs up / aging).*
> 📝 **F19 Q3.3**：*MLFQ approximates SJF because short (interactive, I/O-bound) jobs finish within the top high-priority queue(s), while long CPU-bound jobs are demoted to lower queues — so short jobs get served first like SJF. Its major advantage over SJF is that it does not require knowing the CPU burst times in advance.* ⚠️ S26 MC20 ✓「**Each queue can use a different scheduling algorithm**」；✗ processes never move；✗ requires burst times known；✗ non-preemptive。Sp21 MC ✓「**handles interactive jobs well by delivering similar performance as SJF**」；✗ needs prior knowledge；✗ classification permanent；✗ won't suffer starvation。

8.4 MLFQ 做題 checklist ✍️

1. 每個時間點寫低：**Q0、Q1、Q2 內容（同次序）**、running process、佢喺邊個 queue、**剩餘 quantum**、**剩餘 burst**。
2. 事件：**arrival**（入 Q0 tail；如果 running 係 Q1/Q2 → preempt，被 preempt 嘅返 **head** of own queue，保留剩餘 quantum）；**quantum 用完**（未完成 → 降一級，入 tail）；**完成**。
3. Gantt 上標 **process@Queue**（HW2 要求 label queue）。
4. **同一 process 由 Q0 用完 quantum 立即喺 Q1 繼續跑**（因為 Q0 空）→ Gantt 可合併，但計 response time 要用 Q0 quantum 完結時間（S26 P5：49 − 45 = 4）。
5. **題目規則永遠優先**（S26 第 8 條、F20 aging timer、HW2 「retains unused quantum」）。

9. Thread Scheduling ⭐ ⭐ [5.36–5.37]

- **On modern OSes, kernel-level threads are scheduled (by the OS); user-level threads are managed by the thread library.**
- **LWP (lightweight process)** — intermediate data structure；**appears as a virtual processor on which user threads are scheduled to run；each LWP attached to a kernel thread**。

- ⭐ **Process-Contention Scope (PCS) — under many-to-one and many-to-many models, the thread library schedules user-level threads onto an LWP；competition among threads belonging to the same process**；typically **by priority set by programmers**（thread library 通常唔會調整 priority）；PCS typically **preempts the running thread in favour of a higher-priority user thread**。
- ⭐ **System-Contention Scope (SCS) — the kernel schedules kernel threads onto an available CPU；competition among all threads in the system** → 就係 CPU scheduling。
- **One-to-one systems (Windows, Linux, Solaris) schedule threads using only SCS.**

> 📝 HW2 MC8：thread library 揀 user thread run on LWP → **PCS, because the competing threads belong to the same process**。 📝 **F19 Q3.1**：*The OS schedules kernel-level threads. Mapping user threads onto kernel threads/LWPs (PCS) is done by the thread library, based on priorities set by the programmer, among the user threads of the same process; CPU scheduling (SCS) decides which kernel thread runs on a CPU among all kernel threads in the system.*

10. Multiple-Processor Scheduling ⭐ ⭐ [5.38–5.46]

10.1 AMP vs SMP [5.38–5.39]

- Multiprocessor 而家包括 **multicore CPUs, multithreaded cores, NUMA systems, heterogeneous multiprocessing**。
- **Asymmetric multiprocessing (AMP) — only one processor (the master server) accesses kernel data structures**（唔使 data sharing）；**all scheduling decisions, I/O processing, system activities handled by the master**；other processors execute only user code； ⭐ **the master server can become a potential bottleneck**。
- **Symmetric multiprocessing (SMP) — each processor is self-scheduling**，with **one common ready queue** or **a private ready queue per processor**；每個 processor 嘅 scheduler examine ready queue 揀 thread。
    - Common queue：**must ensure two processors don't choose the same thread（race condition）** 同 processes 唔會 lost。
    - **All modern OSes support SMP**（Windows, Linux, macOS, Android, iOS）。

> 📝 **F20 Q2.4 Model answer（common vs per-processor ready queue）**：*A single common ready queue achieves **load balancing** easily — no processor is idle while the queue is non-empty — but it is hard to keep **processor affinity** (a thread may run on a different CPU each time, losing its cache contents) and the shared queue needs locking (possible race condition/contention). Per-processor private ready queues naturally provide processor affinity (and no contention on a shared queue), but need extra mechanisms such as **push/pull migration** to balance the load.*

10.2 Multicore & Multithreaded cores (CMT) ⭐ [5.40–5.44]

- **Multiple cores on the same chip — faster and consume less power, but complicate scheduling.**
- ⭐ **Memory stall：when a processor accesses memory, it spends a significant amount of time waiting for data to become available（modern processors much faster than memory, esp. on cache miss）**。（處理器可能 50% 時間喺等 memory）
- **Chip multithreading (CMT)（Intel = hyperthreading）：multiple hardware threads per core；each hardware thread has its own architectural state (PC, register set) and appears as a logical CPU**。
- **If one thread stalls waiting for memory, the core switches to another hardware thread.**
- **Dual-threaded dual-core → 4 logical processors；quad-core × 2 threads → 8 logical processors；UltraSPARC T3：16 cores × 8 threads = 128 logical processors**。
- **Two levels of scheduling：**
    1. **OS decides which software (kernel) thread runs on each logical CPU（= CPU scheduling）**
    2. **Each core decides which hardware thread runs on the physical core（e.g. RR on UltraSPARC T3）**
- 三層 mapping [5.44]：user thread → LWP/kernel thread（**PCS**, thread library）；kernel (software) thread → logical CPU / hardware thread（**CPU scheduling**, OS）；hardware thread → core（**core 自己，typically RR**）。

> 📝 **F19 Q2.5**：multicore 優點 = **faster + less power**；CMT 原因 = **memory stall** — 一個 hardware thread 等 memory 時 core 可以跑另一個 thread（S26 MC22 ✓「another thread may run when one thread stalls waiting for memory」）。

10.3 Processor Affinity ⭐ ⭐ [5.45]

- **When a thread has run on a processor, that processor's cache holds its memory accesses → the thread has affinity for that processor.**
- **High cost of invalidating and repopulating caches → most SMP systems try to avoid migrating processes between processors.**
- ⭐ **Per-processor ready queues provide processor affinity for free!**
- **Soft affinity — OS attempts to keep a process on the same processor but doesn't guarantee it（可以 migrate during load balancing）**。
- **Hard affinity — a process specifies a subset of processors it may run on.**（Linux：soft affinity + `sched_setaffinity()` for hard affinity）
- **NUMA**：CPU 存取近佢嘅 memory 較快 → affinity to both processor and memory。

> 📝 **HW2 Q2.4**：hard affinity {CPU0, CPU1}，CPU2 idle → **No**，hard affinity must be respected；load balancing 只可以將 T 由 CPU0 搬去 CPU1。

10.4 Load Balancing ⭐ [5.46]

- **Keep workload evenly distributed**（只喺 per-processor private queues 時需要）。
- **Push migration — a specific task periodically checks loads and pushes tasks from overloaded to idle/less-busy CPUs.**
- **Pull migration — an idle processor pulls a waiting task from a busy processor.**
- **Not mutually exclusive — often both implemented (Linux CFS).**
- ⭐ **Load balancing often counteracts the benefits of processor affinity**（natural tension）。
- S26 MC22 ✗「**The motivation of placing multiple cores on the same chip is to simplify the scheduler**」（其實 complicate scheduling）；✓ AMP master bottleneck；✓ memory stall → run another thread；✓ affinity keeps process on same CPU for cache。

---

11. Real-Time CPU Scheduling ⭐⭐⭐ [5.47–5.52]

11.1 Hard vs Soft ⭐

- **Real-time scheduling demands performance guarantee — predictability.**
- **Hard real-time — a task must be serviced by its deadline; service after the deadline is the same as no service.**
- **Soft real-time — no guarantee when a critical real-time process will be scheduled; only guarantees it will be given preference over noncritical processes.**
- **Real-time OS scheduler must support a priority-based algorithm with preemption**（但 preemptive priority-based 只保證 **soft** real-time）。
- Sp21 MC ✓「**Soft real-time systems guarantee only that the process will be prioritized over non-critical processes**」；✗ doesn't demand guarantee；✗ hard RT can service after deadline；✗ priority without preemption。

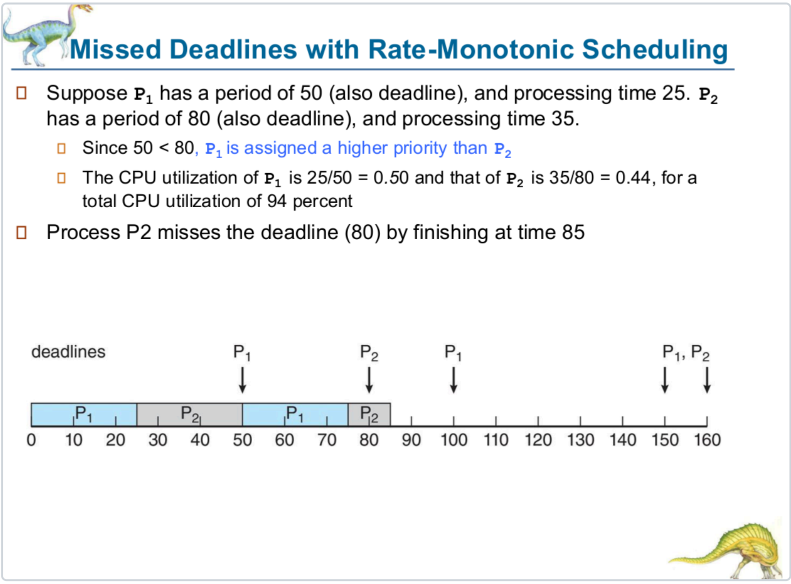
11.2 Periodic tasks [5.48]

- 參數：**processing time** t**, deadline** d**, period** p，$0 \le t \le d \le p$；**rate =** $1/p$。
- **Admission control**：process announces deadline；scheduler **admits only if it can guarantee completion by the deadline**。
- **CPU utilization of task** i **=** t_i/p_i；total（$U > 1$ → 一定有 miss）：

$$U = \sum_i \frac{t_i}{p_i}$$

11.3 Rate-Monotonic (RM) ⭐⭐ [5.49–5.50]

- **Static priority = inverse of period：shorter period → higher priority**（rationale：**tasks requiring CPU more often get higher priority**）。**Preemptive.**
- **RM is optimal among static-priority algorithms：if a set can't be scheduled by RM, no other static-priority algorithm can.**
- **Only handles periodic processes with constant processing time.**
- 例 [5.49]：P1 (p=50, t=20), P2 (p=100, t=35)：U = 0.40 + 0.35 = 75% → OK：`0 P1 20 P2 50 P1 70 P2 75 idle 100`
- 例 [5.50]：P1 (50, 25), P2 (80, 35)：U = 0.50 + 0.44 = **94%** → `0 P1 25 P2 50 P1 75 P2 85` → **P2 misses its deadline (80) by finishing at 85**。



Missed Deadlines with Rate-Monotonic Scheduling

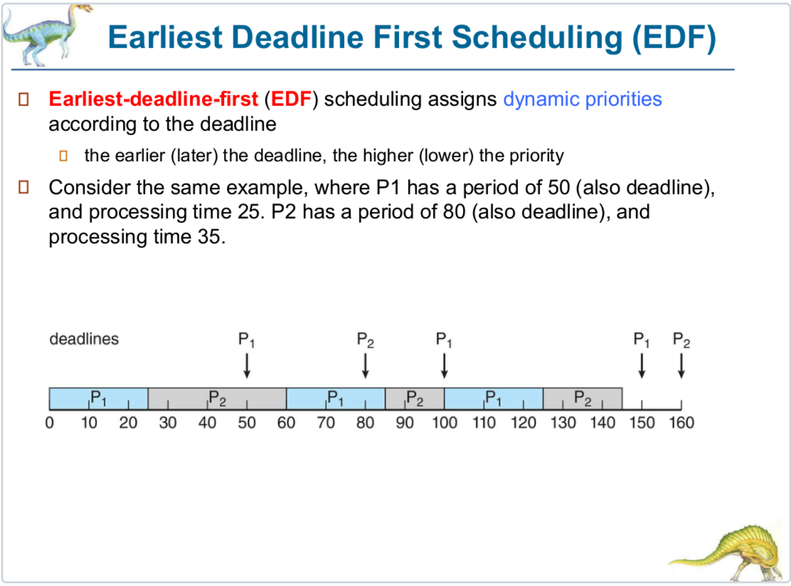
- Suppose P_1 has a period of 50 (also deadline), and processing time 25. P_2 has a period of 80 (also deadline), and processing time 35.
  - Since 50 < 80, P_1 is assigned a higher priority than P_2
  - The CPU utilization of P_1 is 25/50 = 0.50 and that of P_2 is 35/80 = 0.44, for a total CPU utilization of 94 percent
- Process P2 misses the deadline (80) by finishing at time 85

RM: P2 misses its deadline at 80 **[5.50]**

- 課本 bound（slides 冇，但好用）：**RM guaranteed if** $U \le N\left(2^{1/N}-1\right)$：N=1 → 1；**N=2 → 0.828**；N=3 → 0.780；N=4 → 0.757；$N \to \infty \Rightarrow \ln 2 \approx 0.693$。超過 bound **唔一定** miss（要畫 Gantt 確認）。

11.4 EDF — Earliest Deadline First ⭐ ⭐ [5.51–5.52]

- **Dynamic priorities according to deadline：earlier (absolute) deadline → higher priority.** Preemptive。
- 例 **[5.51]**（P1 50/25, P2 80/35）：`0 P1 25 P2 60 P1 85 P2 100 P1 125 P2 145 idle 150` → **no deadline missed**（t=50 時 P2 deadline 80 < P1 新 deadline 100 → P2 繼續）。



Earliest Deadline First Scheduling (EDF)

- **Earliest-deadline-first** (**EDF**) scheduling assigns dynamic priorities according to the deadline
  - the earlier (later) the deadline, the higher (lower) the priority
- Consider the same example, where P1 has a period of 50 (also deadline), and processing time 25. P2 has a period of 80 (also deadline), and processing time 35.

EDF: same tasks, no deadline missed **[5.51]**

- **EDF does not require processes to be periodic, nor a constant CPU time per burst；only requirement：announce its deadline when it becomes runnable.**
- ⭐ **EDF is theoretically optimal — can meet all deadlines with CPU utilization up to 100%**（但實際上 context switching + interrupt handling 令佢做唔到 100%）。
- S26 MC21：assigns priorities according to deadlines → **EDF**。

> 📝 **Sp21 Q2.3(b)**：*EDF is better — unlike RM with static priorities, EDF uses dynamic priorities based on deadlines, which can increase the fraction of met deadlines (schedulable up to 100% utilization).* 📝 **HW2 Q2.5**（A: p=50,t=25；B: p=80,t=35；t=50 時 A 新 instance deadline 100，B 剩 10 deadline 80）→ **No**：**RM 揀 A**（shorter period，static）→ B 75–85 **miss 80**；**EDF 揀 B**（deadline 80 < 100）→ B 50–60 ✓，A 60–85 ✓。

11.5 RM / EDF 做題方法 ✍️

1. 列出每個 task 嘅 **release times**（0, p, 2p, ...）同 **absolute deadlines**（release + d；冇講 d 就 d = p）。
2. 每個 time unit（或者每個事件：release / completion）揀：
   - **RM**：ready jobs 入面 **period 最短**（固定）嗰個。
   - **EDF**：ready jobs 入面 **absolute deadline 最早**嗰個。

3. **Ties**：題目有講就跟（S26：**smaller process ID**）；冇講：**keep the running job（唔 preempt）**（F19、Sp21 官方答案都係咁），再唔得就 smaller ID。寫低 assumption。
4. **Soft real-time：miss 咗 deadline 嘅 job 會繼續跑完**（Sp21 hint；S26 官方亦係咁），但 count 為 missed。
5. 數 **missed deadlines**：每個 deadline ≤ 時限範圍嘅 job，完成時間 > deadline（或者未完成）→ miss。
6. 先計 $U = \sum_i t_i/p_i$ 做 sanity check。

S26 Q4.2（P1 t=1 d=3 p=4；P2 2/4/5；P3 2/6/7；P4 1/5/8；0–16，ties → smaller ID）：

- **RM**（priority P1 > P2 > P3 > P4）：`0 P1 1 P2 3 P3 4 P1 5 P2 7 P3 8 P1 9 P3 10 P2 12 P1 13 P3 14 P3 15 P2 16`
    - P3 job (d=6) finishes 8 ✗；P4 never runs → misses d=5 ✗ and d=13 ✗；P3 job2 (d=13) finishes 14 ✗ → **RM misses 4 deadlines**（U = 0.25+0.4+0.286+0.125 = 1.06 > 1）
- **EDF**：`0 P1 1 P2 3 P4 4 P3 6 P1 7 P2 9 P1 10 P3 12 P4 13 P2 15 P1 16`
    - P2 job3 (d=14) finishes 15 ✗；P1 job4 (d=15) finishes 16 ✗ → **EDF misses 2 deadlines**

Sp21 Q2.3（P1 1/3/3, P2 2/4/4，0–14）：

- **EDF**：`0 P1 1 P2 3 P1 4 P2 6 P1 7 idle 8 P2 10 P1 11 idle 12 P1 13 P2 14`（t=9 tie deadline 12 → keep P2）
- **RM**：`0 P1 1 P2 3 P1 4 P2 6 P1 7 idle 8 P2 9 P1 10 P2 11 idle 12 P1 13 P2 14`
- 兩者都冇 miss（U = 0.833）。

F19 Q3.4（EDF；P1 p=d=10 t=4；P2 16/2；P3 20/8；到 38）：

- `0 P1 4 P2 6 P3 14 P1 18 P2 20 P1 24 P3 32 P1 36 P2 38`（t=10、t=30 deadline tie → keep P3 running）

12. Algorithm Evaluation [5.53–5.60]

- 先定 **criteria**（e.g. maximize CPU utilization **subject to max response time 300 ms**；maximize throughput such that TAT ∝ execution time）。
- **Deterministic modeling — takes a particular predetermined workload and defines the performance of each algorithm for that workload；simple and fast, gives exact numbers, but requires exact input and answers apply only to those cases**。
    - 例 [5.54–5.55]：5 processes at t=0：P1 10, P2 29, P3 3, P4 7, P5 12 → **FCFS avg WT = 28；non-preemptive SJF = 13；RR (q=10) = 23**。
- **Queueing models / analysis** — CPU = server with ready queue；用 **distributions**（exponential, described by mean）；可計 utilization、average queue length、wait time；**classes of algorithms and distributions handled are limited；assumptions often unrealistic**。
    - ⭐ **Little's Formula：$n = \lambda \times W$（n = average queue length, λ = average arrival rate, W = average waiting time in queue）；valid for any scheduling algorithm and arrival distribution**（steady state，arrival rate < service rate）。
    - 例：$\lambda = 7/\text{s},\ n = 14 \Rightarrow W = n/\lambda =$ **2 s**；**HW2 MC10**：$\lambda = 3/\text{s},\ n = 12 \Rightarrow W = 12/3 =$ **4 s**。
- **Simulations** — programming a model of the system；**more accurate**；data from **random number generators (distributions)** 或 **trace files**（real system 記錄）。
- **Implementation** — 真系統測試，最準但 **high cost & high risk**；環境會變；可調 parameters；APIs 改 priority（Java, POSIX, Windows）。

13. Gantt Chart 萬用流程（考試步驟）✍️⭐⭐⭐

1. **抄表**：Process / Arrival / Burst（/ Priority）。
2. **由 t = 0 開始逐個事件推進**（events：arrival、completion、quantum expiry、check point）。每個事件寫低 **ready queue**（同 MLFQ 各層）。
3. **揀 process 規則**：

Algorithm	揀邊個	Preempt?
FCFS	earliest arrival	No
SJF	shortest burst among **arrived**	No
SRTF	shortest **remaining**	Yes, on arrival
RR	head of queue	Yes, quantum expiry
Priority (NP / P)	highest priority	P: on higher arrival
Priority + RR	highest priority；same → RR	Yes
MLFQ	highest non-empty queue	Yes（Q0 arrival preempts Q1/Q2）
RM	shortest period	Yes
EDF	earliest absolute deadline	Yes

4. **CPU idle**：冇 process ready 就畫 **idle**（Sp21 FCFS `0 P1 2 idle 4 P2…`）。
5. **畫 Gantt**：每段標 **start time / process / end time**；所有 boundary 時間都要寫（partial credit 靠呢個）。

6. **計 table**：Completion → **TAT = C − A** → **WT = TAT − Burst** → **RT = end of first run − A**。
7. **Average**：寫出 $\dfrac{a+b+\cdots}{n}=x$（S26：$\dfrac{140}{6}=23.33$）。
8. **Sanity check**：總 burst 加 idle = 最後完成時間；每個 WT ≥ 0。

13.1 Tie-breaking（寫 assumption！）

- 同時到達、同 burst/priority：**FCFS → lower process ID**。
- **SRTF**：新到達 burst = running remaining → 一般 **keep running**（唔 preempt）；F19 官方 preempt 咗，但兩種 avg WT 一樣 2.5。
- **RR**：新到達 + quantum expiry 同一時刻 → **新到達先入 queue**。
- **EDF**：deadline 相同 → **keep running**，否則 smaller ID（或者跟題目）。
- **Dynamic priority（S26 Q4.3）**：題目講「same priority → larger ID first」— **跟題目**！

---

14. Past-paper 計算題全集（全部用程式驗證 ✓）

14.1 S26 Q4.1 MLFQ（Q0 RR4, Q1 RR8, Q2 FCFS；preempted → head of own queue）

P	Arr	Burst
P1	0	9
P2	2	25
P3	6	7
P4	14	18
P5	45	15
P6	58	6

```
0 P1 4 P2 8 P3 12 P1 14 P4 18 P1 21 P2 29 P3 32 P4 40 P2 45 P5 57 P2 58 P6 64 P2 71 P4 77 P5 80
(Q0)(Q0)(Q0)(Q1)(Q0)(Q1)(Q1)(Q1)(Q1)(Q2)(Q0→Q1 at 49)(Q2)(Q0→Q1 at 62)(Q2)(Q2)(Q2)
```

- WT：P1 12, P2 44, P3 19, P4 45, P5 20, P6 0 → **avg** $=\dfrac{140}{6}=23.33$
- RT（end of first run − arrival）：P1 4, P2 6, P3 6, P4 4, P5 4 (49−45), P6 4 (62−58) → **avg** $=\dfrac{28}{6}=4.67$

14.2 S26 Q4.3 Dynamic priority（larger = higher；waiting rate α，running rate β；tie → larger ID）

- P1 (0, 7), P2 (3, 5), P3 (6, 6), P4 (9, 4)
- (a) $\alpha=0,\ \beta=3$（running process priority 上升，waiting 不變 → 一旦 run 就霸住）：`0 P1 7 P3 13 P4 17 P2 22`
  - t=7：P2, P3 都係 0 → tie → larger ID → **P3**；t=13：P2 (0), P4 (0) → **P4**。
- (b) $\alpha=1,\ \beta=0$，只考慮 P1、P2，每 3 個 unit check：`0 P1 3 P2 6 P1 9 P2 11 P1 12`
  - t=3：P1 = 0 (ran 3), P2 = 0 → tie → P2；t=6：P1 = 3, P2 = 0 → P1；t=9：P1 = 3, P2 = 3 → tie → P2；t=11 P2 完 → P1。
- (c) $\alpha=1,\ \beta=-2$，初始 30，每 3 unit check：`0 P1 3 P2 6 P3 9 P4 12 P1 15 P2 17 P3 20 P4 21 P1 22`
  - t=3：P1 = 30−6 = 24, P2 = 30 → P2；t=6：P1 27, P2 24, P3 30 → P3；t=9：P1 30, P2 27, P3 24, P4 30 → tie → P4；t=12：P1 33 → P1；t=15：P1 27, P2 33, P3 30, P4 27 → P2（17 完）；t=17：P1 29, P3 32, P4 29 → P3（20 完）；t=20：P1 32, P4 32 → tie → P4（21 完）；t=21：P1。

14.3 S26 Q2.3 自訂 policy（揀 f$_i$ 最大）

$$f_i=\frac{1}{a\,c_i+b}\qquad(a,b>0)$$

- (a) **Yes** — I/O-bound processes have small average CPU bursts c_i → larger f_i → higher priority.
- (b) **Yes** — a CPU-bound process (large c_i, low f_i) can starve if higher-priority I/O-bound processes keep arriving.

14.4 F19 Q3.5 LUF (Least Used First)（揀用 CPU 最少嘅；tie → FCFS；preempt on arrival）

- P1 (0, 25), P2 (15, 25), P3 (25, 5), P4 (40, 5)
- `0 P1 15 P2 25 P3 30 P2 40 P4 45 P1 55 P2 60`

P	TAT	RT	WT
P1	55	15	30
P2	45	10	20
P3	5	5	0
P4	5	5	0

- LUF vs SRTF：相似 — short jobs 較易有高 priority；分別 — **LUF 令跑咗好耐嘅 process 易 starve**；SRTF 令 **long burst 嘅 process 一開始就 starve**。

14.5 F19 Q3.6（P1 0/8, P2 7/13, P3 9/1, P4 11/5, P5 20/7, P6 27/1）

Alg	Gantt	WT (P1..P6)	Avg
FCFS	0 P1 8 P2 21 P3 22 P4 27 P5 34 P6 35	0,1,12,11,7,7	**6.33**
SJF	0 P1 8 P2 21 P3 22 P4 27 P6 28 P5 35	0,1,12,11,8,0	**5.33**
RR q=4	0 P1 8 P2 12 P3 13 P4 17 P2 21 P4 22 P5 26 P2 30 P5 33 P6 34 P2 35	0,15,3,6,6,6	**6.0**（corrected）
SRTF	0 P1 8 P2 9 P3 10 P2 11 P4 16 P2 20 P5 27 P6 28 P2 35（官方，tie 時 preempt）	0,15,0,0,0,0	**2.5**

- SRTF 另一答案（tie 時 keep P2）：`… 16 P2 27 P6 28 P5 35` → WT 0,7,0,0,8,0 → avg 都係 **2.5**。
- 注意 SJF：t=8 只有 P2 到咗 → P2 跑到 21（non-preemptive）。

14.6 F20 Q4.1（P1 0/9, P2 3/3, P3 8/2, P4 10/6, P5 16/8, P6 20/5）

Alg	Gantt	WT	Avg
FCFS	0 P1 9 P2 12 P3 14 P4 20 P5 28 P6 33	0,6,4,4,4,8	**4.33**
SJF	0 P1 9 P3 11 P2 14 P4 20 P6 25 P5 33	0,8,1,4,9,0	**3.67**
SRTF	0 P1 3 P2 6 P1 8 P3 10 P1 14 P4 20 P6 25 P5 33	5,0,0,4,9,0	**3.0**

14.7 F20 Q4.2 Two-level feedback queue with aging（Q0 RR4；Q1 FCFS；Q1 timer 8 ms → 返 Q0；被 preempt 返 Q1 head 並 reset timer）

- P1 0/9, P2 3/3, P3 8/2, P4 10/9, P5 13/4, P6 18/5
- `0 P1 4 P2 7 P1 8 P3 10 P4 14 P5 18 P1 22 P6 26 P4 30 P6 31 P4 32`
- 關鍵：t=8 P1（Q1）被 P3 preempt → Q1 head，timer reset 8；t=16 P1 timer 到 8 → 升返 Q0；t=22 P4（14 入 Q1）timer 到 8 → 升 Q0。
- WT = 13, 1, 0, 13, 1, 8 → **avg = 6**

14.8 F20 Q4.3 Priority + RR（smaller = higher；preemptive；same priority RR q=4）

- P1 0/9/2, P2 3/3/1, P3 8/5/4, P4 10/2/4, P5 16/8/5, P6 20/5/3
- `0 P1 3 P2 6 P1 12 P3 16 P4 18 P3 19 P5 20 P6 25 P5 32`
- WT = 3, 0, 6, 6, 8, 0 → **avg** $= \frac{23}{6} = 3.83$

14.9 Sp21 Q2.1（P1 0/2, P2 4/12, P3 6/6, P4 9/4, P5 13/5）

Alg	Gantt	WT	Avg
FCFS	0 P1 2 idle 4 P2 16 P3 22 P4 26 P5 31	0,0,10,13,13	**7.2**
SJF	0 P1 2 idle 4 P2 16 P4 20 P5 25 P3 31	0,0,19,7,7	**6.6**
RR q=8	0 P1 2 idle 4 P2 12 P3 18 P4 22 P2 26 P5 31	0,10,6,9,13	**7.6**
SRTF	0 P1 2 idle 4 P2 6 P3 12 P4 16 P5 21 P2 31	0,15,0,3,3	**4.2**

14.10 Sp21 Q2.2 MLFQ（Q0 q=2, Q1 q=4, Q2 FCFS）

- P1 0/10, P2 10/8, P3 8/30, P4 16/2, P5 22/8, P6 6/22
- `0 P1 2 P1 6 P6 8 P3 10 P2 12 P6 16 P4 18 P3 22 P5 24 P2 28 P5 32 P1 36 P6 52 P3 76 P2 78 P5 80`
  - (Q0)(Q1)(Q0)(Q0)(Q0)(Q1)(Q0)(Q1)(Q0)(Q1)(Q1)(Q2)(Q2)(Q2)(Q2)(Q2)
- WT：P1 26, P2 60, P3 38, P4 0, P5 50, P6 24 → **avg = 33**

14.11 HW2（P1 0/7, P2 1/3, P3 4/5, P4 10/2）

- **SRTF**：`0 P1 1 P2 4 P3 9 P1 10 P4 12 P1 17` → WT 10,0,0,0 → **avg WT 2.5；avg TAT 6.75**
- **RR q=3**：`0 P1 3 P2 6 P1 9 P3 12 P1 13 P4 15 P3 17` → WT 6,2,8,3 → **avg WT 4.75；avg TAT 9**
- **MLFQ（Q0 4, Q1 8, Q2 FCFS；P1 0/20, P2 1/5, P3 11/2）**：`0 P1(Q0) 4 P2(Q0) 8 P1(Q1) 11 P3(Q0) 13 P1(Q1) 18 P2(Q1) 19 P1(Q2) 27`
  - t=11 P1 被 P3 preempt → **Q1 head（喺 P2 前面）**，**剩 5 ms quantum**（唔係新 8 ms）。

15. 常見陷阱 & MC 速查

敘述	真/假
Scheduling on running→waiting / terminate = non-preemptive	✓
Timer interrupt (quantum expiry) = preemptive	✓
FCFS is preemptive	✗（**FCFS must be non-preemptive** — Sp21 MC）
SJF gives minimum average waiting time	✓
SJF easy to implement	✗（need next burst length）
SRTF ≥ SJF in performance	✓

敘述	真/假
RR with very large q → FCFS	✓
RR always better TAT than FCFS	✗
RR: no process waits more than (n−1)q	✓
Small q → high context-switch overhead	✓
Priority scheduling problem = starvation；solution = aging	✓
Multilevel queue: processes move between queues	✗（MLFQ 先可以）
MLFQ requires known burst times	✗
MLFQ: each queue different algorithm	✓
Starvation-free: FCFS and RR	✓（Sp21 MC 答案 A：1, 4）
OS schedules user threads (SCS)	✗（PCS 由 thread library；SCS 係 kernel threads）
One-to-one systems only use SCS	✓
AMP: master can be a bottleneck	✓
Multicore simplifies scheduling	✗
Per-processor queues give affinity for free	✓
Load balancing may counteract affinity	✓
Hard RT: service after deadline acceptable	✗
RM: static priority by period	✓
EDF: dynamic priority by deadline	✓
EDF requires periodic tasks	✗
EDF optimal, up to 100% utilization (theory)	✓
Little's formula $n = \lambda W$ only for FCFS	✗（any algorithm）
Response time (course) = first start − arrival	✗（**end of first run − arrival**）

16. 自我測驗

1. P1(0,5), P2(1,3), P3(2,1)：SRTF Gantt？（`0 P1 1 P2 2 P3 3 P2 5 P1 9`；WT = 4,1,0 → 5/3）
2. 10 個 jobs 各 100 s 同時到，RR q=1 vs FCFS 平均 TAT？（RR ≈ 995.5 s；FCFS = 550 s）
3. $\alpha = 0.5, \ \tau_0 = 10$，bursts 6, 4 → τ_2？（$\tau_1 = 8$；$\tau_2 = 6$）
4. 兩個 tasks：A (t=2, p=5)、B (t=4, p=10)：U = 0.8，RM 得唔得？（0.8 ≤ 0.828 → guaranteed）
5. PCS vs SCS？
6. Push vs pull migration？

6 Ch6 Synchronization 同步工具

📌 **考試範圍提示**：Midterm 包到 **Ch6（Synchronization Tools）**。每年都有 **1 大題（8–25 分）+ 3–4 題 MC**。題型：

1. **Race condition — 列出 shared variable 所有可能值**（F19 Q4.3、F20 Q5：每個值 2.5 分！）
2. **Semaphore 次序控制填空**（S26 Q5：wait/signal 填 blank）、**semaphore 輸出序列判斷**（F19 Q4.4）
3. **用 atomic instruction 寫 spinlock**（S26 Q2.4 swap；Sp21 Q3.1 compare_and_reset）
4. **Bounded-waiting test_and_set 算法分析 / 改錯**（S26 MC25、Sp21 Q3.2）
5. 定義：**race condition、critical section 三個條件、spinlock 優缺點、binary semaphore vs mutex**
6. **Pthread mutex + semaphore 程式輸出**（S26 MC23）

1. Background（背景） [6.4–6.8]

- **Processes execute concurrently and may be interrupted at any time, partially completing execution.**
- **Concurrent access to shared data may result in data inconsistency.**
- **Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.**

1.1 Producer–Consumer with `counter` [6.5–6.6]

- 用 `counter` 記 buffer 有幾多 item（可以用晒 BUFFER_SIZE 格）：

```
/* Producer */                          /* Consumer */
while (true) {                          while (true) {
  while (counter == BUFFER_SIZE) ;        while (counter == 0) ;
  buffer[in] = next_produced;             next_consumed = buffer[out];
  in = (in + 1) % BUFFER_SIZE;            out = (out + 1) % BUFFER_SIZE;
  counter++;                              counter--;
}                                       }
```

- 問題：`counter++` / `counter--` **唔係 atomic**。

1.2 Race Condition 例子 ⭐ ⭐ [6.7]

```
counter++  →  register1 = counter;  register1 = register1 + 1;  counter = register1
counter--  →  register2 = counter;  register2 = register2 - 1;  counter = register2
```

counter = 5 initially，interleaving：

Step	Executed	Result
S0	producer: register1 = counter	register1 = 5
S1	producer: register1 = register1 + 1	register1 = 6
S2	consumer: register2 = counter	register2 = 5
S3	consumer: register2 = register2 − 1	register2 = 4
S4	producer: counter = register1	counter = 6
S5	consumer: counter = register2	**counter = 4**（正確應該係 5！）

- 調轉 S4、S5 → counter = 6。**結果取決於執行次序** → **race condition**。
- **Kernel race example** [6.8]：P0、P1 同時 `fork()` → race on kernel variable `next_available_pid` → **same pid could be assigned to two different processes** unless mutual exclusion。

2. Race Condition & Critical-Section Problem ⭐ ⭐ ⭐ [6.9–6.12]

2.1 Race condition 定義（S26 Q2.1 原題，4 分）

- Sp21 MC：「several processes access and manipulate the same data concurrently and the outcome depends on the order」→ **race condition**。
- 根本原因（summary）：**high-level statements (count++) translate into multiple machine instructions, which can be interleaved arbitrarily.**

> 📝 **Model answer**：*A **race condition** is an undesirable situation where several processes (or threads) access and manipulate shared data concurrently, and the final outcome depends on the particular order (timing) in which the accesses take place. Because the execution order is unpredictable (e.g., context switches at untimely points), the result can be incorrect / non-deterministic. **Example**: two threads execute `counter++` on the same shared counter without synchronization; each loads the old value into a register, increments it and stores it back, so one update can be lost and the final value is wrong (e.g., 5 + 1 − 1 gives 4 or 6 instead of 5).*

2.2 Critical Section

- **Critical section：a segment of code in which a process may be changing shared variables, updating a table, writing a file, etc.**
- **When one process is in its critical section, no other may be in its critical section.（mutual exclusion and critical section imply the same thing）**
- **Critical-section problem：design a protocol that processes can use to cooperate.**
- **General structure** [6.10]：

```
do {
    entry section        /* request permission */
        critical section
    exit section
        remainder section
} while (true);
```

2.3 三個 requirements ⭐⭐⭐（F19 Q4.1 原題）[6.11]

1. **Mutual exclusion — if process P_i is executing in its critical section, no other processes can be executing in their critical sections.**
2. **Progress — if no process is executing in its critical section and some processes wish to enter, the selection of the process that will enter next cannot be postponed indefinitely**（只有唔喺 remainder section 嘅 process 先參與決定）。
3. **Bounded waiting — a bound must exist on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.**（i.e. 等待時間有上限，no starvation）

- Assumptions：**each process executes at a nonzero speed；no assumption about the relative speed of the processes.**
- Summary：**bounded waiting also implies progress.**
- Sp21 MC：CS solution **唔需要**滿足 → **synchronization**（三個係 mutual exclusion, progress, bounded waiting）；mutual exclusion 定義 → A。

2.4 Critical sections in the kernel [6.12]

- Kernel data structures（**list of open files**, memory allocation, process table, interrupt handling）都有 race condition。
- **Preemptive kernel** — allows preemption of a process running in kernel mode；**not free from race conditions**，SMP 上更難；但 more responsive（適合 real-time）。
- **Non-preemptive kernel** — runs until it exits kernel mode, blocks, or voluntarily yields → **essentially free of race conditions in kernel mode**（single-processor）。

3. Synchronization Hardware ⭐⭐ [6.13–6.21]

3.1 Disabling interrupts [6.13]

- **Uniprocessor：could simply disable interrupts** → current code runs without preemption。
- **Generally inefficient on multiprocessor/SMP systems**（要 disable 所有 CPU 嘅 interrupts，費時；影響 system clock）。
- 層次：**Hardware**（load/store, disable interrupts, **test&set, compare&swap**）→ **High-level APIs**（locks, semaphores）→ programs。

3.2 Atomic instructions [6.14]

- **Atomic = non-interruptible**；保證 instruction 唔會被中途打斷 → no race condition。
- 即使兩個 CPU 同時執行，**they are forced to execute sequentially (serialization)**。
- **Building blocks for more sophisticated synchronization tools.**
- 兩個常用：`test_and_set()`（test a memory word and set a value）、`compare_and_swap()`（swap contents of two words）。
- S26 MC24：commonly used hardware primitive → **test_and_set()**。

3.3 test_and_set ⭐⭐ [6.15–6.16]

```
boolean test_and_set(boolean *target) {   /* executed atomically */
    boolean rv = *target;
    *target = TRUE;
    return rv;                              /* returns the OLD value */
}
/* lock initialized to FALSE */
do {
    while (test_and_set(&lock)) ;   /* spin while old value was TRUE (lock held) */
        /* critical section */
    lock = false;                   /* release */
        /* remainder section */
} while (true);
```

- 第一個 call 嘅 process 攞到 old = FALSE → 入 CS，同時 lock 變 TRUE；其他人攞到 TRUE → spin。
- ✓ **mutual exclusion**；✗ **bounded waiting**（冇保證輪到邊個）。
- Sp21 MC4（lock init 0）：entry section → **`while (test_and_set(&lock));`**（C）。✗ `while (!test_and_set(&lock))`。

3.4 compare_and_swap (CAS) ⭐ ⭐ [6.17–6.18]

```
int compare_and_swap(int *value, int expected, int new_value) {  /* atomic */
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;                           /* returns the OLD value */
}
/* lock initialized to 0 */
do {
    while (compare_and_swap(&lock, 0, 1) != 0) ;   /* spin until old value was 0 */
        /* critical section */
    lock = 0;
        /* remainder section */
} while (true);
```

- ⚠️ 留意 return 值：**CAS 返回 old value** → 用 `!= 0` 判斷。Sp21 MC4 ✗ `while (compare_and_swap(&lock, 0, 1) == 0)`（相反）、✗ `compare_and_swap(&lock, 1, 0) != 0`。

3.5 Bounded-waiting Mutual Exclusion with test_and_set ⭐ ⭐ ⭐ [6.19–6.20]

```
/* shared: boolean waiting[n] (all false), boolean lock = false */
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;
        /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])        /* scan in cyclic order */
        j = (j + 1) % n;
    if (j == i)
        lock = false;           /* no one is waiting, so release the lock */
    else
        waiting[j] = false;     /* unblock process j (hand the lock over directly) */
        /* remainder section */
} while (true);
```

Sketch proof [6.20]：

- **Mutual exclusion**：P_i enters CS only if `waiting[i] == false` or `key == false`. `key` becomes false only via `test_and_set()` — **only the first process to execute test_and_set finds key == false**；others wait. `waiting[i]` becomes false only when another process leaves its CS, and **only one waiting[i] is set to false** → mutual exclusion。
- **Progress**：a process exiting its CS **either sets lock = false or sets waiting[j] = false** — both let a waiting process proceed。
- **Bounded waiting**：the exiting process **scans waiting[] in cyclic order (i+1, i+2, ..., n−1, 0, ..., i−1)** and designates the first waiting one → **any waiting process enters within n − 1 turns**。

> ⚠️ **S26 MC25**（which is NOT correct）→ **A.「test_and_set(&lock) releases the lock after the critical section」**（錯：釋放係 `lock = false` 或者 `waiting[j] = false`；test_and_set 只會 set lock 為 true）。✓ waiting[i] 表示 P_i 想入 CS；✓ `waiting[j] = false` 表示 P_i 將 lock 交畀 P_j（lock 冇真正釋放）；✓ 執行 `lock = false` 後 P_i 可以再入 CS。

> 📝 **Sp21 Q3.2（改錯題）**：版本用 `j = i + 1; while (j < n && !waiting[j]) j++; if (j < n) waiting[j] = false;`
> **Problems**：(1) **只 scan index 大過 i 嘅 process → index 細過 i 嘅 process 可能 unbounded waiting**（violates bounded waiting）；(2) **冇 process 等嘅時候 lock 永遠唔會 release** → 其他人之後永遠入唔到（violates progress / deadlock）。
> **Correction**：改返 cyclic scan `j = (i + 1) % n; while ((j != i) && !waiting[j]) j = (j + 1) % n;` 同 `if (j == i) lock = false; else waiting[j] = false;`。

3.6 Atomic Variables [6.21]

- **Atomic variable**：**atomic (uninterruptible) updates on basic data types (integers, booleans)**，用 CAS 實現：

```
void increment(atomic_int *v) {
    int temp;
    do {
        temp = *v;
    } while (temp != compare_and_swap(v, temp, temp + 1));   /* retry if someone changed v */
}
```

3.7 用其他 atomic instruction 寫 spinlock（考試題型）⭐⭐

S26 Q2.4 — `swap(x, v)`：atomically sets x = v and returns the old value of x（`locked` = 1 held / 0 free）：

```
while (swap(locked, 1) == 1) ;   /* acquire: spin while the old value was 1 */
/* critical section */
locked = 0;                       /* release (or swap(locked, 0)) */
```

Sp21 Q3.1 — `compare_and_reset(int *a, int old, int new)`：if (*a == old) { *a = new; return 1; } else return 0;

```
struct spinlock { int value = 0; };
void acquire(struct spinlock *lock) {
    while (compare_and_reset(&lock->value, 0, 1) == 0) ;   /* spin until we changed 0 → 1 */
}
void release(struct spinlock *lock) {
    lock->value = 0;
}
```

萬用套路：搵出「**成功攞到 lock**」時 instruction 嘅 return 值 → `while (instruction(...) 表示失敗) ;` → release 就 set 返 free value。

Instruction	Acquire	Release
`test_and_set(&lock)`（return old）	`while (test_and_set(&lock)) ;`	`lock = false;`
`compare_and_swap(&lock, 0, 1)`（return old）	`while (compare_and_swap(&lock, 0, 1) != 0) ;`	`lock = 0;`
`swap(locked, 1)`（return old）	`while (swap(locked, 1) == 1) ;`	`locked = 0;`
`compare_and_reset(&v, 0, 1)`（return 1 = success）	`while (compare_and_reset(&v, 0, 1) == 0) ;`	`v = 0;`
課本 `swap(&lock, &key)`（exchange）	`key = true; while (key == true) swap(&lock, &key);`	`lock = false;`

4. Mutex Locks ⭐⭐ [6.22–6.23]

- **Simplest software tool most OSes use.**
- **Protect a critical region by first `acquire()` a lock, then `release()` it — both must be atomic**（usually via hardware atomic instructions）。
- Boolean variable `available` 表示 lock 有冇得用：

```
acquire() {
    while (!available) ;   /* busy wait */
    available = false;
}
release() {
    available = true;
}
do { acquire lock;  critical section;  release lock;  remainder section; } while (true);
```

- **This solution requires busy waiting → the lock is called a spinlock.**
- Wait if locked — 等耐咗應該 **sleep / block** 而唔係 spin。

4.1 Spinlock 優缺點 ⭐⭐⭐（S26 Q2.4b 原題）

- 單 CPU 用 spinlock 冇乜意思（spin 緊嘅 thread 佔住 CPU，持有 lock 嘅 thread 跑唔到）。

> 📝 **Model answer**：*Disadvantage: a spinlock **wastes CPU cycles due to busy waiting** (the waiting thread keeps looping). Advantage: **no context switch is required** when a process must wait on a lock, and a context switch may take considerable time. So spinlocks are useful when locks are expected to be **held for short times**, especially on **multiprocessor systems**, where one thread can spin on one processor while another thread executes its critical section on another processor.*

5. Semaphores ⭐⭐⭐ [6.24–6.28]

5.1 定義 [6.24]

- **Semaphore S — a (non-negative) integer variable, a generalized lock；Dijkstra (late 1960s)；main synchronization primitive in original UNIX**。
- **Accessed only through two atomic operations (apart from initialization)：`wait()` and `signal()`**（originally **P()** = *proberen*, to test；**V()** = *verhogen*, to increment）。
- **Atomic → no more than one process can execute wait()/signal() on the same semaphore at the same time (serialization).**

```
wait(S) {                      signal(S) {
    while (S <= 0) ;               S++;
    S--;                       }
}
```

5.2 Counting vs Binary [6.25] ⭐

- **Counting semaphore — value ranges over an unrestricted domain；control access to a resource with a finite number of instances（initialize to the number of available resources）**；wait = 攞一個，signal = 還一個。
- **Binary semaphore — value only 0 or 1；can behave like a mutex lock**，但亦可以用嚟做其他同步。
- **Ordering example**（S1 喺 S2 之前執行）：`synch = 0`

```
P1:  S1;  signal(synch);          P2:  wait(synch);  S2;
```

> 📝 **F19 Q4.2：Binary semaphore = mutex lock? → *No.*** *A binary semaphore initialized to 1 can be used as a mutex lock, but a binary semaphore can also be used for other synchronization purposes, e.g., initialized to 0 to enforce execution order / signaling between processes. (Also a mutex lock is owned — the thread that locks it must unlock it — whereas any process can signal a semaphore.)*

5.3 Implementation with no busy waiting ⭐ ⭐ [6.26–6.27]

- **Each semaphore has a waiting queue**（each entry：value + pointer to next record）。
- **block()** — place the invoking process on the waiting queue（suspend）；**wakeup(P)** — remove a process from the waiting queue and put it in the **ready queue**。

```
typedef struct { int value; struct process *list; } semaphore;
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) { add this process to S->list; block(); }
}
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) { remove a process P from S->list; wakeup(P); }
}
```

- ⭐ **Increment/decrement is done BEFORE checking the value**（同 busy-waiting 版相反）。
- ⭐ **Semaphore value may become negative；if negative, its magnitude = number of processes waiting on it.**（busy-waiting classical 版永遠唔會 negative）

5.4 Deadlock and Starvation ⭐ [6.28]

- **Deadlock — two or more processes waiting indefinitely for an event that can be caused only by one of the waiting processes.**

```
S = Q = 1
P0: wait(S); wait(Q); ... signal(S); signal(Q);
P1: wait(Q); wait(S); ... signal(Q); signal(S);
```

P0 wait(S)、P1 wait(Q) 之後：P0 等 Q（要 P1 signal），P1 等 S（要 P0 signal）→ **deadlock**（extremely difficult to debug）。

- **Starvation (indefinite blocking) — a process may never be removed from the semaphore queue**，例如用 **LIFO order** 或者按 priority 移除 queue 入面嘅 processes。

---

6. 做題方法 ✍️ ⭐ ⭐ ⭐

6.1 Race condition 所有可能值（F19 Q4.3、F20 Q5）

步驟：

1. 將每個 thread 拆成 **atomic reads/writes**（`c = b + a` → read b, read a, write c；**if 條件嘅 read 係另一個 read！**）。
2. 另一個 thread 嘅 writes 可以插喺任何 read 之間（**次序保持**）。
3. 按 **「條件 read 嗰陣見到嘅值」分 branch**，再按「每個 read 見到新值定舊值」分 sub-case。
4. 列出所有 **distinct** 值（唔好漏！每個值獨立計分）。

F19 Q4.3：a = 7, b = 2, c = 5；T1：`if (a < 0) c = b - a; else c = b + a;`；T2：`b = 8; a = -6;`

Case	T1 見到	c
T2 全部喺 T1 之前 → a<0 branch	b = 8, a = −6	8 − (−6) = **14**
else branch；T2 喺 c 計之前完成	b = 8, a = −6	8 + (−6) = **2**
else；T2 喺 T1 之後	b = 2, a = 7	**9**
else；write b 喺 read b 前，write a 喺 read a 後	b = 8, a = 7	**15**
else；read b 喺 write b 前，write a 喺 read a 前	b = 2, a = −6	**−4**

→ {14, 2, 9, 15, −4}

F20 Q5：a = 7, b = 2；T1：`if (b < 0) c = b - a; else c = b + a;`；T2：`b = -8; a = 6;`

- b<0 branch（write b 嘅條件 read 之前）：a 新 → −8 − 6 = **−14**；a 舊 → −8 − 7 = **−15**
- else branch，第二次 read b 見到新值（−8）：a 新 → −8 + 6 = **−2**；a 舊 → −8 + 7 = **−1**
- else branch，第二次 read b 見到舊值（2）：a 新 → 2 + 6 = **8**；a 舊 → 2 + 7 = **9** → **{−14, −15, −2, −1, 8, 9}**

6.2 Semaphore 次序（dependency）題（S26 Q5）

規則：「X 必須喺 Y 之前完成」→ 用一個 **init 0** 嘅 semaphore s：**X 完成後 `signal(s)`；Y 開始前 `wait(s)`**。每個 dependency edge 一個 semaphore。

S26 Q5：A depends on B and C；B depends on C（即 C → B → A，且 C → A）；s0, s1, s2 全部 init 0。

```
workerA: /*BLANK1*/ wait(&s2);   wait(&s1);  printf("A");          // 等 B (s2) 同 C (s1)
workerB: /*BLANK2*/ wait(&s0);   printf("B");  signal(&s2);        // 等 C (s0)，完成通知 A
workerC: /*BLANK3*/ /* Nothing */ printf("C"); /*BLANK4*/ signal(&s0); signal(&s1);
```

- **BLANK1 = wait(&s2)，BLANK2 = wait(&s0)，BLANK3 = /* Nothing */，BLANK4 = signal(&s0)** → 輸出一定係 **C B A**。

6.3 Semaphore 輸出序列題（F19 Q4.4）

```
semaphore L = 3, R = 0;
P1: while (true) { wait(L); putc('C'); signal(R); }
P2: while (true) { wait(R); putc('A'); putc('B'); signal(R); }
P3: while (true) { wait(R); putc('D'); }
```

方法：數 token。L = 3 → **P1 最多印 3 個 C**（冇人 signal L）。每個 C 產生 1 個 R token；**P2 借 token 印 AB 後還返；P3 食咗 token 唔還**。

- (a) D 數目 = R token 總數 = **3**
- (b) A：**最少 0**（P3 搶晒 token），**最多 infinite**（P2 無限借還）
- (c) `CABABDCDABCABD` → **No**：印到 `C A B A B D C D` 時 C 印咗 2 次、D 食咗 2 個 token → R = 0，下一個 `A`（P2 需要 wait(R)）冇 token，要等第 3 個 C 先得。
- (d) `CABACDBCABDD` → **Yes**：C(R=1) A(P2 借→0) B(還→1) A(借→0) C(→1) D(食→0) B(還→1) C(→2) A(借→1) B(還→2) D(→1) D(→0) ✓
- **技巧**：逐個字元追蹤 semaphore 值同每個 process「做到邊」；任何時刻 token 唔可以 < 0。

6.4 Pthread 程式輸出題（S26 MC23）

```
int counter = 0; pthread_mutex_t mtx; sem_t sem;   /* sem_init(&sem, 0, 0) */
worker1: lock; counter += 2; printf("A %d;", counter); unlock; sem_post(&sem);
worker2: sem_wait(&sem); lock; counter += 3; printf("B %d;", counter); unlock;
```

- sem 初始 0 → worker2 一定等 worker1 post → **輸出** `A 2; B 5;`（答案 A）。mutex 保證 counter 更新互斥。

7. 補充：Peterson（Lab 6）& classic problems（Ch7 預覽）

7.1 Peterson's Solution（two processes, software-only；Lab 6 有教 + 證明，見 Labs notes L6.1）

```
int turn; boolean flag[2];          /* Pi uses j = 1 - i */
do {
    flag[i] = true;  turn = j;
    while (flag[j] && turn == j) ;
        /* critical section */
    flag[i] = false;
        /* remainder section */
} while (true);
```

- 滿足 mutual exclusion、progress、bounded waiting（兩個 process）；但 **modern architectures may reorder instructions → not guaranteed to work** without memory barriers。

7.2 Classic problems with semaphores（Ch7 預覽）

- **Bounded buffer**：`mutex = 1, full = 0, empty = n`

```
Producer: wait(empty); wait(mutex); add item; signal(mutex); signal(full);
Consumer: wait(full);  wait(mutex); remove item; signal(mutex); signal(empty);
```

- ⚠️ 調轉 `wait(mutex)` 同 `wait(empty)` 次序 → 可能 **deadlock**。
- **First readers–writers**：`rw_mutex = 1, mutex = 1, read_count = 0`；第一個 reader `wait(rw_mutex)`，最後一個 reader `signal(rw_mutex)`；writers 可能 **starve**。
- **Condition variables (POSIX)**：`pthread_cond_wait(&cond, &mutex)`（必須持有 mutex；wait 時**釋放 mutex 並 sleep**，醒後**重新 acquire**）；`pthread_cond_signal(&cond)`。

8. 常見陷阱 & MC 速查

敘述	真/假
Race condition: outcome depends on order of access	✓
counter++ is a single atomic machine instruction	✗
CS requirements: mutual exclusion, progress, bounded waiting	✓
"Synchronization" is one of the 3 CS requirements	✗
Bounded waiting implies progress	✓（summary）
Disabling interrupts works well on multiprocessors	✗（only uniprocessor）
Atomic = non-interruptible	✓
test_and_set returns the new value	✗（old value）
CAS returns old value	✓
Simple TAS lock satisfies bounded waiting	✗
Bounded-waiting TAS: waiting process enters within n−1 turns	✓
test_and_set releases the lock	✗
Spinlock wastes CPU but avoids context switch	✓
Spinlocks useful on multiprocessors for short critical sections	✓
Semaphore accessed only by wait/signal (except init)	✓
P() = wait, V() = signal	✓
Binary semaphore = mutex lock (equivalent)	✗（can be used as one, but more general）
Counting semaphore initialized to # of resources	✓
Blocking semaphore value can be negative = # waiting	✓
Non-preemptive kernel essentially free of race conditions (uniprocessor)	✓
LIFO semaphore queue may cause starvation	✓
Wrong order of wait() on two semaphores may deadlock	✓

9. Past-paper 題目（Ch6）

- **S26**：MC23（A 2; B 5;）、MC24（test_and_set）、MC25（A 錯）、Q2.1 race condition（4 分）、Q2.4 swap spinlock + 優缺點（4 分）、Q5 semaphore 填空（8 分）
- **F19**：Q4.1 三個條件（4 分）、Q4.2 binary semaphore vs mutex（3 分）、Q4.3 c 可能值（8 分）、Q4.4 semaphore 序列（10 分）
- **F20**：Q5 c 可能值（15 分！）
- **Sp21**：MC mutual exclusion、MC CS 唔需要 synchronization、MC race condition、MC4 TAS entry、Q3.1 compare_and_reset spinlock（4 分）、Q3.2 bounded waiting 改錯（7 分）

10. 自我測驗

1. 寫出 CS 三個 requirements 嘅定義（英文）。
2. 用 `test_and_set` 寫 lock；點解唔滿足 bounded waiting？
3. 一個 semaphore value = −3 代表咩？（3 個 processes 喺 waiting queue）
4. x = 0；T1：`x = x + 1`；T2：`x = x * 2`（每句 read/write 分開）→ 可能值？（**2**: T1→T2；**1**: T2→T1；**1**: T1 read 0, T2 read 0 & write 0, T1 write 1；**0**: T2 read 0, T1 write 1, T2 write 0 → {0, 1, 2}）
5. 三個 threads 要 A → B → C 次序，用最少 semaphores 寫出來。（s1 = s2 = 0；A; signal(s1) | wait(s1); B; signal(s2) | wait(s2); C）

L Labs & PA1/PA2 Lab 1–6 • PA 考點 • lab 例題

📌 **點解要睇**：Midterm email 講明「There may be questions related to **PA** or coding questions using some **APIs we discussed in our lab classes**」；Lab 3 slide 亦講「**PA-related questions in exam** — it is crucial to have a thorough understanding of the project assignments」。S26 Q3.3 直接叫你「refer to the **Lab 3 notes** for open, pipe, read, write」。✓ 本頁所有 fork 數目、semaphore 輸出、race 值、Gantt chart 都已用真 C 程式 / 窮舉 / 模擬器驗證，同 lab 官方答案一致。

L1. Lab 1 — Linux 環境 & Shell 指令

- **Terminal** = 畫面/視窗；**Shell** = 解讀指令嘅程式（bash, tcsh...）→ **shell fetches, interprets, executes** commands（Ch2 [2.8]）。
- 常用指令：

指令	用途	指令	用途
`pwd`	顯示目前目錄	`cd dir` / `cd ..` / `cd ~`	轉目錄
`ls -l -h` / `ls -a`	列出檔案（詳細 / 包括隱藏）	`mkdir` / `rmdir`	建立 / 刪除目錄
`cp a b` / `mv a b` / `rm f`	複製 / 搬或改名 / 刪除	`cat` / `less` / `head` / `tail`	睇檔案
`ps` / `ps aux`	列出 processes（PID）	`kill PID`	送 signal 畀 process
`wc -l`	數行數	`sort` / `grep x`	排序 / 搵字
`diff a b`	比較檔案（PA 對答案）	`chmod u+x f`	加執行權限
`du -ah | sort -rh | less`	睇 disk 用量（CS Lab 2 quota 100MB）	`vim` / `nano`	文字編輯器

L2. Lab 2 — C in Linux、編譯流程、Ch1–3 Review

L2.1 Compile pipeline（同 Ch2 linker/loader 掛鈎）

- 最簡：`gcc prog.c -o prog` → `./prog`（**loader** 將 executable 載入 memory）。
- `gcc -c main.c` → `main.o`（relocatable object）；`gcc -o main main.o -lm` → **linker** 連 math library（`-lm`，用 `<math.h>` 時要加）。
- `-g`：加 debugging info → `gdb prog`；Pthreads：`gcc prog.c -lpthread`（或 `-pthread`）。
- **Makefile**：`target: prerequisites` + 下一行 **TAB**（唔可以用空格）+ command；`make` / `make clean`。

L2.2 I/O Subsystem 補充（Lab 2 review slides）

- **Buffering** — store data temporarily while being transferred；**Caching** — faster device holding a **copy** of data。
  - ⭐ **分別：a buffer may hold the only existing copy of a data item, whereas a cache only holds a copy on faster storage of an item that resides elsewhere.**
- **Spooling** — hold output for a device that **cannot accept interleaved data**（e.g. **printer**）。
- I/O hardware：**port**（connection point）、**bus**（PCI / PCIe、expansion bus for slow devices）、**controller (host adapter)**；device registers：**data-in, data-out, status, control**。
- **Device driver** presents a **uniform device-access interface** to the I/O subsystem（abstraction）。
- Interrupt 補充：**maskable vs nonmaskable**；**interrupt vector = table of interrupt-handler addresses**；**multilevel interrupts**（high vs low priority）；⭐ **a trap (software interrupt) has a relatively LOW interrupt priority compared with device interrupts**；multi-CPU systems can process interrupts concurrently。
- **DMA controller operates the memory bus directly — momentarily prevents the CPU from accessing main memory**（cycle stealing）；完成後 interrupt。

L2.3 Lab 2 fork 題（已驗證 ✓）

```
int main() {
    pid_t pid = fork();  int cnt = 0;
    if (pid == 0) {
        cnt += 10;  pid = fork();
        if (pid != 0) { cnt += 10;  pid = fork(); }
    }
    printf("%d \n", pid);  printf("%d \n", cnt);
    return 0;
}
```

- 4 個 processes 印出：P0（pid≠0, cnt 0）、C1（pid≠0, cnt 20）、C2（pid 0, cnt 10）、C3（pid 0, cnt 20）。
- **non-zero PID 印 2 次；'0' 印 3 次（P0 嘅 cnt + C2、C3 嘅 pid）；'10' 印 1 次。**

填空：保證輸出 process 3 → 2 → 1（每個都印 x = 1）：

```
if (fork()) {           // BLANK1: fork() (> 0 亦可)
    x = x + 1;  wait(0); // BLANK2: wait(0) / wait(NULL)
    printf("...process 1 is %d\n", x);
} else if (fork()) {     // BLANK3: fork()
    x = x + 1;  wait(0); // BLANK4: wait(0)
    printf("...process 2 is %d\n", x);
} else { x = x + 1;  printf("...process 3 is %d\n", x); }
```

L3. Lab 3 — Linux Process & Pipe System Calls（⭐⭐⭐ PA1 + 考試 API）

L3.1 本 lab 用到嘅 system calls

- **Process**：`getpid`, `getppid`, `sleep`, `fork`, `wait`, `signal`
- **File**：`open`, `read`, `write`, `close`, `pipe`, `dup`, `dup2`
- **Execute**：`execl`, `execlp`, `execle`, `execv`, `execvp`, `execvpe`

Call	重點
`pid_t getpid(void)` / `getppid(void)`	自己 / parent 嘅 PID
`unsigned int sleep(unsigned int s)`	停 s 秒（`usleep` 用 μs；`nanosleep` 更細）
`pid_t fork(void)`	child 0、parent child PID、−1 error（set errno）；child 得到 parent address space（text, data, bss, heap, stack）同 **open file descriptors** 嘅**獨立 copy**
`pid_t wait(int *status)`	等任何一個 child；`wait(0)` / `wait(NULL)` 唔要 status；返回 child PID；**課程只用 `wait()`**（`waitpid` 唔使）
`signal(SIGINT, handler)`	**signal = software-generated interrupt**；可以用 customized handler 取代 default；**child inherits the customized handler**

L3.2 Zombie / Orphan demo（lab 示範）

- **Orphan**：parent 先完（冇 wait）→ shell prompt 出咗嚟之後 child 先繼續印 → child **被 init (PID 1) adopt & reap**。
- **用 wait() 正確**：parent 印完 `wait(0)` → child 印完先返 shell。
- **Zombie**：child 已完但 parent 仲 `sleep(10)` 未 `wait()` → child 喺嗰段時間係 zombie；parent `wait(0)` 後清走。
- **Exit status**：**voluntary** — normal `exit(0)`（return from main / `exit` / `_exit`）、error `exit(1)`；**involuntary** — fatal error（divide by 0, segmentation fault, core dump）、**killed by another process（`kill()`）**。child 終止時：open files flushed & closed、resources de-allocated、**parent notified via a signal**、exit status via `wait()`。

L3.3 File operations（低階 vs stdio）

```
fd = open("hello.txt", O_CREAT | O_WRONLY, S_IRUSR | S_IWUSR);   /* permission 600 */
write(fd, "hello\n", 6);  close(fd);
fd = open("hello.txt", O_RDONLY);  read(fd, buffer, 6);  close(fd);
```

- `open/read/write/close`（`<fcntl.h>`, `<unistd.h>`）= **low-level system calls**；`fopen/fread/fwrite/fclose`（`<stdio.h>`）= **high-level C library**。
- ⭐ **PA 唔准用 stdio file ops**：`FILE*` **係 C library 嘅 struct，cannot be used in pipe operations**。

L3.4 Pipes

- **Key concept**：**treat a pipe as 2 files**；`int pipe(int pfds[2])` → **pfds[0] read end，pfds[1] write end**。
- Data received **in the order it was sent**；**OS enforces mutual exclusion（only one process at a time）**；**processes sharing the pipe must have the same parent**。
- ⭐ **read/write are blocking**：冇 data 讀 → reader block；pipe 滿 → writer block（pipe size 有限：4 KB page / 64 KB）。
- **Two-way communication** → **create two pipes**（每個方向一條）。
- Shell pipe `ps aux | grep root | sort | wc -l`：前一個 command 嘅 output 做下一個嘅 input。

L3.5 exec family

- **6 variants**：`execl, execlp, execle, execv, execvp, execvpe` — **replace the current process image；成功唔會 return；error return −1**。
- 字母意思：**l** = arguments as a **list**（數目已知）；**v** = arguments in a **vector/array**（compile time 唔知數目，PA1 用 `execvp`）；**p** = search **PATH**；**e** = pass **environment**。

```
execlp("ls", "ls", "-l", "-h", NULL);           /* same as running "ls -l -h" */
char *args[] = {"ls", "-l", "-h", NULL};  execvp("ls", args);
printf("never printed if exec succeeds\n");
```

L3.6 dup / dup2（redirection 核心）

- `int dup(int oldfd)` — copy 到 **first unused (lowest-numbered) fd**；`int dup2(int oldfd, int newfd)` — copy 到指定 newfd（如已開先 close）；成功返回新 fd，失敗 −1。
- Default：**0 = stdin，1 = stdout，2 = stderr**。
- **Lab 題：以下程式輸出係咩？**

```
fd = open("output.txt", O_CREAT | O_WRONLY, S_IRUSR | S_IWUSR);
close(1);                 /* close stdout */
dup(fd);                  /* lowest free fd = 1 → stdout now refers to output.txt */
printf("Hello World!\n"); fflush(stdout);
```

→ **螢幕冇嘢顯示；"Hello World!" 寫入 output.txt**。

- `ls | wc -l` **實作（lab 版本：parent 做 wc）**：

```
pipe(pfds);
if (fork() == 0) {                 /* child → ls */
    close(1); dup(pfds[1]);        /* stdout = pipe write end */
    close(pfds[0]);
    execlp("ls", "ls", NULL);
} else {                           /* parent → wc -l */
    close(0); dup(pfds[0]);        /* stdin = pipe read end */
    close(pfds[1]);                /* MUST close write end, else wc never sees EOF */
    wait(0);
    execlp("wc", "wc", "-l", NULL);
}
```

- **dup vs dup2**：兩個都得；**dup2 更 flexible、易讀**；如果連續兩個 `close` 再兩個 duplicate，**應該用 dup2 指定位置**。

L3.7 Lab 3 fork 例題 + 更多歷屆 fork 題（全部驗證 ✓）

題目	Code（重點）	答案
Ex1	`for (i=0;i<3;i++) fork();`	$2^3 = 8$
Ex2	`for(i<2){ fork(); for(j<3) if(!fork()){} }`	8 個連續 fork → $2^8 = 256$
Ex3	`for(i<10){ if(fork()){fork();fork();} else fork(); }`	每輪 ×6（parent 4 + child 2）→ 6^{10}
Fall 2022 Q3	`for(i<n){ if (fork() == fork()) printf(...); }`	processes 4^n；lines $4^0 + 4^1 + \cdots + 4^{n-1} = \frac{4^n - 1}{3}$（n=1,2,3：1, 5, 21 行）
Fall 2024 P3.1	`for(i<M){ pid = fork(); if(!pid) for(j<N) fork(); }`	(a) **child** 執行 line Y（`!pid` true）；(b) M=N=1 → **3**；(c) $(2^N + 1)^M$
Spring 2025 MC12	`if (fork() || fork()) { if (fork() && !fork()) fork(); }`	**9** processes（新建 8 個）

- **Fall 2022 解釋**：每輪每個 process 變 4 個（2 個 fork）→ 4^n；每輪只有「兩個 fork 都返回 0」嗰個 grandchild 印 → 第 k 輪印 4^{k-1} 行。
- **Fall 2024 解釋**：設 $f(k)$ = 剩 k 輪時一個 process 最終變幾多個：parent 自己繼續 → $f(k-1)$；child 做 N 次 fork 變 2^N 個、每個都繼續 → $2^N f(k-1)$；所以 $f(k) = (1 + 2^N)f(k-1)$，$f(M) = (2^N + 1)^M$。
- **Short-circuit 圖解（lab）**：

Pattern	共幾多 processes	入 if-clause	入 else-clause
`fork() && fork()`	3	**1**（兩個都 > 0）	2
`fork() || fork()`	3	**2**	1（兩個都 = 0）

- S25 MC12：外層 `||` → 2 個入 if、1 個入 else；內層 `fork() && !fork()` → 1 個入 inner if（再 fork → 2）、2 個唔入 → 每個變 4 → $2 \times 4 + 1 = 9$。
- **Tips（lab）**：見到 loop → **展開成冇 loop 嘅等價 statements**；nested loop → 先展開內層；`if (fork())` → 分別諗 parent 同 child 行咩。

L3.8 PA1 — Simplified Linux Shell（考試可能問）

- 6 種 command pattern：`cmd`、`cmd < in`、`cmd > out`、`cmd < in > out`、`cmd > out < in`、`cmd1 | cmd2 | …`（最多 8 段）。
- `cd` **唔可以用 exec 做**（要改 shell 自己嘅 working directory）→ **`chdir(path)`**（0 成功 / −1 失敗）。
- **唔准用 `system()` / `popen()`**（會開另一個 shell 代你做）。
- **Redirection** = `open` + `dup/dup2` 取代 stdin/stdout；**multi-level pipe** = 將 2-level pipe 寫成 for-loop。
- `execvp` 嘅 argv array 尾要放 **NULL**（所以 8 個 arguments 要 size 9）。
- **Ctrl-C** → `signal(SIGINT, handler)`，SIGINT = **2**；start/end PID 要一致（冇多餘 processes）。

- Plagiarism：JPlag 檢測，**提供同抄襲雙方都 0 分**。

L4. Lab 4 — Named Pipes & Pthreads

L4.1 Anonymous vs Named pipes（lab 版本）

- Anonymous：unidirectional；cannot be accessed outside the creating process；parent–child；ceases after processes terminate。
- Named (FIFO)：bidirectional；no parent–child needed；several processes（多 writers）；**appear as files**（`mkfifo()`）；**exist until explicitly deleted**。
- ⭐ FIFO 細節：
  - **open / read / write 係 blocking：FIFO 要兩端都有 process open 先會 unblock**（冇 reader 時 writer 會 block）；用 `O_NONBLOCK` 可以變 non-blocking。
  - **Data read once then discarded**（唔似 regular file 可以重讀）→ **cannot broadcast to multiple recipients**。
  - 實際上 **一條 FIFO 唔適合雙向**（寫完即刻讀可能讀返自己個 message）→ 雙向用兩條。
  - **Writes ≤ PIPE_BUF bytes are atomic**；更大可能同其他 writer 交錯。
- **`popen(cmd, "r"/"w")` / `pclose(fp)`**（`<stdio.h>`）：fork 一個 child 執行 cmd，返回 `FILE*` 連住佢嘅 stdout（"r"）或 stdin（"w"）；`pclose` 等佢完並返回 termination status。（**PA1 禁用**）
- FIFO producer：`if (access(FIFO_NAME, F_OK) == -1) mkfifo(FIFO_NAME, 0777);` → `open(FIFO_NAME, O_WRONLY)` → `write` loop → `close`；consumer：`open(..., O_RDONLY)` → `read` until 0 → `close`。
- `access(path, how)`：`F_OK` 存在、`R_OK` / `W_OK` / `X_OK` 權限；0 = OK，−1 = 唔得。

L4.2 Exam example — 兩條 ordinary pipes 雙向通訊（填空）

```
char buffer[10] = {};  int p2c[2], c2p[2];
pipe(p2c);                                  /* BLANK1 */
pipe(c2p);                                  /* BLANK2 */
pid_t pid = fork();
if (pid == 0) {                             /* child */
    const char msg[] = "Child";
    close(c2p[0]);
    write(c2p[1], msg, 6);                  /* BLANK3（5 亦接受） */
    close(p2c[1]);
    read(p2c[0], buffer, 7);                /* BLANK4（6 亦接受） */
    printf("Child received the message: %s\n", buffer);
} else {                                    /* parent */
    const char msg[] = "Parent";
    close(c2p[1]);
    read(c2p[0], buffer, 6);                /* BLANK5（5 亦接受） */
    close(p2c[0]);
    write(p2c[1], msg, 7);                  /* BLANK6（6 亦接受） */
    printf("Parent received the message: %s\n", buffer);
}
```

- 輸出：`Parent received the message: Child` / `Child received the message: Parent`。

L4.3 Pthreads

```
int sum;                                   /* shared by the threads */
void *runner(void *param) {
    int upper = atoi(param);  sum = 0;
    for (int i = 1; i <= upper; i++) sum += i;
    pthread_exit(0);
}
int main(int argc, char *argv[]) {
    pthread_t tid;  pthread_attr_t attr;
    pthread_attr_init(&attr);                       /* default attributes */
    pthread_create(&tid, &attr, runner, argv[1]);  /* id, attr, start routine, arg */
    pthread_join(tid, NULL);                        /* wait for the thread */
    printf("sum = %d\n", sum);
}
```

- `pthread_t` = thread id；`pthread_attr_t` = attributes（stack size, scheduling info）；compile **`-lpthread`**。
- Join 多個 threads：`for (i = 0; i < 10; i++) pthread_join(workers[i], NULL);`
- **Thread trace**：main 印 "main: begin" → create T1、T2 → join → "main: end"；**threads run independently, order depends on scheduling**（T2 可以喺 T1 之前行）。
- **兩個 threads 各印 "begin of X" 同 "end of X"** → 中間 4 行有 $\binom{4}{2} = 6$ 種次序（每個 thread 自己 begin 先過 end），"main: start" 最先、"main: finish" 最尾 → **6 possible outputs**。

L4.4 Exam example — thread vs process 共享 memory（MC）

```
void *child(void *arg) { int *counter = (int *)arg; (*counter)++; return NULL; }
int main() {
    int counter = 0;  pthread_t thread;
    pthread_create(&thread, NULL, child, (void *)&counter);
    pthread_join(thread, NULL);
    printf("%d\n", counter);                 /* 1: thread shares memory */
    counter = 0;
    pid_t pid = fork();
    if (pid) wait(NULL);
    else { child((void *)&counter); exit(0); }
    printf("%d\n", counter);                 /* 0: child process has its own copy */
}
```

- 選項 A `0 0`、B `1 0`、C `0 1`、D `1 1` → **答案 B（1 / 0）**。**Key：threads share memory space but processes have separate memory spaces.**

L5. Lab 5 — Scheduling Review & PA2（MLFQ）

L5.1 Review 例題（P1 7, P2 4, P3 1, P4 4，全部 t=0，已驗證 ✓）

Algorithm	Gantt	avg WT
FCFS / RR q=8	`0 P1 7 P2 11 P3 12 P4 16`	$\frac{0+7+11+12}{4}=7.5$
RR q=4	`0 P1 4 P2 8 P3 9 P4 13 P1 16`	$\frac{9+4+8+9}{4}=7.5$
RR q=2	`0 P1 2 P2 4 P3 5 P4 7 P1 9 P2 11 P4 13 P1 16`	$\frac{9+7+4+9}{4}=7.25$
SJF	`0 P3 1 P2 5 P4 9 P1 16`（P2、P4 同長 → 先 P2）	$\frac{9+1+0+5}{4}=3.75$
Priority（4,1,2,3）	`0 P2 4 P3 5 P4 9 P1 16`	$\frac{9+0+4+5}{4}=4.5$

- **SRTF**（P1 0/7, P2 2/4, P3 4/1, P4 5/4）：`0 P1 2 P2 4 P3 5 P2 7 P4 11 P1 16` → $\frac{9+1+0+2}{4}=3$。
- 觀察：q 大過最長 burst → RR = FCFS；q 細咗 avg WT 唔一定變差（7.25 < 7.5）。

L5.2 ⭐ Priority + RR 有唔同 arrival time（HW / Exam 熱門）

- ⚠️ **Common mistake：reset the time quantum of a preempted process.** 正確：**被高 priority 新 process preempt 嘅 process 要記住佢剩餘嘅 quantum**（同 MLFQ 規則一樣）。
- **Spring 2023 midterm 官方答案**（q = 2）：`0 P1 1 P2 4 P1 5 P3 7 P1 8 P4 10 P1 11 P3 13 P5 15 P1 16 P3 18 P5 20 P3 22 P5 24`
    - P1 喺 0–1 跑咗 1 unit 被 P2（高 priority）preempt → 4–5 **只跑剩低嘅 1 unit quantum** 然後輪到 P3（同 priority RR）。8–10 同樣：P4 preempt P1，P1 10–11 用返剩低 1 unit。
    - 由 Gantt 推返嘅 table（模擬器重現完全一致 ✓）：P1 (0, 5, pr 2)、P2 (1, 3, pr 1)、P3 (2, 8, pr 2)、P4 (8, 2, pr 1)、P5 (9, 6, pr 2)，細數字 = 高 priority。

L5.3 RM / EDF 例題（deadline < period）

- P1 (t=1, d=2, p=4)、P2 (2, 5, 6)、P3 (3, 7, 9)；$U=\frac{1}{4}+\frac{2}{6}+\frac{3}{9}=0.917$。
- **RM**（P1 > P2 > P3）：`0 P1 1 P2 3 P3 4 P1 5 P3 6 P2 8 P1 9 P3 …` → **P3 first job finishes at 10 > deadline 7 → misses 1 deadline**。
- **EDF**：`0 P1 1 P2 3 P3 4 P1 5 P3 7 P2 8 P1 9 P2 10 P3 12 …` → **no deadline missed**。

L5.4 PA2 — N-level MLFQ simulator

- **N = 1–4 queues，last queue must be FCFS**（input 用大數 e.g. 1000 代表 FCFS），**≤ 10 processes**。
- Input：`queue_num = 3`、`time_quantum = 2 4 1000`、`process_table_size`、`process_table`（`P1 0 7` …）；`#` 開頭 / 空行忽略。
- Assumptions：**sorted by arrival**；**first arrival = 0**；**no two processes arrive at the same time**；**CPU never idle**；Gantt ≤ 300 steps。
- 用法：`./mlfq < input.txt > output.txt`（**input/output redirection**）；用 `diff --side-by-side your-out.txt sample-out.txt`（`--suppress-common-lines`）對答案。
- Output：parsed values + **`Gantt Chart = 0 P1 2 P2 3 P1 4 P3 5 P1 6 P4 8 P1 10 P4 11 P1 12`**。
- **Step-by-step 例子（Q0 q=2, Q1 q=4, Q2 FCFS；P1 0/7, P2 1/1, P3 4/1, P4 6/3）**，已用模擬器驗證 ✓：
    - t=1 P2 到 → **no preemption**（P1 都喺 Q0）；t=2 P1 用完 Q0 quantum → Q1；P2 跑。
    - t=3 P2 完 → P1（Q1，quantum 4）；t=4 P3 到 → **P1 preempted**（剩 burst 4，**剩 quantum 3**）。
    - t=5 P3 完 → P1；t=6 P4 到 → P1 preempted（剩 quantum 2）；P4 6–8 用完 Q0 → Q1 tail。

    - t=8 Q1 = [P1, P4] → P1 跑 2 → quantum 用完（剩 burst 1）→ Q2；t=10 P4 跑 1 完；t=11 P1（Q2）跑到 12。

L6. Lab 6 — Synchronization Review（Sample Questions）

L6.1 Peterson's solution（software-only，2 processes）

```
/* P_i (j = 1 - i) */
do {
    flag[i] = true;  turn = j;
    while (flag[j] && turn == j) ;   /* busy wait */
        /* critical section */
    flag[i] = false;
        /* remainder section */
} while (true);
```

- Shared：`flag[2]`（想入 CS）、`turn`（輪到邊個）；**assumption：load and store are atomic**。
- **Proof**：ME — 兩個都想入時 `turn` 只可以係 0 或 1 → 只有一個過到 while；Progress — 冇人喺 CS 就即刻入；**Bounded waiting — 最多等對方一次**（對方出嚟會設 `flag[j] = false`，再想入就要設 `turn = i`）。
- 缺點：**複雜（只限兩個 process）、每個 thread code 唔同、busy waiting 浪費 CPU** → OS 提供 mutex / semaphore / condition variable。

L6.2 POSIX primitives 速查

- Mutex：`pthread_mutex_init(&m, NULL)` → `lock` → access → `unlock` → `destroy`；**thread 持 lock 時 crash → 其他人 block**。
- Semaphore（`<semaphore.h>`）：`sem_init(&s, pshared, value)`（**pshared = 0：同一 process 嘅 threads 共用**）、`sem_wait`、`sem_post`、`sem_destroy`。

Initial value	行為	用途
1	一個 permit	**mutual exclusion（當 mutex）**
0	wait() 一定 block 直至有人 signal	**ordering**（等其他 thread 做完先）
N	N 個 permits	**管理 N 個相同 resources**

- Condition variable：`pthread_cond_wait(&cond, &mutex)` **atomically releases the mutex and sleeps；signalled → re-acquires the mutex before returning**；用 **`while (!ready)`** 包住；`pthread_cond_signal` / `pthread_cond_broadcast`。
- Lab note：**semaphore ≈ integer S + a mutex protecting it + a condition variable whose condition is "S > 0"**。

L6.3 Sample Questions（官方答案，已驗證 ✓）

- **Q0**：binary semaphore = mutex？→ **No** — init 1 可以當 mutex，但 init 0 可以做其他用途（ordering）。
- **Q1**：T1：`wait(S1); CS; signal(S2)`；T2：`wait(S2); CS; signal(S3)`；T3：`wait(S3); CS; signal(S1)` 要 T1, T2, T3, T1... → **S1 = 1, S2 = 0, S3 = 0**。
- **Q2**（P0 → P1 → P2 循環，填 BLANK）：**S0 = 1, S1 = 0, S2 = 0**；P0：`wait(S0)` ... **`signal(S1)`**；P1：**`wait(S1)`** ... **`signal(S2)`**；P2：**`wait(S2)`** ... `signal(S0)`。
- **Q3**（mutex 填空）：`pthread_mutex_t* m = (pthread_mutex_t*) arg;`、`pthread_mutex_lock(m);`、`pthread_mutex_unlock(m);` → **purpose：protect the shared counter；final counter = 10,000**（兩個 threads 各加 5,000）。（`void*` 要 cast 返 `pthread_mutex_t*`，因為 thread function signature 一定係 `void *f(void *)`。）
- **Q4**（X = Y = 0）：A：`wait(X); "b"; signal(Y); wait(Y); "c"; signal(Y);` B：`signal(X); "a"; wait(Y); "d";`
    - **Both complete：abcd, bacd, bcad（3 種）**；如果 B 先攞到 Y → A 永遠 block → **bad, abd**（A 唔會完成）。
- **Q5**（S0 = 1, S1 = S2 = S3 = 0）：P0 loop：`wait(S0); print '1'; release(S1); release(S2); release(S3);`；P1：`wait(S1); release(S0);`；P2：`wait(S2); release(S0); wait(S3); release(S0);` → **'1' 印 4 次**（S0 permits：initial 1 + P1 1 + P2 2 = 4）。
- **Q6**（readers–writers，用兩個 mutex）：

```
/* reader */
pthread_mutex_lock(&mutex);
read_count++;
if (read_count == 1) pthread_mutex_lock(&rw_mutex);    /* first reader locks out writers */
pthread_mutex_unlock(&mutex);
    /* reading */
pthread_mutex_lock(&mutex);
read_count--;
if (read_count == 0) pthread_mutex_unlock(&rw_mutex);  /* last reader lets writers in */
pthread_mutex_unlock(&mutex);
/* writer */
pthread_mutex_lock(&rw_mutex);  /* writing */  pthread_mutex_unlock(&rw_mutex);
```

- **Q7**（race values）：a = 1, b = 3, c = 0；T1：`a = b + 1; c = a - b;`；T2：`a = 13; b = 10; c = 2;` → **c ∈ {−6, 1, 2, 3, 10}**。
    - 2：T2 嘅 `c = 2` 最後寫；1：T1 讀 a=4、b=3 後 T2 全做完，或者 T2 先全做（a = 11, c = 11 − 10）；−6：T1 讀 a=4 後 T2 全做（b=10）；3：T1 寫 a=4 → T2 全做 → T1 讀 a=13, b=10；10：T1 寫 a=4 → T2 寫 a=13 → T1 讀 a=13, b=3 → T2 寫 b, c → T1 寫 c = 10。
    - 方法：**列出每個 read 可能見到嘅值（a ∈ {4, 11, 13}, b ∈ {3, 10}）+ 邊個最後寫 c**，再剔走不可能組合（a=11 表示 b 已經係 10）。

L7. 常見陷阱（Labs / PA）

敘述	真/假
PA 可以用 `fopen` / `FILE*` 做 pipe	✗（FILE* 唔用得喺 pipe operations）
`wait(0)` 同 `wait(NULL)` 一樣	✓
`dup(fd)` 一定返回 1	✗（返回 lowest unused fd；要先 `close(1)`）
`exec*()` 成功後會返回 0	✗（唔會 return；失敗 −1）
`execvp` 嘅 argv 唔使 NULL 結尾	✗
`cd` 可以用 `execvp("cd", …)` 做	✗（要 `chdir`）
FIFO 開 write end 時冇 reader 會 block	✓
FIFO 啲 data 可以畀多個 reader 各讀一次（broadcast）	✗
Child 會繼承 parent 嘅 customized signal handler	✓
Threads share memory，processes 唔 share	✓
Priority + RR 被 preempt 後 quantum 重新計	✗（保留剩餘 quantum）
PA2：最後一層 queue 係 FCFS	✓
Peterson：assume load/store atomic	✓
`sem_init(&s, 0, 0)` 嘅 0（第二個）表示 threads 共用	✓（pshared = 0）
Condition wait 會放開 mutex	✓（醒返會重新攞）

Q Past Papers Solved 歷屆長題全解

Spring 2026 Midterm（lecturer 提供）

S26-Q2.1 Race condition (4 pts)

CH6

What is a race condition? Give an example.

Answer.
A race condition occurs when two or more processes/threads access and manipulate shared data concurrently, and the final result depends on the order in which the accesses take place; since the order is unpredictable, the result may be incorrect or inconsistent. (2.5) Example: two threads increment the same `counter` without synchronization — `counter++` is load/add/store, so both may load the same old value and one update is lost. (1.5)

S26-Q2.2 Caching (4 pts)

CH1

What is the fundamental idea of caching? Is a high cache hit ratio alone enough to guarantee good performance? Explain by considering hit time and miss penalty.

Answer.
Caching temporarily copies a subset of information from a slower but larger storage into a faster but smaller storage so that frequently used data can be accessed more quickly. (2) No — hit ratio is a major criterion but not sufficient (1); performance is the **average access time = hit ratio × hit time + miss ratio × miss time**, which also depends on the hit time and the miss penalty (1). A slow cache or a huge miss penalty can still give poor performance.

S26-Q2.3 Custom policy $f_i = 1/(a \cdot c_i + b)$ (4 pts)

CH5

The OS keeps c_i = average past CPU burst of P_i and picks the process with the highest $f_i = \dfrac{1}{a\,c_i + b}$ $(a, b > 0)$, running it for one quantum or until it gives up the CPU. (a) Does this prioritize I/O-bound over CPU-bound processes? (b) Can it cause starvation?

Answer.
(a) **Yes** — I/O-bound processes have small CPU bursts c_i, hence a larger f_i and higher priority. (b) **Yes** — a CPU-bound process (large c_i, low f_i) can starve if higher-priority I/O-bound processes keep arriving.

S26-Q2.4 swap spinlock (4 pts)

CH6

`swap(x, v)` atomically sets x to v and returns the old value of x. `locked` = 1 when held, 0 otherwise. (a) Write the line that acquires the spinlock with a busy-spin loop. (b) Advantage and disadvantage of a spinlock.

Answer.
(a) `while (swap(locked, 1) == 1);` (release: `locked = 0;`) (b) Disadvantage: wastes CPU cycles due to busy waiting. Advantage: no context switch is needed when a process must wait on a lock (a context switch can take considerable time) — good for short critical sections on multiprocessors.

S26-Q3.1 fork tree with break (8 pts)

CH3

```
for (int i = 0; i < n; i++) {
    if (fork()) fork();
    else break;
}
```

(a) n = 2: draw the process tree; total processes and number of breaks. (b) Relationship between total processes and n.

Answer.
(a) **7 processes, 3 breaks.** Tree: P0 → P1 (break), P2, P3 (break), P4; P2 → P5 (break), P6. (b) Each active process per iteration creates 2 children: the first breaks, the second keeps looping → active count doubles: $A_k = 2^k$. After n iterations: active $= 2^n$, stopped $= 1 + 2 + \cdots + 2^{n-1} = 2^n - 1$. **Total** $= 2^{n+1} - 1$ (breaks $= 2^n - 1$).

S26-Q3.2 fork + exec scenarios (8 pts)

CH3

```
int ret = fork();
if (ret == 0) { printf("Hello1\n"); exec("some_executable"); printf("Hello2\n"); }
else if (ret > 0) { wait(NULL); printf("Hello3\n"); }
else { printf("Hello4\n"); }
```

List all possible scenarios and outputs.

Answer.

- Case I — fork succeeds, exec succeeds: **Hello1, Hello3** (exec replaces the image; Hello2 never runs).
- Case II — fork succeeds, exec fails (returns −1): **Hello1, Hello2, Hello3**.
- Case III — fork fails: **Hello4**. (Hello3 always last because the parent waits.)

S26-Q3.3 pipe + dup fill-in (8 pts)

CH3

Child counts even numbers in 1.txt, parent in 2.txt (file: length then elements, read with scanf); child sends its count via pipe; parent prints the total.

Answer.

- Blank 1: `close(0)` · Blank 2: `dup(fin)` (stdin now refers to the file)
- Blank 3: `if (number % 2 == 0) cnt++`
- Blank 4.1: `fd[1]` · 4.2: `&cnt` (child: `write(fd[1], &cnt, sizeof(int))`)
- Blank 5.1: `fd[0]` · 5.2: `&child_cnt` (parent: `read(fd[0], &child_cnt, sizeof(int))`)

S26-Q4.1 MLFQ Q0 RR4 / Q1 RR8 / Q2 FCFS (8 pts)

CH5

P1 (0, 9), P2 (2, 25), P3 (6, 7), P4 (14, 18), P5 (45, 15), P6 (58, 6). New jobs → Q0; full quantum → demote; Q0 arrival preempts Q1/Q2 immediately; preempted job → head of its queue; no demotion when preempted.

Answer.

Gantt: `0 P1 4 P2 8 P3 12 P1 14 P4 18 P1 21 P2 29 P3 32 P4 40 P2 45 P5 57 P2 58 P6 64 P2 71 P4 77 P5 80` (P5 45–49 Q0 then 49–57 Q1; P6 58–62 Q0 then 62–64 Q1). Waiting = completion − arrival − burst: P1 21−0−9 = 12; P2 71−2−25 = 44; P3 32−6−7 = 19; P4 77−14−18 = 45; P5 80−45−15 = 20; P6 64−58−6 = 0 → **avg** $= \frac{140}{6} = 23.33$. Response (end of first run − arrival): 4, 6, 6, 4, 4 (49−45), 4 (62−58) → **avg** $= \frac{28}{6} = 4.67$ (6.33 also accepted if P5/P6 merged).

S26-Q4.2 RM / EDF with D < T (10 pts)

CH5

(t, d, p): P1 (1, 3, 4), P2 (2, 4, 5), P3 (2, 6, 7), P4 (1, 5, 8); all arrive at 0; draw 0–16; ties → smaller ID; count missed deadlines.

Answer.

- **RM** (P1 > P2 > P3 > P4 by period): `0 P1 1 P2 3 P3 4 P1 5 P2 7 P3 8 P1 9 P3 10 P2 12 P1 13 P3 14 P3 15 P2 16` → missed: P3 (d 6, finished 8), P4 (d 5, never), P3 (d 13, finished 14), P4 (d 13, never) → **4 missed**. (U = 1.06 > 1.)
- **EDF**: `0 P1 1 P2 3 P4 4 P3 6 P1 7 P2 9 P1 10 P3 12 P4 13 P2 15 P1 16` → missed: P2 (d 14, finished 15), P1 (d 15, finished 16) → **2 missed**.

S26-Q4.3 Dynamic priority (9 pts)

CH5

Larger value = higher priority; waiting changes priority at rate α, running at rate β; tie → larger ID. P1 (0, 7), P2 (3, 5), P3 (6, 6), P4 (9, 4).

Answer.

- (a) $\alpha = 0,\ \beta = 3$, initial 0: `0 P1 7 P3 13 P4 17 P2 22`
- (b) $\alpha = 1,\ \beta = 0$, only P1, P2, check every 3: `0 P1 3 P2 6 P1 9 P2 11 P1 12`
- (c) $\alpha = 1,\ \beta = -2$, initial 30, check every 3: `0 P1 3 P2 6 P3 9 P4 12 P1 15 P2 17 P3 20 P4 21 P1 22`

S26-Q5 Semaphore ordering (8 pts)

CH6

A depends on B and C; B depends on C. s0, s1, s2 initialised to 0. workerA: BLANK1; wait(&s1); print A. workerB: BLANK2; print B; signal(&s2). workerC: BLANK3; print C; BLANK4; signal(&s1).

Answer.

BLANK1 `wait(&s2)` · BLANK2 `wait(&s0)` · BLANK3 `/* Nothing */` · BLANK4 `signal(&s0)` → order always C, B, A.

Fall 2019 Midterm

F19-Q1.1 Linker and loader (2 pts) CH2

What do the linker and loader do, respectively?

Answer.
The linker combines relocatable object files (and libraries) into a single binary executable file; the loader loads the binary executable file into memory so it can run.

F19-Q1.2 Locality (3 pts) CH1

What are temporal locality and spatial locality?

Answer.
Temporal locality: a memory location accessed recently is likely to be accessed again in the near future. Spatial locality: locations near a recently accessed location are likely to be accessed soon.

F19-Q1.3 Why separate API and system calls (2 pts) CH2

Two reasons for separating APIs from the underlying system calls?

Answer.
(1) Program portability — the API stays the same across platforms. (2) Hide the complex details of system calls from programmers.

F19-Q1.4 LKM vs layered (3 pts) CH2

Similarity and difference between the loadable-kernel-module approach and the layered approach?

Answer.
Similarity: functionality is divided into modules, each with a well-defined interface. Difference: LKM is more flexible — any module can call any other module (no higher/lower layer restriction), and modules can be loaded/removed at run time.

F19-Q2.1 State transitions (3 pts) CH3

Fill in the actions for: running→ready, ready→running, running→waiting, waiting→ready.

Answer.
running→ready: **interrupt**; ready→running: **scheduler dispatch (CPU scheduling)**; running→waiting: **I/O or event wait**; waiting→ready: **I/O or event completion**.

F19-Q2.2 Socket (2 pts) CH3

Four pieces of information that uniquely define a socket connection?

Answer.
Source IP address, source port number, destination IP address, destination port number.

F19-Q2.3 Thread unique vs shared (3 pts) CH4

Three elements unique to each thread and three shared.

Answer.
Unique: program counter, registers (state), stack (thread ID). Shared: code, data (globals/heap), open files.

F19-Q2.4 Shell system calls (3 pts) CH3

Two system calls a shell must execute to start a new process on UNIX?

Answer.
`fork()` — clones the calling process (creates a child); `exec()` — overlays/replaces the child's address space with a new program (loads the program to run).

F19-Q2.5 Multicore vs SMP, CMT (4 pts) CH5

Two advantages of multicore vs conventional SMP; reason for chip multithreading (hyper-threading).

Answer.
Multicore is faster (on-chip communication) and consumes less power than separate single-core chips. CMT exploits **memory stall**: processors are much faster than memory, so a core may spend up to ~50% of its time waiting for data; with multiple hardware threads the core switches to another thread while one waits.

F19-Q2.6 Lines printed vs n (5 pts)

CH3

```
for (i = 0; i < n; ++i) { if (fork()) printf("Fork!\n"); else printf("Fork!\n"); }
```

Answer.
Each iteration doubles the processes and every process prints one line: iteration i prints 2^i lines → total $2 + 4 + \cdots + 2^n = 2^{n+1} - 2$ ✓

F19-Q2.7 fork()==fork() || fork()!=fork() (5 pts)

CH3

```
if (fork() == fork() || fork() != fork()) fork();
printf("Fork!\n");
```

Answer.
23. After `fork()==fork()` there are 4 processes; only the one that got 0 twice is true → it skips the rest of `||` and forks → 2 lines. The other 3 run `fork()!=fork()` → 12 processes; 9 are true and fork again → 18; 3 false → 3. Total 2 + 18 + 3 = 23 ✓

F19-Q3.1 Which threads does the OS schedule (4 pts)

CH5

Which threads are scheduled by the OS? How does mapping user threads onto kernel threads/LWPs differ from CPU scheduling?

Answer.
The OS schedules **kernel-level threads**. Scheduling user threads onto LWPs/kernel threads (PCS) is done by the thread library, usually by programmer-set priority, among threads of the **same process**; CPU scheduling (SCS) chooses which kernel thread runs on a CPU among **all** kernel threads in the system.

F19-Q3.2 FCFS problem; RR worse than FCFS (3 pts)

CH5

Answer.
FCFS: **convoy effect** — short processes stuck behind a long one. RR has worse turnaround than FCFS when all processes have the **same CPU burst length that is larger than the quantum** (they all finish near the end).

F19-Q3.3 MLFQ approximates SJF (3 pts)

CH5

Answer.
Short interactive jobs finish in the top (high-priority) queue, while long CPU-bound jobs sink to lower queues — so short jobs are served first, like SJF. Advantage: **no prior knowledge of CPU burst times is needed**.

F19-Q3.4 EDF to t = 38 (6 pts)

CH5

P1 (period = deadline 10, t 4), P2 (16, 2), P3 (20, 8).

Answer.
`0 P1 4 P2 6 P3 14 P1 18 P2 20 P1 24 P3 32 P1 36 P2 38` (deadline ties at t = 10 and 30 → the running P3 continues). No deadline missed.

F19-Q3.5 Least Used First (12 pts)

CH5

LUF picks the process that has used the fewest CPU cycles; tie → FCFS; preempted process goes to the end of the queue; scheduler runs on completion and on new arrival. P1 (0, 25), P2 (15, 25), P3 (25, 5), P4 (40, 5).

Answer.
(a) `0 P1 15 P2 25 P3 30 P2 40 P4 45 P1 55 P2 60` (b) TAT / RT / WT: P1 55 / 15 / 30; P2 45 / 10 / 20; P3 5 / 5 / 0; P4 5 / 5 / 0. (c) Similar to SRTF: short jobs tend to get high priority. Different: under LUF a process that has already run a long time is likely to starve; under SRTF a process with a long burst is likely to starve from the beginning.

F19-Q3.6 FCFS / SJF / RR4 / SRTF (12 pts) CH5

P1 (0, 8), P2 (7, 13), P3 (9, 1), P4 (11, 5), P5 (20, 7), P6 (27, 1).

Answer.

- FCFS `0 P1 8 P2 21 P3 22 P4 27 P5 34 P6 35` — WT 0,1,12,11,7,7 → **6.33**
- SJF `0 P1 8 P2 21 P3 22 P4 27 P6 28 P5 35` — WT 0,1,12,11,8,0 → **5.33**
- RR q=4 `0 P1 8 P2 12 P3 13 P4 17 P2 21 P4 22 P5 26 P2 30 P5 33 P6 34 P2 35` — WT 0,15,3,6,6,6 → **6.0** (corrected official)
- SRTF `0 P1 8 P2 9 P3 10 P2 11 P4 16 P2 20 P5 27 P6 28 P2 35` — WT 0,15,0,0,0,0 → **2.5** (official; keeping P2 on the tie at t=20 also gives 2.5)

F19-Q4.1 CS requirements (4 pts) CH6

Answer.

(1) **Mutual exclusion** — if a process is executing in its CS, no other process can be in its CS. (2) **Progress** — if no process is in its CS and some want to enter, the selection of the next one cannot be postponed indefinitely. (3) **Bounded waiting** — there is a bound on the number of times other processes may enter their CS after a process has requested entry and before that request is granted.

F19-Q4.2 Binary semaphore vs mutex (3 pts) CH6

Answer.

No. A binary semaphore initialised to 1 can be used as a mutex lock, but it can also be used for other synchronization purposes (e.g., initialised to 0 for ordering/signalling).

F19-Q4.3 Possible values of c (8 pts) CH6

a = 7, b = 2, c = 5. T1: `if (a < 0) c = b - a; else c = b + a;` T2: `b = 8; a = -6;`

Answer.

14, 2, 9, 15, −4 — (a<0 branch: 8−(−6) = 14); else-branch: T2 done before c computed: 8+(−6) = 2; T2 after: 2+7 = 9; b written before read but a after: 8+7 = 15; b read before write, a written before read: 2+(−6) = −4.

F19-Q4.4 Semaphores L = 3, R = 0 with putc (10 pts) CH6

P1: wait(L); putc('C'); signal(R). P2: wait(R); putc('A'); putc('B'); signal(R). P3: wait(R); putc('D').

Answer.

(a) **3** D's. (b) A's: min **0**, max **infinite**. (c) `CABABDCDABCABD` — **No** (after "CABABDCD", R = 0, so P2 cannot print the next A before a third C). (d) `CABACDBCABDD` — **Yes**.

Fall 2020 Midterm

F20-Q2.1 Microkernel (4 pts) CH2

Answer.

Main idea: remove all nonessential components from the kernel and implement them as system/user-level programs in separate address spaces; communicate by message passing. Advantage: small kernel, easier to extend and port, more secure and reliable. Disadvantage: performance suffers due to increased system-function overhead (user↔kernel message passing).

F20-Q2.2 Linker and loader (4 pts) CH2

Answer.

Linker: combines several relocatable object modules into a single binary executable image (stored on disk). Loader: loads the executable into memory for execution.

F20-Q2.3 Heavyweight vs lightweight (4 pts) CH4

Answer.

A process is heavyweight because it owns an address space and resources; creating one duplicates program and data. A thread is lightweight — only an execution unit (PC, registers, stack) that shares code, data and files with other threads of its process, so creating it needs no duplication.

F20-Q2.4 Common vs per-processor ready queue (4 pts)

CH5

Answer.
Common queue: easy load balancing (no CPU idle while the queue is non-empty) but hard to keep processor affinity (and needs locking). Per-processor queues: natural processor affinity but need push/pull migration for load balancing.

F20-Q2.5 MLFQ benefits (4 pts)

CH5

Answer.
(1) Resembles SJF/SRTF performance without prior knowledge of the next CPU burst (2 pts); (2) RR-like good response time for interactive jobs (1); (3) fair — all processes make progress (1).

F20-Q3.1 Possible outputs (10 pts)

CH3

```
if (fork() == 0) {
    if (fork() > 0) { printf("1"); wait(NULL); printf("2"); }
    else printf("3");
} else printf("4");
```

Answer.
Unbuffered: **8** outputs — "2" after "1" and "3"; "4" anywhere: 1342, 1432, 3142, 4132, 3412, 4312, 1324, 3124. Buffered ("1" and "2" flushed together): 3 outputs — 4312, 3412, 3124.

F20-Q3.2 Count "a" (13 pts)

CH3

```
if (!fork()) { for (int i = 0; i < n; i++) if (fork()) fork(); }
fork(); printf("a");
```

Answer.
First fork: the child enters the loop; each iteration every process becomes 3 → 3^n processes from the child branch. Plus the original → $3^n + 1$. The final fork doubles → $2(3^n + 1)$ **"a"s ✓**

F20-Q4.1 FCFS / SJF / SRTF (9 pts)

CH5

P1 (0, 9), P2 (3, 3), P3 (8, 2), P4 (10, 6), P5 (16, 8), P6 (20, 5).

Answer.

- FCFS `0 P1 9 P2 12 P3 14 P4 20 P5 28 P6 33` → (0+6+4+4+4+8)/6 = **4.33**
- SJF `0 P1 9 P3 11 P2 14 P4 20 P6 25 P5 33` → (0+8+1+4+9+0)/6 = **3.67**
- SRTF `0 P1 3 P2 6 P1 8 P3 10 P1 14 P4 20 P6 25 P5 33` → (5+0+0+4+9+0)/6 = **3.0**

F20-Q4.2 Two-level feedback queue with aging (10 pts)

CH5

Q0 RR 4 ms (high), Q1 FCFS; new → Q0; Q0 arrival preempts Q1; preempted Q1 process → head of Q1 with timer reset; a Q1 process whose timer reaches 8 ms rejoins Q0. P1 (0, 9), P2 (3, 3), P3 (8, 2), P4 (10, 9), P5 (13, 4), P6 (18, 5).

Answer.
`0 P1 4 P2 7 P1 8 P3 10 P4 14 P5 18 P1 22 P6 26 P4 30 P6 31 P4 32` → WT (13+1+0+13+1+8)/6 = **6**. (P1 ages back to Q0 at 16; P4 at 22.)

F20-Q4.3 Priority + RR (8 pts)

CH5

Smaller number = higher priority; preemptive; same priority → RR q = 4. P1 (0, 9, 2), P2 (3, 3, 1), P3 (8, 5, 4), P4 (10, 2, 4), P5 (16, 8, 5), P6 (20, 5, 3).

Answer.
`0 P1 3 P2 6 P1 12 P3 16 P4 18 P3 19 P5 20 P6 25 P5 32` → (3+0+6+6+8+0)/6 = **3.83**

F20-Q5 Possible values of c (15 pts)

CH6

a = 7, b = 2. T1: `if (b < 0) c = b - a; else c = b + a;` T2: `b = -8; a = 6;`

Answer.
−14, −15, −2, −1, 8, 9 (2.5 marks each).

Spring 2021 Midterm

Sp21-Q1.1 pid1/pid2 (5 pts)

CH3

```
pid1 = fork(); pid2 = fork();
if (pid1 > 0) if (pid2 == 0) fork();
```

Answer.
5 — 4 processes after two forks; only the child created by the parent's second fork has pid1 > 0 and pid2 == 0, so one more process ✓

Sp21-Q1.2 Nested loop (5 pts)

CH3

```
if (fork() == 0) { for (int i = 0; i < 5; i++) if (fork() == 0) fork(); }
fork();
```

Answer.
$(3^5 + 1) \times 2 = 488$ ✓

Sp21-Q1.3 Output orderings (8 pts)

CH3

```
printf("A\n");
if (fork() == 0) printf("B\n");
else { printf("C\n"); ___A___ }
printf("D\n");
```

Answer.
(a) empty: ABDCD, ABCDD, ACBDD, ACDBD. (b) `wait(NULL)` : ABDCD, ABCDD, ACBDD. (c) `printf("E\n")` : ABDCED, ABCDED, ABCEDD, ACBDED, ACBEDD, ACEBDD, ACEDBD.

Sp21-Q2.1 FCFS / SJF / RR8 / SRTF (10 pts)

CH5

P1 (0, 2), P2 (4, 12), P3 (6, 6), P4 (9, 4), P5 (13, 5).

Answer.
FCFS 7.2 · SJF 6.6 · RR(8) 7.6 · SRTF 4.2 (Gantt charts in Ch5 §14.9; note the idle gap 2–4).

Sp21-Q2.2 MLFQ q=2, q=4, FCFS (8 pts)

CH5

P1 (0, 10), P2 (10, 8), P3 (8, 30), P4 (16, 2), P5 (22, 8), P6 (6, 22).

Answer.
`0 P1 2 P1 6 P6 8 P3 10 P2 12 P6 16 P4 18 P3 22 P5 24 P2 28 P5 32 P1 36 P6 52 P3 76 P2 78 P5 80` → WT 26, 60, 38, 0, 50, 24 → **33**.

Sp21-Q2.3 EDF vs RM (8 pts)

CH5

P1 (1, 3, 3), P2 (2, 4, 4), 0–14, soft real-time.

Answer.
EDF `0 P1 1 P2 3 P1 4 P2 6 P1 7 idle 8 P2 10 P1 11 idle 12 P1 13 P2 14`; RM `0 P1 1 P2 3 P1 4 P2 6 P1 7 idle 8 P2 9 P1 10 P2 11 idle 12 P1 13 P2 14`. EDF is better: dynamic deadline-based priorities can meet more deadlines (up to 100% utilization) than RM's static priorities.

Sp21-Q3.1 compare_and_reset spinlock (4 pts)

CH6

Answer.
`struct spinlock { int value = 0; };` · acquire: `while (compare_and_reset(&lock->value, 0, 1) == 0) ;` · release: `lock->value = 0;`

Sp21-Q3.2 Fix the bounded-waiting algorithm (7 pts)

CH6

Answer.
Problems: (1) only processes with index > i are checked → lower-index processes can wait unboundedly; (2) the lock is never released when nobody is waiting. Fix: cyclic scan `j = (i + 1) % n; while ((j != i) && !waiting[j]) j = (j + 1) % n; if (j == i) lock = 0; else waiting[j] = false;`

Homework 1（Fall 2026）

HW1-Q2.1 AMAT + locality (6 pts)

CH1

Hit ratio 90%, hit time 2 ns, miss time 50 ns.

Answer.

$0.9 \times 2 + 0.1 \times 50 =$ **6.8 ns**. Temporal: recently accessed items are likely accessed again (e.g., loop counter, loop body instructions). Spatial: items near recent accesses are likely accessed soon (e.g., sequential array elements a[i], a[i+1]).

HW1-Q2.2 Multiprogramming vs time sharing (6 pts)

CH1

Answer.

Multiprogramming keeps several jobs in memory; when one waits for I/O, the CPU switches to another, so the CPU always has work → higher utilization. Time sharing switches among jobs so frequently (timer + CPU scheduling) that users can interact with each program while it runs → short response time, interactive computing.

HW1-Q2.3 State sequence (6 pts)

CH3

Answer.

New → Ready → Running → Waiting → Ready → Running → Ready → Running → Terminated. When leaving Running, the PCB saves the process state, program counter, CPU registers, plus updated scheduling, accounting and I/O-status information.

HW1-Q2.4 read(): user → kernel → user (6 pts)

CH2

Answer.

(1) read() in libc puts the system-call number and parameters in registers and executes a trap instruction. (2) The trap switches to kernel mode; the kernel indexes the system-call table and runs the read service. (3) The kernel stores the return value and executes return-from-trap, which switches back to user mode and resumes after the call.

HW1-Q2.5 Parent → child through a pipe (6 pts)

CH3

Answer.

Call `pipe(fd)` **before** `fork()`; after fork, the parent (writer) closes `fd[0]` and writes to `fd[1]`; the child (reader) closes `fd[1]` and reads from `fd[0]`; close the used ends when done.

HW1-Q3.1 Values of x (15 pts)

CH3

Answer.

C1: 3, C2: 5, P: 2 (each process has its own copy of x). Outputs: "C1: 3, C2: 5, P: 2" or "C2: 5, C1: 3, P: 2". Moving the first wait before `x *= 2` guarantees C1, C2, P order; values unchanged.

HW1-Q3.2 exec worker (15 pts)

CH3

Answer.

3 processes (parent → child → grandchild; the grandchild execs worker). X is not printed (successful exec never returns). Outputs: A P W C D · A W P C D · A W C P D.

HW1-Q3.3 Pipe parent → child (10 pts)

CH3

Answer.

(a) Parent closes fd[0], writes to fd[1]; child closes fd[1], reads fd[0]; parent → child. (b) "Child: 12" then "Parent: done" — read blocks until data arrives, and the parent waits for the child. (c) No — separate pipe() calls after fork create two unrelated pipes, with no shared descriptors.

Homework 2（Fall 2026）

HW2-Q2.1 Different PC/registers (6 pts)

CH4

Answer.

Yes — each thread has its own PC, register set and stack; saved in its **TCB** when not running.

HW2-Q2.2 Many-to-one on 4 cores (6 pts) CH4

Answer.
No — all user threads map to one kernel thread, which runs on one core at a time; only concurrency, no parallelism (and one blocking call blocks all).

HW2-Q2.3 Ctrl+C (6 pts) CH4

Answer.
Asynchronous (generated by an event external to the process). Yes — install a user-defined handler with `signal(SIGINT, handler)` to override the default (terminate).

HW2-Q2.4 Hard affinity (6 pts) CH5

Answer.
No — hard affinity restricts T to CPU 0 and CPU 1; load balancing may move it between those two only.

HW2-Q2.5 RM vs EDF at t = 50 (6 pts) CH5

Answer.
No. RM picks A (shorter period, static) → B finishes at 85 and misses 80. EDF picks B (deadline 80 < 100) → B finishes at 60, A at 85; both meet deadlines.

HW2-Q3.1 Amdahl (10 pts) CH4

Answer.
(a) $\frac{1}{0.2 + 0.8/4} = \mathbf{2.5}$. (b) $N \to \infty$: $\frac{1}{0.2} = \mathbf{5}$.

HW2-Q3.2 SRTF and RR q=3 (20 pts) CH5

P1 (0, 7), P2 (1, 3), P3 (4, 5), P4 (10, 2).

Answer.
SRTF `0 P1 1 P2 4 P3 9 P1 10 P4 12 P1 17` → avg WT 2.5, avg TAT 6.75. RR(3) `0 P1 3 P2 6 P1 9 P3 12 P1 13 P4 15 P3 17` → avg WT 4.75, avg TAT 9.

HW2-Q3.3 MLFQ (10 pts) CH5

Q0 RR4, Q1 RR8, Q2 FCFS; P1 (0, 20), P2 (1, 5), P3 (11, 2); preempted Q1 job keeps the unused part of its quantum.

Answer.
`0 P1(Q0) 4 P2(Q0) 8 P1(Q1) 11 P3(Q0) 13 P1(Q1) 18 P2(Q1) 19 P1(Q2) 27`. At t = 11, P1 goes to the head of Q1 (ahead of P2) with 5 ms of its Q1 quantum left.

Fall 2022 Midterm（lab slides 引用）

F22-Q3 fork() == fork() in a loop (10 pts) CH3

```
int main() {
    int i, n;
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        if (fork() == fork()) {
            printf("fork() == fork()!\n");
            fflush(stdout);
        }
    }
    return 0;
}
```

(a) Number of processes and of printed lines for n = 1, 2, 3. (b) In terms of n.

Answer.
(a) n = 1: 4 processes, 1 line; n = 2: 16, 5; n = 3: 64, 21. (b) Each iteration every process does two forks → ×4, so **4^n processes**. In each iteration only the grandchild that received 0 from both forks sees equal return values, so the iteration that starts with 4^k processes prints 4^k lines. Total lines $= 4^0 + 4^1 + \cdots + 4^{n-1} = \frac{4^n - 1}{3}$. ✓ (verified)

Fall 2024 Midterm（lab slides 引用）

F24-P3.1 fork in nested loops (10 pts) CH3

```
for (i = 0; i < M; i++) {
    pid_t pid = fork();          // line X
    if (!pid) {
        for (j = 0; j < N; j++)  // line Y
            fork();
    }
}
```

(a) Which process executes line Y? (b) M = N = 1: how many processes? (c) In terms of M and N.

Answer.
(a) The **child** — `!pid` is true only where `fork()` returned 0. (b) **3** — the parent forks a child (2), the child forks once in the inner loop (3). (c) Let $f(k)$ be the number of processes one process becomes with k outer iterations left: the parent continues ($f(k-1)$) and the child becomes 2^N processes that all continue ($2^N f(k-1)$), so $f(k) = (2^N + 1)\, f(k-1)$ and the total is $(2^N + 1)^M$. ✓ (verified)

Spring 2025 Midterm（lab slides 引用）

S25-MC12 Nested short-circuit forks CH3

```
if (fork() || fork()) {
    if (fork() && !fork()) {
        fork();
    }
}
```

How many processes are there in total (suppose all fork() succeed)?

Answer.
9. `fork() || fork()`: 2 processes enter the if-clause, 1 the else-clause. Inside, `fork() && !fork()`: 1 process enters the inner if and forks once (→ 2), 2 processes skip it → each outer-if process becomes 4. Total $2 \times 4 + 1 = 9$ (8 newly created). ✓ (verified)

Spring 2023 Midterm（lab slides 引用）

S23-PRR Preemptive priority + RR with different arrivals (q = 2) CH5

Smaller number = higher priority; processes with equal priority share the CPU round-robin with q = 2; a higher-priority arrival preempts. Process table reconstructed from the official answer: P1 (0, 5, 2), P2 (1, 3, 1), P3 (2, 8, 2), P4 (8, 2, 1), P5 (9, 6, 2).

Answer.
`0 P1 1 P2 4 P1 5 P3 7 P1 8 P4 10 P1 11 P3 13 P5 15 P1 16 P3 18 P5 20 P3 22 P5 24` (official Gantt). A preempted process **keeps the remaining part of its quantum** — P1 runs only 1 unit at 4–5 and at 10–11. Resetting the quantum is the common mistake. Avg WT $= \dfrac{11+0+12+0+9}{5} = 6.4$.

Lab 2 Exercises

Lab2-Q1 Counting printed values CH3

```
pid_t pid = fork();  int cnt = 0;
if (pid == 0) {
    cnt += 10;  pid = fork();
    if (pid != 0) { cnt += 10;  pid = fork(); }
}
printf("%d \n", pid);  printf("%d \n", cnt);
```

How many non-zero PIDs, how many '0's and how many '10's are printed?

Answer.
4 processes print (pid, cnt): P0 (≠0, 0), C1 (≠0, 20), C2 (0, 10), C3 (0, 20) → **2** non-zero PIDs, **3** zeros, **1** ten. ✓ (verified)

Lab2-Q2 Force the order process 3 → 2 → 1

CH3

```
int x = 0;
if (BLANK1) { x = x + 1; BLANK2; printf("The value x in process 1 is %d\n", x); }
else if (BLANK3) { x = x + 1; BLANK4; printf("The value x in process 2 is %d\n", x); }
else { x = x + 1; printf("The value x in process 3 is %d\n", x); }
```

Fill the blanks so the output is always process 3, then 2, then 1 (each printing x = 1).

> **Answer.**
> BLANK1 `fork()` · BLANK2 `wait(0)` · BLANK3 `fork()` · BLANK4 `wait(0)` (`fork() > 0` / `wait(NULL)` also fine). Each process waits for the child it created, so 3 prints before 2, and 2 before 1; every process has its own copy of x, so each prints 1.

Lab 4 Exam Example

Lab4-EX1 Two ordinary pipes for two-way communication

CH3

Parent sends "Parent" to the child and the child sends "Child" to the parent using `p2c[2]` and `c2p[2]`. Fill BLANK1–BLANK6 (create pipes; child writes then reads; parent reads then writes).

> **Answer.**
> BLANK1 `pipe(p2c)` · BLANK2 `pipe(c2p)` · BLANK3 `write(c2p[1], msg, 6)` · BLANK4 `read(p2c[0], buffer, 7)` · BLANK5 `read(c2p[0], buffer, 6)` · BLANK6 `write(p2c[1], msg, 7)` (5 / 6 bytes also accepted). Output: "Parent received the message: Child" and "Child received the message: Parent".

Lab 6 Sample Questions（官方答案）

Lab6-Q1 Semaphores for strict thread order

CH6

T1: `wait(S1); CS; signal(S2);` T2: `wait(S2); CS; signal(S3);` T3: `wait(S3); CS; signal(S1);` — initial values for the order T1, T2, T3, T1, ...?

> **Answer.**
> **S1 = 1, S2 = 0, S3 = 0** (only T1 can start; each thread passes the turn to the next).

Lab6-Q2 P0 → P1 → P2 ordering blanks

CH6

S0 = BLANK1, S1 = BLANK2, S2 = BLANK3; P0: `wait(S0); CS; BLANK4;` P1: `BLANK5; CS; BLANK6;` P2: `BLANK7; CS; signal(S0);`

> **Answer.**
> S0 = **1**, S1 = **0**, S2 = **0**; BLANK4 `signal(S1)`; BLANK5 `wait(S1)`; BLANK6 `signal(S2)`; BLANK7 `wait(S2)`.

Lab6-Q3 Mutex blanks

CH6

`void *add(void *arg) { pthread_mutex_t* m = BLANK1; for (5000 times) { BLANK2; ++counter; BLANK3; } }` — two threads run `add` with `&mutex`. Missing code? Purpose? Final counter?

> **Answer.**
> BLANK1 `(pthread_mutex_t*) arg` · BLANK2 `pthread_mutex_lock(m)` · BLANK3 `pthread_mutex_unlock(m)`. Purpose: protect the shared variable `counter` (critical section). Final value **10,000**.

Lab6-Q4 Possible outputs with semaphores X, Y = 0

CH6

Process A: `wait(X); print "b"; signal(Y); wait(Y); print "c"; signal(Y);` Process B: `signal(X); print "a"; wait(Y); print "d";`

> **Answer.**
> If both complete: **abcd, bacd, bcad**. If B takes Y first, A blocks forever at its second `wait(Y)` → **bad, abd** (A never finishes).
> ✓ (verified by exhaustive search)

Lab6-Q5 Counting semaphore permits

CH6

S0 = 1, S1 = S2 = S3 = 0. P0: `while(true){ wait(S0); print '1'; release(S1); release(S2); release(S3); }` P1: `wait(S1); release(S0);` P2: `wait(S2); release(S0); wait(S3); release(S0);` — how many '1's?

Answer.
4 — S0 permits: initial 1 + P1 gives 1 + P2 gives 2. After the first print P0 releases S1, S2, S3, so P1 and P2 can produce all of them; then P0 blocks forever. ✓ (verified)

Lab6-Q6 Readers–writers with two mutexes

CH6

Fill the reader's blanks (mutex protects read_count; rw_mutex protects the data).

Answer.
Entry: `pthread_mutex_lock(&mutex); read_count++; if (read_count == 1) pthread_mutex_lock(&rw_mutex); pthread_mutex_unlock(&mutex);` Exit: `pthread_mutex_lock(&mutex); read_count--; if (read_count == 0) pthread_mutex_unlock(&rw_mutex); pthread_mutex_unlock(&mutex);` The first reader locks writers out and the last reader lets them in.

Lab6-Q7 Race condition values

CH6

a = 1, b = 3, c = 0. Thread 1: `a = b + 1; c = a - b;` Thread 2: `a = 13; b = 10; c = 2;` All possible final values of c?

Answer.
−6, 1, 2, 3, 10 — 2 if T2's `c = 2` is last; 1 (T1 reads a=4, b=3 then T2 runs; or T2 first then a = 11, c = 11 − 10); −6 (T1 reads a=4, then T2 sets b=10); 3 (T2 runs after T1's `a = 4`, T1 reads a=13, b=10); 10 (T1 reads a=13 and b=3 between T2's writes). ✓ (verified by exhaustive search)

Q

MC Answer Key MC 題庫 + 答案

歷屆 MC（S26 / F20 / Sp21 / HW1 / HW2 / Lab 4）

S26-MC1 Interrupts → C

Which of the following statements about interrupts is **false**?

A. Interrupts may be generated by hardware devices **B.** Interrupts may be generated by software exceptions or traps **C. All interrupts are serviced in exactly the same way, with no difference in priority** **D.** Interrupts are handled by the kernel

interrupt 有 **priority levels**，high-priority 可以 preempt low-priority interrupt **[3.24]**。

S26-MC2 DMA → B

Which of the following statements is **true** about DMA?

A. DMA is used to move data for slow devices, while CPU is used for fast devices **B. DMA allows data transfer between device and memory with less CPU involvement** **C.** DMA eliminates interrupts completely **D.** CPU and DMA can't execute concurrently

DMA 用於 **fast devices / large data**（A 相反）；完成後仍然 **interrupt CPU**（C 錯）；CPU 同 DMA 可以 concurrent（D 錯）**[3.25]**。

S26-MC3 Storage hierarchy → D

Which of the following statements is true about storage hierarchy?

A. Registers are slower than main memory **B.** Main memory is nonvolatile **C.** Hard-disk drives are smaller than volatile memory **D. Main memory is byte addressable**

each address refers to one byte **[1.30]**。Registers 最快；main memory volatile；HDD 大過 memory。

S26-MC4 von Neumann → B

Which of the following is a core characteristic of the von Neumann architecture?

A. Instructions and data are stored in separate physical memories **B. Instructions and data are stored in the same memory space, accessed by address** **C.** The CPU does not contain an ALU; all computations are done by an external co-processor **D.** The Program Counter (PC) is stored in external mass storage devices

stored-program architecture **[1.26]**。A 係 Harvard architecture。

S26-MC5 OS services → C

Which of the following is **false** about services of operating systems?

A. User interface can be CLI or GUI **B.** The logging service of OS should keep track of computer resources that each user uses **C. Errors only occur in memory and I/O devices** **D.** Resource allocation service should allocate resources like CPU cycles, memory, file storage

errors 可以喺 **CPU and memory, I/O devices, user programs** **[2.5]**。

S26-MC6 API → A

Which of the following is an advantage of using APIs instead of directly invoking system calls?

A. APIs provide a more convenient programming interface **B.** APIs always execute faster than system calls **C.** APIs can bypass kernel protection **D.** APIs eliminate the need for linkers and loaders

API 好處 = **portability + hide complex details（convenient）** **[2.17]**。

S26-MC7 Microkernel → B（C 亦接受）

Which is **false** about microkernel system structure?

A. It is easier to extend a microkernel operating system **B. It typically has better performance than a monolithic kernel** **C. It runs memory management in user space** **D.** It is easier to port a microkernel operating system to new architectures

microkernel **performance suffers**（message passing overhead）**[2.48]**；microkernel 保留 minimal memory management 喺 kernel，所以 C 都算錯。

S26-MC8 printf vs write → B

What's the relationship between printf() and write()?

A. Both printf() and write() are system calls provided directly by the kernel **B. printf() invokes write() system call to get service from operating system** **C.** The program enters kernel mode immediately when printf() is called **D.** write() system call invokes printf() to perform the print function

printf() 係 C library function，佢 call write() system call **[2.26]**。

S26-MC9 Layered design → B

Which of the following statements about a layered operating-system design is correct?

A. Any layer may directly access any other layer **B. The interfaces between adjacent layers are usually well defined** **C.** Layered design eliminates all performance overhead **D.** Pure layered design is the dominant structure in all modern operating systems

每層只用下層服務、interface well defined（information hiding）**[2.44–2.45]**。A 係 LKM 特色；layered 有 overhead；好少 pure layered。

S26-MC10 fork count → C

```
int main() {
   int i;
   for (i = 0; i < 3; i++) {
      if (fork() == 0) {
         fork();
         printf("X ");
         return 1;
      }
   }
   printf("Y ");
   return 0;
}
```

What is the total number of processes (including the original), the number of "X " printed, and the number of "Y " printed?

A. Total processes = 4, "X " = 3, "Y " = 1 **B.** Total processes = 4, "X " = 1, "Y " = 3 **C. Total processes = 7, "X " = 6, "Y " = 1** **D.** Total processes = 7, "X " = 1, "Y " = 6

只有 original 繼續 loop；每輪生 1 個 child，child 再 fork → 2 個印 X 然後 return。3 輪 × 2 = **6 X**；original 印 **1 Y**；total = 1 + 6 = **7**（已用 C 程式驗證）。

S26-MC11 Context switch → B

Which statement is true about context switch?

A. Context switch means switching from user mode to kernel mode **B. During context switch, OS saves the state of one process/thread and restores another** **C.** Context switch requires the same hardware on all CPU architectures **D.** Context switch is shorter if the OS is more complex

[3.20]。A 係 mode switch；越複雜 OS/PCB → context switch **越長**。

S26-MC12 Ordinary pipes → B

Which statement about ordinary pipes is **false**?

A. They can be accessed like file descriptors **B. They can be accessed by arbitrary processes** **C.** They disappear after the communicating processes terminate **D.** They are typically used between parent and child processes

ordinary pipe cannot be accessed from outside the process that creates it（只限 parent–child）**[3.65–3.66]**。

S26-MC13 Zombie → B

Which statement about zombie processes is correct?

A. A zombie process is still executing user code normally **B. A zombie process has terminated but its parent has not yet collected its status** **C.** A zombie process is created when exec() fails **D.** A zombie process owns no process identifier

[3.49] terminated 但 parent 未 call wait()；**PID、process-table entry、PCB 仍然存在**。

S26-MC14 Leaving running state → A

Which event typically causes a process to leave the running state?

A. It issues an I/O request **B.** It calls a function **C.** It increases a local variable **D.** It reads from cache

I/O request → running → **waiting** **[3.9]**。

S26-MC15 Multithreaded process

→ C

Which of the following is true about the multithreaded process?

A. Each thread has its own address space and file descriptors **B.** Threads can't run concurrently on multiple cores if they belong to the same process **C. Thread creation is usually faster than process creation** **D.** It can't take advantage of processing capabilities on multicore systems

Economy — thread creation cheaper **[4.4, 4.7]**；threads share address space & files。

S26-MC16 Concurrency vs parallelism

→ C

Which statement is correct about concurrency and parallelism in multicore systems?

A. Concurrency means a system can perform more than one task at exactly the same time **B.** Parallelism means tasks only make progress over time on a single processor core **C. Concurrency can be supported on a single processor core through time multiplexing by the scheduler** **D.** Testing and debugging become easier in multicore systems because execution paths are fixed

[4.9]。A 係 parallelism 定義；debugging **變難**。

S26-MC17 Amdahl's Law

→ C

Which combination gives a speedup between 1.6 and 1.7?

A. parallel 81%, 3 cores **B.** parallel 81%, 5 cores **C. parallel 61%, 3 cores** **D.** parallel 61%, 5 cores

$\text{speedup} = \frac{1}{S + (1 - S)/N}$。A：$\frac{1}{0.19 + 0.27} = 2.17$；B：2.84；**C**：$\frac{1}{0.39 + 0.2033} = 1.69$；D：1.95。

S26-MC18 Signals

→ C

Which statement about signal handling is **false**?

A. Signals can be received synchronously or asynchronously **B.** Users can define custom signal handlers for certain signals **C. All signals should be delivered to all threads in a process** **D.** Different signals can have different handlers in the same process

synchronous signals 只 deliver 畀 **causing thread**；只有部分 asynchronous（e.g. Ctrl-C terminate）先畀所有 threads **[4.30–4.31]**。

S26-MC19 Scheduling criteria

→ C

Which statement about scheduling criteria is correct?

A. Waiting time is the amount of time from process arrival until the process finishes execution **B.** Turnaround time is the amount of time a process spends waiting in the ready queue **C. Response time is the amount of time from when a request is submitted until the first response is produced** **D.** Throughput is the fraction of time the CPU is busy

A 係 turnaround；B 係 waiting；D 係 CPU utilization **[5.8, 5.12]**。

S26-MC20 MLFQ

→ B

Which statement about MLFQ is correct?

A. Processes never move between queues **B. Each queue can use a different scheduling algorithm** **C.** It requires burst times to be known exactly in advance **D.** It is a non-preemptive algorithm

[5.30–5.31]。MLFQ 可以搬 queue、唔需要知 burst、preemptive。

S26-MC21 EDF

→ B

Which scheduling algorithm assigns priorities according to deadlines?

A. RM **B. EDF** **C.** FCFS **D.** RR

EDF = dynamic priority by deadline；RM = static by period **[5.51]**。

S26-MC22 Multiprocessor scheduling

→ C

Which statement about multiprocessor scheduling is **false**?

A. For asymmetric multiprocessing, the master server can become a potential bottleneck **B.** In a multithreaded system, another thread may run when one thread stalls while waiting for memory **C. The motivation of placing multiple processor cores on the same physical chip is to simplify the scheduler** **D.** Processor affinity tries to keep a process on the same CPU to improve cache performance

multicore 係為咗 **faster + less power**，而且令 scheduling **更複雜** **[5.40]**。

S26-MC23 Mutex + semaphore output

→ A

```
int counter = 0; pthread_mutex_t mtx; sem_t sem;   /*
sem_init(&sem, 0, 0) */
void *worker1(void *arg) { pthread_mutex_lock(&mtx); counter
= counter + 2;
    printf("A %d;", counter); pthread_mutex_unlock(&mtx);
sem_post(&sem); return NULL; }
void *worker2(void *arg) { sem_wait(&sem);
pthread_mutex_lock(&mtx); counter = counter + 3;
    printf("B %d;", counter); pthread_mutex_unlock(&mtx);
return NULL; }
/* main creates t1 (worker1) and t2 (worker2), then joins
both */
```

What is the output?

A. A 2; B 5; **B.** B 3; A 5; **C.** A 3; B 5; **D.** B 2; A 4;

sem 初始 0 → worker2 一定等 worker1 `sem_post` → 先 A（0+2=2）後 B（2+3=5）。

S26-MC24 Hardware primitive

→ B

Which hardware primitive is commonly used to build synchronization tools?

A. exec() **B. test_and_set()** **C.** fork() **D.** open()

atomic hardware instructions：**test_and_set()**、**compare_and_swap()** **[6.14]**。

S26-MC25 Bounded-waiting TAS

→ A

In the bounded-waiting mutual exclusion algorithm using test_and_set() (waiting[], key, lock; exit section scans j = (i+1)%n ... and either sets lock = false or waiting[j] = false), which statement is **not correct**?

A. test_and_set(&lock) releases the lock after the critical section **B.** waiting[i] is used to indicate that process i wants to enter the critical section **C.** waiting[j] = false indicates that process i releases the lock (hands it to j) **D.** Process i can re-enter the critical section after executing lock = false

test_and_set 只會 **set lock = true（acquire）**；release 係 `lock = false` 或者 `waiting[j] = false` **[6.19]**。

F20-MC1 Locality

→ B

Locality is essential to ___

A. Memory **B. Cache** **C.** Hard disks **D.** Non-volatile memory (NVM)

caching works because of **temporal/spatial locality** **[1.33]**。

F20-MC2 Virtualization

→ D

___ allows a guest OS to run as an application on a native OS.

A. Operating system **B.** Emulation **C.** User interface **D. Virtualization**

[1.43] "allows an OS to run as an application within another OS"。

F20-MC3 Interrupt

→ C

Which of the following statements about interrupts is **NOT** true?

A. Interrupts enable hardware to interact with the operating system **B.** Interrupt alerts the CPU that some event requires attention **C. Interrupt allows software to access hardware including the CPU** **D.** Interrupt can be caused by both hardware and software

C 講緊 **system call** 嘅功能。

F20-MC4 API

→ C

Programmers interface with ___ to obtain services from the OS.

A. Shell **B.** System call interfaces **C. APIs (application programming interfaces)** **D.** System calls

programmers 用 **API**（best answer） **[2.15]**。

F20-MC5 Parameter passing

→ D

How can parameters be passed to the OS in system calls?

A. Registers **B.** Stack **C.** Block or table **D. All of the above**

三種方法 **[2.20]**。

F20-MC6 LKM

→ B

___ refers to the OS design that maintains a set of core components in the kernel and adds additional services via modules during run time.

A. Layered approach **B. Loadable kernel modules (LKMs)** **C.** Microkernel **D.** Monolithic

[2.49]。

F20-MC7 fork **→ A**

Which of the following statements on fork() is **NOT** true?

A. The parent process waits until the completion of the child process **B.** The child process duplicates the entire address space of the parent process **C.** The parent and child processes run the same program **D.** The parent and child processes run concurrently in the system

等 child 係 **wait()** 嘅工作，fork 本身唔會等。

F20-MC8 Address space **→ D**

Which of the following is contained in the address space of a process?

A. Stack **B.** Code **C.** Data **D. All of the above**

text (code)、data、heap、stack [3.5]。

F20-MC9 Named pipes **→ C**

___ enables communication for multiple processes with no parent-child relationship, independent from network.

A. RPCs **B.** Ordinary pipes **C. Named pipes** **D.** Socket

named pipes 唔需要 parent-child、多 processes 用 [3.69]；socket/RPC 要 network。

F20-MC10 Medium-term scheduler **→ D**

___ swaps a process out of memory and temporarily stores it on disk.

A. The long-term scheduler **B.** A device scheduler **C.** The CPU scheduler **D. The medium-term scheduler**

swapping [3.18]。

F20-MC11 Not shared by threads **→ B**

Which item below is **NOT** shared by threads of the same process?

A. Code **B. Stack** **C.** Files **D.** Data

每個 thread 有自己 **stack, registers, PC** [4.6]。

F20-MC12 Thread library **→ A**

___ provides the programmer with an API for creating and managing threads.

A. Thread library (such as POSIX Pthreads) **B.** Signals **C.** System calls **D.** Inter-process communications

[4.20]。

F20-MC13 Multithreading models **→ C**

Which of the following statements is **NOT** true?

A. Kernel threads are scheduled by the OS **B.** Programmers or users manage user threads **C. A user thread can be mapped to more than one kernel thread** **D.** One user thread can be mapped to at most one kernel thread

冇 one-to-many mapping；many-to-many 係 many user → **smaller or equal** kernel threads。

F20-MC14 Minimum average waiting time **→ C（D 亦接受）**

Given a set of processes, which algorithm produces the minimum average waiting time?

A. FCFS **B.** RR **C. SJF** **D. SRTF**

SJF optimal among non-preemptive；SRTF optimal among preemptive [5.17, 5.24]。

F20-MC15 Hard to implement **→ A**

Which of the following scheduling algorithms is difficult to implement in practice?

A. SJF **B.** RR **C.** MLFQ **D.** FCFS

好難知道 **next CPU burst length** [5.17]。

Sp21-MC1 von Neumann **→ A**

In a von Neumann architecture, a processing unit contains a/an ___ and processor registers.

A. arithmetic logic unit (ALU) **B.** instruction register (IR) **C.** program counter (PC) **D.** all of the mentioned

[1.26] "a CPU that contains an ALU and processor registers"；IR 屬 control unit。

Sp21-MC2 Interrupt trigger **→ D**

Which of the following events will directly trigger an interrupt?

A. kernel code execution **B.** I/O job execution **C.** user code execution **D. I/O job completion**

device controller 完成操作 → **interrupt** CPU [3.21]。

Sp21-MC3 Dual mode **→ C**

The two separate modes of operation in an operating system are ___.

A. supervisor mode and system mode **B.** physical mode and logical mode **C. user mode and kernel mode** **D.** kernel mode and privileged mode

[3.27]。

Sp21-MC4 System calls **→ C**

___ provide an interface to the services provided by an operating system.

A. Inter-process communications **B.** Remote procedure calls **C. System calls** **D.** User interfaces

[2.14]。

Sp21-MC5 Original UNIX **→ C**

Which operating system structure is adopted by the original UNIX?

A. layered structure **B.** microkernel **C. monolithic structure** **D.** modular approach

[2.40–2.41]。

Sp21-MC6 Degree of multiprogramming **→ B**

The ___ refers to the number of processes in the memory.

A. program counter **B. degree of multi-programming** **C.** long-term scheduler **D.** CPU scheduler

[3.14]。

Sp21-MC7 Amdahl **→ B**

Using Amdahl's Law, what is the speedup for an application that is 30% serial on 7 processing cores?

A. 2 **B. 2.5** **C.** 3 **D.** 3.5

$\frac{1}{0.3+0.7/7} = \frac{1}{0.4} = 2.5$。

Sp21-MC8 Process owns **→ D**

In an operating system, each process has its own ___.

A. address space **B.** local/global variables and open files **C.** pending alarms, signals and signal handlers **D. all of the above**

process = address space + resources。

Sp21-MC9 Stack **→ D**

Which section of a process contains temporary data like function parameters, return addresses and local variables?

A. text section **B.** data section **C.** heap **D. stack**

[3.5]。

Sp21-MC10 Short-term scheduler **→ A**

Which of the following selects from among the processes in the ready queue to execute on the CPU?

A. Short-term scheduler **B.** Medium-term scheduler **C.** Long-term scheduler **D.** Context switcher

= CPU scheduler [3.17]。

Sp21-MC11 fork incorrect **→ B**

During process creation with fork(), which of the following is **incorrect**?

A. The child process can run concurrently with the parent **B. The child process can call wait() in order to finish earlier than the parent** **C.** The child process can call exec() to load a new program into it **D.** The child can be an exact duplicate of the parent

wait() 係等 **child** 完成，唔係令自己早完。

Sp21-MC12 Pipes **→ C**

Which of the following statements about pipes is **not** true?

A. Named pipes allow bi-directional communication **B.** Only the parent and child processes can use ordinary pipes for communication **C. Named pipes cease to exist after the communicating processes have finished** **D.** Reading and writing to ordinary pipes on both UNIX and Windows can be performed like ordinary file I/O

named pipes **continue to exist until explicitly deleted** [3.69]。

Sp21-MC13 Leave the CPU **→ D**

Which of the following events might force a process to leave the CPU?

A. make a fork system call **B.** make an I/O request **C.** interrupt **D. all of the above**

system call / I/O / interrupt 都可以令 process 離開 CPU。

Sp21-MC14 fork with ||

→ C

```
if (fork() || fork()) {
   fork();
}
```

How many processes are created in total (including the original)?

A. 3 **B.** 4 **C. 5** **D.** 6

P0（true，跳過第二個 fork）→ fork → 2；C1（false）→ fork2：C1 true → fork → 2；C2 false → 1。**2 + 2 + 1 = 5** ✓

Sp21-MC15 fork in loop

→ C

```
for (int i = 0; i < 3; ++i) {
   if (fork() == 0) fork();
}
```

How many processes in total?

A. 3 **B.** 8 **C. 27** **D.** 64

每輪每個 process 變 3 個 → $3^3 = 27$ ✓

Sp21-MC16 TCB

→ D

Which of the following is **not** in the Thread Control Block (TCB)?

A. Execution state **B.** Scheduling information **C.** Pointer to enclosing process **D. Process identifier**

TCB 有 execution state、scheduling info、accounting、pointers、**pointer to enclosing PCB** [4.15]；PID 喺 PCB。

Sp21-MC17 Concurrency

→ B

Which refers to the capability to allow multiple tasks to make progress on a single processor?

A. parallelism **B. concurrency** **C.** data parallelism **D.** task parallelism

[4.9]。

Sp21-MC18 Multithreading models

→ C

Which of the following is **not** true about multithreading models?

A. Many-to-one means many user-level threads are mapped to a single kernel thread **B.** One-to-one model provides more concurrency than many-to-one model **C. The many-to-many model is easy to implement in practice** **D.** Most operating systems now use the one-to-one model

many-to-many **difficult to implement** [4.26]。

Sp21-MC19 Thread-local storage

→ D

Thread-local storage is data that ____.

A. is not associated with any process **B.** has been modified by the thread, but not yet updated to the parent process **C.** is generated by the thread independent of the thread's process **D. is unique to each thread**

TLS = **unique to each thread**（Windows TEB 入面有 TLS [4.36]）。

Sp21-MC20 When scheduling occurs

→ D

CPU scheduling decisions may take place when ____.

A. a process switches from running to waiting state **B.** a process terminates **C.** a process switches from running to ready state **D. all of the above**

四種情況 [5.6]。

Sp21-MC21 Priority problem

→ B

The main problem with a priority scheduling algorithm is ____.

A. how to set the priority of a process **B. some processes may never execute** **C.** how to determine the length of the next CPU burst **D.** how to determine the length of the time quantum

starvation；solution = aging [5.25]。

Sp21-MC22 RR large quantum

→ B

For RR scheduling, when the time quantum q gets too large, it degenerates to ____.

A. SJF **B. FCFS** **C.** Shortest-remaining-time-first **D.** Multilevel queue

[5.14, 5.16]。

Sp21-MC23 Must be non-preemptive

→ A

Which of the following scheduling algorithms must be non-preemptive?

A. FCFS **B.** SJF **C.** Priority scheduling **D.** Multi-level feedback queue

FCFS 一定 non-preemptive；SJF / priority 有 preemptive 版本；MLFQ preemptive [5.11]。

Sp21-MC24 MLFQ true statement

→ C

In the multilevel feedback queue algorithm, which statement is true?

A. Prior knowledge on the next CPU burst time is required for scheduling **B.** Classification of the ready queue is permanent **C. It handles interactive jobs well by delivering similar performance as SJF** **D.** It won't suffer from starvation problem

[5.35]。B 係 multilevel queue（冇 feedback）；MLFQ 都可能 starvation（要 reshuffle/aging）。

Sp21-MC25 Real-time

→ C

Which statement about real-time CPU scheduling is true?

A. Real-time scheduling doesn't demand performance guarantee **B.** In hard real-time systems, a task can be serviced after its deadline **C. Soft real-time systems guarantee only that the process will be prioritized over non-critical processes** **D.** The scheduler for a real-time system must support a priority-based algorithm without preemption

[5.47]。RT scheduler 要 **priority-based with preemption**。

Sp21-MC26 Starvation-free

→ A

Which algorithms are free from starvation? 1. FCFS 2. Non-preemptive SJF 3. Preemptive SJF 4. Round Robin 5. Multi-level Queue

A. 1 4 **B.** 1 2 4 **C.** 1 3 4 **D.** 1 2 4 5

SJF/SRTF 長 job 可能 starve；multilevel queue（fixed priority）低 queue 可能 starve；**FCFS**、**RR** 唔會。

Sp21-MC27 Mutual exclusion

→ A

Mutual exclusion implies that ____.

A. if a process is executing in its critical section, then no other process can be executing in their critical sections **B.** if a process is executing in its critical section, then only one of the other processes can be executing in its critical section **C.** if a process is executing in its critical section, then all the resources of the system must be blocked until it finishes execution **D.** none of the above

[6.11]。

Sp21-MC28 CS requirements

→ C

A solution to the critical section problem does **not** have to satisfy which requirement?

A. mutual exclusion **B.** progress **C. synchronization** **D.** bounded waiting

三個 requirements = ME + progress + bounded waiting。

Sp21-MC29 Race condition

→ B

A situation where several processes access and manipulate the same data concurrently and the outcome depends on the particular order of access is called ____.

A. data consistency **B. race condition** **C.** aging **D.** starvation

[6.9]。

Sp21-MC30 Entry section

→ C

Suppose the binary variable lock is initialized to 0. Which can be an implementation of the entry section to solve the critical-section problem?

A. while (compare_and_swap(&lock, 0, 1) == 0) ; **B.** while (compare_and_swap(&lock, 1, 0) != 0) ; **C. while (test_and_set(&lock)) ;** **D.** while (!test_and_set(&lock)) ;

TAS 返回 old value；lock = 0 時第一個 process 得 0 → 跳出 loop 入 CS [6.16]。A 嘅條件相反（應該 `!= 0`）。

HW1-MC1 DMA

→ C

Which statement about I/O operations and DMA is **NOT** true?

A. In programmed I/O, the CPU executes special I/O instructions to move data between a device controller and memory **B.** A DMA controller can transfer a block of data between an I/O device and memory without continuous CPU involvement **C. During a DMA transfer, the DMA controller sends an interrupt to the CPU after every byte is transferred** **D.** After a DMA block transfer completes, the DMA controller can interrupt the CPU

DMA **block 完成先 interrupt 一次** [3.25–3.26]。

HW1-MC2 Interrupts and traps

→ B

Which statement about interrupts and traps is **NOT** true?

A. Hardware interrupts are generally asynchronous to the instruction currently being executed **B. Interrupts can only be generated by I/O devices** **C.** A trap or exception may be caused by an error or by a user request such as a system call **D.** Interrupts may have different priority levels

trap/exception = **software-generated interrupt** [3.24]。

HW1-MC3 Resource allocator

→ C

Several jobs are ready to run on a shared computer. The OS decides which job receives CPU time and how memory is allocated. Which role does this illustrate?

A. The OS provides a user interface **B.** The OS provides a communication service **C. The OS acts as a resource allocator** **D.** The OS acts as an application program

[1.23] ◦

HW1-MC4 NUMA

→ D

Which statement about multiprocessor, multicore, and NUMA systems is **NOT** true?

A. In SMP, processors have their own registers and local caches but share physical memory **B.** A multicore system places multiple computing cores on a single physical chip **C.** In NUMA, a CPU can access its local memory faster than remote memory connected to another CPU group **D. In NUMA, every CPU accesses every part of memory with exactly the same latency**

NUMA = **non-uniform** latency [1.40] ◦

HW1-MC5 OS structures

→ B

Which statement about operating-system structures is **true**?

A. A monolithic kernel moves most OS services into user-space processes that communicate by message passing **B. A microkernel keeps only essential functions in the kernel and provides other services outside the kernel through communication mechanisms such as message passing** **C.** In a layered OS, each layer may directly call any other layer without restriction **D.** A loadable kernel module always requires the entire kernel to be recompiled and the machine to be rebooted

[2.46–2.47] ◦ A 描述 microkernel 唔係 monolithic；C 錯；LKM 唔使 recompile/reboot ◦

HW1-MC6 read() path

→ C

A C program calls read() to obtain data from a file. Which sequence best describes how the request reaches the OS?

A. The application directly executes the device's privileged I/O instructions while remaining in user mode **B.** The read() API completes the entire file operation in user mode without invoking the kernel **C. The API and run-time environment use the system-call interface to invoke the corresponding kernel service, and control later returns to the application** **D.** The OS creates a new process to execute every read() request

[2.18–2.19] ◦

HW1-MC7 Privileged instruction

→ A

What normally happens if a user program attempts to execute a privileged instruction in user mode?

A. The hardware treats the instruction as illegal and traps to the operating system **B.** The CPU automatically changes the instruction into a non-privileged instruction **C.** The user program changes the mode bit by itself and continues in kernel mode **D.** The instruction is always executed, but its result is discarded

[3.28] ◦

HW1-MC8 Virtualization

→ B

Which statement about virtualization is **NOT** true?

A. A virtual machine manager creates and manages virtual machines **B. An application always runs faster inside a virtual machine than directly on the physical machine** **C.** Each guest OS runs in its own VM and behaves as if it has access to hardware **D.** Virtualization allows multiple operating systems to run on one physical machine

VM 只會 **minor performance decrease** [1.46] ◦

HW1-MC9 fork

→ C

Which statement about fork() is **NOT** true?

A. After a successful fork(), the child receives 0 and the parent receives the child's PID **B.** The parent and child both continue execution at the instruction following fork() **C. If the child changes a global variable after fork(), the parent's copy changes automatically** **D.** The execution order of parent and child depends on scheduling unless synchronization is used

各自有 **own copy** ◦

HW1-MC10 Pipes

→ D

Which statement about pipes is **NOT** true?

A. An ordinary pipe is typically used between processes with a parent-child relationship **B.** A named pipe can be used by processes that do not have a parent-child relationship **C.** An ordinary pipe is typically unidirectional **D. A named pipe ceases to exist as soon as the communicating processes terminate**

named pipe 要 **explicitly deleted** ◦

HW2-MC1 Responsiveness

→ B

An interactive application continues responding to the user while one of its threads is blocked. Which benefit does this illustrate?

A. Economy **B. Responsiveness** **C.** Resource sharing **D.** Scalability

[4.7] ◦

HW2-MC2 Data parallelism

→ D

Which arrangement is an example of data parallelism?

A. Threads on different cores each perform a different operation **B.** One thread processes all elements of an array sequentially **C.** One thread handles user input while another thread performs spelling checks **D. Threads on different cores perform the same operation on different subsets of an array**

A、C = task parallelism；B = serial [4.11] ◦

HW2-MC3 Signal classification

→ A

Which statement correctly classifies signals?

A. Division by zero can generate a synchronous signal, whereas timer expiration can generate an asynchronous signal **B.** Division by zero can generate an asynchronous signal, whereas timer expiration can generate a synchronous signal **C.** Both generate synchronous signals **D.** Both generate asynchronous signals

[4.29] ◦

HW2-MC4 Deferred cancellation

→ C

Which statement describes deferred thread cancellation?

A. The target thread is terminated immediately when cancellation is requested **B.** Requesting cancellation terminates every thread in its process **C. The target thread periodically checks whether it should terminate, allowing it to terminate in an orderly manner** **D.** The target thread can be canceled only after it has finished its normal execution

A 係 asynchronous cancellation [4.32] ◦

HW2-MC5 Preemptive scheduling

→ C

Which event is an example of preemptive CPU scheduling?

A. A running process completes its execution and terminates **B.** A running process requests I/O and enters the waiting state **C. A time quantum expires, and the running process is moved back to the ready queue** **D.** A running process calls wait() and enters the waiting state

running → ready（involuntary）= preemptive [5.6] ◦

HW2-MC6 Convoy effect

→ B

Which situation illustrates the convoy effect in FCFS?

A. A waiting process gradually gains a higher priority **B. Several short processes wait behind a long process that arrived first** **C.** A process moves to a lower-priority queue after using its time quantum **D.** A running process is preempted when its RR quantum expires

[5.10] ◦ A = aging；C = MLFQ demotion ◦

HW2-MC7 Aging

→ A

What is the purpose of aging in priority scheduling?

A. To address starvation by gradually increasing the priority of a waiting process **B.** To keep each process permanently in the same priority queue **C.** To predict the length of a process's next CPU burst **D.** To give higher priority to periodic tasks with shorter periods

[5.25] ◦ D = RM ◦

HW2-MC8 PCS vs SCS

→ D

In a many-to-many model, a thread library selects a user thread to run on an available LWP. Which scheduling scope applies?

A. SCS, because several user threads are competing **B.** SCS, because the library schedules kernel threads onto CPUs **C.** PCS, because all kernel threads in the system are competing **D. PCS, because the competing threads belong to the same process**

[5.37] ◦

HW2-MC9 Exponential averaging → C

The predicted CPU burst is 10 ms, the actual burst is 6 ms, $\alpha = 0.25$. What is the next prediction?

A. 6 ms **B.** 7 ms **C. 9 ms** **D.** 10 ms

$\tau_{n+1} = 0.25 \times 6 + 0.75 \times 10 = \mathbf{9}$。

HW2-MC10 Little's formula → B

3 processes arrive per second on average; average queue length is 12. Average waiting time?

A. 0.25 s **B. 4 s** **C.** 9 s **D.** 36 s

$W = n/\lambda = 12/3 =$ **4 s** **[5.57]**。

Lab4-MC Threads share memory, processes do not → B

```
void *child(void *arg) { int *counter = (int *) arg;
(*counter)++; return NULL; }
int main() {
    int counter = 0;  pthread_t thread;
    pthread_create(&thread, NULL, child, (void *)&counter);
    pthread_join(thread, NULL);
    printf("%d\n", counter);
    counter = 0;
    pid_t pid = fork();
    if (pid) wait(NULL);
    else { child((void *)&counter); exit(0); }
    printf("%d\n", counter);
}
```

What is the output?

A. 0 then 0 **B. 1 then 0** **C.** 0 then 1 **D.** 1 then 1

thread 同 main 共用 `counter` → 印 1；fork 出嚟嘅 child 改嘅係自己嗰份 copy → parent 印 0。**Threads share memory space, processes have separate memory spaces.**（Lab 4）

新練習 MC（針對 Fall 2026 slides）

P-ch1-1 Storage managed by → A

According to the storage characteristics table, which level of storage is managed by the **compiler**?

A. Registers **B.** Cache **C.** Main memory **D.** Solid-state disk

registers – compiler；cache – hardware；main memory / SSD / disk – OS **[1.34]**。

P-ch1-2 Bootstrap → C

Where is the bootstrap program (the first program run at power-on) typically stored?

A. DRAM **B.** Cache **C. EEPROM** **D.** Hard disk

[1.30] bootstrap program stored on EEPROM（firmware）。

P-ch1-3 Multiprocessor speedup → B

Why is the speed-up ratio with N processors less than N?

A. Each processor runs a different instruction set **B. Overhead, e.g., contention for shared resources such as the bus or memory** **C.** Only one processor can execute user processes **D.** Processors must share a single register set

[1.37]。

P-ch1-4 Multiprocessor advantages → D

Which is **NOT** an advantage of multiprocessor systems listed in the lecture?

A. Increased throughput **B.** Economy of scale **C.** Increased reliability (graceful degradation) **D. Simpler parallel programming**

multiprocessor 令 parallel programming **更複雜** [1.37, summary]。

P-ch1-5 NUMA drawback → B

What is the potential drawback of a NUMA system?

A. All CPUs must share a single slow bus **B. Increased latency when a CPU must access remote memory across the system interconnect** **C.** CPUs cannot share one physical address space **D.** It consumes more power than multicore chips

[1.40]。

P-ch1-6 Cloud models → B

A database server provided as a ready-to-use software stack over the Internet is an example of:

A. SaaS **B. PaaS** **C.** IaaS **D.** MaaS

SaaS = applications；**PaaS = software stack (e.g. database server)**；IaaS = servers/storage **[1.48]**。

P-ch1-7 VMM → B

Which statement about virtualization components is correct?

A. The guest OS creates and manages the virtual machine manager **B. The VMM (hypervisor) creates and manages VMs, providing an interface identical to the host** **C.** The host is the virtual copy given to each guest **D.** Programs in a VM usually run several times faster than natively

[1.44]。

P-ch1-8 Timesharing → B

Which statement is correct?

A. Multiprogramming is an extension of timesharing **B. Timesharing switches jobs so frequently that users can interact with each job while it runs** **C.** Timesharing requires that only one program be in memory **D.** Multiprogramming is not used in modern systems

timesharing = logical extension of multiprogramming **[1.42]**。

P-ch1-9 Mechanical vs electrical storage → B

Compared with mechanical storage (e.g., HDD), electrical storage (NVM) is generally:

A. Larger and cheaper per byte **B. More costly, smaller, more reliable and faster** **C.** Slower but more reliable **D.** Volatile

[1.31]。

P-ch1-10 AMAT → C

A cache has hit ratio 95%, hit time 1 ns and miss time 100 ns. The average access time is:

A. 1.0 ns **B.** 5.0 ns **C. 5.95 ns** **D.** 100 ns

$0.95 \times 1 + 0.05 \times 100 = 0.95 + 5 =$ **5.95 ns**。

P-ch2-1 ioctl → C

In UNIX, `ioctl()` belongs to which category of system calls?

A. Process control **B.** File management **C. Device management** **D.** Protection

[2.25] device management：ioctl()、read()、write()。

P-ch2-2 chmod → C

`chmod()`, `umask()` and `chown()` are examples of which category of system calls?

A. Information maintenance **B.** Communications **C. Protection** **D.** File management

[2.25]。

P-ch2-3 Parameter passing limit → B

Which parameter-passing methods do **not** limit the number or length of parameters?

A. Registers only **B. Block (table) and stack methods** **C.** Registers and stack **D.** None of them

[2.20]。

P-ch2-4 Policy vs mechanism → B

Deciding how long the timer should be set for a particular user is an example of:

A. A mechanism **B. A policy** **C.** A system call **D.** A system program

timer 本身 = mechanism；設幾耐 = policy **[2.34]**。

P-ch2-5 Shell → B

What is the primary functionality of a shell?

A. Schedule kernel threads **B. Fetch a command from the user, interpret it and execute it** **C.** Manage virtual memory **D.** Load device drivers

[2.8]。

P-ch2-6 DLL **→ B**

What is a benefit of dynamically linked libraries (DLLs)?

A. They are linked statically into every executable **B. They are loaded when needed and shared by all programs using the same version** **C.** They remove the need for a loader **D.** They make executables larger

[2.31]。

P-ch2-7 MS-DOS **→ B**

Which statement about MS-DOS is correct?

A. It is a multitasking system with dual mode **B. Application programs could access I/O directly, and it had no dual mode or hardware protection** **C.** It is a microkernel design **D.** It creates a new process for every program

[2.36, 2.39]。

P-ch2-8 Microkernel services **→ B**

A microkernel typically provides:

A. File systems and device drivers only **B. Minimal process and memory management, plus a communication facility** **C.** The GUI and shells **D.** All OS services in kernel mode

[2.46]。

P-ch2-9 Darwin **→ B**

Darwin, the kernel environment of macOS/iOS, consists of:

A. Linux kernel and glibc **B. Mach microkernel and BSD UNIX kernel** **C.** Windows NT kernel and personalities **D.** ART and JNI

[2.54, 2.57] （hybrid，XNU kernel）。

P-ch2-10 Android **→ B**

Which statement about Android is **false**?

A. Java applications execute on the Android RunTime (ART) **B. Programs written using JNI are generally portable across hardware** **C.** Android uses the Bionic C library instead of glibc **D.** The HAL abstracts hardware to provide a consistent view

JNI programs **generally not portable** **[2.58]**。

P-ch2-11 Windows structure **→ B**

Windows is described as:

A. Pure microkernel **B. Largely monolithic with microkernel-like subsystems (personalities) and loadable modules** **C.** Pure layered **D.** Simple structure like MS-DOS

[2.52]。

P-ch2-12 High-level language **→ B**

Which is a main advantage of implementing an OS in a high-level language?

A. Always faster than assembly **B. Easier to port to other hardware** **C.** Less storage needed **D.** No compiler needed

[2.35]；缺點 = reduced speed、more storage。

P-ch3-1 Default file descriptors **→ A**

On UNIX, each process by default has three open file descriptors. Which are they?

A. stdin (0), stdout (1), stderr (2) **B.** stdin (1), stdout (2), stderr (3) **C.** pipe read end, pipe write end, stderr **D.** stdin, stdout, swap

[3.7]。

P-ch3-2 Linux PCB **→ B**

In Linux, the PCB is represented by:

A. struct pcb in a hash table **B. struct task_struct elements in a circular doubly linked list (task list)** **C.** TCB in a binary tree **D.** ETHREAD

[3.13]。

P-ch3-3 Long-term scheduler **→ B**

Which statement about the long-term scheduler is correct?

A. It is invoked every few milliseconds **B. It controls the degree of multiprogramming and is mainly used in mainframes, not PCs** **C.** It swaps processes out to disk **D.** It selects the next process from the ready queue

[3.17]。C = medium-term；D = short-term。

P-ch3-4 Register sets **→ B**

Some processors (e.g., SUN UltraSPARC) provide multiple register sets. What is the effect on context switch?

A. Context switch becomes impossible **B. A context switch may just change the pointer to a different register set** **C.** The PCB is no longer needed **D.** Context switch time increases

[3.20]。

P-ch3-5 Copy-on-write **→ B**

In Linux, fork() uses copy-on-write. What does this achieve?

A. Parent and child never share any pages **B. Copying of the address space is delayed until a page is written, speeding up process creation** **C.** The child always runs exec() first **D.** All pages are copied immediately

[3.46]。

P-ch3-6 Mode bit **→ B**

The hardware mode bit is ___ in user mode and ___ in kernel mode.

A. 0, 1 **B. 1, 0** **C.** 1, 1 **D.** 0, 0

[3.28]。

P-ch3-7 Protection rings **→ B**

On Intel processors with four protection rings:

A. Ring 3 is kernel mode and ring 0 is user mode **B. Ring 0 is kernel mode and ring 3 is user mode** **C.** All four rings are heavily used **D.** Ring 1 is for the VMM only

[3.29]；rings 1、2 rarely used。

P-ch3-8 Trap **→ C**

A segmentation fault (protection violation) causes which type of mode transfer?

A. System call **B.** Interrupt **C. Trap or exception** **D.** Context switch only

trap/exception = **internal synchronous event** **[3.31]**。

P-ch3-9 Bounded buffer **→ B**

The shared-memory bounded-buffer solution with `in` and `out` and no counter can hold at most:

A. BUFFER_SIZE items **B. BUFFER_SIZE − 1 items** **C.** BUFFER_SIZE + 1 items **D.** Unlimited items

[3.54]。

P-ch3-10 Direct communication **→ B**

Which is a property of direct communication in message passing?

A. Messages go through mailboxes **B. A link is associated with exactly one pair of processes** **C.** A link may be associated with more than two processes **D.** It provides better modularity than indirect communication

[3.59]；缺點 = limited modularity。

P-ch3-11 Zero capacity **→ C**

With a zero-capacity message queue:

A. The sender never waits **B.** The sender waits only if the queue is full **C. The sender must wait for the receiver (rendezvous)** **D.** Messages are lost

[3.64]。

P-ch3-12 UNIX FIFO **→ B**

Which statement about named pipes (FIFOs) in UNIX is correct?

A. They support full-duplex transmission over networks **B. They are created with mkfifo(), appear as files, support half-duplex, and processes must be on the same machine** **C.** They disappear when the creating process exits **D.** They require a parent-child relationship

[3.69]；Windows named pipes 先係 full-duplex、可跨機。

P-ch3-13 Ports **→ B**

Which statement about client–server sockets is correct?

A. Servers initiate the connection to clients **B. All ports below 1024 are well known; clients pick a port above 1024** **C.** HTTP uses port 22 **D.** A socket is identified by the MAC address only

[3.71]；HTTP 80、SSH 22、FTP 21。

P-ch3-14 Orphan **→ B**

When a parent terminates without calling wait(), its child process:

A. Is immediately deleted by the kernel in all OSes **B. Becomes an orphan and is re-parented to init/systemd, which periodically calls wait()** **C.** Becomes a zombie forever **D.** Takes over the parent's PID

[3.50]。

P-ch3-15 exec **→ C**

Which is true about a successful exec() call?

A. It returns the PID of the new program **B.** It returns 0 to the child **C. It does not return; the process image is replaced** **D.** It creates a new process

exec **唔會建立新 process**，只係 replace 現有 process 嘅 image **[3.41]**。

P-ch3-16 fork count **→ C**

How many processes are there in total after: `fork(); fork() && fork(); fork();`

A. 8 **B.** 10 **C. 12** **D.** 16

2 × 3 × 2 = **12**（`fork() && fork()` 令每個 process 變 3 個）。

P-ch3-17 Interrupt vector **→ B**

The interrupt vector is:

A. A list of processes waiting for I/O **B. A table of addresses of interrupt handlers, indexed by interrupt number** **C.** The register that holds the next instruction address **D.** A queue of DMA requests

Ch3 summary。

P-ch3-18 Nonmaskable interrupt **→ B**

A nonmaskable interrupt is reserved for events such as:

A. Keyboard input **B. Unrecoverable memory errors and power failure** **C.** Timer expiry for RR **D.** Disk I/O completion

Ch3 summary；maskable 用於 device controllers。

P-ch4-1 Multicore challenges **→ D**

Which is **NOT** one of the multicore programming challenges listed in the lecture?

A. Dividing tasks **B.** Data dependency **C.** Testing and debugging **D. Reducing the number of system calls**

五大：dividing tasks、balance、data splitting、data dependency、testing & debugging **[4.9]**。

P-ch4-2 Windows TEB **→ C**

In Windows, which thread data structure lives in **user space** and contains the thread id, user-mode stack and thread-local storage?

A. ETHREAD **B.** KTHREAD **C. TEB** **D.** PCB

[4.36]。ETHREAD、KTHREAD 喺 kernel space。

P-ch4-3 clone() **→ B**

Which call creates a Linux task that behaves like a thread?

A. clone(SIGCHLD, 0) **B. clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0)** **C.** fork() followed by exec() **D.** pthread_kill()

share address space、fs info、files、signal handlers **[4.38]**；A = normal fork。

P-ch4-4 Linux threads **→ B**

How does Linux treat threads?

A. As lightweight processes with separate TCB and PCB structures **B. As tasks — a thread is a process that shares certain resources with other tasks** **C.** Only user-level threads are supported **D.** Through the many-to-one model

[4.37]。

P-ch4-5 Signal delivery **→ B**

A synchronous signal such as division by zero should be delivered to:

A. Every thread in the process **B. The thread that caused the signal** **C.** A random thread **D.** The parent process

[4.31]。

P-ch4-6 Pthread cancellation default **→ B**

In Pthreads, the default cancellation type is:

A. Asynchronous **B. Deferred** **C.** Disabled permanently **D.** Immediate

[4.32]。

P-ch4-7 Amdahl limit **→ B**

An application is 50% parallel. As the number of cores approaches infinity, the maximum speedup is:

A. 1.5 **B. 2** **C.** 4 **D.** Infinite

$1/S = 1/0.5 =$ **2**。

P-ch4-8 One-to-one drawback **→ C**

What is the main drawback of the one-to-one model?

A. One blocking call blocks the whole process **B.** Threads cannot run in parallel **C. Creating a user thread requires creating a kernel thread, so the number of threads may be restricted** **D.** It is difficult to implement

[4.23]。A、B 係 many-to-one；D 係 many-to-many。

P-ch4-9 Thread context switch **→ B**

Why is a context switch between two threads of the same process cheaper than between processes?

A. Threads have no registers **B. The address space stays the same** **C.** Threads do not have stacks **D.** The OS does not save any state

[4.14]。

P-ch4-10 fork in multithreaded process **→ B**

If exec() is called immediately after fork() in a multithreaded process, which fork() version is appropriate?

A. Duplicate all threads **B. Duplicate only the calling thread** **C.** Duplicate no threads **D.** It does not matter for correctness or efficiency

exec 會 replace 成個 process，複製所有 threads 係浪費 **[4.28]**。

P-ch5-1 Dispatcher **→ D**

Which is **NOT** a function of the dispatcher?

A. Switching context **B.** Switching to user mode **C.** Jumping to the proper location in the user program **D. Selecting the next process using the scheduling algorithm**

揀 process 係 **CPU scheduler**；dispatcher 只負責交 CPU 畀佢 **[5.7]**。

P-ch5-2 RR waiting bound **→ B**

With n processes and quantum q under RR (ignoring context switches), no process waits more than:

A. nq **B. (n − 1)q** **C.** q / n **D.** n + q

[5.11]。

P-ch5-3 Quantum rule of thumb **→ B**

A common rule of thumb for choosing the RR quantum is:

A. q should be smaller than the context-switch time **B. 80% of CPU bursts should be shorter than q** **C.** q should equal the longest CPU burst **D.** q should be 1 ms

[5.16]。

P-ch5-4 Exponential averaging α = 0 **→ B**

With exponential averaging, if $\alpha = 0$ then:

A. Only the last CPU burst counts **B. Recent history does not count; the prediction never changes** **C.** All bursts are weighted equally **D.** The prediction is always 0

$\tau_{n+1} = \tau_n$ **[5.22]**。

P-ch5-5 SJF as priority **→ B**

SJF is a special case of priority scheduling where the priority is:

A. The arrival time **B. The inverse of the predicted next CPU burst** **C.** The process ID **D.** The period of the task

[5.25]。

P-ch5-6 MLFQ preemption rule **→ C**

In the lecture's MLFQ, when a job running from Q1 is preempted by a new arrival in Q0, it:

A. Is demoted to Q2 **B.** Joins the tail of Q1 with a fresh quantum **C. Joins the head of Q1 with its remaining quantum** **D.** Is moved to Q0

[5.31]。

P-ch5-7 Logical CPUs **→ D**

A quad-core processor with 2 hardware threads per core appears to the OS as how many logical processors?

A. 2 **B.** 4 **C.** 6 **D. 8**

4 × 2 = 8 **[5.42]**。

P-ch5-8 Push migration **→ B**

Push migration means:

A. An idle processor pulls a waiting task from a busy processor **B. A specific task periodically checks loads and moves tasks from overloaded to idle/less-busy CPUs** **C.** A process specifies the CPUs it may run on **D.** A thread is pinned to one core forever

A = pull migration **[5.46]**；C = hard affinity **[5.45]**。

P-ch5-9 Affinity system call **→ A**

In Linux, which system call supports hard affinity?

A. sched_setaffinity() **B.** setpriority() **C.** clone() **D.** nice()

[5.45]。

P-ch5-10 RM optimality

→ B

Rate-monotonic scheduling is optimal in the sense that:

A. It always achieves 100% CPU utilization **B. If a task set cannot be scheduled by RM, it cannot be scheduled by any other static-priority algorithm** **C.** It never misses deadlines **D.** It handles aperiodic tasks best

[5.52] 。

P-ch5-11 EDF requirement

→ C

What is the only requirement EDF places on a process?

A. It must be periodic **B.** It must have a constant CPU burst **C. It must announce its deadline to the scheduler when it becomes runnable** **D.** Its period must be a power of 2

[5.52] 。

P-ch5-12 RM utilization

→ B

Two periodic tasks: P1 (period 50, burst 25), P2 (period 80, burst 35). Under RM:

A. Total utilization is 75% and no deadline is missed **B. Total utilization ≈ 94% and P2 misses its deadline at 80 (finishes at 85)** **C.** P1 misses its deadline **D.** EDF would also miss a deadline

[5.50–5.51]；EDF 可以全部 meet。

P-ch5-13 Deterministic modeling

→ B

Which statement about deterministic modeling is correct?

A. It uses probability distributions of CPU bursts **B. It takes a predetermined workload and gives exact numbers, but results apply only to that workload** **C.** It requires running the real system **D.** It is the most accurate method

[5.54] 。

P-ch5-14 Response time (course definition)

→ C

RR with q = 4; P1 (24), P2 (3), P3 (3) all arrive at 0 in that order. Using the course definition, the average response time is:

A. 3.67 **B.** 5.67 **C. 7** **D.** 15.67

end of first run − arrival = 4, 7, 10 → avg **7** **[5.13]** 。(3.67 係課本「first start」定義；5.67 係 avg WT；15.67 係 avg TAT)

P-ch5-15 Non-preemptive kernel / scheduling

→ B

Scheduling that happens only when a process switches from running to waiting or terminates is called:

A. Preemptive **B. Non-preemptive (cooperative)** **C.** Real-time **D.** Multilevel

[5.6] 。

P-ch5-16 SMP common queue issue

→ A

With a single common ready queue in SMP, what must the system guard against?

A. Two processors choosing to schedule the same thread (race condition) **B.** Processor affinity being too strong **C.** Too many push migrations **D.** Memory stalls

[5.39] 。

P-ch6-1 Disabling interrupts

→ B

Why is disabling interrupts not a good critical-section solution on multiprocessors?

A. It is not allowed in kernel mode **B. It is inefficient — interrupts must be disabled on all processors** **C.** It causes race conditions on uniprocessors **D.** It requires semaphores

[6.13] 。

P-ch6-2 CAS return value

→ C

compare_and_swap(&value, expected, new_value) returns:

A. new_value **B.** expected **C. The original value of *value** **D.** TRUE if the swap happened

[6.17] 。

P-ch6-3 Negative semaphore

→ B

In the blocking (no busy waiting) semaphore implementation, a value of −3 means:

A. The semaphore is broken **B. Three processes are waiting on the semaphore** **C.** Three resources are available **D.** Three signals were lost

[6.26] 。

P-ch6-4 Spinlock usage

→ B

Spinlocks are most appropriate when:

A. Locks are held for a long time on a uniprocessor **B. Locks are held for short times on multiprocessor systems** **C.** Processes must sleep while waiting **D.** Context switches are free

[6.22] 。

P-ch6-5 Preemptive kernel

→ B

Which statement is correct?

A. A preemptive kernel is free from race conditions **B. A non-preemptive kernel is essentially free from race conditions in kernel mode (on a single processor)** **C.** Non-preemptive kernels are harder to make correct on SMP than preemptive ones **D.** Kernel code never has race conditions

[6.12] 。

P-ch6-6 Kernel race example

→ B

The lecture's kernel race-condition example involves two processes calling fork() concurrently. Which variable is at risk?

A. counter **B. next_available_pid** **C.** lock **D.** turn

same PID could be assigned to two processes **[6.8]** 。

P-ch6-7 P and V

→ B

In Dijkstra's notation, P() and V() correspond to:

A. signal() and wait() **B. wait() and signal()** **C.** acquire() and test_and_set() **D.** lock() and fork()

P = proberen (test)，V = verhogen (increment) **[6.24]** 。

P-ch6-8 Semaphore deadlock

→ B

P0: wait(S); wait(Q); ... P1: wait(Q); wait(S); ... with S = Q = 1. What can happen?

A. Starvation only **B. Deadlock** **C.** Race condition on S only **D.** Nothing; this is always safe

[6.28] 。

P-ch6-9 Starvation in semaphore queue

→ A

Starvation (indefinite blocking) can occur with semaphores when:

A. Processes are removed from the waiting queue in LIFO order **B.** The semaphore is initialized to 1 **C.** signal() is atomic **D.** The waiting queue is FIFO

[6.28] 。

P-ch6-10 Atomic increment

→ B

In `do { temp = *v; } while (temp != compare_and_swap(v, temp, temp + 1));`, why might the loop repeat?

A. Because CAS always fails the first time **B. Because another thread changed *v between reading temp and the CAS, so the CAS returned a different value** **C.** Because temp + 1 overflows **D.** It never repeats

[6.21] 。

P-ch6-11 Counting semaphore initial value

→ C

A counting semaphore controlling access to 5 identical printers should be initialized to:

A. 0 **B.** 1 **C. 5** **D.** −5

initialized to the number of resources available **[6.25]** 。

P-ch6-12 Simple TAS lock

→ B

The simple lock `while (test_and_set(&lock)); /*CS*/ lock = false;` satisfies:

A. Mutual exclusion and bounded waiting **B. Mutual exclusion but not bounded waiting** **C.** Bounded waiting but not mutual exclusion **D.** None of the requirements

要用 waiting[] 版本 **[6.19]** 先有 bounded waiting。

A C / Pthreads API Lab / PA API 速查

Announcement：「There may be questions related to PA or coding questions using some APIs we discussed in our lab classes」→ 呢頁就係用嚟填空 / 寫 code。

1. Process APIs

Call	Header	Return	重點
`pid_t fork(void)`	`<unistd.h>`	child: **0**；parent: **child PID (>0)**；fail: **−1**	child 由下一句開始；複製 address space（COW）
`int execlp(const char *file, const char *arg0, ..., (char *)NULL)`	`<unistd.h>`	**success: no return**；fail: −1	用 PATH 搵 program；**arg0 = program name**；最後一定要 `NULL`
`int execvp(const char *file, char *const argv[])`	`<unistd.h>`	同上	`argv` 最後一個元素要係 **NULL**（PA1：所以 array size 係 9 而唔係 8）
`int execl(path, arg0, ..., NULL)` / `execv(path, argv)`	`<unistd.h>`	同上	要 full path（冇 p = 唔搵 PATH）
`pid_t wait(int *status)`	`<sys/wait.h>`	terminated child 嘅 PID；冇 child → −1	`wait(NULL)` 唔要 status；等**任何一個** child
`pid_t waitpid(pid_t pid, int *status, int options)`	`<sys/wait.h>`	該 child PID；error −1	`waitpid(pid, &status, 0)` 等指定 child；`WNOHANG` 唔 block
`void exit(int status)`	`<stdlib.h>`	—	結束 process，status 經 wait 交畀 parent（`WEXITSTATUS(status)`）
`pid_t getpid(void)` / `getppid(void)`	`<unistd.h>`	自己 / parent PID	
`int chdir(const char *path)`	`<unistd.h>`	0 / −1	PA1 `cd`（唔可以用 exec 做 cd，因為要改 shell 自己嘅 cwd）
`sighandler_t signal(int signum, sighandler_t handler)`	`<signal.h>`	舊 handler	`signal(SIGINT, handler)`；`SIG_IGN` 忽略；`SIG_DFL` default
`int kill(pid_t pid, int sig)`	`<signal.h>`	0 / −1	send signal to process
`unsigned int sleep(unsigned int s)`	`<unistd.h>`	剩餘秒數	停 s 秒；`usleep` = μs；`nanosleep`（`<time.h>`）更細

exec family（Lab 3）：六個 — `execl, execlp, execle, execv, execvp, execvpe`；全部 **replace the current process image，成功唔會 return，失敗 −1**。

字母	意思	例
l (list)	arguments 逐個列出，數目 compile time 已知	`execlp("ls", "ls", "-l", "-h", NULL);`
v (vector)	arguments 放喺 array（尾要 NULL）；數目唔知時用	`char *a[] = {"ls", "-l", NULL}; execvp("ls", a);`
p (path)	用 **PATH** 搵 program（唔使 full path）	`execlp` / `execvp` / `execvpe`
e (env)	額外傳 environment array	`execle` / `execvpe`

1.1 經典 fork–exec–wait

```
pid_t pid = fork();
if (pid < 0) { perror("fork"); exit(1); }
else if (pid == 0) {                     /* child */
    execlp("ls", "ls", "-l", (char *)NULL);
    perror("exec failed");                /* only reached if exec fails */
    exit(1);
} else {                                 /* parent */
    int status;
    waitpid(pid, &status, 0);            /* or wait(NULL) */
}
```

1.2 Signal handler（PA1 Feature 2）

```
void handle_sigint(int signum) {
    printf("Myshell (pid=%d) terminates by Ctrl-C with signal: %d\n", getpid(), signum);
    exit(0);
}
int main() { signal(SIGINT, handle_sigint); /* ... */ }
```

- SIGINT = **2**（Ctrl-C）；SIGKILL (9)、SIGSTOP 唔可以 catch。

2. File I/O（POSIX，唔係 stdio）

Call	Return	重點
`int open(const char *path, int flags[, mode_t mode])` `<fcntl.h>`	**lowest unused fd** / −1	flags：`O_RDONLY`、`O_WRONLY`、`O_RDWR`、`O_CREAT`、`O_TRUNC`、`O_APPEND`；建立檔案要 mode（e.g. `0644`）
`ssize_t read(int fd, void *buf, size_t count)`	bytes read；**0 = EOF**；−1 error	blocking（pipe 冇 data 會等；所有 write ends close 晒先 return 0）
`ssize_t write(int fd, const void *buf, size_t count)`	bytes written / −1	
`int close(int fd)`	0 / −1	**用完要 close**，尤其 pipe write end
`int dup(int oldfd)`	新 fd = **lowest unused**	`close(0); dup(fin);` → fd 0 指向 file
`int dup2(int oldfd, int newfd)`	newfd	newfd 已開 → 先 close；`dup2(fd, 1)` = stdout 指向 fd

- **fd 0 = stdin，1 = stdout，2 = stderr**（`STDIN_FILENO`, `STDOUT_FILENO`, `STDERR_FILENO`）。
- Lab 3 permission 寫法：`open("hello.txt", O_CREAT | O_WRONLY, S_IRUSR | S_IWUSR)`（**600**：owner read + write）；`O_RDONLY` 開讀。
- ⚠ **唔好用 stdio 嘅 `fopen/fread/fwrite/fclose`（`FILE*`）做 PA**：`FILE*` 係 C library struct，**cannot be used in pipe operations**。
- **Output redirection**（`cmd > out.txt`）：

```
int fout = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(fout, STDOUT_FILENO);  close(fout);   /* then exec */
```

- **Input redirection**（`cmd < in.txt`）：

```
int fin = open("in.txt", O_RDONLY);
dup2(fin, STDIN_FILENO);  close(fin);      /* or: close(0); dup(fin); */
```

3. Pipes

Call	重點
`int pipe(int fd[2])` `<unistd.h>`	**fd[0] = read end，fd[1] = write end**；0 / −1；**一定喺 fork() 之前 call**
`int mkfifo(const char *path, mode_t mode)` `<sys/stat.h>`	named pipe（FIFO），之後用 `open()` / `read()` / `write()`；要 `unlink()` 刪除；0 / −1
`int access(const char *path, int how)` `<unistd.h>`	`F_OK` 存在、`R_OK` / `W_OK` / `X_OK` 權限；0 = OK，−1 = 唔得（Lab 4：`if (access(FIFO, F_OK) == -1) mkfifo(FIFO, 0777);`）
`FILE *popen(const char *cmd, const char *type)` / `int pclose(FILE *fp)` `<stdio.h>`	fork 一個 child 跑 cmd；`"r"` 讀佢 stdout、`"w"` 寫佢 stdin；`pclose` 等佢完返回 status（**PA1 禁用**）

- **FIFO 特性（Lab 4）**：`open/read/write` **blocking，要兩端都有 process open 先繼續**（可用 `O_NONBLOCK`）；data **讀咗就冇**（唔可以 broadcast）；≤ `PIPE_BUF` **bytes 嘅 write 係 atomic**；一條 FIFO 實際唔適合雙向（可能讀返自己寫嘅）→ 雙向用兩條。
- **兩條 ordinary pipes 做雙向**：`pipe(p2c); pipe(c2p);` → child：`close(c2p[0]); write(c2p[1], …); close(p2c[1]); read(p2c[0], …);`；parent：`close(c2p[1]); read(c2p[0], …); close(p2c[0]); write(p2c[1], …);`（Lab 4 exam example）。

3.1 Parent → child

```
int fd[2]; pipe(fd);
if (fork() == 0) {              /* child: reader */
    close(fd[1]);
    int n; read(fd[0], &n, sizeof(n));
    close(fd[0]); exit(0);
} else {                        /* parent: writer */
    close(fd[0]);
    int n = 6; write(fd[1], &n, sizeof(n));
    close(fd[1]); wait(NULL);
}
```

3.2 Shell pipeline `ls | wc -l`

```
int fd[2]; pipe(fd);
if (fork() == 0) {               /* child 1: ls → pipe */
    dup2(fd[1], STDOUT_FILENO);
    close(fd[0]); close(fd[1]);
    execlp("ls", "ls", (char *)NULL);
}
if (fork() == 0) {               /* child 2: pipe → wc */
    dup2(fd[0], STDIN_FILENO);
    close(fd[0]); close(fd[1]);
    execlp("wc", "wc", "-l", (char *)NULL);
}
close(fd[0]); close(fd[1]);      /* parent MUST close both, else wc never sees EOF */
wait(NULL); wait(NULL);
```

- **Multi-level pipe（PA1 Feature 5）**：n segments → n−1 pipes；segment i：`dup2(fds[i-1][0], 0)`（if i > 0）、`dup2(fds[i][1], 1)`（if i < n−1）；每個 child close 所有 pipe fds；parent close 所有 fds 再 wait 全部。

3.3 常見錯誤

1. `pipe()` 喺 `fork()` 之後 call → 兩條唔相關嘅 pipe（HW1 Q3.3c）。
2. 冇 close 未用嘅 write end → reader `read()` 永遠唔會得到 EOF（hang）。
3. `write(fd[0], …)` / `read(fd[1], …)` 搞錯方向（**0 讀 1 寫**：記「0 = stdin-like，1 = stdout-like」）。
4. `read` / `write` 第二個參數要 **address**（`&cnt`），第三個係 **size**（`sizeof(int)`）。

---

4. Pthreads（`#include <pthread.h>`，compile with `-pthread`）

Call	重點
`int pthread_create(pthread_t *tid, const pthread_attr_t *attr, void *(*start)(void *), void *arg)`	return 0 = success；attr 用 `NULL`；start routine signature `void *f(void *)`
`int pthread_join(pthread_t tid, void **retval)`	等 thread 結束（類似 wait）；`retval` 可 NULL
`void pthread_exit(void *retval)`	thread 自己結束
`int pthread_cancel(pthread_t tid)`	request cancellation（default **deferred**）
`pthread_t pthread_self(void)`	自己 tid
`int pthread_kill(pthread_t tid, int sig)`	signal 指定 thread
`int pthread_attr_init(pthread_attr_t *attr)`	default attributes（stack size, scheduling info）→ 再傳畀 `pthread_create`（Lab 4 sum example）
`pthread_cond_broadcast(&cond)`	叫醒**所有**等緊嘅 threads（`pthread_cond_signal` 只叫醒一個）

```
void *worker(void *arg) { int id = *(int *)arg; printf("thread %d\n", id); return NULL; }
int main() {
    pthread_t t[3]; int id[3];
    for (int i = 0; i < 3; i++) { id[i] = i; pthread_create(&t[i], NULL, worker, &id[i]); }
    for (int i = 0; i < 3; i++) pthread_join(t[i], NULL);
    return 0;
}
```

4.1 Mutex

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;   /* or pthread_mutex_init(&m, NULL) */
pthread_mutex_lock(&m);   /* critical section */   pthread_mutex_unlock(&m);
pthread_mutex_destroy(&m);
```

4.2 POSIX unnamed semaphore（`#include <semaphore.h>`）

```
sem_t s;
sem_init(&s, 0, 1);    /* pshared = 0 → shared between threads of this process; initial value 1 */
sem_wait(&s);          /* P(): if value > 0 decrement, else block */
sem_post(&s);          /* V(): increment, wake a waiter */
sem_destroy(&s);
```

- **Ordering**：`sem_init(&s, 0, 0)`；先做嘅 thread 做完 `sem_post(&s)`，後做嘅 thread 開頭 `sem_wait(&s)`。
- **Mutual exclusion**：init 1；`sem_wait` → CS → `sem_post`。
- **Counting resources**：init = 資源數量 n。

4.3 Condition variables

```
pthread_mutex_t m; pthread_cond_t c; int ready = 0;
/* waiter */
pthread_mutex_lock(&m);
while (!ready) pthread_cond_wait(&c, &m);   /* releases m while sleeping, re-acquires on wake */
pthread_mutex_unlock(&m);
/* signaller */
pthread_mutex_lock(&m); ready = 1; pthread_cond_signal(&c); pthread_mutex_unlock(&m);
```

- 一定要持有 mutex 先 call；用 **while**（唔係 if）再 check 條件。

---

5. Compile / Run（Lab 1–2）

```
gcc -pedantic -Wall -Wextra -std=c99 -o myshell myshell.c
gcc prog.c -o prog -lpthread       # pthread programs (or -pthread)
gcc buggy.c -o buggy -g            # debug info, then: gdb buggy
./mlfq < input.txt > output.txt    # PA2: input/output redirection
diff --side-by-side my-out.txt sample-out.txt   # compare outputs (--suppress-common-lines)
gcc main.c -o main -lm             # link math library (linker!)
./prog
```

- `gcc -c main.c` → `main.o`（compiler）；`gcc -o main main.o -lm`（linker）；`./main`（loader）[2.32]。

G Glossary A-Z 術語表

Term	Definition（exam English）	中文	Ref
Accounting information	PCB field: CPU used, clock time elapsed, time limits	記帳資訊	3.10
Address space	The memory a program executes in, distinct from other programs' (text, data, heap, stack)	位址空間	3.4–3.5
Admission control	Real-time scheduler admits a task only if it can guarantee its deadline	准入控制	5.48
Aging	Gradually increasing the priority of a process that waits a long time, to prevent starvation	老化	5.25
Amdahl's Law	$\text{speedup} \le \dfrac{1}{S + (1 - S)/N}$; as $N \to \infty$, speedup $\to 1/S$	阿姆達爾定律	4.12
API	A set of functions available to programmers (parameters + return values) that invoke system calls on their behalf	應用程式介面	2.15
Asymmetric multiprocessing (AMP)	Master processor handles scheduling, I/O and kernel data; others run user code only; master may bottleneck	非對稱多處理	1.37, 5.38
Asynchronous cancellation	Terminates the target thread immediately	非同步取消	4.32
Asynchronous signal	Signal generated by an event external to the running process (Ctrl-C, timer expiry)	非同步信號	4.29
Atomic	Non-interruptible; executes as one uninterruptible unit	原子性	6.14
Atomic variable	Provides atomic updates on basic data types (e.g., increment via CAS)	原子變數	6.21
Average access time	Hit ratio × hit time + miss ratio × miss time	平均存取時間	1.32
Binary semaphore	Semaphore whose value is only 0 or 1; can act like a mutex	二元信號量	6.25
Blocking (synchronous) send/receive	Sender blocks until the message is received; receiver blocks until a message is available	阻塞式	3.62
Bootstrap program	First program run at power-on, stored in EEPROM	開機程式	1.30
Bounded buffer	Fixed-size buffer shared by producer and consumer	有界緩衝區	3.53
Bounded waiting	A bound exists on how many times others may enter their CS after a process requests entry	有限等待	6.11
Busy waiting	Looping continuously while waiting for a condition; wastes CPU	忙碌等待	6.22
Cache	Faster, smaller storage holding a copy of a subset of slower, larger storage	快取	1.32
Cascading termination	When a process terminates, all its children are also terminated by the OS	級聯終止	3.48
Chip multithreading (CMT) / hyperthreading	Multiple hardware threads per core, each appearing as a logical CPU; hides memory stalls	晶片多執行緒	5.40–5.42
clone()	Linux call creating a task that shares selected resources with its parent (flags)	複製	3.46, 4.38
Cloud computing	Delivers computing, storage and apps as a service over a network; extension of virtualization	雲端運算	1.47
Compare-and-swap (CAS)	Atomically: if *value == expected set *value = new; return original value	比較並交換	6.17
Concurrency	More than one task making progress (possibly by time multiplexing on one core)	並行（並發）	4.9
Context switch	Saving the state of the old process/thread and loading the saved state of the new one; pure overhead	上下文切換	3.20
Convoy effect	Short processes stuck waiting behind a long process (FCFS)	護航效應	5.10
Cooperating process	A process that can affect or be affected by other processes	協作進程	3.51
Copy-on-write (COW)	Parent and child share pages until one writes; then the page is copied	寫入時複製	3.46
CPU burst	A period of CPU execution between I/O waits	CPU 爆發	5.4
CPU-bound process	Spends most time computing; few long CPU bursts	CPU 密集	3.14
CPU scheduler (short-term)	Selects a process from the ready queue and allocates a CPU core	短期排程器	3.17, 5.6
CPU utilization	Fraction of time the CPU is busy	CPU 使用率	5.8
Critical section	Code segment in which shared data is accessed/modified	臨界區	6.9
Data parallelism	Same operation on different subsets of data across cores	數據並行	4.11
Deadlock	Two or more processes wait indefinitely for an event only one of the waiting processes can cause	死結	6.28
Deferred cancellation	Target thread periodically checks whether it should terminate (orderly)	延遲取消	4.32
Degree of multiprogramming	Number of processes currently in memory	多道程式度	3.14
Deterministic modeling	Evaluates algorithms on a predetermined workload; exact but specific	確定性建模	5.54
Device driver	Kernel module that controls a device	裝置驅動程式	1 (summary)

Term	Definition（exam English）	中文	Ref
Direct communication	Processes name each other explicitly; one link per pair	直接通訊	3.59
Dispatch latency	Time for the dispatcher to stop one process and start another	分派延遲	5.7
Dispatcher	Gives CPU control to the selected process: context switch, switch to user mode, jump to restart location	分派器	5.7
DLL (dynamically linked library)	Library loaded when needed and shared by all programs using it	動態連結庫	2.31
DMA	Controller transfers blocks directly between device and memory without CPU; interrupts once when done	直接記憶體存取	3.25
Dual-mode operation	User mode vs kernel mode distinguished by a hardware mode bit (1 = user, 0 = kernel)	雙模式	3.27–3.28
dup / dup2	Duplicate a file descriptor (dup → lowest free fd; dup2 → specified fd)	複製描述符	Lab/PA1
EDF	Earliest-deadline-first: dynamic priority, earlier deadline = higher priority; theoretically optimal	最早截止期限優先	5.51
Exec()	Replaces the current process image with a new program; returns only on failure	執行新程式	3.41
Exponential averaging	$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$: predicts the next CPU burst	指數平均	5.19
FCFS	First-come first-served; non-preemptive; convoy effect	先到先得	5.9
fork()	Creates a child duplicating the parent's address space; returns 0 to child, child PID to parent, −1 on failure	分叉	3.35–3.37
Hard affinity	Process specifies the subset of CPUs it may run on	硬親和性	5.45
Hard real-time	Task must be serviced by its deadline; late = no service	硬實時	5.47
Heap	Memory dynamically allocated at run time	堆積	3.5
Hybrid system	OS combining several structures (Linux, Windows, macOS)	混合式	2.52
Indirect communication	Messages sent to/received from mailboxes (ports)	間接通訊	3.60
Interrupt	Signal from hardware/software requiring CPU attention; handled by an ISR	中斷	3.21–3.24
Interrupt vector	Table of interrupt-handler addresses indexed by interrupt number	中斷向量	3 (summary)
I/O-bound process	Spends more time on I/O; many short CPU bursts	I/O 密集	3.14
IPC	Mechanism for cooperating processes to exchange data: shared memory or message passing	進程間通訊	3.51
Kernel	The one program running at all times on the computer	核心	1.24
Kernel thread	Thread supported and scheduled by the OS	核心執行緒	4.19
Layered approach	OS divided into layers (0 = hardware … N = UI); each uses only lower layers	分層結構	2.44
Linker	Combines relocatable object files and libraries into one executable	連結器	2.31
Little's formula	$n = \lambda \times W$ (queue length = arrival rate × waiting time)	利特爾公式	5.57
Load balancing	Keep workload evenly distributed across CPUs (push / pull migration)	負載平衡	5.46
Loadable kernel module (LKM)	Core kernel plus modules linked in at boot or run time; any module can call any other	可載入核心模組	2.49–2.50
Loader	Loads the executable file into memory for execution	載入器	2.31
Locality (temporal / spatial)	Recently accessed items are reused soon / nearby items are accessed soon	局部性	1.33
Long-term scheduler (job scheduler)	Selects which jobs are brought into memory; controls degree of multiprogramming	長期排程器	3.17
LWP (lightweight process)	Virtual processor between user threads and a kernel thread	輕量級進程	5.36
Many-to-one / one-to-one / many-to-many	Mappings from user threads to kernel threads	多對一 / 一對一 / 多對多	4.21–4.24
Medium-term scheduler	Swaps processes out to disk and back to reduce degree of multiprogramming	中期排程器	3.18
Memory stall	Processor waits for data from memory (e.g., cache miss)	記憶體停頓	5.40
Message passing	IPC via explicit send()/receive() without shared memory	訊息傳遞	3.57
Microkernel	Minimal kernel; other services run in user space and communicate via message passing	微核心	2.46–2.48
MLFQ	Multilevel feedback queue: processes move between queues; approximates SRTF without burst knowledge	多級回饋佇列	5.30–5.35
Mode bit	Hardware bit: 1 = user mode, 0 = kernel mode	模式位元	3.28
Monolithic	All kernel functionality in one static binary in a single address space; fast but hard to maintain	單體式	2.41
Multicore	Multiple computing cores on a single chip; faster on-chip communication, less power	多核心	1.39

Term	Definition（exam English）	中文	Ref
Multilevel queue	Processes permanently assigned to queues by type; each queue its own algorithm	多級佇列	5.28–5.29
Multiprogramming	Keeping several jobs in memory so the CPU always has one to execute	多道程式	1.42
Mutex lock	Lock acquired before and released after a critical section (acquire/release atomic)	互斥鎖	6.22
Mutual exclusion	If one process is in its CS, no other process can be in its CS	互斥	6.11
Named pipe (FIFO)	Bidirectional (UNIX half-duplex) pipe with a file name; no parent–child needed; persists until deleted	具名管道	3.69
Non-preemptive scheduling	Process keeps the CPU until it terminates or blocks	非搶佔式	5.6
NUMA	Each CPU has fast local memory; remote memory access slower	非一致記憶體存取	1.40
Ordinary pipe	Unidirectional pipe between parent and child; fd[0] read, fd[1] write; gone after use	普通管道	3.66
Orphan	Child whose parent terminated without wait(); adopted by init/systemd	孤兒進程	3.50
Parallelism	Performing more than one task simultaneously (needs multiple cores)	平行	4.9
PCB	Process control block: state, PC, registers, scheduling, memory, accounting, I/O info	進程控制塊	3.10
PCS (process-contention scope)	Thread library schedules user threads onto LWPs, among threads of the same process	進程競爭範圍	5.37
Pipe	IPC channel treated as a special file; read()/write()	管道	3.65
Policy vs mechanism	Policy = what will be done; mechanism = how to do it	策略與機制	2.34
Preemptive scheduling	CPU can be taken away involuntarily (interrupt, higher-priority arrival)	搶佔式	5.6
Priority scheduling	CPU to highest-priority process (smallest number); starvation → aging	優先權排程	5.25
Privileged instruction	Instruction executable only in kernel mode (I/O, timer, interrupt management)	特權指令	3.28
Process	A program in execution; active entity with PC and resources	進程	3.4–3.5
Processor affinity	Tendency to keep a process on the CPU whose cache holds its data	處理器親和性	5.45
Program	Passive entity — executable file on disk	程式	3.5
Progress	If no one is in the CS, selection of the next entrant cannot be postponed indefinitely	前進	6.11
Pull / push migration	Idle CPU pulls tasks / a task pushes work from overloaded CPUs	拉 / 推遷移	5.46
Race condition	Outcome depends on the order of concurrent access to shared data	競爭條件	6.9
Ready queue	Processes in memory ready and waiting to run	就緒佇列	3.15
Rate-monotonic (RM)	Static priority = shorter period higher priority; optimal among static-priority algorithms	速率單調	5.49
Response time	(Course) end of the first run − arrival; time until the first response is produced	回應時間	5.12
Round robin (RR)	FCFS with a time quantum; preempted process to tail; wait ≤ (n−1)q	輪轉	5.11
RPC	Remote procedure call: message-based procedure call between networked systems	遠端程序呼叫	3.70
SCS (system-contention scope)	Kernel schedules kernel threads onto CPUs, among all threads in the system	系統競爭範圍	5.37
Semaphore	Integer accessed only via atomic wait() (P) and signal() (V)	信號量	6.24
Shared memory	IPC by reading/writing a shared region	共享記憶體	3.51
Signal	UNIX notification to a process that an event occurred	信號	4.29
SJF	Shortest-job-first: optimal minimum average waiting time; hard to know next burst	最短工作優先	5.17
SMP	Symmetric multiprocessing: each processor self-schedules; own registers & cache, shared memory	對稱多處理	1.38, 5.39
Socket	Endpoint for communication identified by IP address + port	套接字	3.70
Soft affinity	OS tries but does not guarantee to keep a process on the same CPU	軟親和性	5.45
Soft real-time	Critical tasks get preference over non-critical ones; no guarantee	軟實時	5.47
Spinlock	Lock that busy-waits; no context switch, wastes CPU; good for short CS on multiprocessors	自旋鎖	6.22
SRTF	Shortest-remaining-time-first: preemptive SJF	最短剩餘時間優先	5.23
Stack	Temporary data: function parameters, return addresses, local variables	堆疊	3.5
Starvation	A process waits indefinitely (low priority / semaphore queue)	飢餓	5.25, 6.28

Term	Definition（exam English）	中文	Ref
System call	Programming interface to OS services	系統呼叫	2.14
System program	Utilities shipped with the OS providing a convenient environment (not part of kernel)	系統程式	2.27
Task parallelism	Different threads performing unique operations on cores	任務並行	4.11
TCB	Thread control block: registers, PC, stack pointer, scheduling info, pointer to PCB	執行緒控制塊	4.15
Test-and-set	Atomically returns the old value of a boolean and sets it to TRUE	測試並設置	6.15
Thread	Basic unit of CPU utilization: thread ID, PC, register set, stack	執行緒	4.6
Thread-local storage	Data unique to each thread	執行緒區域儲存	(Sp21 MC)
Throughput	Number of processes completed per unit time	吞吐量	5 (summary)
Timesharing (multitasking)	CPU switches among jobs so frequently that users can interact with each	分時	1.42
Trap / exception	Software-generated interrupt caused by an error or a system call request	陷阱 / 例外	3.24, 3.31
Turnaround time	Completion time − arrival time	周轉時間	5.8
User thread	Thread managed by a user-level library, invisible to the kernel	使用者執行緒	4.19
Virtualization	Abstracts one computer's hardware into multiple execution environments (VMs)	虛擬化	1.43
VMM / hypervisor	Creates and manages VMs, presenting an interface identical to the host	虛擬機監控器	1.44
von Neumann architecture	Stored-program: instructions and data in the same memory; fetch–decode–execute	馮紐曼架構	1.26
Waiting time	Time spent in the ready queue = turnaround − burst	等待時間	5.8
Zombie	Terminated process whose parent has not yet called wait(); PID & PCB remain	殭屍進程	3.49

I Index 索引

A

Accounting information · 36, 47, 102
Address space · 32, 39, 45, 46, 52, 104, 160
Admission control · 21, 60, 126
Aging · 56, 57, 58, 123, **130**, 161, 162
Amdahl's Law · 15, 21, 46, 47, 52, **114**
AMP · 20, 21, 25, 27, 59, **125**
Android · 20, 23, 28, 34, 35, 96, 164
API · 29, 34, **90**, **117**, **140**, **167**
ART · 20, 34, 96, 97, 164
Asymmetric multiprocessing (AMP) · 59, 85, 125, 159
Asynchronous cancellation · 16, 51, 53, 116, 118, 119, 162
Asynchronous signal · 51, 52, 118, 162
Atomic · 64, 65, 66, 72, **133**, **134**, **135**
Atomic variable · 65, 72, 134
Average access time · 15, 20, 24, 27, **85**, 88

B

Binary semaphore · 16, 21, 66, 72, 136, 138, 150
blocking · 21, 42, 43, 49, 51, 56, 109
Blocking (synchronous) send/receive · 21, 30, 39, 41, 42, 108, 109
bootstrap · 20, 24, 27, 84, 163
Bootstrap program · 24, 27, 84, 163
Bounded buffer · 15, 21, 22, 42, 80, **108**
bounded capacity · 42, 109
Bounded waiting · 64, 65, 68, 69, 72, 138, 166
buffering · 20, 23, 29, 42, 87, **108**, 109
Busy waiting · 21, 64, 66, 67, 135, **136**
byte addressable · 15, 24, 84, 87, 88, 158

C

Cache · 23, 24, 25, 27, 84, 85, 88
Cascading termination · 41, 45, 105
chdir · 40, 45, 73, 77, 78, 141, 145
Chip multithreading (CMT) / hyperthreading · 59, 125
CLI · 20, 28, 89, 90, 92, 158
clone() · 20, 21, 39, 52, **104**, 165
close() · 18, 43, 75, 76, 77, 110, 168
cloud · 20, 26, 27, 34, 82, **87**
Cloud computing · 20, 26, 34, **87**
CMT · 59, **125**, 148
compare_and_reset · 19, 132, 135, 138, 152
compare_and_swap · 19, 64, 65, 66, 69, **134**
Concurrency · 46, 49, **113**, 114, 118, 159
condition variable · 71, 144
consumer · 42, 63, 70, 80, **108**, **132**
Context switch · 16, 20, 37, **103**, 158, 164
control program · 20, 22, 82
Convoy effect · 18, 21, 53, 54, 58, 121, 162
copy-on-write · 20, 39, 40, 45, **104**, 164
Copy-on-write (COW) · 20, 39, 40, 45, **104**, 164
counter++ · 21, 63, 72, 79, 132, 138, 146
counting semaphore · 21, 66, 67, 70, 72, 136, 166
CPU burst · 53, 54, 55, 58, **122**, 124
CPU scheduler (short-term) · 36, 39, 53, 60, 73, 120, 160
CPU utilization · 26, 36, 54, 55, 60, 61, 120
CPU-bound · 36, 47, 53, 54, 57, 103, 124
CPU-bound process · 53, 57, 103, 129
Critical section · 63, 65, 66, 68, **133**, 161
Ctrl-C · 21, 22, 41, 51, 73, 77, 167
cycle stealing · 139

D

Darwin · 20, 33, 34, 35, 94, 95, 164
Data parallelism · 46, 52, 114, 118, 161, 162
deadline · 60, 61, 126, **127**, 128, **143**
Deadlock · 21, 67, 68, 70, 71, **136**
Deferred cancellation · 21, 51, 116, 117, 162
Degree of multiprogramming · 36, 37, 44, 103, 111, 160
Deterministic modeling · 61, 128, 166
Device driver · 23, 37, 38, 73, 83, 87, 99
Direct communication · 109, 164
Dispatch latency · 21, 53, 60, 73, 120
Dispatcher · 21, 53, 60, 61, **120**, 165
DLL · 35, 164
DLL (dynamically linked library) · 35, 164
DMA · 38, 73, **99**, 100, 158, 161
dual mode · 32, 35, 94, **99**, 160, 164
Dual-mode operation · 20, 38, **100**, 101
dup / dup2 · **18**, 44, 76, **110**, **141**
dup() · 18, 43, 44, 76, 110, 141, 168
dup2 · 18, 44, 76, **110**, **141**
dynamic priority · 15, 18, 21, 56, 120, **129**

E

economy · 25, 46, 85, 88, 113, 159, 162
EDF · 60, 61, **127**, 128, **143**, 166
EEPROM · 20, 24, 27, 84, 163
entry section · 63, 64, 65, 79, 133, 134, 161
exception · 36, 38, 45, 99, 100, 161, 164
Exec() · 39, 40, 45, 50, 104, 105, **116**
execl · 140, 167
execle · 140, 167
execlp · 18, 40, 76, 105, 107, 140, 167
execv · 140, 167
execvp · 40, 44, 74, 75, 77, 140, 167
execvpe · 140, 167
exit section · 63, 65, 72, 79, 133, 159
exit() · 30, 36, 38, 40, 41, 48, 105
Exponential averaging · 15, 21, 55, 62, **122**, 165

F

FCFS · 55, 58, **121**, **124**, **129**, **130**
fflush · 40, 44, 77, 141, 154
FIFO · 43, 45, 54, 78, 111, 142, 168
fopen · 41, 140, 145, 168
fork() · **17**, **104**, **105**, 106, **116**, 141
full-duplex · 43, 45, 109, 111, 164

G

Gantt · 56, 57, **122**, **123**, **128**, 143
gcc · 31, 48, 73, 79, 92, 139, 170
getpid · 20, 30, 37, 39, 74, 140, 167
getppid · 39, 74, 81, 140, 167
guest · 20, 26, 27, 38, 86, 87, 163
GUI · 20, 28, 30, 34, 89, 90, 92

H

half-duplex · 16, 43, 45, 109, 111, 164
Hard affinity · 59, 125, 126, 154, 165
Hard real-time · 60, 62, 126, 161
HDD · 27, 83, 84, 88, 158, 163
Heap · 35, 39, 45, 52, 101, 110, 112
heavyweight · 45, 46, 112, **113**, 118, 150
hit ratio · 20, 24, 27, 85, 88, 146, 153
hybrid · 20, 26, 33, 34, 46, **96**
hyperthreading · 59, 125
hypervisor · 26, 86, 88, 163

I

I/O-bound · 53, 68, 78, **87**, **99**, **110**, **120**
I/O-bound process · 65, 68, 78, **87**, **99**, **110**, **120**
IaaS · 20, 27, 87, 163
Indirect communication · 21, 42, 109
Interrupt · 37, 38, 60, 73, **99**, 139
Interrupt vector · 20, 37, 99, 139, 165
IPC · **35**, 41, 42, 96, **108**

J

JNI · 20, 34, 35, 96, 164

K

Kernel · 33, 49, **96**, **100**, **115**, **133**
Kernel thread · 49, 52, 59, 115, 116, 125, 160
kill() · 41, 51, 53, 105, 116, 140, 167

L

layered · 32, 33, **95**, 96, 97, 158
Layered approach · 20, 32, **95**, 148, 159
Least Used First · **129**, 149
lightweight · 45, 46, 49, 59, **113**, 150
Linker · 31, 73, 92, 93, 97, **139**, 148
Linux · 33, 52, 94, **117**, **139**, **140**, **141**
Little's formula · 15, 16, 21, 61, 62, 120, 128
LKM · 15, 20, 33, 89, 96, 97, 148
Load balancing · 21, 59, 60, 62, **126**, 151
loadable kernel module · 33, 162
Loadable kernel module (LKM) · 33, 162
Loader · 31, 35, 73, 92, 93, **139**, 148
locality · 24, 27, **85**, 88, 148, 159
Locality (temporal / spatial) · 24, 27, **85**, 88, 148, 159
long-term scheduler · 36, 73, 111, 160, 164
Long-term scheduler (job scheduler) · 36, 73, 111, 160, 164
LUF · 18, 120, **129**, 149
LWP · 21, 49, 59, 62, 116, 124, 125
LWP (lightweight process) · 21, 49, 59, 62, 116, 124, 125

M

Mach · 20, 33, 34, 35, 94, 95, 96
mailbox · 21, 42, 109
main memory · 23, 24, 25, **84**, 88, 158
Makefile · 139
many-to-many · 47, 49, 52, 115, 116, 118, 161
many-to-one · 21, 47, 49, 116, 118, 161, 165
Many-to-one / one-to-one / many-to-many · 47, 49, 52, 116, 118, 161, 165
maskable · 20, 37, 44, 99, 139, 165
mechanism · 20, 31, 35, **93**, 97, 163
Medium-term scheduler · 20, 36, 37, 44, 103, 111, 160
Memory stall · 21, 59, 125, 126, 148
Message passing · 21, 33, 42, 96, 97, **108**
Microkernel · 32, 33, **95**, 96, 97, 158, 164
middleware · 20, 22, 23, 27, 82
mkfifo · 43, 78, 109, 111, 142, 164, 168
MLFQ · 21, 57, 58, **123**, **124**, **129**, **130**
Mode bit · 16, 20, 37, 38, 45, 73, 164
Monolithic · 32, 33, 35, **94**, 96, 97
MS-DOS · 20, 32, 35, 93, **94**, 164
Multicore · 26, 59, **85**, **113**, **125**, 159
multilevel feedback queue · 57, **123**, 161
Multilevel queue · 16, 21, 57, 58, **123**, 161
multiprocessor · 20, 25, 26, 64, 66, **85**, 163
Multiprogramming · 20, 26, 36, **86**, 103, 163
multitasking · 26, 27, 32, 86, 93, 164
mutex · 66, 70, 71, 80, **135**, 144, **169**
Mutex lock · 21, 66, 72, 84, 136, 138, 150

Mutual exclusion · 63, 64, 65, 66, 68, 70, **134**

N

named pipe · 16, 22, 43, 78, 109, 111, 162
Named pipe (FIFO) · 16, 22, 43, 78, 109, 111, 162
next_available_pid · 63, 132, 166
non-blocking · 21, 42, 43, 109, 142
non-preemptive · 21, 53, 54, 56, 63, 120, **122**
Non-preemptive scheduling · 21, 53
nonmaskable · 20, 37, 44, 99, 139, 165
NUMA · 26, 59, **86**, 88, 125, 162
NVM · 20, 23, 24, 27, 83, 84, 159

O

O_CREAT · 18, 44, 73, 76, 140, 141, 168
O_RDONLY · 18, 44, 76, 78, 110, 140, 168
O_WRONLY · 18, 44, 73, 76, 78, 140, 168
one-to-one · 21, 47, 49, 52, 116, 118, 161
open() · 18, 30, 44, 76, 78, 91, 168
Ordinary pipe · 16, 43, 45, 75, 76, 78, 162
Orphan · 21, 41, 81, **105**, **140**

P

PA1 · 21, 40, 41, **110**, **139**, **140**, **141**
PA2 · **139**, **143**, 145, 170
PaaS · 20, 27, 87, 163
Parallelism · 46, 52, **113**, **114**, 118
parameter passing · 20, 29, 89, **91**, 97, 159
PCB · 36, 37, 47, 73, **102**, 164
pclose · 142, 168
PCS · 21, 26, 49, 59, 125, 131, 162
PCS (process-contention scope) · 21, 26, 49, 59, 125, 131, 162
Peterson · 21, 64, 68, 69, 72, **137**, **144**
Pipe · **18**, 43, 76, **110**, **140**
policy · 16, 31, 35, **93**, **129**, 163
Policy vs mechanism · 20, 31, 89, **93**, 98, 163
popen · 21, 41, 78, 141, 142, 168
port · 31, 33, 35, 44, 110, 148, 164
possible values · 22, 69, 150, 151
preemptive · 53, 56, 58, 63, **122**, **130**, 166
Preemptive scheduling · 21, 53, 123, 162
printf · 17, 29, 40, 92, 107, 108, 158
Priority scheduling · 21, 56, 58, 62, **122**, 123, 161
Privileged instruction · 38, 45, 73, 97, 100, 111, 162
Process · 36, 53, 73, **100**, **101**, **102**, **104**
process state · 16, 21, 36, 37, 47, **101**, 102
process tree · 17, 99, 106, 107, 146
Processor affinity · 21, 59, 60, 86, **125**, 151
producer · 42, 63, 70, 80, **108**, **132**
Program · 22, 29, 32, 35, 38, **101**
programmed I/O · 36, 53, 54, **87**, **99**, **110**, **120**
Progress · 46, 64, 65, 68, 72, 133, 161
ps aux · 139, 140
pthread_cancel · 21, 51, 117, 169
pthread_cond_wait · 71, 72, 137, 144, 170
pthread_create · 21, 48, 50, 51, 79, 117, 169
pthread_exit · 21, 48, 78, 79, 117, 142, 169
pthread_join · 48, 50, 51, 79, 80, 81, 117
pthread_mutex_init · 79, 117, 144, 169
pthread_mutex_lock · 63, 71, 80, 144, 157, 159, 170
Pthreads · 21, 48, 51, **117**, **142**, **167**, **169**
Pull / push migration · 21, 59, 60, 125, 126, 131, 165
pull migration · 60, 125, 126, 131, 151, 165
push migration · 60, 126, 165

R

Race condition · 21, 63, **132**, **136**, 138, 146
rate-monotonic · 21, 60, **126**, 166
Rate-monotonic (RM) · 21, 60, **126**, 166
read() · 29, 30, 43, 73, 91, 97, 162
read_count · 71, 80, 81, 137, 144, 157
Ready queue · 36, 54, 62, 73, 103, 120, 125
rendezvous · 21, 42, 67, 109, 164
resource allocator · 20, 22, 82, 88, 162
resource sharing · 28, 39, 46, **89**, 104, 162
Response time · 26, 53, 54, 55, 120, 121, 124
responsiveness · 46, 113, 118, 162
RM · 60, 61, 62, **126**, **127**, **143**
round robin · **121**, 161
Round robin (RR) · **121**, 161
RPC · 21, 44, 109, **110**, 160
RR · 54, 55, 58, **121**, **122**, **130**, **143**
rw_mutex · 71, 80, 81, 137, 144, 157

S

SaaS · 20, 27, 87, 163
scalability · 46, 113, 162
SCS · 21, 59, 118, 120, 125, 131, 162
SCS (system-contention scope) · 21, 59, 118, 120, 125, 131, 162
sem_destroy · 79, 80, 144, 169
sem_init · 79, 80, 81, 117, 137, 144, 169
sem_post · 66, 71, 79, 80, 81, 159, 169
sem_wait · 79, 80, 81, 117, 137, 144, 169
Semaphore · 66, 67, 69, 136, **137**, 138, **169**
Shared memory · 21, 23, 28, 42, 45, **108**
shell · 29, 32, 40, 73, **139**, **141**, **169**
short-circuit · 105, **106**, 141, 155
short-term scheduler · 36, 37, 44, 73, 103, 111, 160
SIGCHLD · 52, 117, 118, 165
SIGINT · 21, 41, 73, 77, 116, 141, 167
Signal · 51, 66, 67, 70, **105**, **116**, **167**
signal handler · 51, 73, 145, **167**
SJF · 55, 56, 58, **122**, 124, 130
SMP · 25, 59, **85**, **125**, 166
Socket · 44, 45, 109, 110, 111, 148, 160
Soft affinity · 59, 62, 125
Soft real-time · 15, 21, 60, 61, 62, 120, 126
spatial locality · 24, 27, 85, 88, 148, 159
speedup · 21, 24, 46, 47, 52, **114**
Spinlock · 66, 69, 132, **135**, 138, 146
spooling · 20, 23, 27, 87, 139
SRTF · 55, 56, 58, **122**, 124, 130
Stack · 29, 35, 45, 52, 101, 112, 160
Starvation · 54, 56, 58, 68, 123, **136**, 161
STDOUT_FILENO · 76, 77, 168, 169
storage hierarchy · 20, 23, 25, 82, **83**, 84, 158
swap() · 19, 69, 135, 146
swapping · 20, 25, 26, 37, 44, 55, 70
synchronous signal · 162, 165
System call · 29, 38, **90**, **91**, 97, 100
System program · 20, 29, 35, 97

T

Task parallelism · 21, 46, 52, 112, **114**, 162
task_struct · 20, 21, 36, 52, 53, 102, 117
TCB · 45, 47, 52, **114**, **115**, 118
temporal locality · 24, 85, 148
test_and_set · 64, 65, **133**, **134**, 159
Thread · 48, 49, 52, **112**, **115**, **116**, **117**
thread cancellation · 51, 112, **116**, 162
thread library · 48, 49, 59, 62, 115, 125, 160
Thread-local storage · 21, 52, 53, **117**, 161, 165
Throughput · 21, 25, 53, 55, 61, 85, 88
time quantum · 21, 54, 62, 73, **121**, 161
timesharing · 82, **86**, 88, 163
Timesharing (multitasking) · 82, **86**, 88, 163
trap · 37, 38, 91, 99, 111, 153, 164
Trap / exception · 38, 91, 99, 100, 111, 161, 164
turnaround · 53, 54, 55, 56, 62, **121**
Turnaround time · 54, 55, 61, 62, 120, 121, 122
two-level · 21, 48, 49, 52, **130**, 151

U

unbounded capacity · 21, 42, 45, 109
UNIX · 29, 32, 43, 90, **94**, 109
User thread · 49, 52, 59, 115, 116, 118, 125
usleep · 140, 167
utilization · 21, 34, 54, 55, 60, 61, 166

V

Virtualization · 20, 26, **86**, 87, 88, 162
VMM · 20, 26, 38, 86, 87, 100, 163
VMM / hypervisor · 20, 26, 38, 86, 87, 88, 163
volatile · 15, 23, 24, 27, 84, 88, 158
von Neumann · 15, 20, 23, **83**, 88, 158, 160
von Neumann architecture · 20, 23, **83**, 160

W

wait queue · 36, 44, 73, 103
wait() · 41, 66, 67, 70, 72, 105, 140
Waiting time · 53, 54, 55, 56, 62, 121, 122
waitpid · 18, 73, 74, 75, 78, 110, 167
wc -l · 22, 43, 44, 73, 76, 139, 141
Windows · 33, 35, 49, 52, 94, **117**
write() · 20, 29, 30, 38, 43, 92, 158

Z

zero capacity · 42, 109, 164
Zombie · 41, 81, **105**, 111, **140**, 158